

考虑预测更新与运行平稳性的微网购电调控策略

摘要

负载、光伏与电价的变化使微网购电同时面临费用控制、供电保障和储能运行强度的权衡。本文以十分钟为调度间隔，统一交流侧电量、储电递推与实际结算口径，根据各问可获得的信息建立日前计划和日内滚动策略。年度方案均从1月1日6000 kWh初始储电量开始连续运行，题目报表统计2月1日至12月31日的334日结果。

对于问题一，建立两层混合整数规划，在功率爬坡、最短开启与关闭时间等约束下，先求最低购电费，再允许费用增加0.1%以减小功率总变差。典型日第二层购电量为57560.6438 kWh，费用为34115.03元，初末储电量均为6000 kWh。相对只考虑经济性的基准，费用增加0.93%，功率总变差下降67.66%，充放电开启次数由12次减至4次。

对于问题二，先等权融合直方图梯度提升树与极端随机树，再进行共同岭回归校准和前日负载偏差半量修正。利用最近28日的整日预测误差保留时间关联，依次执行模式规划、固定模式购电修正和当槽储能反馈。评价期总购电费为13195653.64元，实际非空充放电方向反转2535次；每日计划模式变化预算与实际反转次数分别核验，不将二者混同。

对于问题三，对官方光伏预报作同发布时间节点校准，以负载已观察前缀修正日内偏差，构造共享购电量的情景线性规划。模型在0、6、12、18时更新剩余计划，下调按取消部分退回原款一半，并按相对午夜计划的最终净差额结算。评价期费用为12788960.26元，其中下调退款77458.97元，紧急购电费202487.48元。

对于问题四，按价格逐步揭示的条件分别预测未来电价，将无日内更新和有日内更新的完整方案用于同一波动价格序列。两方案评价期费用分别为14008609.54元和13539823.62元，后者低3.35%。两种方案同时包含预测、模式限制和执行规则等差异，费用差不能全部归因于预报更新。逐槽复核表明各方案实际运行满足供需平衡、储电边界与功率上限；敏感性分析和固定计划扰动回放进一步说明结果的适用范围。本研究属于开发年度的因果回测，运行强度指标用于描述调控特征，不直接推算电池寿命。

关键词：微网购电；储能调度；混合整数规划；情景线性规划；预测校准

一、 问题背景与问题重述

1.1 问题背景

分布式光伏为社区提供了就地获取电能的途径，但其电力供给的时间分布与居民用电需求并不天然一致：发电充裕时可能出现剩余电量，而负载集中或光伏回落时又需要依赖外部电网。配置储能电池可以将不同时段电能供需连接起来，却也引入了充放电损耗、功率上限和安全储电范围等限制。题目给出的储能设备最大容量为 12000 kWh，最大充放电功率为 5000 kW，运行过程中储电量须保持在 1200–10800 kWh 之间，充放电效率为 90%，并给定了评价期初始储电量。这些参数使得相邻时段的调度决策相互关联，购电、充电、放电和剩余电量必须同时满足能量平衡与设备运行约束。

题目同时提供了不同层次的外部信息：附件 1 给出典型日的分时电价、社区负载和光伏预测，附件 2 给出全年按时段记录的负载与光伏实际功率，附件 3 给出每天多个时刻发布的未来 24 小时光伏预报，附件 4 进一步给出随日期和时段波动的外网电价。因而，调度者面对的并不是一次性已知的全天数据，而是随着时间推进逐步获得信息的决策过程。若计划购电量不足以覆盖实际负载，就需要以交易时刻电价的 5 倍进行紧急购电；若频繁根据新预报调整计划，又会产生计划与调整电量之间的违约或超额购电费用。

因此，微网运营的关键并非单纯提高某一时段的自给率，而是在持续满足负载的前提下，协调购电时机、储能状态与预测风险，控制整个运行周期的购电支出。现实调度还存在“计划先制定、信息后到达”的矛盾。购电计划偏乐观可能引发高价紧急购电，过度保守则可能为未被利用的电量支付费用；中途调整购电安排有助于修正计划，但调整本身也有代价。因此，设计一个“预测—计划—修正—执行—结算”一体化的调度分析模型，对建设兼具经济性与可靠性的微网发电系统具有重要意义。

1.2 问题重述

本文以光伏电源、社区负载、储能电池及外网组成的系统为对象，以十分钟时段的购电量和储能动作作为主要决策变量，将原题四问归纳为以下递进任务。四个问题由理想化的典型日逐步推广到全年实测数据、滚动预测和波动电价，分别考察确定性调度、预测误差下的日前计划、信息更新下的动态调整以及价格与供需同时变化时的综合决策。

问题一要求在每天 0:00 利用典型日的电价、社区负载和给定的光伏功率预测，确定全天各十分钟时段的计划购电量以及储能设备的充放电安排。微网提供的电能不得低于社区负载，并要求储能设备在 0:00 和 24:00 的储电量相同。我们建立两层混合整数线性规划模型，在日初日末储电量一致的条件下，先求最低购电费，再在小幅费用让步范围内改善电池充放电的平稳性。购电量作为题目要求的结果进行统计，不再单独最小化，并按题目给定的表 1、表 2 格式汇总指定时段和全天结果。

问题二要求在每天 0:00 制定全天计划，而当天的社区负载和光伏实际功率将在运行过程中逐步实现。我们先依据历史负载和光伏数据构建预测模型，根据预测得到的每日负载和发电数据制定计划购电量与储能安排；执行时，若计划供电低于实际负载，则需要在相应时段按交易电价 5 倍进行紧急购电。除紧急购电费用外，其余时段按照 0:00

锁定的计划购电量结算。该问题需要利用附件 1 和附件 2 完成 2025 年全年滚动评价，并按题目给定格式报告指定日期的计划购电、充放电、全天费用及紧急购电区间。

问题三引入每日 0、6、12、18 时发布的未来 24 小时整点光伏预报，研究是否以及如何利用新信息修正尚未执行的购电计划。根据 0:00 预报制定的计划与后续调整之间存在不对称费用：计划购电量高于调整购电量的部分按交易时刻电价的 50% 计收违约费用，调整购电量超过计划购电量的部分按交易时刻电价的 1.5 倍计收；此外还要计入正常计划购电费用和可能发生的紧急购电费用。该问题重点衡量预报提前量、未来更新机会与调整成本之间的关系，判断是否有必要使用 6:00、12:00 和 18:00 的额外预报，并评价这些更新信息的实际决策价值。

问题四将外网的固定日内价格扩展为随日期和时段变化的实时电价，在明确价格可见性的前提下，分别重新研究无额外光伏预报和有日内预报的两类策略。我们利用附件 2 中的社区负载与光伏实际功率、附件 3 中的光伏预报以及附件 4 中的全年电价数据，重新计算问题二和问题三，比较波动价格、供需预测误差及信息更新时间共同作用下的购电安排、储能运行和总费用变化，并将两类策略的完整结果分别保存为题目要求的文件。

二、问题分析

本文大致可分为两大模型：第一是依据过往数据，建立神经网络模型，用以预测每天每个时段的小区负载、光伏发电、波动电价等数据；第二是在得到预测数据的基础上，建立一个动态规划模型，制定使费用最小化的购电方案，并根据最新信息实时滚动调整。本文模型在四个问题的逻辑关系见图 ??。

问题 1 分析。在固定每日负载、发电和电价的基础上，我们需要制定一个使购电成本最小化的购电安排。然而，考虑到实际电网运行中，如果储能设备的充放电功率发生频繁的、碎片化的跳变，会严重影响电池寿命，带来不容忽视的设备折旧损失。因此，我们建立了双层混合整数线性规划，在尽可能减少购电成本的同时，限制储能设施的功率波动与启动次数，使得模型更加符合现实应用场景。

问题 2 分析。我们采用了卷积神经网络模型得到 2-12 月份的小区负载和光伏发电的预测数据，并通过滚动法，仅将被预测日之前的真实数据输入模型，避免循环论证的逻辑问题。以条件误差树描述净需求偏差，再通过储能动态规划制定日前购电。计划全天锁定，电池根据当槽实际供需调整充放电，缺口按 5 倍电价补购。关键矛盾是计划提前锁定与供需逐步揭晓，少购需高价补购，多购需承担剩余电量费用。先融合梯度提升树与极端随机树预测供需并校准偏差；再以整日历史误差构造相关情景，刻画连续缺口对储能的累积影响；据此先规划共同充放电模式，再固定模式修正购电量，按实测供需反馈并跨日传递状态。

问题 3 分析。关键在于新预报的纠偏机会与计划调整代价之间的权衡。先校准各发布时间的光伏预报并修正负载偏差；再以同发布时间历史误差构造情景，建立计及上调加价、下调退款和补购风险的线性规划。每次从实际储电量出发，仅重排未执行时段并考虑后续更新机会，结合误差与费用分析日内预报的必要性。

问题 4 分析。关键在于电价与供需同时不确定，使低价采购和缺口防范相互牵制。先以历史电价及周期特征建立加权岭回归，再扩展问题二、三的框架：无日内预报时采用小时共同模式与购电修正，有日内预报时更新价格并滚动调整。最后按实际电价结算，从费用和运行强度比较完整策略，并结合预测与控制差异界定比较范围。

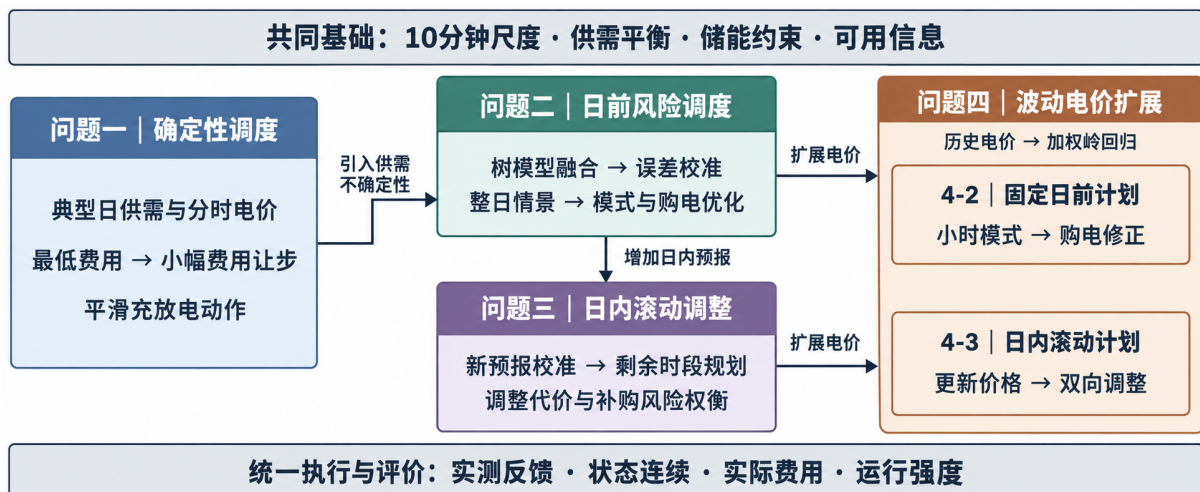


图 1 我们的工作

三、模型假设

- 假设 1：以单母线聚合系统描述微网。**将光伏电源、社区负载、储能设备与外网接入点视为统一的能量交换系统，以 10 分钟区间平均功率建立供需平衡，忽略内部线路损耗及电压、频率的暂态变化。附件时标按区间右端点处理，整点光伏预报经插值、积分后转换为相同时间尺度的平均功率。
- 假设 2：储能效率与可用容量在研究期内保持不变。**将题面 90% 的充放电效率解释为往返效率，并假设两个方向对称，取 $\eta_c = \eta_d = \sqrt{0.9}$ 。储电量以电池内部电量计，充放电量以交流侧电量计，同一时段不同时充电和放电。调度始终满足题设容量、功率和安全储电范围，不单独计入自放电、辅助耗电及容量衰减。
- 假设 3：日初、日末储电量相等的条件仅适用于问题一。**问题一按典型日循环运行，取 0:00 与 24:00 的储电量均为 6000 kWh。问题二至问题四按连续日期运行，各方案从 2025 年 1 月 1 日 0:00 的 6000 kWh 出发，使用各自的预测、规划和反馈规则完成一月份预热。每槽依据实际供需执行充放电并递推内部储电量，将 1 月 31 日 24:00 的实际运行状态作为 2 月 1 日 0:00 的初始状态。此后以前一日末储电量作为次日初值，不再施加每日初末相等的条件；不同方案的二月初始储电量可因一月运行轨迹不同而不同。
- 假设 4：将抑制不必要的功率波动和充放电切换作为运行要求。**考虑到频繁启停、短时反复换向及过大的功率变化可能增加设备运行负担，并影响电池寿命、供电运行效率和长期成本，在满足负载与储能安全边界的基础上，尽量减少碎片化充放电。各问依据模型结构，采用功率变化上限、最短运行时间、模式变化预算或功率总变差与切换惩罚实现这一要求。
- 假设 5：负载不可主动削减，外网能够正常补充供电。**社区用电需求须得到满足，不考虑需求削减及停电情形，也不另设外网购电容量约束。紧急购电仅补足常规供电与储能放电后仍存在的负载缺口，不用于充电；无法利用的剩余电量允许舍弃，其来源可以是剩余光伏或已付费的计划购电。
- 假设 6：购电决策遵守信息发布顺序，区间内反馈足够及时。**在各问允许的发布时刻，只依据此前实测记录、已知参数及已发布预报确定或调整购电量，已执行时段不再修改。假设测量和控制能够在十分钟区间内及时响应，使储能动作及紧急补购随当前实际供需确定。问题四的未来电价按逐步揭示处理，规划采用当时的价格预测，结算采用实际成交时刻的电价。
- 假设 7：近期历史误差能够反映短期预测的不确定性。**在考虑日历周期与近期状态后，假设最近至多 28 日的已实现预测误差可用于后续校准和情景构造，并保留整日误差路径的时间关联。该假设不要求全年误差同分布，也不要求各时段误差独立或服从正态分布；历史不足时按各问的冷启动规则处理。

四、符号说明

公共符号见表 ??，回归系数及各问局部辅助变量在使用处定义。 E 表示内部储电量， e 表示紧急购电量。

表 1 主要符号及单位

符号	含义	单位
d, t	日期与日内槽索引, $t = 1, \dots, 144$	—
Δt	槽长, 取 1/6	h
τ	预报发布小时, 取 0、6、12、18	h
$L_{d,t}, V_{d,t}$	区间平均负载、光伏功率	kW
$p_{d,t}, \hat{p}_{d,t}$	实际基础电价与预测电价	元/kWh
$g_{d,t}$	午夜原计划购电量	kWh
$q_{d,t}$	该槽最后有效版本的常规购电总量	kWh
$a_{d,t}^+, a_{d,t}^-$	相对午夜原计划的最终净上调、净下调量	kWh
$e_{d,t}$	实际紧急购电量	kWh
$c_{d,t}, d_{d,t}$	交流侧充电、放电量	kWh
$P_{d,t}^c, P_{d,t}^d$	交流侧充电、放电功率	kW
$E_{d,t}$	第 t 槽开始时的内部储电量	kWh
$E_{\min}, E_{\max}$	内部储电量下、上界	kWh
$P_{\max}$	单方向最大功率	kW
η_c, η_d	单向充、放电效率, 均为 $\sqrt{0.9}$	—
$w_{d,t}$	总未利用电量	kWh
$\mathcal{I}_{d,\tau}$	购电发布时刻可用信息集	—
s, π_s	情景索引与权重	—
C	实际购电费用	元
J	各问规划代理目标	见各问定义

功率与交流侧电量按 $c = P^c \Delta t$ 、 $d = P^d \Delta t$ 转换。以 $n = (L - V) \Delta t$ 表示净需求电量，实际执行满足

$$q + d + e - c - w = n, \quad E_{t+1} = E_t + \eta_c c_t - d_t / \eta_d. \quad (1)$$

普通购电量 q 不含紧急购电，外网总购电量为 $q + e$ 。问题二和 4-2 中 $q = g$ ；问题三和 4-3 中，未执行区间采用最后一次有效更新，令 $a^+ = (q - g)^+$ 、 $a^- = (g - q)^+$ ，实际费用统一为

$$C = \sum_{d,t} p_{d,t} (g_{d,t} + 1.5a_{d,t}^+ - 0.5a_{d,t}^- + 5e_{d,t}). \quad (2)$$

工作簿的“调整购电量”填写 q ，不是 $a^+ - a^-$ ，不再与 g 重复相加。吞吐、启动、功率变化惩罚与日末库存估值只进入 J ，不属于式 (??) 的真实账单。文中费用保留 2 位小数、电量保留 4 位小数，差额和比例均由未舍入结果计算。

五、问题一的模型建立与求解

问题一给定典型日负载、光伏出力和分时电价，要求在满足供电需求及储能运行限制的条件下，确定全天购电与充放电计划，并保持日初、日末储电量一致。储能可通过转移购电时段降低电费，但仅以购电费最小为目标，可能产生频繁启停和较大的相邻时段功率变化，因此还需考虑调度方案的运行特征。

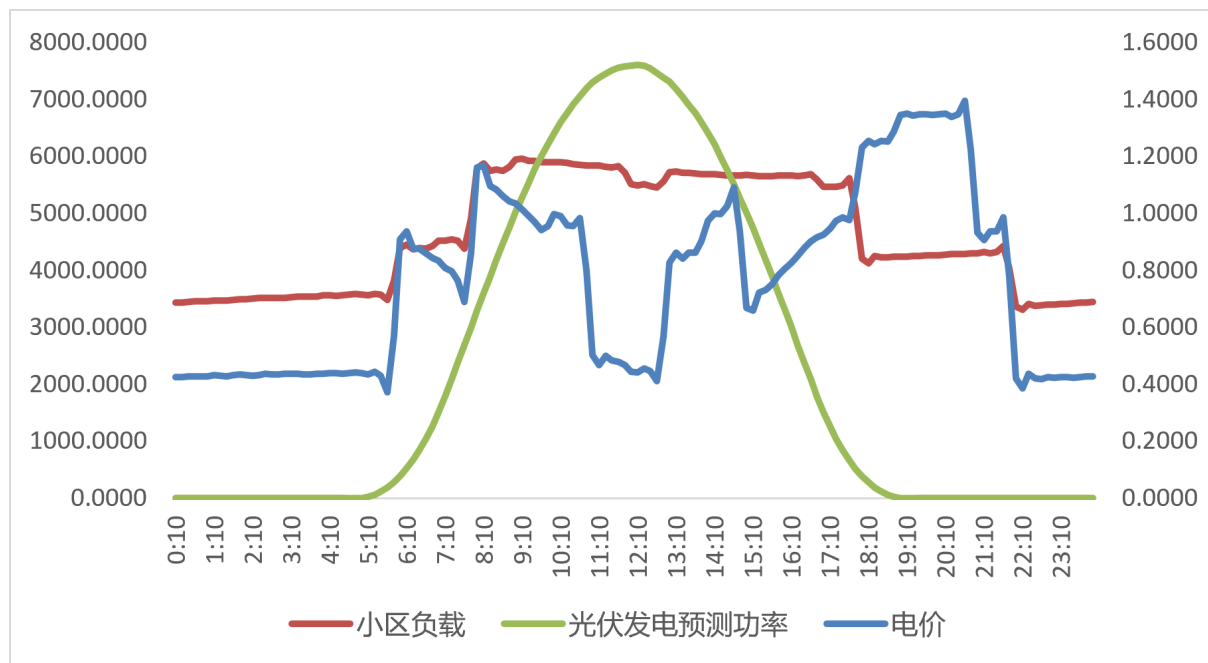


图2 典型日负载、光伏与电价

电池老化是储能运行的实际成本来源。已有研究表明，电池退化与循环深度、荷电状态及温度等因素有关[?]，不利充电条件下的析锂还会造成活性锂损失[?]。Xu等将循环老化成本纳入电力市场调度，指出忽略该成本会影响储能收益评价¹。因此，购电费最低并不等同于全寿命成本最低。不过，动态功率对寿命的影响取决于具体工况，部分动态放电实验反而观察到寿命延长[?]，不能将功率跳变直接等同于寿命损失。本文据此将限制功率变化和减少启停作为抑制不必要调节的工程措施，以小幅电费增量换取更平稳的运行；由于缺少电芯老化及更换成本数据，不将平稳性指标折算为折旧费。

5.1 混合整数线性规划模型

本文采用混合整数线性规划（mixed-integer linear programming, MILP），以连续变量表示电量，以二元变量表示充放电状态。建模过程包括输入时序数据、建立供需与运

¹Xu B, Zhao J, Zheng T, et al. Factoring the Cycle Aging Cost of Batteries Participating in Electricity Markets. arXiv:1707.04567, 2017. DOI: 10.48550/arXiv.1707.04567.

行约束、设置优化目标和求解调度计划。为兼顾电费与运行平稳性，采用 ε 约束法²：先求最低购电费，再将允许的费用增幅转化为约束，优化功率变化与启停次数。

将全天分为 $T = 144$ 个十分钟区间， $\Delta t = 1/6$ h。附件以区间右端点记录功率，将其作为该区间平均功率；10:00–10:10 对应右端点 10:10 的记录。记 L_t, V_t 为负载与光伏功率，净需求电量为 $n_t = (L_t - V_t)\Delta t$ 。典型日输入见图 ??。

记 g_t, c_t, d_t, w_t 分别为购电、交流侧充电、放电及未利用剩余电量， E_t 为区间初内部储电量，单位均为 kWh。根据题目设定取往返效率 90%，对称分解为 $\eta_c = \eta_d = \sqrt{0.9}$ ，得到

$$\begin{aligned} g_t + d_t &= n_t + c_t + w_t, & g_t, c_t, d_t, w_t &\geq 0, \\ E_{t+1} &= E_t + \eta_c c_t - d_t / \eta_d, \\ 1200 &\leq E_t \leq 10800, & E_1 &= E_{T+1} = 6000. \end{aligned} \quad (3)$$

以 $z_t^c, z_t^d \in \{0, 1\}$ 表示充、放电模式，额定功率为 $P_{\max} = 5000$ kW。功率上限与互斥约束为

$$0 \leq c_t \leq P_{\max} \Delta t z_t^c, \quad 0 \leq d_t \leq P_{\max} \Delta t z_t^d, \quad z_t^c + z_t^d \leq 1. \quad (4)$$

多余电量用 w_t 记录。

第一层只最小化实际购电费用：

$$C_1^* = \min \sum_{t=1}^T p_t g_t. \quad (5)$$

第二层沿用第一层的全部约束，并增加费用预算

$$\sum_t p_t g_t \leq (1 + 0.001) C_1^* + 10^{-4}, \quad (6)$$

再最小化归一化运行指标

$$F_2 = \frac{\sum_t v_t}{2P_{\max}T} + \frac{\sum_{t,a} s_t^a}{2T} + \frac{\sum_{t,a} \sum_{k=1}^2 b_{t,k}^a}{4T}, \quad v_t \geq \pm(P_t - P_{t-1}). \quad (7)$$

其中 $b_{t,k}^a$ 表示在 t 槽启动后未能持续到第 $k+1$ 槽的缺额标记。当前最短开启 3 槽的设置使其取零，保留该项便于与放宽最短持续时间的经济基准比较。三个分母用于归一化功率变化、启动次数与短段缺额，均不改变真实购电账单的定义。

5.2 求解结果与典型日策略

采用 HiGHS 混合整数规划求解器完成两层优化 [?]。第一层最低购电费为 34080.95 元，第二层费用为 34115.03 元，两层归档 MIP 相对间隙均为 0。第一层与第二层求解时

²Mavrotas G. Effective implementation of the ε -constraint method in Multi-Objective Mathematical Programming problems. Applied Mathematics and Computation, 2009, 213(2): 455–465. DOI: 10.1016/j.amc.2009.03.037.

间分别为 0.35 s 和 7.58 s, 仅表示对应环境中的单次求解耗时。表 ?? 给出分层结果, 表 ?? 和表 ?? 均由第二层得到。

表 2 两层混合整数规划求解结果

指标	经济基准	两层规划方案
购电费/元	33801.50	34115.03
购电量/kWh	57526.2435	57560.6438
循环功率总变差/kW	102684.4368	33209.1661
最大相邻净功率变化/kW	8546.8412	1000.0000
运行段启动次数/次	12	4
短于 30 分钟的运行段数/段	6	0
转换损耗/kWh	1984.2710	2018.6714

表 3 指定时段购电量及全天购电结果

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.0000	12:00–12:10	480.3510	14:00–14:10	0.0000
16:00–16:10	445.4317	18:00–18:10	621.8476	20:00–20:10	2.2591
全天购电量: 57560.6438 kWh			全天购电费: 34115.03 元		

表 4 储能分时充放电量与起止储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	5059.6443	0.0000	4:00–8:00	0.0000	7357.6857
8:00–12:00	5071.6806	1702.9970	12:00–16:00	4995.7446	0.0000
16:00–20:00	0.0000	6729.2480	20:00–24:00	5059.6443	2378.1117
0:00 储电量: 6000.0000 kWh			24:00 储电量: 6000.0000 kWh		

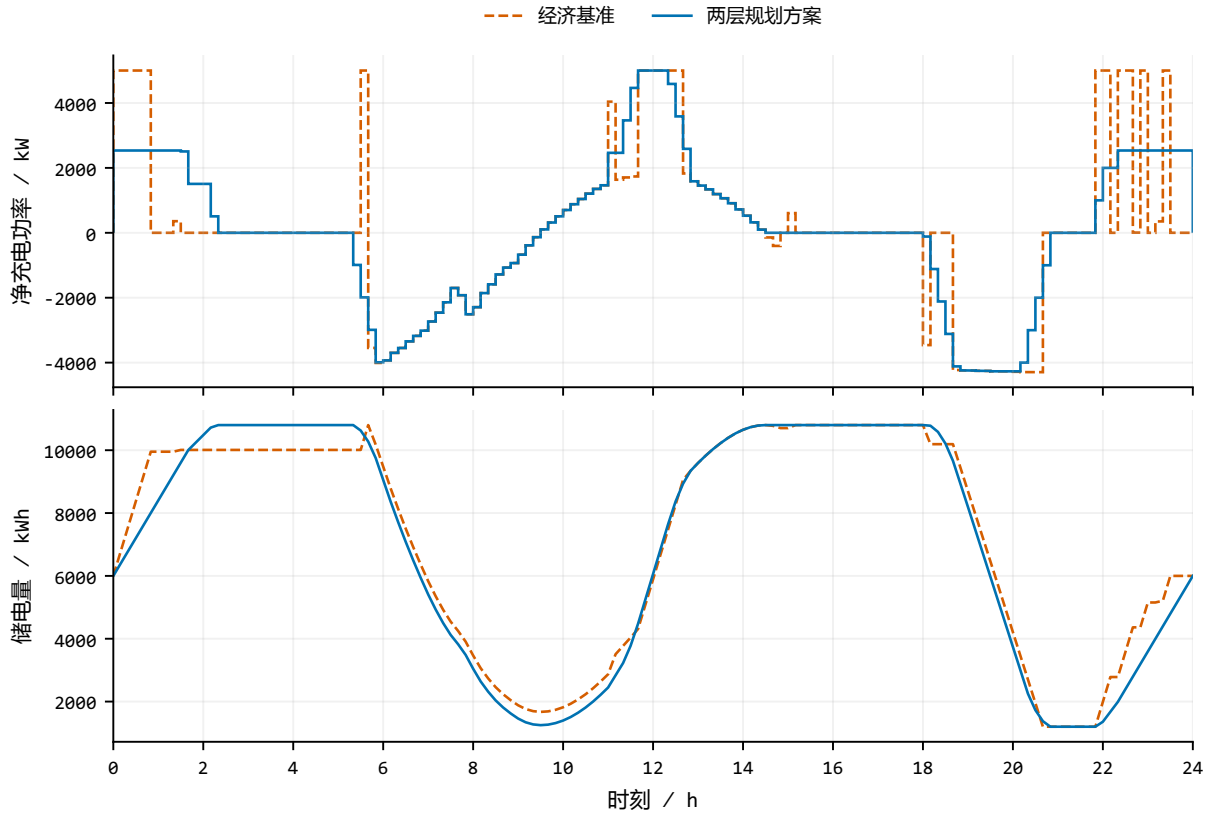


图 3 典型日电池净充电功率与储电轨迹

从图 ?? 和表 ?? 可以发现，双层优化相对基准的单层优化增加了购电费 313.54 元，约 0.93%，但电池充放电循环净功率总变差为 33209.1661 kW，相对经济基准减少 67.66%，运行段启动次数由 12 次降为 4 次，短于 30 分钟的运行段由 6 段降为 0 段。这表明附加设置确实抑制了短时碎片化动作，恰恰证明了我们模型的现实价值和必要性。

六、问题二的模型建立与求解

问题二在每日 0 时锁定全天购电量。计划不足需按 5 倍电价紧急补购，未利用的计划电量不能退款。本文先预测供需，再以历史整日误差构造路径，依次确定充放电模式、修正购电量，并按当槽实况执行储能动作。

6.1 日前预测与整日误差路径

负载与光伏分别采用直方图梯度提升树（HGB）和极端随机树（ExtraTrees），以昨日、上周同槽、近期统计量和日历周期等 31 个历史特征预测功率。两类树按月训练、月内参数冻结，随机种子均为 42；输入仅取发布前已观测数据。先将两类原始输出等权融合，再用最近至多 28 个已完成日的原始融合误差作共同岭回归校准，以加权残差平方和与系数正则惩罚之和最小为目标，最后叠加前日负载平均残差的一半。该残差相对半

量修正前的岭回归输出计算，不递归累加；功率预测截为非负，光伏限制在由历史数据确定的发电时段内。

预测从 1 月 1 日开始：首日使用附件 1 先验；一月负载取上周同槽，未满一周时取昨日同槽，光伏取昨日同槽；二月起接入上述树模型。每日将最近至多 28 个已完成日当时发布的净需求预测误差整条加到当天预测上，保留连续偏高或偏低的时间关联。一月已完成日同样进入路径集合，无历史时使用一条零误差路径。

6.2 模式规划与购电修正

从历史路径中按日期等间隔选取至多 3 条，建立共享购电量和充放电模式的情景混合整数规划。各路径分别计算储能响应，满足供需平衡、储电量与功率边界，并限制每日计划模式变化：

$$\sum_{t=1}^T |m_t - m_{t-1}| \leq 8, \quad (8)$$

式中， t 为日内槽索引， $T = 144$ ； m_t 为共同计划模式，取 1 允许充电、取 0 允许放电， m_{t-1} 为前槽模式。 m_0 承接前一日的计划末模式，故午夜边界也计入 8 次预算。模式最短保持 1 槽，计划模式变化与实际非空动作反转分别统计。

模式规划最小化代理目标

$$J_{\text{MIP}} = \sum_t p_t g_t + \frac{1}{K} \sum_{h=1}^K \left[\sum_t \{5p_t e_t^{(h)} + 0.002(c_t^{(h)} + d_t^{(h)})\} - v_{\text{MIP}}(E_{T+1}^{(h)} - 1200) \right]. \quad (9)$$

式中， J_{MIP} 为代理费用， p_t 为已知电价（元/kWh）， g_t 为共享计划购电量（kWh）； K 为所选路径数， h 为其编号。 $e_t^{(h)}$ 、 $c_t^{(h)}$ 、 $d_t^{(h)}$ 分别为路径中的紧急购电、充电、放电量（交流侧 kWh）， $E_{T+1}^{(h)}$ 为日末内部储电量。 v_{MIP} 为每 kWh 内部期末库存的估值，普通日取当日最低电价除以充电效率 η_c ，12 月 31 日取 0；1200 为最低储电量，0.002 为吞吐代理权重。

随后固定整日模式，在全部至多 28 条路径上回放储能反馈，以 L-BFGS-B 修正 144 个非负购电量。修正目标仍含计划费、平均紧急费及 0.002 元/kWh 吞吐权重，普通日期末库存估值改为 0.45 元/kWh，末日为 0。模式规划每次限时 5 s、目标相对间隙 0.2%；购电修正最多 120 次迭代，采用有限迭代返回值，不保证全年费用全局最优。

6.3 实际执行与结算

购电量在 0 时修正完成后锁定。运行时，在计划模式允许的方向上，根据当槽实际余电或缺口、功率上限和可用储电空间确定充放电量；充电死区为 0，不以规划路径中的充放电意图量作为上限。仍有缺口时紧急补购，余电允许舍弃；紧急购电不用于充电，同槽不同时充放电。内部储电量按式 (??) 递推。

真实费用只计锁定计划购电费与 5 倍电价的紧急购电费，吞吐权重和期末库存估值不进入账单，也不形成退款或残值收入。运行从 1 月 1 日 6000 kWh 开始，每日传递实际末储电量、计划末模式及已保持槽数，普通日不要求恢复到 6000 kWh。全年连续运行 365 日，附件 5 工作簿汇总 2-12 月 334 日结果。

6.4 指定日期与全年结果

表 ?? 列出四日计划购电量与全天费用，表 ?? 汇总四日充放电及起止储电量，表 ?? 给出紧急购电区间。购电量为 0 时锁定值，费用含紧急补购；时间段按左闭右开区间统计。

表 5 2025 年四个指定日期的计划购电量及全天费用

时间段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	48.2652	0.0000	0.0000	45.6628
12:00–12:10	606.0250	0.0000	491.0172	948.0275
14:00–14:10	0.0000	0.0000	0.0000	0.0000
16:00–16:10	439.5483	220.1115	503.9053	868.6242
18:00–18:10	673.0022	132.0644	746.5827	684.9746
20:00–20:10	25.2301	0.0000	127.3036	33.5085
全天计划购电量/kWh	65277.1470	30903.9138	67847.2652	93111.4419
全天购电费/元	40076.53	19195.23	43261.35	60110.89

表 6 2025 年四个指定日期的充放电量及起止储电量（kWh）

时间段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
充电量（kWh）				
00:00–04:00	6395.6805	0.0000	7093.1118	7517.4410
04:00–08:00	2500.0565	787.9293	716.6349	3426.9275
08:00–12:00	5809.0850	10119.2885	5277.7118	5176.1069
12:00–16:00	3657.8022	0.0000	2459.3218	6372.7058
16:00–20:00	400.6455	0.0000	0.0000	0.0000
20:00–24:00	1150.3048	2055.9982	1918.6147	0.0000
放电量（kWh）				
00:00–04:00	0.0000	1438.8347	0.0000	0.0000
04:00–08:00	6254.0246	3286.6826	4990.4167	2445.3267
08:00–12:00	2732.1701	0.0000	1896.6157	6897.5587
12:00–16:00	671.0733	0.0000	407.6353	2685.3895
16:00–20:00	5806.4536	5634.7636	6126.8605	5941.8612
20:00–24:00	2818.9397	3340.3250	2778.3055	2895.6742
0:00 储电量	2851.9662	5433.6372	3391.0237	2140.0098
24:00 储电量	2472.0243	3289.9172	2884.0282	1484.4197

表 7 四个指定日期的连续紧急购电区间及电量

3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
01:00–01:20	14.9026	06:40–07:20	247.6871	14:50–15:10	20.5010	03:30–03:50	73.6481
–	–	–	–	20:50–21:00	26.1799	10:40–10:50	36.2066

表 ?? 同时给出全年账单与附件 5 要求区间的账单，二者不混用。

表 8 问题二不同统计区间的购电量与实际费用

指标	1–12 月（365 日）	2–12 月（334 日）
计划购电量/kWh	23465106.4174	20519293.0944
紧急购电量/kWh	154624.4448	104033.4282
外网合计购电量/kWh	23619730.8622	20623326.5225
计划购电费/元	14518936.61	12636712.82
紧急购电费/元	811967.83	558940.83
合计费用/元	15330904.44	13195653.64

预测误差采用 2–12 月每日午夜最终输出，所有槽各计一次，样本数均为 48096；结果见表 ??。MAE 为平均绝对误差，RMSE 为均方根误差，偏差取预测功率减实测功率

的均值。

表 9 午夜最终供需预测的误差（2-12 月，不重叠日窗口）

变量	MAE/kW	RMSE/kW	偏差/kW
负载	136.9731	182.9870	-3.5806
光伏	153.2594	307.0504	6.1798
净负载	242.0206	357.1618	-9.7604

一月费用为 2135250.79 元，2 月 1 日初始储电量为 3743.6008 kWh，由同一策略的实际末状态连续承接。四个指定日期共出现 6 个连续紧急购电区间；全年 52560 槽均满足供需平衡、功率与储电量约束，逐槽账单复算最大残差为 9.09e-13 元。

2-12 月实际反转 2535 次、活动槽 37259 个，交流侧吞吐 11340559.8068 kWh。反转剔除空闲动作后在评价区间内连续计数，不含一、二月连接边界，也不等同于计划模式变化次数。

365 次模式规划均取得可行解，113 次达到时限，最大实际相对间隙为 7.5327%；固定模式修正中 60 次按求解器成功条件停止。模式规划与购电修正累计求解时间分别为 1043.50 s 和 640.54 s，不含预测训练，亦非多进程总墙钟时间。图 ?? 给出各月计划费与紧急费；月度差异同时受天数和季节需求影响。

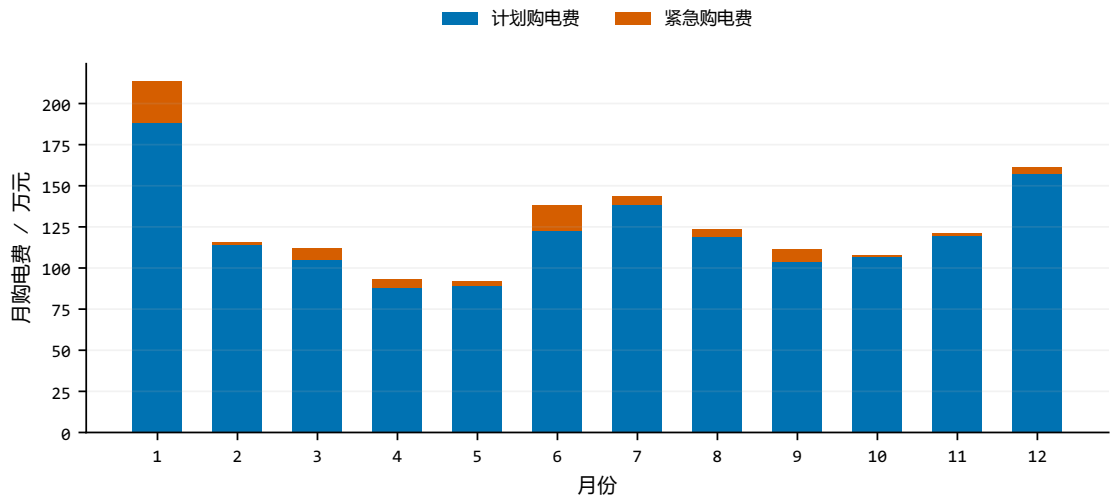


图 4 问题二全年各月计划购电费与紧急购电费

七、问题三的模型建立与求解

问题三在 0、6、12、18 时获得新预报，并允许调整尚未执行的购电计划。本文校准各次光伏预报，以同发布时间的历史误差构造情景，通过滚动线性规划权衡提前采购、日内调整和紧急补购。

7.1 预测更新与历史情景

午夜负载采用单 HGB，经近 28 日岭校准和前一日偏差半量修正，与问题二的两树融合不同。光伏从首日 0 时起使用当时发布的官方预报；预测和购电只能利用此前实况、已经发布的预报及已知分时价，未来实况在执行时才可见。

光伏节点校准。选取近 28 日同发布时间、完整 24 小时标签已到达的历史预报，以 14 日半衰期计算平均功率残差。先求 24 个节点的未收缩偏差解：目标为区间平均功率修正与平均残差之差的平方和，加 12 倍二阶差分平方惩罚及 2 倍修正幅度平方惩罚；再将该解额外乘以 $N/(N+7)$ ，其中 N 为可用历史发布数。收缩是求解后的独立步骤，无历史时修正置零。保留官方零发电节点，并将其余节点截为非负；以发布前最后实测值作锚点，线性插值并积分得到十分钟平均功率。

负载修正与情景。日内取最近至多 18 个已观察槽的负载误差，以 6 槽半衰期加权，并按样本数与样本数加 12 之比收缩；修正量以 36 槽为尺度向未来指数衰减，午夜修正取零。将负载减光伏换算为十分钟净需求电量后，叠加近 28 个完整历史日各自在同一时刻发布的误差路径，按日期等距选取至多 7 条、赋予相同权重。这样保留整日误差的时间关联；无历史路径时使用零偏差情景。

7.2 退款计费与滚动规划

最终调整以午夜原计划为基准，仅按该槽最后生效的版本结算：

$$a_{d,t}^+ = [q_{d,t} - g_{d,t}^0]_+, \quad a_{d,t}^- = [g_{d,t}^0 - q_{d,t}]_+. \quad (10)$$

式中 d 为日期， $t = 1, \dots, 144$ 为日内十分钟槽； $g_{d,t}^0$ 为午夜原计划， $q_{d,t}$ 为最终常规购电总量， $a_{d,t}^+, a_{d,t}^-$ 分别为净上调、净下调量，单位均为 kWh。实际账单为

$$C_{\text{tot}} = \sum_{d,t} p_t \left(g_{d,t}^0 + 1.5a_{d,t}^+ - 0.5a_{d,t}^- + 5e_{d,t} \right). \quad (11)$$

式中 C_{tot} 为统计区间的总购电费（元）， p_t 为分时电价（元/kWh）， $e_{d,t}$ 为实际紧急购电量（kWh）。上调部分收取 1.5 倍电价，下调取消部分退回原费用的 50%；中间被覆盖的版本不重复收费，后次调整不受前次计划单调增加的限制。

对每次发布至当日日末的剩余时段，求解情景线性规划：

$$\begin{aligned} \min \quad & \Phi_\tau + \sum_{s=1}^S \pi_s \left[\sum_{t \in \mathcal{T}_\tau} (\gamma_{\tau,t} p_t e_{s,t} + \lambda_H (c_{s,t} + d_{s,t})) \right. \\ & \left. + \lambda_{\text{TV}} \sum_{t=r_\tau+2}^{144} v_{s,t} - \nu_{d,\tau} (E_{s,145} - E_{\min}) \right]. \end{aligned} \quad (12)$$

式中 τ 为发布小时， $r_\tau = 6\tau$ 为已执行槽数， $\mathcal{T}_\tau$ 为当天剩余槽集合； $s = 1, \dots, S$ 为情景索引， $\pi_s = 1/S$ 为权重。 Φ_τ 在午夜为原计划购电费，日内为 1.5 倍上调费减 0.5 倍下

调退款，已付原计划费作为常数省略。 $e_{s,t}, c_{s,t}, d_{s,t}$ 分别为场景紧急购电、充电、放电量； $E_{s,145}$ 为日末内部储电量， $E_{\min} = 1200 \text{ kWh}$ 为下界；上述电量均以 kWh 计。 $v_{s,t}$ 为相邻净充电功率差绝对值的非负上界，单位为 kW。

$\gamma_{\tau,t}$ 为场景短缺费用倍数，下一发布前取 5，其后取 1.5，以近似后续上调机会；18 时后均取 5。吞吐权重 $\lambda_H = 0.005$ 元/kWh，功率总变差权重 $\lambda_{TV} = 0.001$ 元/kW。 $\nu_{d,\tau}$ 为日末库存估值系数（元/kWh），普通日取剩余时段最低电价除以单向充电效率，评价末日取零。这些代理惩罚和库存价值均不计入实际账单。

约束包括各场景供需平衡、考虑充放电效率的 SOC 递推、1200–10800 kWh 储电边界与单方向 5000 kW 功率界。各情景共享非负常规购电量；日内以“原计划加上调减下调”的线性关系引入非负调整量，午夜则直接确定原计划。场景初态继承本次发布时的实际储电量。这些条件构成储能响应的线性近似，单次代理目标的最优解不等同于全年真实费用的全局最优策略。

实际执行。每次只执行至下一发布时刻：当槽净盈余用于充电，净缺口先由电池放电、再紧急补购，动作同时受 SOC 和功率界限制；充电量不足 20 kWh 时取零。实际 SOC 跨发布时刻及自然日连续传递，已执行购电槽保持冻结，后续发布只覆盖未来计划。

7.3 求解结果与更新价值

从 1 月 1 日 6000 kWh 起连续运行本策略；一月负载采用问题二的周期冷启动安排，二月起接入单 HGB 预测。2 月 1 日实际储电量为 1779.6212 kWh，评价区间为 2 月 1 日至 12 月 31 日 334 日、48096 槽，末日实际储电量为 1200.0000 kWh。采用 HiGHS 求解，每次预算 20 s；全年 1460 次 LP 均获得可行解，规划累计 125.17 s，单次最长 0.41 s，运行与归档共 127.74 s。

表 ?? 给出评价期账单，总费用为 12788960.26 元。下调量 244156.6289 kWh 表明，新预报到达后可取消部分早期过量采购；真实轨迹满足物理边界，实际同时充放电和紧急购电用于充电的槽数均为零，账单按式 (??) 复算一致。

表 10 问题三评价期购电费用与运行结果

指标	数值
原计划费/元	11619999.62
上调费/元	1043932.12
下调退款/元	-77458.97
紧急购电费/元	202487.48
总费用/元	12788960.26
最终常规购电量/kWh	19832388.1712
下调电量/kWh	244156.6289
紧急购电量/kWh	40952.4203
非空方向反转/次	5767

指定日期结果。表 ?? 并列午夜原计划与最终常规购电量，全天费用对应最终结算。表 ?? 汇总四个指定日期的四小时充放电量及起止储电量；表 ?? 给出四日完整紧急购电区间。时间段按左闭右开区间表示，四小时量为其中 24 槽电量之和。

表 11 指定日期的原计划与最终常规购电量（kWh）

时段/指标	3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
	原计划	最终量	原计划	最终量	原计划	最终量	原计划	最终量
10:00–10:10	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
12:00–12:10	447.5773	500.6572	0.0000	0.0000	294.9785	168.0894	977.8658	942.1100
14:00–14:10	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
16:00–16:10	611.9552	611.9552	96.9398	96.9398	528.3145	528.3145	839.5792	868.0448
18:00–18:10	673.7367	706.8097	114.9568	136.8373	742.1825	782.7903	675.9807	675.9807
20:00–20:10	0.0000	0.0000	0.0000	0.0000	0.3603	26.3649	0.0000	0.0000
全天电量/kWh	61349.0929	63872.1053	29446.2346	30369.8186	64336.0938	65296.2037	90241.3602	92689.3004
全天费用/元	–	41499.47	–	18009.86	–	42067.72	–	61935.90

表 12 问题三四个指定日期的充放电量及起止储电量 (kWh)

时间段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
充电量 (kWh)				
00:00–04:00	7662.8115	157.7376	7701.4622	8621.9853
04:00–08:00	1835.3594	1094.2404	1687.1562	3299.7274
08:00–12:00	2492.6697	9783.1738	4906.2508	5042.3042
12:00–16:00	6300.9854	0.0000	4745.8939	8006.9484
16:00–20:00	555.0210	0.0000	0.0000	0.0000
20:00–24:00	327.9287	1029.2523	1366.7910	346.2147
放电量 (kWh)				
00:00–04:00	78.3045	878.2834	0.0000	124.9033
04:00–08:00	6431.5328	2058.6613	6537.9082	2580.0109
08:00–12:00	2677.3195	0.0000	1896.6157	8024.6273
12:00–16:00	254.7228	0.0000	444.6634	2636.9678
16:00–20:00	5144.0722	6403.8908	5741.0755	6478.2932
20:00–24:00	3105.5341	2748.0573	3355.9961	2758.4698
0:00 储电量	1873.3574	3426.9473	1913.5068	1200.0000
24:00 储电量	1415.6833	2129.4342	2325.1720	1392.0451

表 13 四个指定日的紧急购电区间及电量 (kWh)

3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
09:30–10:00	100.0577	06:30–07:20	176.8897	20:30–20:40	28.2227	07:50–08:00	4.4020
–	–	–	–	20:50–21:00	7.7398	10:40–10:50	6.8387
–	–	–	–	–	–	20:30–20:50	215.7174
合计	100.0577	合计	176.8897	合计	35.9624	合计	226.9581

图 ?? 与图 ?? 按相同日期顺序展示购电调整及储电轨迹。四小时内充放电累计量可同时非零，实际十分钟槽内仍只取一个方向。

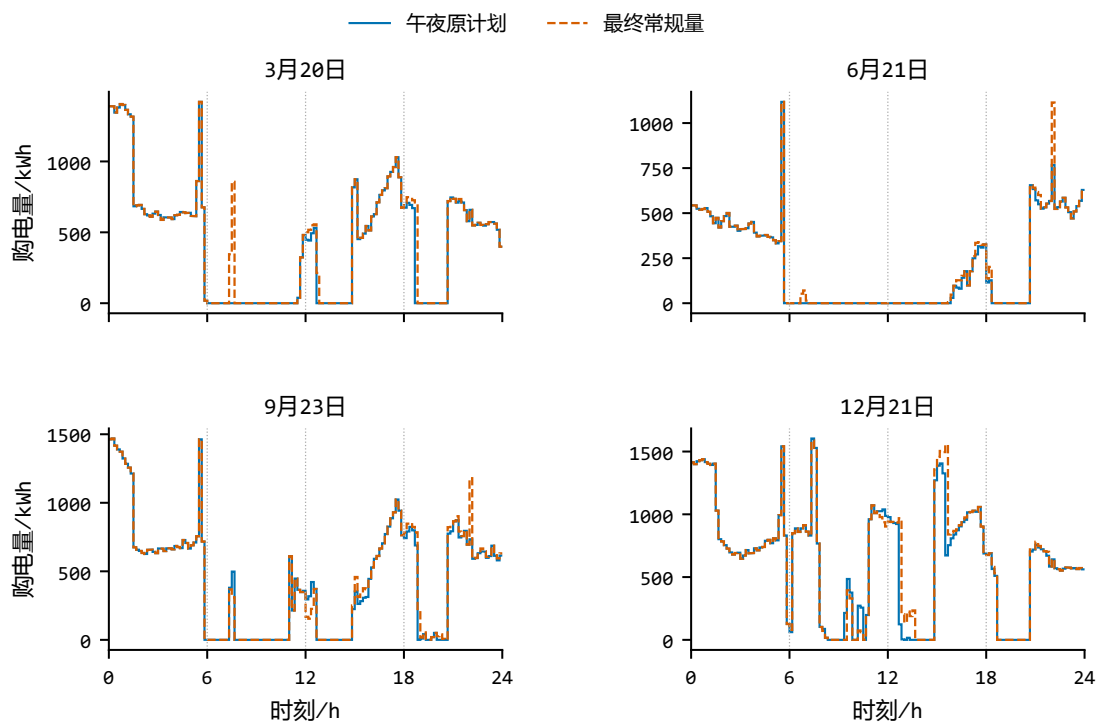


图 5 四个指定日的原计划与最终常规购电量。竖线为预报发布时间。

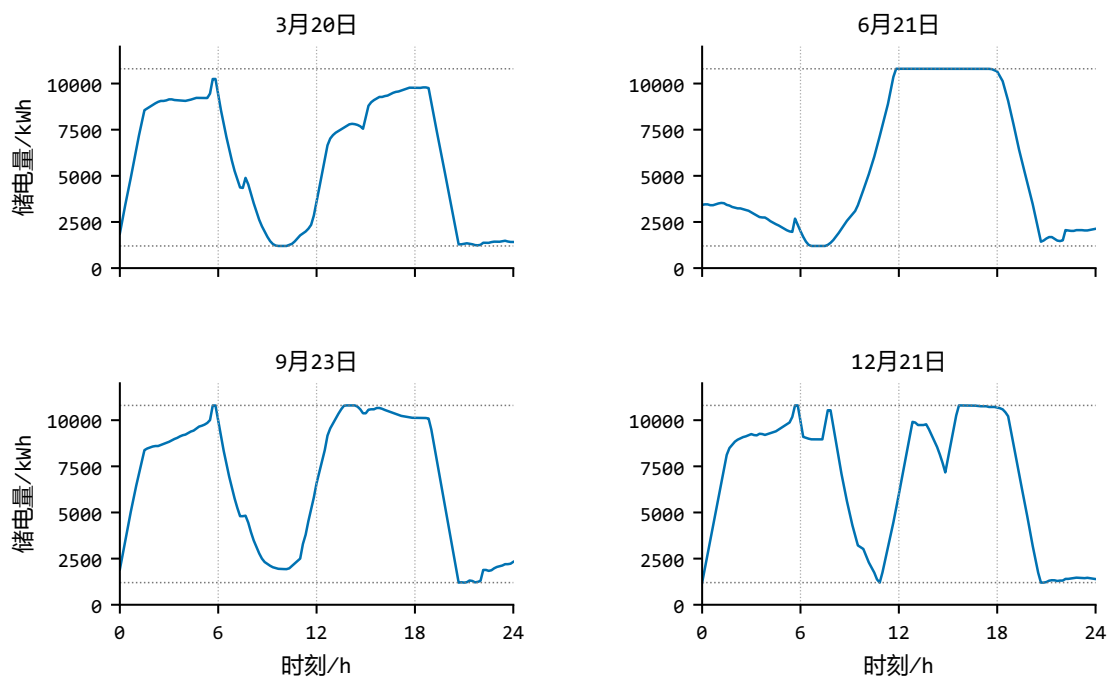


图 6 对应日期的储电量轨迹。水平虚线为 1200 与 10800 kWh 安全边界。

图 ?? 展示评价期的最终调整量：正值为上调，负值为下调；0 至 6 时已经按午夜计划执行，该段最终调整量为零。

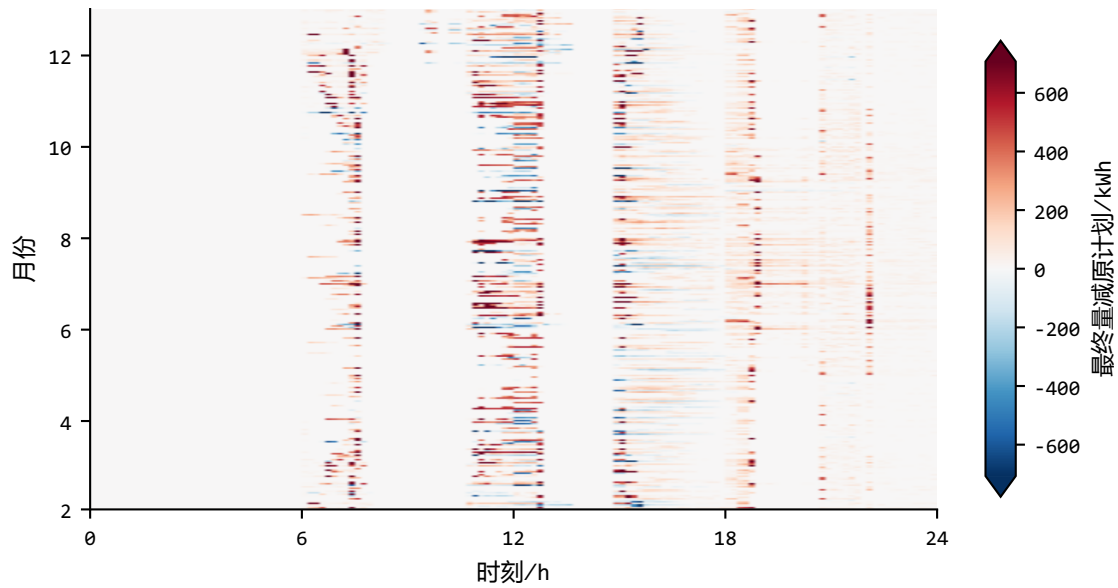


图 7 2 月至 12 月最终常规量减午夜原计划。色轴在绝对调整量 99.5% 分位处截断，原始值均保留参与费用计算。

是否需要其他时刻的预报。表 ?? 比较同一 48096 槽的午夜保持与日内更新预测，按发布后的六小时组成不重叠窗口。午夜光伏已使用 0 时官方预报，因此误差改善反映后续预报和负载前缀修正的共同作用。

表 14 同一 48096 槽上午夜保持与日内更新的预测误差（单位：kW）

变量	午夜 MAE	更新 MAE	午夜 RMSE	更新 RMSE
负载	144.7736	139.6079	192.8553	184.0852
光伏	198.8857	91.2404	379.4724	171.7355
净需求	288.1254	187.2121	428.9432	251.8011

后续预报降低了实际执行窗口的预测误差，并支持取消早期过量采购，因而有理由用于调整。是否每次都必须使用，还取决于调整代价、储能余量与信息变化；本次未逐一取消各发布时间进行经济对照，不能据预测改善断言每次发布都有相同的费用收益。完整策略保存于 result3.xlsx，调整表填写最终常规总电量。

八、问题四的模型建立与求解

针对波动电价，分别构建无日内预报的方案 4-2 和有日内预报的方案 4-3。两者先预测尚未揭示的价格，再按各自信息条件制定购电策略；设备边界与储电递推沿用公共模型。

8.1 价格预测与两类调度策略

因果电价预测。共同的八维特征包括截距、附件 1 基准价格、前日与前周同槽价格及日内一、二阶谐波。方案 4-2 另加入以 MW 计的已发布负载、光伏预测，形成十维模型；历史训练行使用其当时发布的预测。采用近 28 日、14 日半衰期的加权先验岭回归：

$$\hat{\beta}_o = \arg \min_{\beta} \left\{ \sum_{j \in \mathcal{H}_o} \omega_j (p_j - \mathbf{x}_j^T \beta)^2 + (\beta - \beta_0)^T \Lambda (\beta - \beta_0) \right\}, \quad (13)$$

$$\hat{p}_{i|o} = \max\{10^{-4}, \mathbf{x}_{i|o}^T \hat{\beta}_o\}.$$

式中， o 为发布时刻的全局槽号， i 为目标槽， $j \in \mathcal{H}_o$ 为发布前近 28 日的历史槽； p_j 为实际价格， $\mathbf{x}_j$ 和 $\mathbf{x}_{i|o}$ 分别为历史及目标特征。 ω_j 为时间衰减权重， β 与 $\hat{\beta}_o$ 为待估及估计系数， $\hat{p}_{i|o}$ 为预测价格，单位为元/kWh。 β_0 为系数先验，日、周滞后项各取 0.5，其余为零； $\Lambda = \text{diag}(10^{-6}, 5, \dots, 5)$ 为惩罚矩阵。

方案 4-2：小时模式规划。供需预测采用单 HGB 及问题二的校准、前日偏差修正。由近 28 日完整净需求误差中选取至多三条情景，以预测购电费、五倍短缺代价、方向切换罚和吞吐罚之和减去日末库存价值为目标，求解共享购电量的混合整数规划。每小时六槽共享充放电方向，并满足供需、SOC 和功率边界，禁止同槽充电与紧急补购。切换罚为 50 元/次，吞吐罚为 0.002 元/kWh；普通日末库存按最低预测价除以单向效率计值。随后固定小时模式，在全部历史路径上按实际反馈回放，用 L-BFGS-B 修正购电量，库存估值改为 0.45 元/kWh。评价末日库存价值均为零；计划于午夜锁定，执行充电死区为零。

方案 4-3：随预报滚动调整。负载基线为单 HGB，光伏从 0 时起采用官方预报。在 0、6、12、18 时更新供需和八维价格预测，将问题三式 (??) 中的分时价格换为当前预测价格，沿用同发布时间的至多七条误差路径、短缺权重、吞吐与总变差惩罚以及 20 kWh 充电死区，按实际储电量重求未执行计划。

两方案的实际账单均按式 (??) 以已实现价格结算；方案 4-2 无日内调整，方案 4-3 按相对午夜原计划的最终净差额结算。方法差异汇总于表 ??，因此方案间费用差反映预测与控制的作用。

表 15 问题二、三与波动电价方案的方法对应关系

设置	问题二	方案 4-2	问题三	方案 4-3
基础预测器	HGB/ExtraTrees 融合	单 HGB	单 HGB 负载	单 HGB 负载
官方光伏预报	不使用	不使用	0/6/12/18 时	0/6/12/18 时
模式规划	十分钟共同模式	小时共同模式	情景 LP	情景 LP
额外模式设置	每日变化至多 8 次	切换罚 50 元/次	无硬切换预算	无硬切换预算
日内调整	不允许	不允许	双向退款调整	双向退款调整
价格输入	已知分时价	10 维岭回归	已知分时价	8 维岭回归

8.2 问题四模型的求解与结果

两方案均从 1 月 1 日 6000 kWh 按各自策略连续运行，评价区间为 2 月 1 日至 12 月 31 日 334 日、48096 槽。2 月 1 日期初储电量分别为 6699.9484 和 1790.8119 kWh，均由一月实际轨迹承接；年末储电量均为 1200.0000 kWh。方案 4-2 的 MIP 单次限时 5 s、目标间隙 0.2%，固定模式修正最多 120 次迭代；365 次规划均得到可行解，实际最大相对间隙为 2.7128%。方案 4-3 的 1460 次线性规划均得到可行解。单次代理规划间隙不代表年度账单的最优性界。

图 ?? 按相同 48096 个午夜预测槽比较价格模型，八维与十维模型的 RMSE 分别为 0.06195、0.05614 元/kWh。评价期账单见表 ??：方案 4-3 比 4-2 少支出 468785.91 元 (3.35%)，紧急购电量减少 58.87%；非空方向反转分别为 6679 和 1757 次，表明费用与运行强度仍有权衡。

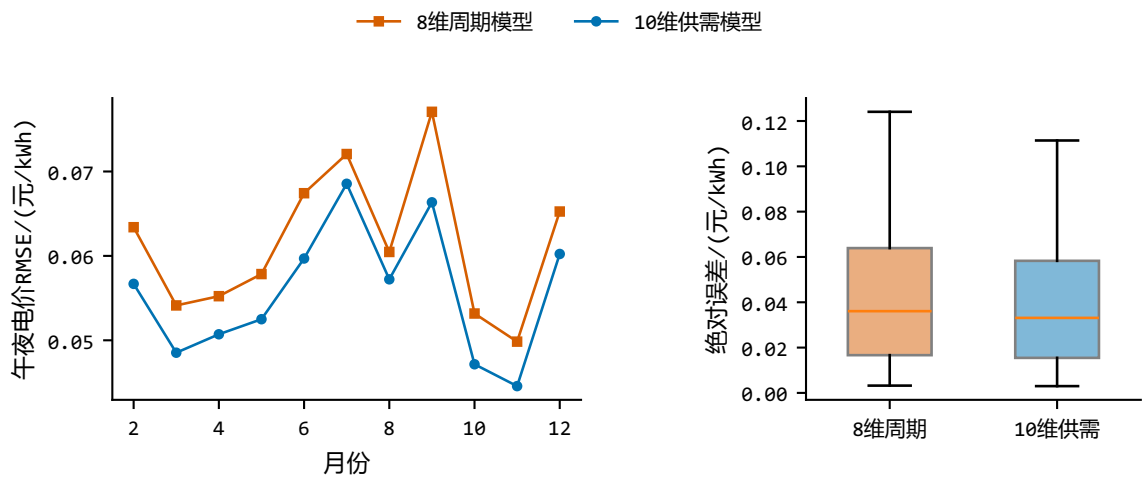


图 8 两种午夜价格模型在相同 48096 槽上的预测误差。右图箱体为四分位区间，横线为中位数，须为 5% 与 95% 分位数，不表示置信区间。

表 16 波动电价下两类完整方案的评价期结果

指标	方案 4-2	方案 4-3
原计划费/元	13402862.29	12266424.93
上调费/元	0.00	1132763.34
下调退款/元	0.00	-83052.64
紧急购电费/元	605747.24	223687.99
总费用/元	14008609.54	13539823.62
最终常规购电量/kWh	20506017.8625	19865431.5040
紧急购电量/kWh	112042.9064	46082.4095
非空方向反转/次	1757	6679
电芯等效满循环/次	463.6987	525.4197

图 ?? 比较月度费用及紧急费占比，图 ?? 展示每日费用差异，正值表示方案 4-3 费用较低；月总额同时受天数与季节需求影响。

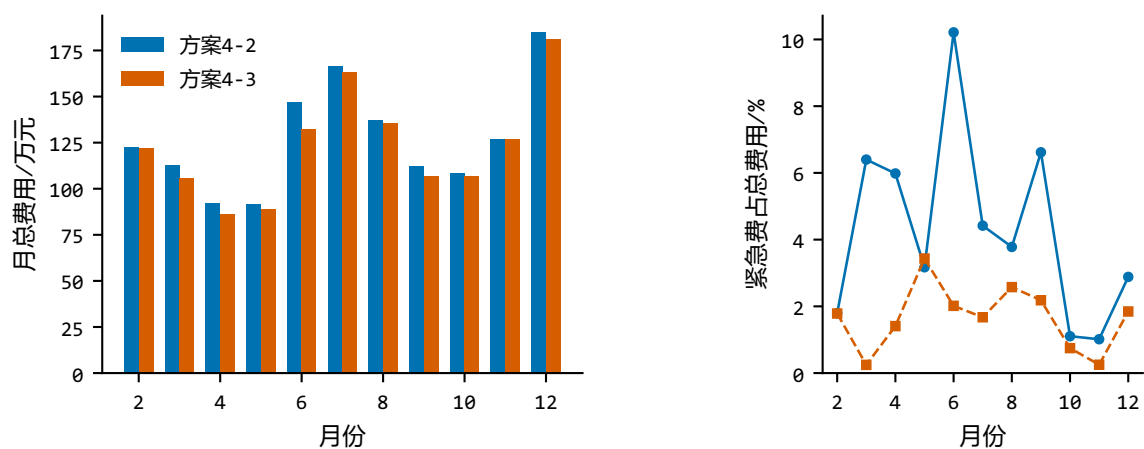


图 9 两类完整方案的月度费用与紧急费占比。

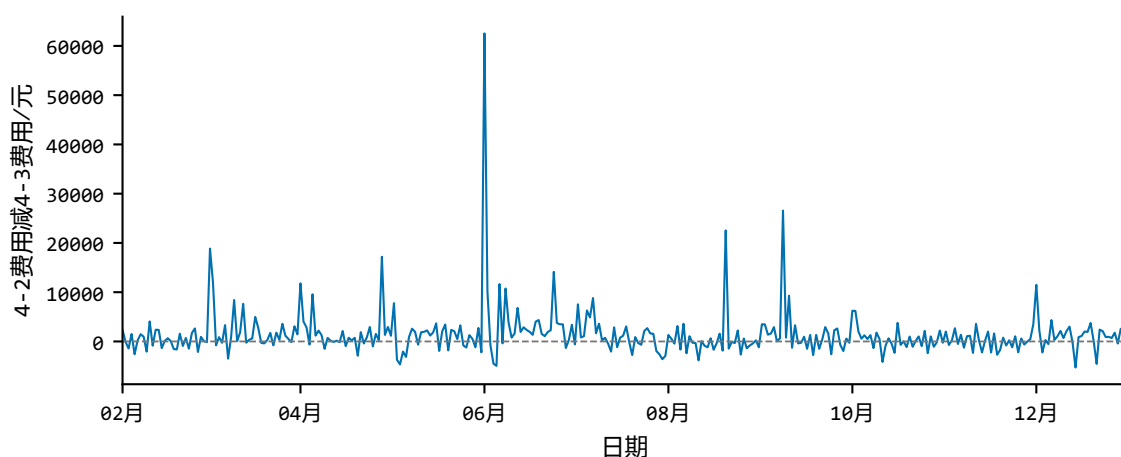


图 10 评价期每日总费用差（方案 4-2 减方案 4-3）。该差额反映两套完整策略的共同差异。

指定日期调度。方案 4-2 的指定槽与全天购电量见表 ??，方案 4-3 的午夜计划与最终量见表 ??、??。两方案四个指定日的储能结果分别见表 ??、??、??、??和表 ??、??、??、??。紧急购电完整区间见表 ??和表 ??。

表 17 问题 4-2 指定日期的最终常规购电量（单位：kWh）

时间段	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	56.8021	0.0000	14.2668	0.0000
12:00–12:10	564.5003	0.0000	504.4275	999.2309
14:00–14:10	0.0000	0.0000	0.0000	0.0000
16:00–16:10	431.9505	252.1208	499.5770	1165.4219
18:00–18:10	675.7274	382.0389	718.5905	691.9762
20:00–20:10	121.2289	0.0000	97.0970	0.0000
全天电量	66523.2925	31152.4188	68627.2874	88185.5065
全天总费用/元	42360.91	16636.03	45477.18	68042.02

表 18 方案 4-2：3 月 20 日储能（kWh）

时间段	充电量	放电量
00:00–04:00	7396.6319	87.3354
04:00–08:00	1757.3799	5318.2836
08:00–12:00	5819.2437	2777.3772
12:00–16:00	3399.7565	519.5389
16:00–20:00	0.0000	6015.2029
20:00–24:00	1515.8655	2572.6178
0:00 储电量	1995.4658	
24:00 储电量	2638.0763	

表 19 方案 4-2：6 月 21 日储能（kWh）

时间段	充电量	放电量
00:00–04:00	0.0000	2467.5629
04:00–08:00	0.0000	1870.8452
08:00–12:00	9757.9626	0.0000
12:00–16:00	0.0000	0.0000
16:00–20:00	0.0000	4428.1966
20:00–24:00	0.0000	3174.2822
0:00 储电量	6115.8675	
24:00 储电量	2786.2837	

表 20 方案 4-2：9 月 23 日储能（kWh）

时间段	充电量	放电量
00:00–04:00	8044.4697	0.0000
04:00–08:00	0.0000	4995.7313
08:00–12:00	5059.8420	1893.2268
12:00–16:00	2952.2254	592.1208
16:00–20:00	0.0000	6067.2634
20:00–24:00	1810.3827	2574.9749
0:00 储电量	3168.3459	
24:00 储电量	3122.9258	

表 21 方案 4-2：12 月 21 日储能（kWh）

时间段	充电量	放电量
00:00–04:00	4706.9660	193.1802
04:00–08:00	0.0000	988.5520
08:00–12:00	4424.1685	7041.4419
12:00–16:00	7692.5936	3036.8150
16:00–20:00	394.3369	5638.3036
20:00–24:00	0.0000	3093.9648
0:00 储电量	6334.5799	
24:00 储电量	1595.3809	

表 22 方案 4-2 四个指定日期的紧急购电区间（kWh）

3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
21:20–22:00	116.1019	–	–	15:00–15:10	7.7027	17:20–17:30	4.1823
合计	116.1019	合计	0.0000	合计	7.7027	合计	4.1823

表 23 问题 4-3 指定日期的原计划购电量（单位：kWh）

时间段	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	0.0000	0.0000	0.0000	0.0000
12:00–12:10	447.5773	0.0000	294.9785	977.8658
14:00–14:10	0.0000	0.0000	0.0000	0.0000
16:00–16:10	642.1417	96.9398	524.6990	865.8055
18:00–18:10	632.4041	0.0000	723.2164	680.6723
20:00–20:10	114.6849	0.0000	0.3603	0.0000
全天电量	61441.5401	29640.7183	64616.1447	90372.9344

表 24 问题 4-3 指定日期的最终常规购电量（单位：kWh）

时间段	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00–10:10	0.0000	0.0000	0.0000	0.0000
12:00–12:10	500.6572	0.0000	294.9785	942.1100
14:00–14:10	0.0000	0.0000	0.0000	0.0000
16:00–16:10	642.1417	96.9398	524.6990	1412.1259
18:00–18:10	678.6308	0.0000	725.6931	705.7614
20:00–20:10	719.6935	0.0000	2.0019	0.0000
全天电量	63775.7995	30514.7960	65529.3019	92829.0596
全天总费用/元	42751.71	15553.81	43840.15	72598.29

表 25 方案 4-3：3 月 20 日储能（kWh）

时间段	充电量	放电量
00:00–04:00	7383.9974	310.9833
04:00–08:00	2543.5155	6656.0298
08:00–12:00	2464.9699	2677.3195
12:00–16:00	6123.8121	254.7228
16:00–20:00	549.2050	6628.0990
20:00–24:00	313.2254	1410.7146
0:00 储电量	1947.9534	
24:00 储电量	1424.0526	

表 26 方案 4-3：6 月 21 日储能（kWh）

时间段	充电量	放电量
00:00–04:00	901.4645	2485.0043
04:00–08:00	1613.5323	1485.0959
08:00–12:00	9783.1738	0.0000
12:00–16:00	0.0000	0.0000
16:00–20:00	25.8726	6420.2901
20:00–24:00	2751.3906	4539.5586
0:00 储电量	3317.7840	
24:00 储电量	1882.0483	

表 27 方案 4-3：9 月 23 日储能（kWh）

时间段	充电量	放电量
00:00–04:00	9446.8317	6.2947
04:00–08:00	793.1483	7126.9436
08:00–12:00	4906.2508	1896.6157
12:00–16:00	4744.6378	443.0756
16:00–20:00	0.0000	6359.8540
20:00–24:00	1186.5006	2706.0787
0:00 储电量	1731.7495	
24:00 储电量	2185.8209	

表 28 方案 4-3：12 月 21 日储能（kWh）

时间段	充电量	放电量
00:00–04:00	10107.0507	150.7402
04:00–08:00	1792.0683	2520.6360
08:00–12:00	4308.2839	7911.5199
12:00–16:00	9238.4489	3404.2588
16:00–20:00	228.1659	6181.9629
20:00–24:00	1139.8274	3708.6294
0:00 储电量	1210.4470	
24:00 储电量	1478.9384	

表 29 方案 4-3 四个指定日期的紧急购电区间（kWh）

3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
时间段	电量	时间段	电量	时间段	电量	时间段	电量
09:30–10:00	100.0577	06:50–07:20	176.8897	20:10–20:20	10.2344	07:50–08:00	4.4020
–	–	–	–	–	–	20:40–20:50	215.7174
合计	100.0577	合计	176.8897	合计	10.2344	合计	220.1194

图 ?? 展示 9 月 23 日的电价、购电及储能轨迹，反映小时模式与日内更新下的运行差异。图中价格预测均为午夜版本，方案 4-3 的实际规划仍在后续发布时重新估价。

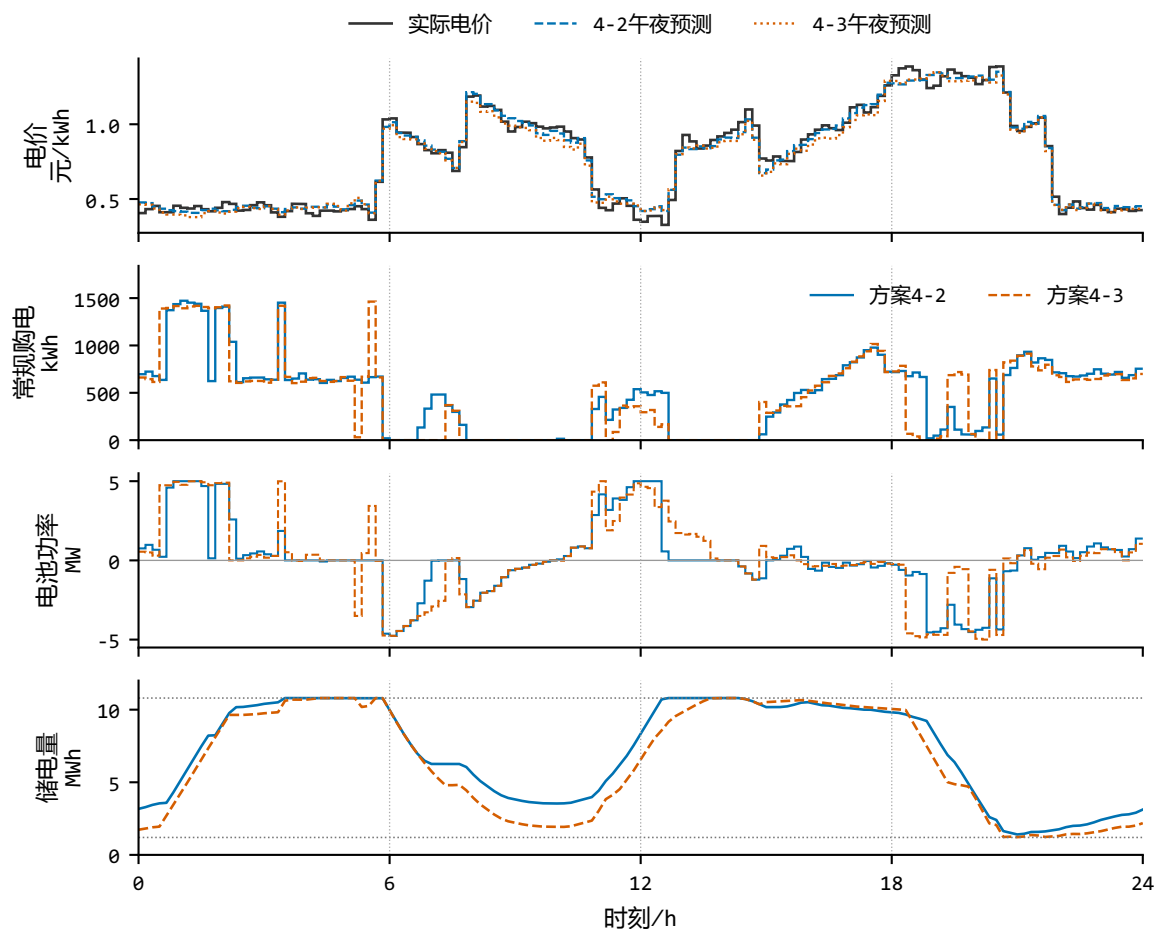


图 11 9月23日的电价、购电与储能运行轨迹。功率正值表示充电，负值表示放电；竖线为6、12、18时发布时刻，储电量水平虚线为安全边界。

结果检验。两方案均通过逐槽物理约束与实际账单核验，同时充放电及紧急购电用于充电的槽数均为零；统一核验方法见第九节。完整策略分别保存于 result4-2.xlsx、result4-3.xlsx。

九、模型分析与检验

9.1 物理、账单与信息边界核验

核验从逐槽实际执行轨迹出发，而非只检查优化目标值。对各问重新计算式 (??) 的能量平衡和储电递推，逐槽检查安全储电范围、单向功率上限以及实际充放电不同时发生；对年度方案另外检查相邻自然日的储电量完全衔接。问题一核对循环边界和两层费用预算，问题二核对每日计划模式变化预算，问题四无日内预报方案核对每小时共同模式。

费用核验以原计划、最终常规购电、紧急购电和实际价格逐槽重建式 (??)，下调项按负值退款计入。先汇总未舍入槽值再显示，分别检查指定槽、四小时累计、指定日和评价期总额。工作簿保留原题模板，以区间右端点对应记录；年度数组中的充放电量为交流侧 kWh，问题一原始程序的充放电功率需先除以 6。

信息核验以发布时刻为截点，改变尚未揭示的未来实测值，确认当前发布预测和计划不随之改变。日内发布之后，只覆盖尚未执行槽；区间内储能反馈使用当槽测量。该检验支持所建离散能量管理策略的取数顺序，不扩大为通信延迟或秒级设备验证。完整运行、费用与模板联查结果见各问结果表及支撑材料。

9.2 问题一的参数敏感性

为区分参数变化与执行扰动，先对问题一两层模型逐组重新优化。以正文采用的参数为中心，改变费用让步、爬坡上限、最短开启和关闭时间、往返效率及标称容量，共 15 组；其他参数保持原值。容量改变时保持 10%–90% 储电范围与 50% 初始比例，逆变器功率仍为 5000 kW。表 ?? 列示全部结果，各组均得到最优解且通过相应参数下的物理检查。

表 30 问题一两层模型的单参数重求解结果

参数设置	购电费/元	循环总变差/kW	费用变化/%
基准	34115.03	33209.17	+0.000
费用让步 0.05%	34097.99	34345.00	-0.050
费用让步 0.09%	34111.62	34436.05	-0.010
费用让步 0.11%	34118.44	32900.37	+0.010
费用让步 0.20%	34149.11	31089.99	+0.100
爬坡上限 900 kW	34153.52	33045.79	+0.113
爬坡上限 1100 kW	34085.24	33952.71	-0.087
最短开启 2 槽	34115.00	33212.14	-0.000
最短开启 4 槽	34121.31	32652.08	+0.018
最短关闭 1 槽	34115.03	33209.17	+0.000
最短关闭 3 槽	34115.03	33209.17	+0.000
往返效率 85.5%	34780.07	32867.93	+1.949
往返效率 94.5%	33488.06	38699.56	-1.838
容量 90%	34893.42	32259.51	+2.282
容量 110%	33406.40	33805.05	-2.077

费用让步由 0.05% 增至 0.20% 时, 循环总变差从 34345.00 kW 降至 31089.99 kW, 表明允许小幅增加购电费用能够扩大平稳调度的选择范围。往返效率从 90% 降低至 85.5% 时, 费用增加 1.949%; 提高至 94.5% 时, 费用减少 1.838%。容量减少 10% 使费用增加 2.282%, 增加 10% 使费用减少 2.077%。相较之下, 最短关闭时间在 1 至 3 槽内改变并未明显影响本典型日结果, 说明各方向原有的关闭间隔已经满足这些设置。

以上比较均对新参数重新求解, 反映模型与参数的关系, 不是原计划面对未知扰动的鲁棒性保证。单个典型日、单因素试验也不刻画参数交互作用或统计置信区间。

9.3 第二问执行扰动的补充检验

对先前完成的 334 日计划进行固定购电、固定模式的执行压力回放, 以观察误差增大后电池反馈和紧急购电的作用。该试验保留当时的计划与起点, 仅改变执行供需, 逐槽连续传递储电量; 未重新训练预测器或重新优化后续购电, 也不将其费用当作本次全年冷启动策略的重训练检验。

乘性噪声取 $\exp(\sigma z_t - \sigma^2/2)$, 其中 z_t 为相关系数 0.95 的平稳 AR(1) 过程, 负载与光伏两通道相关系数为 -0.3 , 光伏零值保持为零。每个噪声幅度重复 30 次, 使用相同

随机数以比较不同强度，结果见表 ??。

表 31 固定第二间历史计划的相关噪声执行回放

对数噪声标准差	费用平均变化/%	5% 分位/%	95% 分位/%
0.01	0.368	0.094	0.697
0.02	1.608	0.980	2.237
0.05	10.099	8.589	12.046
0.10	31.676	28.312	36.693

噪声标准差增至 0.10 时，费用平均上升 31.676%。即使物理反馈仍能借助外网补足负载，经济表现也会随预测误差恶化。表中分位数描述设定扰动下的 30 次结果，不是未来真实费用的置信区间；后续计划未随扰动历史重求，因此结论限定为执行层压力响应。

9.4 评价范围与求解精度

问题一的循环总变差连接典型日首尾；年度非空反转删除空闲槽后统计相邻方向变化，涉及评价边界时按相应表注处理。计划模式变化预算与实际非空反转具有不同定义，不将 8 次预算直接当作实测次数。电芯等效满循环由电芯侧吞吐除以 2×12000 计算。上述指标用于描述运行强度，不直接推导电池折旧或寿命。

混合整数规划的相对间隙描述该次代理问题的可行解与界，不代表年度实际费用误差。固定模式购电修正为局部数值优化，日内线性规划仅在本次发布的剩余时域求解；全年滚动执行不作全局最优性声明。年度数据参与过模型开发，本文保留合法信息截点和实际运行检验，不以单个随机种子或月度结果代替独立年份验证。

十、模型的评价、改进与推广

10.1 模型的优点

首先，模型将预测、购电发布、储能执行和费用结算连接起来。不同发布时刻只使用当时可获取的信息，实际储电量逐槽递推并跨日连续，避免通过每日重置电池库存形成不真实的节费。供需误差不仅以单槽均方误差度量，还以整日路径进入调度，使连续缺口对储能的影响得到体现。

其次，费用与运行要求分层处理。问题一先确定满足运行设置的最低费用，再用明确的 0.1% 费用让步改善功率变化与启动；问题二通过有限的计划模式变化约束组织充放电，再固定模式修正购电量。这些设置使费用代价与操作偏好可以分别检查，而非把任意惩罚直接计入购电账单。

最后，日内更新将新光伏信息和当前储电量共同用于剩余计划，并统一处理上调加价与下调退款。实际执行将购电盈余、可用库存和功率上限同时纳入充放电裁剪，剩余缺口才触发紧急购电。指定日期、年度汇总和工作簿均由同一执行轨迹产生，便于核对题目要求的结果。

10.2 模型的局限

预测与参数选择使用同一开发年度，当前结果说明在所给数据及信息条件下的表现，尚不足以保证跨年或不同小区的预测精度。价格采用点预测，其突变对购电与储能的联合影响未充分进入情景分布。

求解也存在范围限制。固定模式的购电修正采用局部优化；日内规划用分段短缺权重近似后续更新机会，未完整求解未来所有发布决策。各问的预测器、预报权限及运行设置并不完全相同，跨问费用差反映完整方案差异，不能全部归因于某一次预报或单独的价格变化。

平稳性指标与电池寿命之间仅有定性联系。反转次数、活动槽和等效循环能够描述运行强度，却未包含温度、荷电状态停留时间和循环深度等全部因素。部分动态工况对寿命的作用并非单调 [?]，因此本文不将减少某一指标直接换算为延寿或折旧收益。

10.3 模型的改进

后续可在独立年度或新小区上验证预测与调度参数，并通过保持预测器、初始状态和控制器相同的配对实验，分离价格变化与预报更新的作用。对波动电价，可进一步用已观察的价格误差构造联合情景；在通信与控制数据可得后，再评估测量延迟和执行偏差。若需要将寿命纳入费用，应引入可识别的循环老化模型及电芯参数 [?]，据实际退化数据标定，不沿用当前操作惩罚作为折旧价格。

10.4 模型的推广

在保留负载保障、储电递推和发布时序的基础上，可将模型应用于园区光储系统、社区共享储能及充电站能量管理。具体应用时需重新确定外网容量、可转移负载、售电规则和价格可见范围；多储能单元可增加设备索引并分别施加功率、容量和效率约束。若系统存在孤岛运行或网络安全要求，还需增加备用、失负荷或潮流约束后重新求解。

AI 工具使用声明

本参赛队使用 OpenAI Codex 辅助进行模型讨论、程序编写与调试、计算结果整理及论文表述和排版。模型取舍与冲突处理依据团队人工决定，数值由实际程序运行并核验；具体使用环节、主要提示、采纳修改和核验范围见支撑材料《AI 工具使用详情》。

参考文献

- [1] HUANG W, YE Y, CHEN H, et al. Onboard early detection and mitigation of lithium plating in fast-charging batteries[J]. Nature Communications, 2022, 13: 7091. DOI: <https://doi.org/10.1038/s41467-022-33486-4>.
- [2] XU B, ZHAO J, ZHENG T, et al. Factoring the Cycle Aging Cost of Batteries Participating in Electricity Markets[EB/OL]. arXiv:1707.04567, 2017. <https://arxiv.org/abs/1707.04567>.
- [3] GESLIN A, XU L, GANAPATHI D, et al. Dynamic cycling enhances battery lifetime[J]. Nature Energy, 2025, 10: 172–180. DOI: <https://doi.org/10.1038/s41560-024-01675-8>.
- [4] SciPy contributors. scipy.optimize.milp: Mixed-integer linear programming[EB/OL]. SciPy 1.15.3 documentation. <https://docs.scipy.org/doc/scipy-1.15.3/reference/generated/scipy.optimize.milp.html>.

附录 A 文件列表

表 ?? 列出正文结果与支撑文件的对应关系。

正式结果目录为 `data/results/exp009/`。五份工作簿集中保存在此处，年度完整轨迹同时保留一月连续运行及题目要求的 2–12 月评价部分。旧示例程序和历史第三问结果不列入正式交付。

表 32 论文结果及可运行支撑文件

文件或目录	功能
<code>result1.xlsx</code>	问题一第二层计划、充放电与起止储电量。
<code>result2.xlsx</code>	问题二的计划、实际储能与紧急购电结果。
<code>result3.xlsx</code>	问题三原计划、最终调整后总量、实际储能与紧急购电。
<code>result4-2.xlsx</code>	波动电价下无日内预报的购电结果。
<code>result4-3.xlsx</code>	波动电价下含日内预报的购电结果。
<code>q1/、q2/、q3/、q4_2/、q4_3/</code>	各问轨迹、实际费用、指定日期及运行检查；年度 NPZ 中各槽电量与储电量以 kWh 记录。
<code>experiments/exp008/</code>	基础预测、问题一两层求解、共同模式与反馈优化模块。
<code>experiments/exp009/</code>	全年冷启动与退款调度入口、结果导出和绘图程序。
<code>experiments/common/</code>	原始数据读取、稀疏线性约束构建等公共模块。
<code>data/raw/、预测文件</code>	题目输入及按当时信息形成的负载、光伏预测。
<code>requirements.txt</code>	本次实际运行所用的数值与导出依赖版本。
<code>verify_exp009_delivery.py</code>	逐槽物理、费用、工作簿及文件一致性检查。
<code>AI 工具使用详情.pdf</code>	工具用途、主要提示、人工决定与实际核验范围。

完整程序见附录 ??。读取已发布预测并重新求解调度，与从原始数据重新训练预测器是两种不同运行方式；具体命令、预测来源及文件字典见支撑材料说明。

附录 B 完整求解程序

下列程序按支撑材料中的相对目录组织。问题一可直接运行；年度策略读取附件 CSV 与已发布预测文件，从 1 月 1 日开始执行完整调度。复现时保留目录结构，在支撑包根目录安装所列依赖后运行：

```
python scripts/prepare_exp009_support.py
python Xelatex/code/q1_reproduce.py --quick --out q1_output
python -m experiments.exp009.q12_run --scenario 2 --days 365 --out data/results/
    exp009/reproduced/q2
python -m experiments.exp009.q12_run --scenario 4-2 --days 365 --out data/results/
    exp009/reproduced/q4_2
python -m experiments.exp009.q34_run --scenario 3 --days 365 --label reproduction
python -m experiments.exp009.q34_run --scenario 4-3 --days 365 --label reproduction
```

以下列出求解入口及所需本地模块的完整内容。绘图、工作簿导出及核验程序随支撑包提供；预测发布值与重新训练预测器分别说明，不将读取已保存预测的运行称为重新训练。支撑包保留全部 365 日轨迹，为减少重复文件，准备命令从完整数组恢复一月与评价期切片，并逐文件核对原归档哈希，不重新计算调度。

Xelatex/code/q1_reproduce.py

```
1  """问题一完整复现: Python >=3.10; pip install numpy scipy matplotlib。
2  直接运行 python q1_reproduce.py; 输入内嵌为附件1原始144行。
3  可选 --input 附件1.csv / 附件1.xlsx; 损坏文件默认报错, --fallback才使用内嵌数据。
4  全部输出放入 --out 指定的新目录; 不覆盖原始附件或队友工作簿。
5  """
6  import argparse # 解析命令行参数。
7  import csv # 读写不依赖Excel软件的表格。
8  import io # 将内嵌文本作为文件读取。
9  import json # 保存完整数值和求解状态。
10 import os # 将绘图缓存限制在本次输出目录。
11 import sys # 向终端返回失败状态。
12 import time # 记录真实求解耗时。
13 from pathlib import Path # 使用跨平台路径, 不写入个人绝对路径。
14 try:
15     import numpy as np # 数值计算。
16     import scipy # 记录实际使用的库版本。
17     from scipy.optimize import milp, Bounds, LinearConstraint # 混合整数规划。
18     from scipy.sparse import coo_matrix # 稀疏约束矩阵。
19 except ImportError as exc:
20     raise SystemExit("请先安装: pip install numpy scipy matplotlib") from exc
21
22 # 以下是附件1逐行数据, 顺序为区间右端、电价、负载、光伏功率。
23 EMBEDDED_CSV = """时间,电价,小区负载,光伏发电预测功率
24 00:10:00,0.4248,3439.8466,0
25 00:20:00,0.4245,3437.9792,0
26 00:30:00,0.4269,3444.3031,0
27 00:40:00,0.4272,3454.4437,0
28 00:50:00,0.4271,3458.8273,0
29 01:00:00,0.4285,3459.8401,0
30 01:10:00,0.4311,3467.1582,0
31 01:20:00,0.4302,3467.9632,0
32 01:30:00,0.4279,3466.9683,0
33 01:40:00,0.4312,3482.4927,0
34 01:50:00,0.4333,3495.0841,0
35 02:00:00,0.4311,3498.4443,0
36 02:10:00,0.4293,3505.2295,0
37 02:20:00,0.4324,3512.6387,0
```

38	02:30:00,0.4367,3516.8685,0
39	02:40:00,0.4348,3517.9246,0
40	02:50:00,0.4335,3518.5518,0
41	03:00:00,0.4357,3521.0976,0
42	03:10:00,0.4374,3526.9261,0
43	03:20:00,0.4371,3537.2854,0
44	03:30:00,0.4356,3544.8808,0
45	03:40:00,0.4353,3541.2787,0
46	03:50:00,0.436,3546.3976,0
47	04:00:00,0.4378,3560.207,0
48	04:10:00,0.4401,3561.9581,0
49	04:20:00,0.4389,3556.7183,0
50	04:30:00,0.4364,3560.0272,0
51	04:40:00,0.4393,3580.654,0.5994
52	04:50:00,0.4422,3586.916,0.2345
53	05:00:00,0.4395,3571.2635,0.2885
54	05:10:00,0.4333,3562.3171,18.6781
55	05:20:00,0.4439,3591.3809,54.4698
56	05:30:00,0.4301,3579.7045,108.9718
57	05:40:00,0.3713,3486.119,184.1603
58	05:50:00,0.5708,3823.8994,277.0582
59	06:00:00,0.9099,4398.0727,388.5494
60	06:10:00,0.9368,4454.0123,521.2803
61	06:20:00,0.8758,4371.7919,672.7677
62	06:30:00,0.8759,4392.953,844.2261
63	06:40:00,0.8601,4384.5956,1039.6744
64	06:50:00,0.8442,4434.7757,1259.3375
65	07:00:00,0.8334,4523.4561,1508.6318
66	07:10:00,0.8092,4524.0721,1791.4107
67	07:20:00,0.7972,4545.8867,2085.4865
68	07:30:00,0.7644,4523.2418,2381.2611
69	07:40:00,0.6899,4383.3566,2683.1752
70	07:50:00,0.86,4910.4599,2986.3552
71	08:00:00,1.1617,5799.7327,3289.204
72	08:10:00,1.1683,5884.826,3593.3613
73	08:20:00,1.0956,5748.0713,3890.7332
74	08:30:00,1.0849,5769.0581,4181.3417
75	08:40:00,1.061,5754.2105,4471.7657
76	08:50:00,1.043,5824.1807,4752.8272
77	09:00:00,1.036,5954.2601,5021.9074
78	09:10:00,1.0147,5958.9696,5285.4126
79	09:20:00,0.9922,5929.5554,5543.1742
80	09:30:00,0.97,5923.7828,5788.4093
81	09:40:00,0.9422,5907.586,6012.1304
82	09:50:00,0.9569,5899.1262,6216.1528
83	10:00:00,0.9977,5902.2196,6409.5941
84	10:10,0.9919,5897.4793,6599.3808
85	10:20,0.9588,5886.7568,6766.7204
86	10:30,0.9549,5873.4708,6914.5923
87	10:40,0.9839,5858.0308,7064.2053
88	10:50,0.799,5844.9948,7197.4315
89	11:00,0.5025,5839.0587,7303.1142
90	11:10,0.4662,5838.7103,7387.6848
91	11:20,0.5007,5823.4378,7459.227
92	11:30,0.4827,5814.7932,7520.7191
93	11:40,0.4785,5832.3951,7568.2039
94	11:50,0.4662,5712.901,7592.1539
95	12:00,0.4432,5519.4607,7601.0433
96	12:10,0.4411,5494.7907,7612.316
97	12:20,0.4551,5514.4524,7602.6524
98	12:30,0.4463,5485.829,7554.1872
99	12:40,0.4113,5462.3628,7472.7384
100	12:50,0.5682,5566.1066,7393.4172
101	13:00,0.8279,5725.7548,7312.7979
102	13:10,0.8636,5739.4957,7194.541
103	13:20,0.8416,5714.6836,7053.1371
104	13:30,0.8618,5717.8505,6910.4064
105	13:40,0.8623,5708.2486,6772.4075
106	13:50,0.9026,5696.9182,6607.36
107	14:00,0.9754,5692.2983,6412.5085

```

108 14:10,1.0002,5687.1291,6214.0158
109 14:20,0.9978,5678.962,6000.7112
110 14:30,1.0271,5669.4508,5766.9816
111 14:40,1.0906,5664.1948,5521.5066
112 14:50,0.9376,5672.7011,5268.781
113 15:00,0.6694,5682.2544,5009.7152
114 15:10,0.66,5669.0195,4743.4484
115 15:20,0.7217,5656.3651,4471.224
116 15:30,0.7308,5657.0867,4188.2025
117 15:40,0.7534,5662.6008,3889.3931
118 15:50,0.7823,5669.2596,3594.5946
119 16:00,0.8055,5673.4767,3303.8878
120 16:10,0.8271,5671.0146,2998.4247
121 16:20,0.8519,5661.5926,2687.4131
122 16:30,0.8791,5665.8922,2383.1779
123 16:40,0.9024,5689.0372,2089.1246
124 16:50,0.9163,5603.6614,1791.0829
125 17:00,0.9264,5466.7124,1501.7283
126 17:10,0.9462,5467.3939,1255.1723
127 17:20,0.9742,5464.727,1036.3323
128 17:30,0.9862,5492.4458,836.7775
129 17:40,0.976,5621.7308,665.2313
130 17:50,1.0742,5091.2126,516.6758
131 18:00,1.2313,4208.5688,386.6731
132 18:10,1.2561,4122.7611,275.7933
133 18:20,1.2447,4255.9358,182.9526
134 18:30,1.2545,4225.9524,108.3414
135 18:40,1.2522,4233.3384,54.3121
136 18:50,1.2881,4246.4625,18.6569
137 19:00,1.3466,4239.1795,0.2753
138 19:10,1.3523,4244.6684,0.2296
139 19:20,1.3445,4251.9187,0.6007
140 19:30,1.3483,4256.6813,0
141 19:40,1.349,4267.3947,0
142 19:50,1.3473,4269.975,0
143 20:00,1.3482,4268.6702,0
144 20:10,1.3513,4282.2245,0
145 20:20,1.34,4291.7049,0
146 20:30,1.3489,4291.3611,0
147 20:40,1.3952,4293.9182,0
148 20:50,1.2226,4295.2135,0
149 21:00,0.9332,4302.0307,0
150 21:10,0.9068,4320.9618,0
151 21:20,0.9371,4302.3292,0
152 21:30,0.9371,4322.6758,0
153 21:40,0.9871,4428.8083,0
154 21:50,0.7764,4030.1614,0
155 22:00,0.4209,3362.7419,0
156 22:10,0.3859,3309.3934,0
157 22:20,0.4368,3412.9563,0
158 22:30,0.4198,3381.9313,0
159 22:40,0.4186,3386.5592,0
160 22:50,0.4241,3400.4846,0
161 23:00,0.4237,3399.2426,0
162 23:10,0.4248,3406.9809,0
163 23:20,0.424,3416.1518,0
164 23:30,0.422,3421.9811,0
165 23:40,0.424,3429.6923,0
166 23:50,0.4271,3439.061,0
167 0:00+1,0.4276,3444.7259,0""
168 T, DT = 144, 1 / 6 # 一天144个10分钟区间, 功率转电量乘DT。
169 PMAX, PMIN, EMIN, EMAX, EINIT = 5000., 1., 1200., 10800., 6000.
170 EPS_C, EPS_S = 1e-4, 1e-7 # 费用和无量纲第二目标的绝对容差。
171
172
173 def read_data(path=None, fallback=False):
174     """只读输入; 任何缺失、负数、时间错位都显式报错。"""
175     try:
176         if path is None:
177             rows = list(csv.reader(io.StringIO(EMBEDDED_CSV.strip())))[1:]

```

```

178 elif Path(path).suffix.lower() == ".xlsx":
179     from openpyxl import load_workbook # 仅外部XLSX输入需要该可选库。
180     book = load_workbook(path, data_only=True, read_only=True)
181     rows = list(book.active.values)[1:] # 跳过标题; 不会保存或修改源文件。
182     book.close()
183 else:
184     rows = None
185     for encoding in ("utf-8-sig", "gb18030"): # 兼容常用中文CSV编码。
186         try:
187             with open(path, encoding=encoding, newline="") as handle:
188                 rows = list(csv.reader(handle))[1:]
189                 break
190         except UnicodeDecodeError:
191             continue
192     if rows is None:
193         raise ValueError("CSV编码无法识别")
194     if len(rows) != T:
195         raise ValueError(f"应有144行数据, 实际{len(rows)}行")
196     for i, row in enumerate(rows): # 检查右端时刻, 避免错位十分钟。
197         label = str(row[0]).strip()
198         if i == T - 1 and label in ("0:00+1", "00:00+1", "24:00", "24:00:00", "00:00:00"):
199             continue
200         pieces = label.split(":")
201         if len(pieces) < 2 or int(pieces[0]) * 60 + int(pieces[1]) != (i + 1) * 10:
202             raise ValueError(f"第{i+1}行时间不连续: {label}")
203     data = np.asarray([[float(x) for x in row[1:4]] for row in rows])
204     if data.shape != (T, 3) or not np.isfinite(data).all():
205         raise ValueError("存在空值、非数值或非有限数")
206     if (data < 0).any() or (data[:, 0] <= 0).any():
207         raise ValueError("本模型要求正电价以及非负负载、光伏")
208     return data
209 except Exception as exc: # 同时兼容Excel压缩包损坏、编码和格式解析异常。
210     if path is not None and fallback:
211         print(f"警告: 读取失败({exc}), 明确回退到内嵌附件1", file=sys.stderr)
212         return read_data()
213     raise ValueError(f"输入读取失败: {exc}; 可用--fallback允许内嵌回退") from exc
214
215
216 def solve(data, delta=.001, ramp=1000., up=3, down=2, eta_rt=.9):
217     """与 main 最新正文一致: 只求费用、平稳性两层, 不添加第三层。"""
218     if not (0 < eta_rt <= 1 and delta >= 0 and 1 <= up <= T and 1 <= down <= T):
219         raise ValueError("效率、让步比例或持续时间不合法")
220     eta = np.sqrt(eta_rt) # 本问保持往返效率口径。
221     names = ("g", "c", "d", "w", "E", "zc", "zd", "sc", "sd", "oc", "od", "v",
222             "bc1", "bc2", "bd1", "bd2") # b为旧基准短段缺额线性化变量。
223     ids, offset = {}, 0
224     for name in names:
225         size = T + 1 if name == "E" else T # 储电量包含145个边界。
226         ids[name] = np.arange(offset, offset + size)
227         offset += size
228     lo, hi, integer = np.zeros(offset), np.full(offset, np.inf), np.zeros(offset)
229     hi[ids["v"]] = 2 * PMAX # 物理上界; 避免第一层未入目标的辅助量无界。
230     for name in ("c", "d"):
231         hi[ids[name]] = PMAX
232     lo[ids["E"]], hi[ids["E"]] = EMIN, EMAX
233     lo[ids["E"]][0, -1]] = EINIT
234     hi[ids["E"]][0, -1]] = EINIT # 日初日末均6000kWh。
235     for name in ("zc", "zd", "sc", "sd", "oc", "od", "bc1", "bc2", "bd1", "bd2"):
236         hi[ids[name]] = 1
237     integer[ids["zc"]], integer[ids["zd"]] = 1, 1 # 仅模式变量必须声明整数。
238     ri, ci, av, lower, upper = [], [], [], [], []
239     def add(items, lb=-np.inf, ub=np.inf):
240         """将一行稀疏线性约束加入模型。"""
241         row = len(lower)
242         for col, val in items:
243             ri.append(row); ci.append(int(col)); av.append(float(val))
244             lower.append(float(lb)); upper.append(float(ub))
245     for t in range(T):
246         prev = (t - 1) % T # 循环日跨午夜连接。
247         ix = lambda key, j=t: int(ids[key][j])

```

```

248 net = [(ix("c"), 1), (ix("d"), -1), (ix("c", prev), -1), (ix("d", prev), 1)]
249 rhs = (data[t, 1] - data[t, 2]) * DT
250 add([(ix("g"), 1), (ix("d"), DT), (ix("c"), -DT), (ix("w"), -1)], rhs, rhs)
251 add([(ix("E", t+1), 1), (ix("E"), -1), (ix("c"), -eta*DT), (ix("d"), DT/eta)], 0, 0)
252 add([(ix("zc"), 1), (ix("zd"), 1)], ub=1) # 禁止同时充放电。
253 add(net, -ramp, ramp) # 运行功率变化上限, 基准可设为10000。
254 add([(ix("v"), 1)] + [(k, -v) for k, v in net], lb=0)
255 add([(ix("v"), 1)] + net, lb=0) # v不小于净功率差的绝对值。
256 for mode in ("c", "d"):
257     z, oldz, start, stop = ix("z"+mode), ix("z"+mode, prev), ix("s"+mode), ix("o"+mode)
258     add([(ix(mode), 1), (z, -PMAX)], ub=0)
259     add([(ix(mode), 1), (z, -PMIN)], lb=0)
260     add([(start, 1), (z, -1), (oldz, 1)], lb=0)
261     add([(start, 1), (z, -1)], ub=0)
262     add([(start, 1), (oldz, 1)], ub=1)
263     add([(stop, 1), (oldz, -1), (z, 1)], lb=0)
264     add([(stop, 1), (oldz, -1)], ub=0)
265     add([(stop, 1), (z, 1)], ub=1) # s, o精确表示0-1转换。
266     add([(ix("z"+mode, (t+j)%T), 1) for j in range(up)] + [(start, -up)], lb=0)
267     add([(ix("z"+mode, (t+j)%T), 1) for j in range(down)] + [(stop, down)], ub=down)
268     for k in (1, 2): # 30分钟参考长度的两级短段缺额。
269         b = ix("b"+mode+str(k))
270         add([(b, 1), (start, -1)], ub=0)
271         for j in range(1, k+1):
272             add([(b, 1), (start, -1), (ix("z"+mode, (t+j)%T), 1)], lb=0)
273             add([(b, 1)] + [(ix("z"+mode, (t+j)%T), 1) for j in range(1, k+1)], ub=k)
274 cost, smooth, quantity = np.zeros(offset), np.zeros(offset), np.zeros(offset)
275 cost[ids["g"]], quantity[ids["g"]] = data[:, 0], 1
276 smooth[ids["v"]] = 1 / (2 * PMAX * T)
277 for name in ("sc", "sd"):
278     smooth[ids[name]] = 1 / (2 * T)
279 for name in ("bc1", "bc2", "bd1", "bd2"):
280     smooth[ids[name]] = 1 / (4 * T)
281 records, solutions = [], []
282 for stage, objective in enumerate((cost, smooth * 1e6), 1):
283     matrix = coo_matrix((av, (ri, ci)), shape=(len(lower), offset)).tocsc()
284     began = time.perf_counter()
285     result = milp(objective, integrality=integer, bounds=Bounds(lo, hi),
286                  constraints=LinearConstraint(matrix, lower, upper),
287                  options={"mip_rel_gap": 1e-9, "time_limit": 180.})
288     if not result.success or result.x is None:
289         raise RuntimeError(f"第{stage}层未证最优: {result.message}")
290     x = result.x # 不对连续解取整, 不静默接受超时解。
291     pnet = x[ids["c"]] - x[ids["d"]]
292     physical_smooth = (np.abs(pnet-np.roll(pnet,1)).sum()/(2*PMAX*T)
293                      +sum(x[ids[k]].sum()/(2*T) for k in ("sc", "sd"))
294                      +sum(x[ids[k]].sum()/(4*T) for k in ("bc1", "bc2", "bd1", "bd2")))
295     rec = dict(stage=stage, cost=float(np.sum(cost*x)), quantity=float(np.sum(quantity*x)),
296              smooth=float(physical_smooth), gap=float(result.mip_gap),
297              seconds=time.perf_counter()-began, nodes=int(result.mip_node_count))
298     records.append(rec); solutions.append({k: x[v] for k, v in ids.items()})
299     if stage == 1:
300         budget = (1 + delta) * rec["cost"] + EPS_C
301         add(list(zip(ids["g"], data[:, 0])), ub=budget)
302     elif stage == 2:
303         add([(i, v*1e6) for i, v in enumerate(smooth) if v], ub=(rec["smooth"]+EPS_S)*1e6)
304     config = dict(delta=delta, ramp=ramp, up=up, down=down, eta_rt=eta_rt)
305     result = dict(config=config, stages=records, trajectory=solutions[-1], budget=budget)
306     result["metrics"] = audit(data, result) # 从物理量独立复算, 不只看求解器成功标志。
307     return result
308
309
310 def lengths(z, value=1):
311     """读取循环0-1序列的段长, 合并跨午夜的同模式段。"""
312     starts = [i for i in range(T) if z[i] == value and z[(i-1)%T] != value]
313     if not starts:
314         return [T] if np.all(z == value) else []
315     return [next(k for k in range(1, T+1) if z[(i+k)%T] != value) for i in starts]
316
317

```

```

318 def audit(data, result):
319     """独立检验供需、SOC、功率、互斥、段长、费用预算及层次目标。"""
320     q, cfg = result["trajectory"], result["config"]
321     g, c, d, w, E = (q[k] for k in ("g", "c", "d", "w", "E"))
322     eta = np.sqrt(cfg["eta_rt"])
323     net = c-d
324     on, off = [], []
325     for key in ("zc", "zd"):
326         z = np rint(q[key]).astype(int) # 仅用于诊断整数变量, 不更改求解值。
327         on.extend(lengths(z)); off.extend(lengths(z, 0))
328         balance = g + data[:, 2]*DT + d*DT - data[:, 1]*DT - c*DT - w
329         transition = np.diff(E)-eta*c*DT+d*DT/eta
330         tv = float(np.abs(net-np.roll(net, 1)).sum())
331         short = sum(max(0, 3-k) for k in on)
332         smooth = tv/(2*PMAx*T)+len(on)/(2*T)+short/(4*T)
333         metrics = dict(cost=float(data[:, 0]@g), quantity=float(g.sum()), tv=tv, starts=len(on),
334             short_deficit=short, short_runs=sum(k<3 for k in on),
335             min_on=min(on, default=T)*10, min_off=min(off, default=T)*10,
336             max_step=float(np.abs(net-np.roll(net, 1)).max()),
337             charge=float(c.sum()*DT), discharge=float(d.sum()*DT),
338             surplus=float(w.sum()), emin=float(E.min()), emax=float(E.max()),
339             balance=float(np.abs(balance).max()), state=float(np.abs(transition).max()),
340             endpoint=float(max(abs(E[0]-EINIT), abs(E[-1]-EINIT))),
341             simultaneous=float(np.minimum(c, d).max()), smooth=smooth,
342             max_c=float(c.max()), max_d=float(d.max()),
343             low=int(np.sum((np.abs(net)>1e-5)&(np.abs(net)<50))))
344         metrics["daily_balance"] = float(abs(g.sum() - (data[:, 1]-data[:, 2]).sum()*DT
345             - (c.sum()-d.sum())*DT - w.sum()))
346         metrics["cost_recompute"] = abs(metrics["cost"]-result["stages"][-1]["cost"])
347         tolerance = 2e-5 # kW/kWh诊断容差, 目标函数另用各自量尺核查。
348         violations = [
349             metrics["balance"]>tolerance, metrics["state"]>tolerance,
350             metrics["endpoint"]>tolerance, metrics["simultaneous"]>tolerance,
351             E.min()<EMIN-tolerance, E.max()>EMAX+tolerance,
352             c.max()>PMAx+tolerance, d.max()>PMAx+tolerance,
353             min(g.min(), c.min(), d.min(), w.min()) < -tolerance,
354             metrics["max_step"]>cfg["ramp"]+tolerance,
355             metrics["min_on"]<cfg["up"]*10, metrics["min_off"]<cfg["down"]*10,
356             metrics["cost"]>result["budget"]+1e-5,
357             smooth>result["stages"][-1]["smooth"]+EPS_S+1e-9,
358             np.max(np.abs(q["zc"]-np rint(q["zc"])))>1e-6,
359             np.max(np.abs(q["zd"]-np rint(q["zd"])))>1e-6,
360             np.max(q["zc"]+q["zd"])>1+1e-6,
361             np.max(PMIN*q["zc"]-c)>tolerance, np.max(c-PMAx*q["zc"])>tolerance,
362             np.max(PMIN*q["zd"]-d)>tolerance, np.max(d-PMAx*q["zd"])>tolerance,
363             metrics["daily_balance"]>tolerance, metrics["cost_recompute"]>1e-5,
364         ]
365         metrics["violations"] = sum(bool(v) for v in violations)
366         if metrics["violations"]:
367             raise RuntimeError(f"独立约束检验失败: {metrics}")
368         return metrics
369
370
371 def save_case(out, name, data, result):
372     """导出144段完整计划及每层状态, 不覆盖用户原始表。"""
373     def serial(value):
374         if isinstance(value, np.ndarray):
375             return value.tolist()
376         if isinstance(value, np.generic):
377             return value.item()
378         raise TypeError(f"无法序列化: {type(value)}")
379     (out/(name+".json")).write_text(json.dumps(result, ensure_ascii=False, indent=2,
380         default=serial), encoding="utf-8")
381     q = result["trajectory"]
382     with (out/(name+".csv")).open("w", newline="", encoding="utf-8-sig") as handle:
383         writer = csv.writer(handle)
384         writer.writerow(["起点/h", "终点/h", "电价/元每kWh", "负载/kW", "光伏/kW",
385             "购电量/kWh", "充电功率/kW", "放电功率/kW", "剩余/kWh",
386             "期初储电/kWh", "期末储电/kWh", "区间电费/元"])
387         for t in range(T):

```

```

388         writer.writerow([t*DT, (t+1)*DT, *data[t], q["g"][t], q["c"][t], q["d"][t],
389                           q["w"][t], q["E"][t], q["E"][t+1], data[t, 0]*q["g"][t]])
390
391
392 def draw(out, data, cases, sweep):
393     """依据真实解绘制阶梯功率、储电轨迹和费用-总变差图。"""
394     os.environ.setdefault("MPLCONFIGDIR", str((out/"mpl-cache").resolve()))
395     import matplotlib
396     matplotlib.use("Agg") # 无图形界面的机器也能输出。
397     import matplotlib.pyplot as plt
398     plt.rcParams.update({"font.sans-serif": ["Microsoft YaHei", "SimHei", "DejaVu Sans"],
399                          "axes.unicode_minus": False, "font.size": 10,
400                          "axes.spines.top": False, "axes.spines.right": False})
401     edges = np.arange(T+1)*DT
402     fig, axes = plt.subplots(2, 1, figsize=(10, 6.4), sharex=True, layout="constrained")
403     for key, label, color in [("baseline", "严格经济基准", "#8061A8"), ("revised", "平稳运行方案",
404                               "#138B99")]:
405         q = cases[key]["trajectory"]
406         axes[0].stairs(q["c"]-q["d"], edges, label=label, color=color, linewidth=1.6)
407         axes[1].plot(edges, q["E"], label=label, color=color, linewidth=1.6)
408     axes[0].axhline(0, color="#777777", linewidth=.6)
409     axes[0].set_ylabel("净充电功率 / kW") # 标题与图例分别留出空间。
410     axes[0].set_title("典型日电池净功率与储电量", pad=34)
411     axes[1].axhline(EMIN, color="#B27838", linestyle="--", label="安全储电边界")
412     axes[1].axhline(EMAX, color="#B27838", linestyle="--")
413     axes[1].set(xlabel="时刻 / h", ylabel="储电量 / kWh", xlim=(0,24), xticks=np.arange(0,25,2))
414     for ax in axes:
415         ax.legend(ncol=3, fontsize=9, loc="lower center",
416                  bbox_to_anchor=(.5, 1.01), frameon=False) # 图例不遮挡数据轨迹。
417     ax.grid(alpha=.16)
418     fig.savefig(out/"q1_dispatch.png", dpi=320); plt.close(fig)
419     rows = sorted(sweep, key=lambda row: row["delta"])
420     fig, axes = plt.subplots(1, 2, figsize=(10, 3.5), layout="constrained")
421     x = [row["delta"]*100 for row in rows]
422     for ax, key, ylabel, color in [(axes[0], "cost", "购电费 / 元", "#B27838"),
423                                   (axes[1], "tv", "循环功率总变差 / kW", "#138B99")]:
424         ax.plot(x, [row[key] for row in rows], "o-", color=color, label="单因素重求解")
425         ax.axvline(.1, color="#8061A8", linestyle="--", label="选定让步比例")
426         ax.set(xlabel="电费让步比例 / %", ylabel=ylabel); ax.grid(alpha=.16); ax.legend(fontsize=8)
427     fig.suptitle("经济让步比例对电费与运行平稳性的影响")
428     fig.savefig(out/"q1_tradeoff.png", dpi=320); plt.close(fig)
429
430 def main():
431     parser = argparse.ArgumentParser(description=__doc__)
432     parser.add_argument("--input", help="可选CSV或XLSX; 省略则读取内嵌附件1")
433     parser.add_argument("--fallback", action="store_true", help="外部输入失败时显式回退")
434     parser.add_argument("--out", default="q1_output", help="生成结果目录")
435     parser.add_argument("--quick", action="store_true", help="兼容参数; 默认仅求正式两层方案与经济基准")
436     args = parser.parse_args()
437     data = read_data(args.input, args.fallback)
438     out = Path(args.out)
439     out.mkdir(parents=True, exist_ok=True)
440     cases = {}
441     tasks = [{"revised", {}}, {"baseline", dict(delta=0, ramp=10000, up=1, down=1)}]
442     for name, settings in tasks:
443         print(f"求解 {name}", flush=True)
444         result = solve(data, **settings)
445         cases[name] = result
446         save_case(out, name, data, result)
447         print(json.dumps(result["metrics"], ensure_ascii=False), flush=True)
448     final = cases["revised"]["trajectory"]
449     summary = dict(
450         environment=dict(python=sys.version, numpy=np.__version__, scipy=scipy.__version__),
451         stages=cases["revised"]["stages"], metrics=cases["revised"]["metrics"],
452         baseline_metrics=cases["baseline"]["metrics"],
453         specified=[dict(start_hour=t/6, purchase_kwh=float(final["g"][t]))
454                    for t in (60, 72, 84, 96, 108, 120)],
455         storage_blocks=[dict(start_hour=4*j, end_hour=4*(j+1),
456                             charge_kwh=float(final["c"][24*j:24*(j+1)].sum()*DT),

```

```

457         discharge_kwh=float(final["d"][24*j:24*(j+1)].sum()*DT)) for j in range(6)],
458         initial_kwh=float(final["E"][0]), final_kwh=float(final["E"][-1]))
459     (out/"summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf
460     -8")
461     print("完成; 正式第二层轨迹与经济基准位于", out.resolve(), flush=True)
462
463 if __name__ == "__main__":
464     try:
465         main()
466     except (ValueError, RuntimeError, OSError, ImportError) as exc:
467         print(f"运行终止: {exc}", file=sys.stderr)
468         sys.exit(1) # 不输出虚构的成功结论。

```

experiments/exp009/q12_run.py

```

1  """Run Q2 or Q4-2 continuously from 2025-01-01 with initial SOC 6000 kWh.
2
3  Use --scenario 2 or --scenario 4-2 and --days 365 for a full-year run.
4  Daily files retain boundary states and support resuming the same output directory.
5  Use --out to select a new directory when recomputing archived results.
6  """
7  import argparse
8  import hashlib
9  import json
10 import platform
11 from pathlib import Path
12 from time import perf_counter
13
14 import numpy as np
15 import pandas as pd
16 import scipy
17
18 from experiments.common.neural_v2.data import ROOT
19 from experiments.exp008.closed_loop import optimize
20 from experiments.exp008.mode_budget_hold1_physical import plan as q2_plan
21 from experiments.exp008.mode_planning_physical import plan as q42_plan
22 from experiments.exp008.planner import execute
23 from experiments.exp008.verify import verify_arrays, battery_metrics
24 from experiments.exp009.q12_forecast import YearForecasts
25
26
27 def write(path, value):
28     Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False)+'\n')
29
30
31 def read(path):
32     with np.load(path, allow_pickle=False) as z:
33         return {k: z[k].copy() for k in z.files}
34
35
36 def run(scenario, days, output=None):
37     out = ROOT/"data/results/exp009"/('q2' if scenario == '2' else 'q4_2') if output is None else
38     Path(output)
39     out.mkdir(parents=True, exist_ok=True)
40     chunks = out/"daily_chunks"
41     chunks.mkdir(exist_ok=True)
42     fc = YearForecasts(scenario)
43     config = {'scenario': scenario, 'start_day': 0, 'initial_soc_kwh': 6000.,
44             'initial_mode': 1, 'initial_mode_age': 1, 'mode_boundary': 'planned mode and age' if
45             scenario=='2' else 'previous nonidle actual mode',
46             'seed': 42, 'mip_seconds': 5., 'target_mip_gap': .002, 'scenario_count_up_to': 3,
47             'refinement_history_days': 28, 'refinement_maxiter': 120, 'throughput': .002,
48             'deadband': 0., 'terminal_value_refinement': .45, 'year_end_terminal_value': 0.,
49             'forecast_path': str(fc.store.path.relative_to(ROOT)), 'forecast_sha256': fc.store.sha256,
50             'cold_start': 'day0 attachment1; January load lag7/fallback lag1 and PV lag1; own completed
51             daily errors including day0; no dispatch warmup',

```

```

49     'source_sha256': {str(p.relative_to(ROOT)): hashlib.sha256(p.read_bytes()).hexdigest()
50         for p in (Path(__file__).resolve(), ROOT/'experiments/exp009/q12_forecast.py',
51             ROOT/'experiments/exp008/closed_loop.py', ROOT/'experiments/exp008/
mode_budget_hold1_physical.py',
52             ROOT/'experiments/exp008/mode_planning_physical.py', ROOT/'experiments/exp008/
planner.py'))},
53     'raw_input_sha256': fc.data.hashes, 'python': platform.python_version(),
54     'numpy': np.__version__, 'scipy': scipy.__version__, 'threads': 1}
55 if (out/'protocol.json').exists():
56     assert json.loads((out/'protocol.json').read_text()) == config, 'changed protocol; choose
fresh output'
57 else:
58     write(out/'protocol.json', config)
59 parts, audits, rows = [], [], []
60 soc, mode, age = 6000., 1, 1
61 began = perf_counter()
62 for day in range(days):
63     part_path, audit_path = chunks/f'day_{day:03d}.npz', chunks/f'day_{day:03d}.json'
64     if part_path.exists() and audit_path.exists():
65         detail, record = read(part_path), json.loads(audit_path.read_text())
66         assert abs(float(detail['states'][0])-soc)<1e-7
67         assert record['boundary_before'] == [mode, age]
68     else:
69         start = perf_counter()
70         issue, history = fc.get(day), fc.net_error_paths(day)
71         paths = (issue['load_kw']-issue['pv_kw'])[None, :]/6+history['errors_kwh']
72         selected = np.linspace(0, len(paths)-1, min(3, len(paths))).astype(int)
73         price = issue['price']
74         if scenario == '2':
75             planned = q2_plan(paths[selected], price, soc, mode, age, final=day==364)
76         else:
77             planned = q42_plan(paths[selected], price, soc, mode, block_slots=6,
switching=50., wear=.002, seconds=5., gap=.002, final=day==364)
78             refined = optimize(planned['purchase'], paths, price, soc,
charge_mask=planned['allowed_charge'], throughput=.002, variation=0.,
81             terminal=0. if day==364 else .45, deadband=0., maxiter=120)
82             q = refined['purchase']
83             observed = fc.data.actual[day*144:(day+1)*144]
84             detail = execute(q, observed[:, :2], price, soc,
charge_mask=planned['allowed_charge'], charge_deadband=0.)
85             real_price = price if scenario == '2' else observed[:, 2]
86             detail.update(original=q, final=q.copy(), actual=observed.copy(), price=real_price.copy
87             ()),
88             allowed_charge=planned['allowed_charge'].copy())
89             detail['fees'] = np.stack((q*real_price, np.zeros(144), np.zeros(144),
5*detail['emergency']*real_price), axis=-1)
90             if scenario == '2':
91                 next_mode, next_age = planned['metadata']['final_planned_mode'], planned['metadata'
][['final_planned_run_slots']]
92             else:
93                 signs = np.sign(detail['charge']-detail['discharge'])
94                 next_mode, next_age = int(signs[signs!=0][-1]) if np.any(signs) else mode, 1
95             record = {'day': day, 'information_cutoff': day*144, 'forecast': issue['audit'],
'history': history['audit'], 'mip': planned['metadata'], 'refinement': refined['
97             metadata'],
98                 'boundary_before': [mode, age], 'boundary_after': [next_mode, next_age],
99                 'seconds': perf_counter()-start, 'purchase_locked_before_current_actual_read': True}
100             np.savez_compressed(chunks/f'planning_{day:03d}.npz',
**{k: v for k, v in planned.items() if k != 'metadata'},
102             all_net_paths=paths, selected_scenario_indices=selected,
103             predicted_price=price, refined_purchase=q, initial_soc=soc)
104             np.savez_compressed(part_path, **detail)
105             write(audit_path, record)
106             soc = float(detail['states'][-1])
107             mode, age = record['boundary_after']
108             parts.append(detail); audits.append(record)
109             rows.append({'day': day, 'date': str((pd.Timestamp('2025-01-01')+pd.Timedelta(days=day)).
date()),
110                 'total_cost': float(detail['fees'].sum()), 'planned_cost': float(detail['fees'][:,0].sum
()),

```

```

111         'emergency_cost': float(detail['fees'][:,3].sum()), 'emergency_kwh': float(detail['
emergency'].sum()),
112         'initial_soc': float(detail['states'][0]), 'final_soc': soc,
113         'mip_gap': record['mip']['mip_gap'], 'mip_seconds': record['mip']['seconds'],
114         'refinement_seconds': record['refinement']['planning_seconds'])
115     if day < 3 or (day+1)%10 == 0:
116         print(scenario, 'DAY', day, 'cost', rows[-1]['total_cost'], 'seconds', round(
perf_counter()-began, 1), flush=True)
117     arrays = {k: np.stack([part[k] for part in parts]) for k in parts[0]}
118     arrays['days'] = np.arange(days)
119     np.savez_compressed(out/'dispatch.npz', **arrays)
120     write(out/'planning_audit.json', audits)
121     pd.DataFrame(rows).to_csv(out/'daily.csv', index=False)
122     actual = fc.data.actual[:days*144].reshape(days, 144, 3)
123     check = verify_arrays(arrays, scenario, expected_days=days, start_day=0, initial_soc=6000.,
initial_mode=0, initial_power_kw=0., source_actual=actual,
124         source_price=fc.data.fixed_price if scenario=='2' else actual[:, :, 2], audit_records=audits)
125     assert check['passed'], check['errors']
126     write(out/'verification.json', check)
127     summary = {'scenario': scenario, 'days': days, 'complete': days==365,
128         'full_year_cost': float(arrays['fees'].sum()), 'january_cost': float(arrays['fees'][:31].sum
129         ()),
130         'feb_dec_cost': float(arrays['fees'][31:].sum()), 'feb_initial_soc': float(arrays['states'
][31,0]) if days>31 else None,
131         'last_soc': soc, 'battery': battery_metrics(arrays, initial_mode=0, initial_power_kw=0.),
132         'mip_seconds_sum': sum(r['mip_seconds'] for r in rows),
133         'refinement_seconds_sum': sum(r['refinement_seconds'] for r in rows),
134         'mip_gap_max': max(r['mip_gap'] for r in rows), 'mip_gap_mean': float(np.mean([r['mip_gap']
for r in rows])),
135         'verified': True, 'forecast_retrained': False}
136     if days>31:
137         subset={k:v[31:] for k,v in arrays.items()}
138         summary['feb_dec_battery'] = battery_metrics(subset, initial_mode=0, initial_power_kw=0.)
139     write(out/'summary.json', summary)
140     print('DONE', json.dumps(summary, ensure_ascii=False), flush=True)
141     return summary
142
143
144 if __name__ == '__main__':
145     parser=argparse.ArgumentParser(description=__doc__)
146     parser.add_argument('--scenario', choices=['2', '4-2'], required=True)
147     parser.add_argument('--days', type=int, default=365)
148     parser.add_argument('--out', type=Path)
149     args=parser.parse_args()
150     run(args.scenario, args.days, args.out)

```

experiments/exp009/q12_forecast.py

```

1 """Causal year-start adapters for the accepted Q2 and Q4-2 predictors.
2
3 January's periodic fallback is a declared part of the same forecast pipeline.
4 From February the accepted issued predictions are reused byte-for-byte; no
5 dispatch warmup archive or current/future observations are used as features.
6 """
7 from functools import lru_cache
8 import hashlib
9 from pathlib import Path
10
11 import numpy as np
12
13 from experiments.common.neural_v2.data import Data, ROOT
14 from experiments.exp008.forecast import Forecasts, _integration_basis
15 from experiments.exp008.hgb_linked_price_forecast import LinkedPriceForecasts
16
17
18 class FrozenStore:
19     def __init__(self, path):

```

```

20     self.path = Path(path)
21     self.sha256 = hashlib.sha256(self.path.read_bytes()).hexdigest()
22     with np.load(self.path, allow_pickle=False) as z:
23         self.origins = z['origins'].copy()
24         self.values = z['values'].copy()
25     assert np.array_equal(self.origins, np.arange(31, 365)*144)
26     assert self.values.shape == (334, 144, 2)
27     self.lookup = {int(x): i for i, x in enumerate(self.origins)}
28
29     def get(self, origin):
30         return self.values[self.lookup[int(origin)]].copy()
31
32
33 class YearForecasts(Forecasts):
34     def __init__(self, scenario, data=None):
35         self.data = Data() if data is None else data
36         self.seed = 42
37         self.scenario = scenario
38         path = ('forecast_hgb_extra_trees_half/hgb_extra_trees_half_ridge28_memory.npz'
39                 if scenario == '2' else
40                 'forecast_absolute_hgb/direct_hgb_ridge28_memory_half.npz')
41         self.store = FrozenStore(ROOT/'data/results/exp008'/path)
42         self._basis = _integration_basis()
43         difference = np.diff(np.eye(24), n=2, axis=0)
44         self._pv_penalty = 12*np.einsum('ni,nj->ij', difference, difference) + 2*np.eye(24)
45
46     design = LinkedPriceForecasts.design
47
48     @lru_cache(maxsize=400)
49     def _price(self, origin, stop):
50         if origin < 144:
51             return Forecasts._price(self, origin, stop)
52         if origin == 144:
53             # No training labels have valid one-day-lag regressors yet. The
54             # unchanged regression prior is .5 daily + .5 weekly (daily fallback).
55             indices = np.arange(origin, stop)
56             value = self._observed(0, origin, origin)[indices-144, 2].copy()
57             return value, {'price_method': 'lag_regression_prior_without_valid_training_rows',
58                           'price_last_label': origin-1, 'price_training_rows': 0,
59                           'known_future_price': False, 'price_feature_columns': 10}
60         return LinkedPriceForecasts._price(self, origin, stop)
61
62     def get(self, day, slot=0, scenario=None):
63         scenario = self.scenario if scenario is None else scenario
64         out = Forecasts.get(self, day, slot, scenario)
65         out['audit'].update(base_forecast=('
66 raw_HGB_ExtraTrees_half_then_common_Ridge28_then_half_memory'
67         if scenario == '2' else 'single_HGB_then_Ridge28_then_half_memory') if day >= 31
68         else 'causal_periodic_cold_start', load_method='same_issued_midnight_pipeline',
69         no_historical_dispatch_warmup=True, seed=42)
70         return out
71
72     def net_error_paths(self, day, scenario=None, limit=28):
73         scenario = self.scenario if scenario is None else scenario
74         history = np.arange(max(0, day-limit), day, dtype=int)
75         if len(history):
76             errors, prices = [], []
77             for old in history:
78                 issue = self.get(int(old), scenario=scenario)
79                 observed = self._observed(int(old)*144, (int(old)+1)*144, int(day)*144)
80                 errors.append(observed[:, :2]-np.column_stack((issue['load_kw'], issue['pv_kw'])))
81                 prices.append(observed[:, 2]-issue['price'])
82             errors = np.asarray(errors)
83             net = (errors[:, :, 0]-errors[:, :, 1])/6
84         else:
85             errors, net, prices = np.zeros((1, 144, 2)), np.zeros((1, 144)), np.zeros((1, 144))
86         return {'errors_kwh': net, 'errors_kw': errors, 'price_errors': np.asarray(prices),
87               'origins': history*144,
88               'audit': {'information_cutoff_exclusive': day*144,
89                        'max_observed_index': day*144-1 if day else None,

```

```

89         'training_origins': (history*144).tolist(),
90         'fallback_days': history[history<31].tolist(),
91         'source': 'own_issued_pipeline_complete_daily_errors',
92         'no_history_zero_path': day == 0,
93         'history_includes_day_zero_once_complete': True}}

```

experiments/exp009/q34_run.py

```

1  """Run the accepted Q3/Q4-3 LP with refund billing and own January startup.
2
3  Examples (from repository root):
4      .venv/bin/python -m experiments.exp009.q34_run --scenario 3 --days 2 --label smoke
5      .venv/bin/python -m experiments.exp009.q34_run --scenario 3
6      .venv/bin/python -m experiments.exp009.q34_run --scenario 4-3
7  """
8  import argparse
9  import copy
10 import hashlib
11 import json
12 import platform
13 from dataclasses import asdict
14 from pathlib import Path
15 from time import perf_counter
16 import numpy as np
17 import pandas as pd
18 import scipy
19
20 from experiments.exp008.planner import execute
21 from experiments.exp008.verify import battery_metrics
22 from experiments.exp009.q34_forecast import SelectedForecasts, residual_paths, DIRECTORY, MODEL
23 from experiments.exp009.q34_planner import plan, Settings, ETA, LOW, HIGH, LIMIT
24
25 ROOT = Path(__file__).resolve().parents[2]
26 OUT = ROOT / 'data/results/exp009'
27 SETTINGS = Settings(future_shortfall_weight=1.5)
28
29
30 def save(path, value):
31     path = Path(path)
32     path.parent.mkdir(parents=True, exist_ok=True)
33     path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False)+'\n')
34
35
36 def digest(path):
37     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
38
39
40 def settle(original, final, emergency, price):
41     return np.stack((original*price, 1.5*price*np.maximum(final-original, 0),
42                     -0.5*price*np.maximum(original-final, 0), 5*price*emergency), axis=-1)
43
44
45 def verify(a, initial=6000.):
46     """Slotwise physical and signed-refund accounting checks, no optimization."""
47     c, d, e, w, q, g, s, p = (a[k] for k in
48                               ('charge', 'discharge', 'emergency', 'surplus', 'final', 'original', 'states', 'price'))
49     balance = q+(a['actual'][..., 1]-a['actual'][..., 0])/6+d+e-c-w
50     state_error = np.diff(s, axis=1)-ETA*c+d/ETA
51     fees = np.stack((g*p, 1.5*p*np.maximum(q-g, 0), -0.5*p*np.maximum(g-q, 0), 5*p*e), axis=-1)
52     max_balance, max_soc = float(np.abs(balance).max()), float(np.abs(state_error).max())
53     continuity = float(np.abs(s[1:, 0]-s[:-1, -1]).max(initial=0))
54     checks = {
55         'all_finite': all(np.isfinite(a[k]).all() for k in a),
56         'nonnegative_energy': min(float(a[k].min()) for k in ('charge', 'discharge', 'emergency', 'surplus', 'final', 'original')) >= -1e-6,
57         'balance': max_balance < 1e-6, 'soc_recurrence': max_soc < 1e-6,
58         'soc_continuity': continuity < 1e-6, 'initial_soc': abs(s[0, 0]-initial)<1e-6,

```

```

59     'soc_bounds': bool(s.min()>=LOW-1e-6 and s.max()<=HIGH+1e-6),
60     'power_bounds': bool(max(c.max(), d.max())<=LIMIT+1e-6),
61     'no_simultaneous_actions': not bool(((c>1e-6)&(d>1e-6)).any()),
62     'no_emergency_charging': not bool(((c>1e-6)&(e>1e-6)).any()),
63     'signed_refund_billing': bool(np.max(np.abs(fees-a['fees']))<1e-6),
64     'day_identifiers': bool(np.array_equal(a['days'], np.arange(int(a['days'][0]),int(a['days']
    ][0])+len(s))))}
65 if 'versions' in a:
66     expected = np.concatenate([a['versions'][:, j, j*36:(j+1)*36] for j in range(4)],axis=1)
67     checks['last_legal_version_per_executed_slot'] = bool(np.max(np.abs(expected-q))<1e-6)
68     checks['original_is_midnight_version'] = bool(np.max(np.abs(a['versions'][:,0]-g))<1e-6)
69     checks = {key: bool(value) for key, value in checks.items()}
70     result = {'passed': all(checks.values()), 'checks': checks,
71             'max_balance_error_kwh': max_balance, 'max_soc_error_kwh': max_soc,
72             'max_cross_day_soc_error_kwh': continuity,
73             'max_slot_fee_error_yuan': float(np.max(np.abs(fees-a['fees']))),
74             'billing': dict(zip(('planned_cost', 'up_cost', 'down_refund', 'emergency_cost'), map(float,
    fees.sum(axis=(0,1)))))},
75     'total_cost': float(fees.sum()), 'planned_kwh': float(g.sum()),
76     'final_regular_kwh': float(q.sum()), 'up_kwh': float(np.maximum(q-g,0).sum()),
77     'down_kwh': float(np.maximum(g-q,0).sum()), 'emergency_kwh': float(e.sum()),
78     'battery': battery_metrics(a, initial_mode=0, initial_power_kw=0.))
79 assert result['passed'], result
80 return result
81
82
83 def causal_checks():
84     rows=[]
85     for scenario in ('3','4-3'):
86         for day,slot in ((0,0),(0,36),(1,0),(1,36),(31,0),(32,36),(60,72),(180,108)):
87             before=SelectedForecasts()
88             changed=copy.copy(before.data)
89             cutoff=day*144+slot
90             changed.actual=before.data.actual.copy()
91             changed.actual[cutoff:]+=100000.,50000.,1000.]
92             changed._forecasts={k:np.asarray(v).copy()+(90000. if k>cutoff else 0.) for k,v in
    before.data.forecasts.items()}
93             after=SelectedForecasts(changed)
94             a,b=before.get(day,slot,scenario),after.get(day,slot,scenario)
95             for key in ('load_kw','pv_kw','price'):
96                 np.testing.assert_array_equal(a[key],b[key])
97             ra,aa=residual_paths(before,day,slot,scenario)
98             rb,ab=residual_paths(after,day,slot,scenario)
99             np.testing.assert_array_equal(ra,rb)
100             assert aa==ab and all(x<=day*144 for x in aa['label_stops_exclusive'])
101             rows.append({'scenario':scenario,'day':day,'slot':slot,'future_mutation_identical':True,
102                 'same_issued_history_identical':True,'label_stop_max':max(aa['label_stops_exclusive']
    ],default=None)})
103 save(OUT/'q34_causality.json',{'passed':True,'rows':rows,
104     'scope':'adapter/current official issue/price/cold-start/residual paths; HGB training
    provenance inherited unchanged'})
105
106
107 def summarize(a, directory, scenario, rows):
108     whole=verify(a)
109     save(directory/'verification_365.json',whole)
110     formal_mask=a['days']>=31
111     if formal_mask.any():
112         formal={k:v[formal_mask] for k,v in a.items()}
113         warm={k:v[~formal_mask] for k,v in a.items()}
114         np.savez_compressed(directory/f'dispatch_{scenario}.npz',**formal)
115         np.savez_compressed(directory/'warmup.npz',**warm)
116         summary=verify(formal, float(formal['states'][0,0]))
117         summary.update(scenario=scenario, days=len(formal['days']), intervals=len(formal['days'])
    *144,
118             evaluation_start='2025-02-01',evaluation_end=str(pd.Timestamp('2025-01-01')+pd.Timedelta
    (days=int(formal['days'][-1]))),
119             initial_soc_january1_kwh=6000., initial_soc_february1_kwh=float(formal['states'][0,0]),
120             january_cost=whole['total_cost']-summary['total_cost'],
121             same_selected_controller_throughout_year=True, inherited_exp002_state=False,

```

```

122     billing_rule='p*(g0+1.5*(q-g0)+-0.5*(g0-q))+5*e)',
123     solver='7 representative historical paths shared-purchase LP, released every 6 hours',
124     final_day_terminal_value=0., charge_deadband_kwh=20.)
125 save(directory/'summary.json',summary)
126 daily=pd.DataFrame(rows).query('day>=31')
127 daily.to_csv(directory/'daily.csv',index=False)
128 selected=daily[daily.date.isin(['2025-03-20','2025-06-21','2025-09-23','2025-12-21'])]
129 selected.to_csv(directory/'specified_daily.csv',index=False)
130 blocks=[]; emergencies=[]
131 for day in (78,171,265,354):
132     ids=np.flatnonzero(a['days']==day)
133     if not len(ids): continue
134     i=int(ids[0]); date=str((pd.Timestamp('2025-01-01')+pd.Timedelta(days=day)).date())
135     for start,stop in zip((0,24,48,72,96,120),(24,48,72,96,120,144)):
136         blocks.append(dict(date=date,start_hour=start//6,end_hour=stop//6,
137             charge_kwh=float(a['charge'][i,start:stop].sum()),discharge_kwh=float(a['
138 discharge'][i,start:stop].sum()),
139             soc_start_kwh=float(a['states'][i,start]),soc_end_kwh=float(a['states'][i,stop])
140 ))
141     mask=a['emergency'][i]>1e-6
142     edges=np.diff(np.r_[False,mask,False].astype(int));begins=np.flatnonzero(edges==1);ends=
143 np.flatnonzero(edges==-1)
144     for start,stop in zip(begins,ends):
145         emergencies.append(dict(date=date,start_slot=int(start),stop_slot=int(stop),
146             emergency_kwh=float(a['emergency'][i,start:stop].sum())))
147     pd.DataFrame(blocks).to_csv(directory/'specified_battery.csv',index=False)
148     pd.DataFrame(emergencies).to_csv(directory/'specified_emergency.csv',index=False)
149     return summary
150 return whole
151
152 def run(scenario, days=365, label=None):
153     directory=OUT/('q3' if scenario=='3' else 'q4_3')
154     if label: directory=directory/label
155     directory.mkdir(parents=True,exist_ok=True)
156     if (directory/'completion.json').exists():
157         raise FileExistsError(f'Completed immutable run exists: {directory}')
158     forecasts=SelectedForecasts()
159     sources={str(p.relative_to(ROOT)):digest(p) for p in Path(__file__).parent.glob('q34*.py')}
160     sources.update({str(p.relative_to(ROOT)):digest(p) for p in (ROOT/'experiments/exp008/planner.py'
161 ,ROOT/'experiments/exp008/forecast.py',ROOT/'experiments/common/neural_v2/physics.py')})
162     command=f'.venv/bin/python -m experiments.exp009.q34_run --scenario {scenario} --days {days}'+(f
163 ' --label {label}' if label else '')
164     protocol={'scenario':scenario,'days':days,'initial_soc':6000.,'seed':42,'settings':asdict(
165 SETTINGS),
166         'charge_deadband_kwh':20.,'sources':sources,'raw_data_sha256':forecasts.data.hashes,
167         'forecast_archive_sha256':digest(DIRECTORY/f'{MODEL}.npz'),'original_commit':'459cdb2',
168         'changes':['downward half refund and q>=0','own selected algorithm January startup'],
169         'command':command,'python':platform.python_version(),'numpy':np.__version__,'scipy':scipy.
170 __version__}
171     save(directory/'protocol.json',protocol)
172     began=perf_counter();soc=6000.;parts=[];rows=[];audits=[]
173     for day in range(days):
174         day_start=perf_counter();original=None;final=np.zeros(144);states=np.empty(145);states[0]=
175 soc
176         components={key:np.zeros(144) for key in ('charge','discharge','emergency','surplus')}
177         versions=[]
178         for slot,stop in zip((0,36,72,108),(36,72,108,144)):
179             forecast=forecasts.get(day,slot,scenario)
180             errors,risk=residual_paths(forecasts,day,slot,scenario)
181             result=plan(forecast,float(states[slot]),errors,None if slot==0 else original[slot:],
182                 settings=SETTINGS,final_day=day==364,next_update_slot=stop-slot)
183             if slot==0:original=result['purchase'].copy()
184             final[slot:]=result['purchase'];versions.append(final.copy())
185             actual=forecasts.data.actual[day*144+slot:day*144+stop]
186             price=actual[:,2] if scenario=='4-3' else forecasts.data.fixed_price[slot:stop]
187             executed=execute(final[slot:stop],actual,price,float(states[slot]),charge_deadband=20.)
188             for key in components:components[key][slot:stop]=executed[key]
189             states[slot:stop+1]=executed['states']
190             audits.append({'day':day,'slot':slot,'forecast':forecast['audit'],'risk':risk,'solver':

```

```

183     result['metadata'])
184     actual=forecasts.data.actual[day*144:(day+1)*144].copy()
185     price=actual[:,2].copy() if scenario=='4-3' else forecasts.data.fixed_price.copy()
186     fees=settle(original,final,components['emergency'],price)
187     parts.append(dict(original=original,final=final,**components,states=states,fees=fees,actual=
188     actual,price=price,versions=np.stack(versions)))
189     soc=float(states[-1]);billing=fees.sum(axis=0)
190     rows.append(dict(day=day,date=str((pd.Timestamp('2025-01-01')+pd.Timedelta(days=day)).date()
191     ),
192     scenario=scenario,planned_cost=float(billing[0]),up_cost=float(billing[1]),down_refund=
193     float(billing[2]),
194     emergency_cost=float(billing[3]),total_cost=float(fees.sum()),planned_kwh=float(original
195     .sum()),
196     final_regular_kwh=float(final.sum()),up_kwh=float(np.maximum(final-original,0).sum()),
197     down_kwh=float(np.maximum(original-final,0).sum()),
198     emergency_kwh=float(components['emergency'].sum()),initial_soc=float(states[0]),
199     final_soc=soc,seconds=perf_counter()-day_start))
200     if (day+1)%10==0 or day+1==days:
201         print(json.dumps({'scenario':scenario,'days':day+1,'elapsed_seconds':perf_counter()-
202         began,'cost':sum(r['total_cost'] for r in rows)}),flush=True)
203         save(directory/'progress.json',{'days':day+1,'elapsed_seconds':perf_counter()-began,'
204         last_day':rows[-1]})
205     arrays={key:np.stack([part[key] for part in parts]) for key in parts[0]}
206     arrays['days']=np.arange(days)
207     np.savez_compressed(directory/'dispatch_365.npz',**arrays)
208     pd.DataFrame(rows).to_csv(directory/'daily_365.csv',index=False)
209     save(directory/'audit.json',audits)
210     summary=summarize(arrays,directory,scenario,rows)
211     wall=perf_counter()-began
212     save(directory/'completion.json',{'complete':True,'full_year':days==365,'wall_seconds':wall,
213     'evaluation_cost':summary['total_cost'],'source_hashes':sources,'archive_sha256':digest(
214     directory/'dispatch_365.npz'),
215     'solver_planning_seconds':sum(a['solver']['planning_seconds'] for a in audits),
216     'solver_feasible_all':all(a['solver']['feasible'] for a in audits),'run_command':command})
217     print(json.dumps({'scenario':scenario,'done':True,'wall_seconds':wall,'summary':summary}),flush=
218     True)
219
220 if __name__=='__main__':
221     parser=argparse.ArgumentParser();parser.add_argument('--scenario',choices=('3','4-3'),default='3
222     ')
223     parser.add_argument('--days',type=int,default=365);parser.add_argument('--label');parser.
224     add_argument('--causality',action='store_true')
225     args=parser.parse_args()
226     if args.causality:causal_checks()
227     else:run(args.scenario,args.days,args.label)

```

experiments/exp009/q34_planner.py

```

1  """Joint historical-path purchase planning; realized dispatch is verified separately.
2
3  The scenario recourse LP is an optimistic planning approximation: its battery
4  states are scenario dependent, not a certified nonanticipative scenario tree.
5  Only purchases are shared. No future realized observations enter this module.
6  """
7  from dataclasses import dataclass
8  from time import perf_counter
9
10 import numpy as np
11
12 from experiments.common.neural_v2.physics import LinearModel
13
14 ETA = float(np.sqrt(.9))
15 LOW, HIGH, LIMIT = 1200., 10800., 5000 / 6
16
17
18 @dataclass(frozen=True)

```

```

19 class Settings:
20     scenarios: int = 7
21     residual_scale: float = 1.
22     throughput_penalty: float = .005
23     variation_penalty: float = .001
24     terminal_value: float = 1.
25     emergency_weight: float = 5.
26     future_shortfall_weight: float | None = None
27
28
29 def plan(forecast, initial_soc, error_paths=None, original=None, settings=Settings(),
30         final_day=False, charge_mask=None, next_update_slot=None,
31         future_shortfall_weight=None):
32     """Return shared purchases plus representative inventory/mode intentions.
33
34     next_update_slot is relative to this remaining horizon: 36 means its first
35     36 ten-minute intervals execute before the next legal information release.
36     The optional future weight applies from that index onward, only when both
37     a boundary and a weight are present. It is an opportunity-value proxy, not
38     an alternative billing rule or a nonanticipative recourse certificate.
39     """
40     began = perf_counter()
41     net = (np.asarray(forecast['load_kw']) - forecast['pv_kw']) / 6
42     price = np.asarray(forecast['price'])
43     n = len(net)
44     shortfall_weights = np.broadcast_to(np.asarray(settings.emergency_weight, float), (n,)).copy()
45     future_weight = (getattr(settings, 'future_shortfall_weight', None)
46                     if future_shortfall_weight is None else future_shortfall_weight)
47     if future_weight is not None and (not np.isfinite(future_weight) or future_weight <= 0):
48         raise ValueError('future_shortfall_weight must be finite and positive')
49     if next_update_slot is not None:
50         if (not isinstance(next_update_slot, (int, np.integer))
51             or not 1 <= next_update_slot <= n):
52             raise ValueError('next_update_slot must be a relative horizon index in 1..n')
53         if future_weight is not None:
54             shortfall_weights[next_update_slot:] = future_weight
55     paths = np.asarray(error_paths) if error_paths is not None else np.zeros((1, n))
56     if not len(paths):
57         paths = np.zeros((1, n))
58     if paths.shape[1] == 144:
59         paths = paths[:, -n:]
60     if len(paths) > settings.scenarios:
61         # Deterministic equally spaced complete historical days, retaining
62         # within-day correlations. No sorting by future or realized cost.
63         paths = paths[np.linspace(0, len(paths)-1, settings.scenarios).astype(int)]
64     demands = net[None, :] + settings.residual_scale * paths
65     k = len(demands)
66     assert demands.shape == (k, n) and price.shape == (n,)
67     assert np.isfinite(demands).all() and np.all(price > 0)
68     assert LOW - 1e-6 <= initial_soc <= HIGH + 1e-6
69     m = LinearModel()
70     g = m.variables((n,), cost=price if original is None else 0., lower=0.)
71     if original is not None:
72         original = np.asarray(original, float)
73         up = m.variables((n,), cost=1.5 * price)
74         down = m.variables((n,), cost=-0.5 * price)
75         for t in range(n):
76             m.constraint([(g[t], 1), (up[t], -1), (down[t], 1)], original[t], original[t])
77         # Since simultaneous up/down costs +p per kWh, the LP itself chooses
78         # their positive parts. The fixed original bill is a dropped constant.
79     charge_limit = LIMIT if charge_mask is None else LIMIT*np.asarray(charge_mask)[None, :]
80     discharge_limit = LIMIT if charge_mask is None else LIMIT*(1-np.asarray(charge_mask))[None, :]
81     c = m.variables((k, n), upper=charge_limit, cost=settings.throughput_penalty/k)
82     d = m.variables((k, n), upper=discharge_limit, cost=settings.throughput_penalty/k)
83     e = m.variables((k, n), cost=shortfall_weights*price[None, :]/k)
84     w = m.variables((k, n))
85     s = m.variables((k, n), lower=LOW, upper=HIGH)
86     tv = m.variables((k, n), cost=settings.variation_penalty*6/k)
87     terminal = 0 if final_day else settings.terminal_value*price.min()/ETA
88     for index in s[:, -1]:

```

```

89     m.objective[int(index)] = -terminal/k
90     for j in range(k):
91         for t in range(n):
92             m.constraint([(g[t], 1), (d[j,t], 1), (e[j,t], 1),
93                           (c[j,t], -1), (w[j,t], -1)], demands[j,t])
94             transition = [(s[j,t], 1), (c[j,t], -ETA), (d[j,t], 1/ETA)]
95             rhs = initial_soc if t == 0 else 0
96             if t:
97                 transition.append((s[j,t-1], -1))
98             m.constraint(transition, rhs, rhs)
99             if t:
100                 diff = [(c[j,t],1),(d[j,t],-1),(c[j,t-1],-1),(d[j,t-1],1)]
101                 m.constraint(diff + [(tv[j,t],-1)], upper=0)
102                 m.constraint([(ids,-v) for ids,v in diff] + [(tv[j,t],-1)], upper=0)
103 x, meta = m.solve(seconds=20, gap=1e-7)
104 if x is None:
105     raise RuntimeError(f'Joint-path LP failed: {meta}')
106 return {'purchase': np.maximum(x[g], 0), 'charge': np.maximum(x[c].mean(0), 0),
107        'discharge': np.maximum(x[d].mean(0), 0),
108        'states': np.r_[initial_soc, x[s].mean(0)],
109        'metadata': {'*meta', 'planning_seconds': perf_counter()-began,
110                     'method': 'joint_path_lp_surrogate', 'scenarios': k,
111                     'nonanticipative_recourse_certificate': False,
112                     'next_update_slot': None if next_update_slot is None else int(
113 next_update_slot),
114                     'future_shortfall_weight': future_weight,
115                     'planning_emergency_weights': shortfall_weights.tolist(),
116                     'opportunity_proxy_active': bool(future_weight is not None
117 and next_update_slot is not None and next_update_slot < n),
118                     'downward_revision_allowed': original is not None,
119                     'settlement': 'original+1.5*up-0.5*down+5*emergency'}}

```

experiments/exp009/q34_forecast.py

```

1  """Selected exp008 HGB pipeline, with its causal January start integrated.
2
3  Reads no exp002 state or exp004 prediction archive. January uses the selected
4  adapter's periodic fallback; February onward uses the unchanged accepted HGB
5  Ridge28/half-memory issued archive. PV and price release rules remain exp008.
6  """
7  import hashlib
8  import json
9  from functools import lru_cache
10 from pathlib import Path
11 import numpy as np
12
13 from experiments.common.neural_v2.data import Data
14 from experiments.exp008.forecast import Forecasts, _integration_basis
15
16 ROOT = Path(__file__).resolve().parents[2]
17 DIRECTORY = ROOT / 'data/results/exp008/forecast_absolute_hgb'
18 MODEL = 'direct_hgb_ridge28_memory_half'
19
20
21 class Store:
22     def __init__(self):
23         path = DIRECTORY / f'{MODEL}.npz'
24         meta = json.loads((DIRECTORY / 'provenance.json').read_text())
25         assert meta['complete'] and hashlib.sha256(path.read_bytes()).hexdigest() == meta['archives'
26 ][MODEL]
27         with np.load(path) as z:
28             self.values, self.origins = z['values'].copy(), z['origins'].copy()
29             self.lookup = {int(o): i for i, o in enumerate(self.origins)}
30
31     def get(self, origin):
32         return self.values[self.lookup[int(origin)]].copy()

```

```

33
34 class SelectedForecasts(Forecasts):
35     def __init__(self, data=None):
36         self.data = Data() if data is None else data
37         self.seed = 42
38         self.store = Store()
39         self.calibration = MODEL
40         self._basis = _integration_basis()
41         difference = np.diff(np.eye(24), n=2, axis=0)
42         self._pv_penalty = 12 * np.einsum('ni,nj->ij', difference, difference, optimize=False) + 2 *
            np.eye(24)
43
44     @lru_cache(maxsize=3000)
45     def _price(self, origin, stop):
46         if origin == 144:
47             return self._observed(0, 144, origin)[: , 2].copy(), {
48                 'price_method': 'observed_previous_day_before_ridge_has_training_rows',
49                 'price_last_label': 143, 'known_future_price': False}
50         return super()._price(origin, stop)
51
52     @lru_cache(maxsize=3000)
53     def get(self, day, slot=0, scenario='3'):
54         result = super().get(day, slot, scenario)
55         result['audit'].update(
56             base_forecast=MODEL if day >= 31 else 'causal_periodic_cold_start',
57             load_method='selected_HGB_with_causal_prefix_bias' if day >= 31 else '
periodic_with_causal_prefix_bias',
58             selected_model_id=MODEL,
59             january_rule='appendix1_day0_then_observed_weekly_load_daily_PV',
60             no_exp002_dispatch_or_state=True)
61         return result
62
63
64 def residual_paths(forecasts, day, slot, scenario, limit=28):
65     """All completed prior target days; zero path when no day is available."""
66     history = np.arange(max(0, day - limit), day, dtype=int)
67     n = 144 - slot
68     errors, prices = [], []
69     for old in history:
70         pred = forecasts.get(int(old), slot, scenario)
71         truth = forecasts._observed(int(old)*144+slot, (int(old)+1)*144, day*144+slot)
72         errors.append(((truth[:, 0]-pred['load_kw'])-(truth[:, 1]-pred['pv_kw']))/6)
73         prices.append(truth[:, 2]-pred['price'])
74     paths = np.asarray(errors) if len(history) else np.zeros((1, n))
75     return paths, {
76         'source': 'same_selected_algorithm_prior_completed_target_days',
77         'model_id': MODEL, 'origins': (history*144+slot).tolist(),
78         'label_stops_exclusive': ((history+1)*144).tolist(),
79         'max_observed_index': int(day*144-1) if day else None,
80         'zero_history_deterministic_path': not len(history),
81         'january_cold_start_days': history[history < 31].tolist(),
82         'price_errors_used_in_planning': False,
83         'errors_kwh_sha256': hashlib.sha256(np.ascontiguousarray(paths).tobytes()).hexdigest()}

```

experiments/common/neural_v2/data.py

```

1 """Issue-time feature construction; attachment 3 is loaded only on demand."""
2 import hashlib
3 import json
4 from pathlib import Path
5
6 import numpy as np
7 import pandas as pd
8
9 ROOT = Path(__file__).resolve().parents[3]
10 HERE = Path(__file__).resolve().parent
11 EPOCH = pd.Timestamp("2025-01-01")

```

```

12 STEPS = 144
13 TARGETS = ("load", "pv", "price", "pv_corrected")
14 CHANNELS = np.array([0, 1, 2, 1])
15 SEEDS = (42, 2026, 3407)
16
17
18 def protocol():
19     return json.loads((HERE / "protocol.json").read_text())
20
21
22 def signature(names, extra=None):
23     digest = hashlib.sha256()
24     for name in names:
25         digest.update((HERE / name).read_bytes())
26     digest.update(json.dumps(extra, sort_keys=True).encode())
27     return digest.hexdigest()
28
29
30 class Data:
31     def __init__(self, root=ROOT):
32         self.root = Path(root)
33         raw = self.root / "data/raw"
34         names = ("附件2_小区负载.csv", "附件2_光伏发电实际功率.csv", "附件4.csv")
35         frames = [pd.read_csv(raw / name) for name in names]
36         for frame in frames:
37             assert frame.shape == (365, 145)
38             assert pd.DatetimeIndex(pd.to_datetime(frame.iloc[:, 0])).equals(
39                 pd.date_range("2025-01-01", "2025-12-31"))
40         self.actual = np.stack([f.iloc[:, 1:].to_numpy(float).ravel() for f in frames], -1)
41         assert np.isfinite(self.actual).all() and (self.actual >= 0).all()
42         self.reference = pd.read_csv(raw / "附件1.csv").iloc[:, 1:].to_numpy(float)
43         self.fixed_price = self.reference[:, 0]
44         self._forecasts = None
45         self.hashes = {p.name: hashlib.sha256(p.read_bytes()).hexdigest()
46                        for p in sorted(raw.glob("*.csv"))}
47
48     @property
49     def forecasts(self):
50         if self._forecasts is None:
51             frame = pd.read_csv(self.root / "data/raw/附件3.csv")
52             frame.iloc[:, 0] = frame.iloc[:, 0].ffill()
53             self._forecasts = {}
54             for row in frame.itertuples(index=False, name=None):
55                 day = (pd.Timestamp(row[0]) - EPOCH).days
56                 hour = int(str(row[1]).split(":")[0])
57                 self._forecasts[day * 144 + hour * 6] = np.asarray(row[2:], float)
58             assert len(self._forecasts) == 1460
59         return self._forecasts
60
61     def issued_points(self, origin):
62         anchor = self.actual[origin - 1, 1] if origin else 0.0
63         return np.maximum(0, np.interp(np.arange(1, 145), np.arange(0, 145, 6),
64                                     np.r_[anchor, self.forecasts[origin]]))
65
66     def integrate_points(self, points, origin):
67         """kW interval averages; multiplying by 1/6 gives exact linear integral."""
68         anchor = self.actual[origin - 1, 1] if origin else 0.0
69         return (np.r_[anchor, points[:-1]] + points) / 2
70
71     def interpolate_hourly(self, hourly, origin):
72         anchor = self.actual[origin - 1, 1] if origin else 0.0
73         points = np.interp(np.arange(1, 145), np.arange(0, 145, 6), np.r_[anchor, hourly])
74         return self.integrate_points(points, origin)
75
76     def lag(self, origin, days):
77         target = origin + np.arange(144)
78         reference = self.reference[target % 144][:, [1, 2, 0]].copy()
79         ids = target - days * 144
80         known = (ids >= 0) & (ids < origin)
81         reference[known] = self.actual[ids[known]]

```

```

82     if days > 1:
83         recent = target - 144
84         fallback = ~known & (recent >= 0) & (recent < origin)
85         reference[fallback] = self.actual[recent[fallback]]
86     return reference
87
88 def baseline(self, origin, kind="periodic", issued=True, points=False):
89     yesterday, weekly = self.lag(origin, 1), self.lag(origin, 7)
90     if kind == "yesterday":
91         output = yesterday.copy()
92     elif kind == "weekly":
93         output = weekly.copy()
94     else:
95         output = weekly.copy(); output[:, 1] = yesterday[:, 1]
96     if issued:
97         pv = self.issued_points(origin)
98         output = np.column_stack((output, pv if points else self.integrate_points(pv, origin)))
99     return output
100
101 def features(self, origins, cutoff, issued=True):
102     assert 0 < cutoff <= max(origins)
103     mean = self.actual[:cutoff].mean(0)
104     scale = np.maximum(self.actual[:cutoff].std(0), 1e-6)
105     features, bases = [], []
106     count = 4 if issued else 3
107     for o in origins:
108         assert o >= 7 * 144
109         base = self.baseline(o, issued=issued, points=True)
110         y, w = self.lag(o, 1), self.lag(o, 7)
111         end = o + np.arange(1, 145)
112         dayphase = end % 144 / 144 * 2 * np.pi
113         weekphase = ((end // 144 + 2) % 7) / 7 * 2 * np.pi
114         issuephase = o % 144 / 144 * 2 * np.pi
115         cal = np.column_stack((np.sin(dayphase), np.cos(dayphase),
116                               np.sin(weekphase), np.cos(weekphase),
117                               np.full(144, np.sin(issuephase)),
118                               np.full(144, np.cos(issuephase)), np.arange(1,145)/144))
119
120         branches = []
121         for j, c in enumerate(CHANNELS[:count]):
122             h = self.actual[max(0, o-168*6):o, c]
123             values = [self.actual[o-1, c], h[-36:].mean(), h[-144:].mean(), h.mean()]
124             stats = np.tile(np.r_[(np.array(values)-mean[c])/scale[c],
125                                   h[-144:].std()/scale[c], h.std()/scale[c]], (144,1))
126             branches.append(np.column_stack((cal, (y[:,c]-mean[c])/scale[c],
127                                                  (w[:,c]-mean[c])/scale[c], stats, (base[:,j]-mean[c])/scale[c])))
128         features.append(np.stack(branches, axis=1)); bases.append(base)
129     return (np.asarray(features, "float32"), np.asarray(bases, "float32"), mean, scale)
130
131 def labels(self, origins):
132     ids = np.asarray(origins[:, None] + np.arange(144))
133     assert ids.max() < len(self.actual)
134     return self.actual[ids][..., CHANNELS].astype("float32")
135
136 def month_origins(month):
137     start = (pd.Timestamp(2025, month, 1) - EPOCH).days * 144
138     end = (pd.Timestamp(2025, month, 1) + pd.offsets.MonthBegin(1) - EPOCH).days * 144
139     return np.arange(start, end, 36, dtype=int)
140
141
142 def split_origins(month):
143     asof = int(month_origins(month)[0]); cutoff = asof - 7 * 144
144     train = np.arange(7*144, cutoff-144+1, 36, dtype=int)
145     validation = np.arange(cutoff, asof-144+1, 36, dtype=int)
146     return train, validation, cutoff

```

experiments/common/neural_v2/physics.py

```

1  """One physical model, exact causal execution, and independent settlement."""
2  import time
3  import warnings
4
5  import numpy as np
6  from scipy.optimize import Bounds, LinearConstraint, milp
7  from scipy.sparse import coo_matrix
8
9  ETA = np.sqrt(0.9)
10 MIN_SOC, MAX_SOC, POWER_ENERGY = 1200.0, 10800.0, 5000/6
11
12
13 class LinearModel:
14     def __init__(self):
15         self.lower=[];self.upper=[];self.integer=[];self.objective=[]
16         self.row=[];self.col=[];self.value=[];self.lb=[];self.ub=[]
17
18     def variables(self, shape, lower=0, upper=np.inf, integer=False, cost=0):
19         n=int(np.prod(shape)); start=len(self.lower)
20         self.lower.extend(np.broadcast_to(lower,shape).ravel())
21         self.upper.extend(np.broadcast_to(upper,shape).ravel())
22         self.integer.extend([int(integer)]*n)
23         self.objective.extend(np.broadcast_to(cost,shape).ravel())
24         return np.arange(start,start+n).reshape(shape)
25
26     def constraint(self, terms, lower=-np.inf, upper=np.inf):
27         row=len(self.lb)
28         for ids,coefficients in terms:
29             ids=np.asarray(ids).ravel(); coefficients=np.broadcast_to(coefficients,ids.shape)
30             self.row.extend([row]*len(ids));self.col.extend(ids);self.value.extend(coefficients)
31             self.lb.append(lower);self.ub.append(upper)
32
33     def solve(self, seconds=2, gap=.01, fixed=None):
34         a=coo_matrix((self.value,(self.row,self.col)),shape=(len(self.lb),len(self.lower))).tocsc()
35         a.indices=a.indices.astype(np.int32);a.indptr=a.indptr.astype(np.int32)
36         constraints=LinearConstraint(a,np.asarray(self.lb),np.asarray(self.ub))
37         began=time.monotonic()
38         lower=np.asarray(self.lower).copy();upper=np.asarray(self.upper).copy()
39         integrality=np.asarray(self.integer)
40         if fixed is not None:
41             ids,values=fixed;lower[ids]=values;upper[ids]=values
42             integrality=np.zeros_like(integrality)
43         with warnings.catch_warnings():
44             warnings.filterwarnings("ignore", message="Unrecognized options detected.*", category=
RuntimeWarning)
45             result=milp(np.asarray(self.objective),integrality=integrality,
46                         bounds=Bounds(lower,upper),constraints=constraints,
47                         options={"time_limit":seconds,"mip_rel_gap":gap,"threads":1})
48         valid=False; residual=None
49         if result.x is not None and np.isfinite(result.x).all():
50             x=result.x;ax=a@x
51             residual=max(float(np.max(np.maximum(np.asarray(self.lower)-x,0))),
52                          float(np.max(np.maximum(x-np.asarray(self.upper),0))),
53                          float(np.max(np.maximum(np.asarray(self.lb)-ax,0))),
54                          float(np.max(np.maximum(ax-np.asarray(self.ub),0))))
55             ints=np.asarray(self.integer,dtype=bool)
56             int_error=float(np.max(np.abs(x[ints]-np.round(x[ints])))) if ints.any() else 0
57             valid=residual<1e-5 and int_error<1e-5
58             metadata={"status":int(result.status),"message":str(result.message),
59                      "seconds":time.monotonic()-began,"feasible":valid,
60                      "mip_gap":float(result.mip_gap) if getattr(result,"mip_gap",None) is not None and
np.isfinite(result.mip_gap) else None,
61                      "node_count":int(getattr(result,"mip_node_count",0) or 0),
62                      "constraint_residual":residual,"variables":len(self.lower),
63                      "binaries":int(sum(self.integer)),"constraints":len(self.lb)}
64             bound=getattr(result,"mip_dual_bound",None)
65             metadata["dual_bound"]=float(bound) if bound is not None and np.isfinite(bound) else None
66             return result.x if valid else None, metadata
67

```

```

68
69 def deterministic(load,pv,price,soc,original=None):
70     n=len(load); net=(np.asarray(load)-pv)/6; price=np.maximum(price,1e-6)
71     m=LinearModel()
72     g=m.variables((n,),upper=np.maximum(net,0)+POWER_ENERGY,cost=price if original is None else 0)
73     c=m.variables((n,),upper=POWER_ENERGY,cost=1e-8)
74     d=m.variables((n,),upper=POWER_ENERGY,cost=1e-8)
75     w=m.variables((n,))
76     s=m.variables((n,),lower=MIN_SOC,upper=MAX_SOC)
77     z=m.variables((n,),upper=1,integer=True)
78     if original is not None:
79         up=m.variables((n,),cost=1.5*price); down=m.variables((n,),cost=.5*price)
80         for t in range(n):m.constraint([(g[t],1),(up[t],-1),(down[t],1)],original[t],original[t])
81     for t in range(n):
82         m.constraint([(g[t],1),(c[t],-1),(d[t],1),(w[t],-1)],net[t],net[t])
83         terms=[(s[t],1),(c[t],-ETA),(d[t],1/ETA)]
84         if t:terms.append((s[t-1],-1))
85         rhs=soc if t==0 else 0
86         m.constraint(terms,rhs,rhs)
87         m.constraint([(c[t],1),(z[t],-POWER_ENERGY)],upper=0)
88         m.constraint([(d[t],1),(z[t],POWER_ENERGY)],upper=POWER_ENERGY)
89     x,meta=m.solve(seconds=2)
90     meta["method"]="deterministic_milp";meta["fallback"]=x is None
91     return (np.maximum(x[g],0) if x is not None else np.maximum(net,0)),meta
92
93
94 def execute(grid,load,pv,initial):
95     n=len(grid);c,d,e,w=(np.zeros(n) for _ in range(4))
96     s=np.empty(n+1);s[0]=initial
97     for t in range(n):
98         balance=grid[t]+(pv[t]-load[t])/6
99         if balance>=0:
100             c[t]=min(balance,POWER_ENERGY,max(0,(MAX_SOC-s[t])/ETA));w[t]=balance-c[t]
101         else:
102             d[t]=min(-balance,POWER_ENERGY,max(0,(s[t]-MIN_SOC)*ETA));e[t]=-balance-d[t]
103             s[t+1]=s[t]+ETA*c[t]-d[t]/ETA
104     return c,d,e,w,s
105
106
107 def settle(original,final,emergency,price):
108     return np.stack((original*price,np.maximum(final-original,0)*1.5*price,
109                     np.maximum(original-final,0)*.5*price,emergency*price*5),axis=-1)
110
111
112 def validate_detail(detail):
113     d=detail
114     for k in ("original","final","charge","discharge","emergency","surplus"):
115         assert np.isfinite(d[k]).all() and d[k].min()>=-1e-7
116     balance=d["final"]+(d["actual"][:,1]-d["actual"][:,0])/6+d["discharge"]+d["emergency"]-d["charge"]-d["surplus"]
117     transition=np.diff(d["states"])-ETA*d["charge"]+d["discharge"]/ETA
118     assert max(np.abs(balance).max(),np.abs(transition).max())<1e-6
119     assert d["states"].min()>=MIN_SOC-1e-6 and d["states"].max()<=MAX_SOC+1e-6
120     assert max(d["charge"].max(),d["discharge"].max())<=POWER_ENERGY+1e-6
121     assert not ((d["charge"]>1e-6)&(d["discharge"]>1e-6)).any()
122     np.testing.assert_allclose(d["fees"],settle(d["original"],d["final"],d["emergency"],d["price"]),
123                                atol=1e-7)
124     return float(np.abs(balance).max()),float(np.abs(transition).max())

```

experiments/exp008/absolute_hgb_forecast_adapter.py

```

1 """Issue-time adapter for the fixed direct HGB pipeline; no fitting or search."""
2 import json
3
4 from experiments.exp008.forecast import Forecasts
5 from experiments.exp008.forecast_absolute_hgb import AbsoluteHGBStore, OUT, PRIMARY
6 from experiments.exp008.forecast_override_adapter import AuditedForecastOverride

```

```

7
8
9 class AbsoluteHGBForecasts(Forecasts):
10     def __init__(self, data=None):
11         super().__init__(data=data, seed=42)
12         self.store = AbsoluteHGBStore()
13         self.calibration = PRIMARY
14         self.training_audit = {row['month']: row for row in json.loads((OUT / 'training_audit.json')
15             .read_text())}
16         self.day_audit = {row['day']: row for row in json.loads((OUT / 'prediction_audit.json').
17             read_text())}
18         self.memory_audit = json.loads((OUT / 'memory_half_audit.json').read_text())
19
20     def get(self, day, slot=0, scenario='2'):
21         result = super().get(day, slot, scenario)
22         result['audit'].update(base_forecast='direct_absolute_load_pv_HGB' if day >= 31 else '
23         periodic_cold_start',
24             output_calibration='same_fixed_Ridge28_then_load_energy_memory_half' if day >= 31 else
25             None,
26             selected_model_id=PRIMARY, base_architecture_unchanged=day < 31)
27         if day >= 31:
28             result['audit']['direct_hgb_prediction'] = self.day_audit[day]
29             result['audit']['load_energy_memory'] = self.memory_audit[day - 31]
30         return result
31
32     def net_error_paths(self, day, scenario='2', limit=28):
33         result = super().net_error_paths(day, scenario, limit)
34         result['audit'].update(source='
35         same_direct_HGB_Ridge28_memory_with_labelled_January_periodic_cold_start',
36             model_id=PRIMARY)
37         return result
38
39     def audited_forecasts():
40         return AuditedForecastOverride(AbsoluteHGBForecasts(), PRIMARY,
41             [OUT / 'provenance.json', OUT / 'protocol.json', OUT / f'{PRIMARY}.npz',
42             OUT / 'training_audit.json', OUT / 'ridge28_audit.json', OUT / 'memory_half_audit.json'])

```

experiments/exp008/audited_store_bridge.py

```

1 """One signed-store bridge to the unchanged tree28/q.8/buffer500 controller."""
2 import copy
3 import hashlib
4 import json
5 from pathlib import Path
6
7 from experiments.exp008.forecast_absolute_hgb import OUT as BASE, array_hash
8 from experiments.exp008.risk_window import implementation_check, replay
9 from experiments.exp008.verify import verify_npz
10 from experiments.problem2.exp003.data import Data
11
12
13 def save(path,value):
14     Path(path).parent.mkdir(parents=True,exist_ok=True)
15     Path(path).write_text(json.dumps(value,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
16
17
18 def run(store,out,evidence):
19     out=Path(out)
20     if out.exists():
21         raise FileExistsError('Existing bridge evidence is immutable')
22     out.mkdir(parents=True)
23     paths=[Path(p).resolve() for p in evidence]
24     hashes={str(p):hashlib.sha256(p.read_bytes()).hexdigest() for p in paths}
25     save(out/'protocol.json',{ 'model_id':store.name,'days':334,
26         'conditioning':'tree','history_days':28,'quantile':.8,'state_buffer':500.,
27         'values_sha256':array_hash(store.values),'origins_sha256':array_hash(store.origins),

```

```

28     'model_evidence_sha256':hashes, 'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).
    hexdigest(),
29     'one_fixed_comparison':True, 'development_not_independent_test':True, 'final_model_selection':
    False})
30 (out/'source_snapshot.py').write_bytes(Path(__file__).read_bytes())
31 data=Data()
32 save(out/'risk_implementation_verification.json',implementation_check(data,store))
33 case=store.name+'_tree28_q08_buffer500'
34 replay(case,28,'tree',334,data,store,out)
35 directory=out/f'{case}_334days'
36 audit=json.loads((directory/'audit.json').read_text());corrected=copy.deepcopy(audit)
37 for row in corrected:
38     row['forecast_calibration']=store.name
39     row['residual_source']='periodic_baseline' if row['fallback'] else store.name+'
    _same_prior_issued_predictions'
40     row['prediction_values_sha256']=array_hash(store.values)
41 save(directory/'source_corrected_audit.json',corrected)
42 save(directory/'audit_metadata_correction.json',{'original_preserved':'audit.json',
43     'corrected_view':'source_corrected_audit.json','numerical_arrays_changed':False,
44     'reason':'shared numerical bridge has fixed legacy ridge28 textual labels; source identity
    follows actual constructor inputs'})
45 new=verify_npz(directory/'dispatch_2.npz',audit_path=directory/'source_corrected_audit.json')
46 old=verify_npz(BASE/'lp_bridge/direct_hgb_ridge28_memory_tree28_q08_buffer500_334days/dispatch_2
    .npz',
47     audit_path=BASE/'lp_bridge/direct_hgb_ridge28_memory_tree28_q08_buffer500_334days
    /audit.json')
48 assert new['passed'] and old['passed']
49 assert all(hashlib.sha256(p.read_bytes()).hexdigest()==hashes[str(p)] for p in paths)
50 result={'complete':True, 'baseline':old, 'candidate':new,
51     'cost_change_yuan':new['recomputed_total_cost']-old['recomputed_total_cost'],
52     'reversal_change':new['battery_metrics']['direction_reversals']-old['battery_metrics']['
    direction_reversals'],
53     'terminal_soc_difference_kwh':new['battery_metrics']['final_soc']-old['battery_metrics']['
    final_soc'],
54     'all_evidence_unchanged':True, 'final_model_selection':False}
55 save(out/'paired_findings.json',result)
56 print(json.dumps({k:v for k,v in result.items() if k not in ('candidate','baseline')},indent=2),
    flush=True)
57 return result

```

experiments/exp008/battery_feedback.py

```

1  """Fixed-purchase stochastic inventory value feedback, with physical projection.
2
3  Backward values use available per-slot distributions. This assumes independent
4  marginal disturbances and is not a perfect-information recourse calculation.
5  Actual actions inspect only the current observation and precomputed values.
6  """
7  from time import perf_counter
8  import numpy as np
9
10 ETA=np.sqrt(.9)
11 LOW,HIGH,LIMIT=1200.,10800.,5000/6
12
13
14 def value_functions(purchase, support, price, *, grid_kwh=100., wear=.002,
15     charge_mask=None, terminal=.45, charge_deadband=0.):
16     began=perf_counter()
17     purchase, support, price=map(np.asarray,(purchase,support,price))
18     grid=np.r_[np.arange(LOW,HIGH,grid_kwh),HIGH]
19     n=len(price)
20     values=np.zeros((n+1,len(grid)))
21     values[-1]=-terminal*(grid-LOW)
22     delta=grid[None,:]-grid[:,None]
23     for t in range(n-1,-1,-1):
24         expected=np.zeros(len(grid))
25         future=values[t+1]

```

```

26     for demand in support[t]:
27         b=purchase[t]-demand
28         positive=b>=0
29         permitted=charge_mask is None or bool(charge_mask[t])==positive
30         if not permitted:
31             expected+=future+5*price[t]*max(-b,0)
32             continue
33         if positive:
34             cmax=np.minimum(np.minimum(b,LIMIT),(HIGH-grid)/ETA)
35             cmax=np.where(cmax>=charge_deadband,cmax,0)
36             endpoint=grid+ETA*cmax
37             eligible=(delta>=-1e-8)&(delta<=ETA*cmax[:,None]+1e-8)
38             if charge_deadband:
39                 eligible&=(delta<1e-8)|(delta>=ETA*charge_deadband-1e-8)
40             costs=future[None,:]+wear*delta/ETA
41             end_value=np.interp(endpoint,grid,future)+wear*cmax
42             if charge_deadband:
43                 minimum=grid+ETA*charge_deadband
44                 minimum_value=np.interp(minimum,grid,future)+wear*charge_deadband
45                 end_value=np.minimum(end_value,np.where(cmax>=charge_deadband,minimum_value,np.
inf))
46         else:
47             dmax=np.minimum(np.minimum(-b,LIMIT),(grid-LOW)*ETA)
48             endpoint=grid-dmax/ETA
49             eligible=(delta<=1e-8)&(delta>=-dmax[:,None]/ETA-1e-8)
50             costs=future[None,:]+5*price[t]*(-b+ETA*delta)-wear*ETA*delta
51             end_value=np.interp(endpoint,grid,future)+5*price[t]*(-b-dmax)+wear*dmax
52             grid_value=np.min(np.where(eligible, costs, np.inf), axis=1)
53             expected+=np.minimum(grid_value, end_value)
54         values[t]=expected/support.shape[1]
55     return grid, values, {'method': 'fixed_purchase_marginal_value_dp',
56                          'seconds': perf_counter()-began, 'grid_kwh': grid_kwh,
57                          'disturbance_assumption': 'independent_marginals',
58                          'terminal_value': terminal, 'wear': wear}
59
60
61 def execute(purchase, actual, price, initial_soc, grid, values, *, wear=.002,
62            charge_mask=None, charge_deadband=0.):
63     n=len(purchase)
64     c,d,e,w=(np.zeros(n) for _ in range(4))
65     states=np.empty(n+1); states[0]=initial_soc
66     for t in range(n):
67         b=purchase[t]+(actual[t,1]-actual[t,0])/6
68         positive=b>=0
69         allowed=charge_mask is None or bool(charge_mask[t])==positive
70         current=states[t]
71         if not allowed:
72             following=current
73         elif positive:
74             full=min(b,LIMIT,max(0,(HIGH-current)/ETA))
75             if full<charge_deadband:
76                 full=0.
77             endpoint=current+ETA*full
78             choices=np.r_[current,grid[(grid>=current)&(grid<=endpoint)],endpoint]
79             if charge_deadband:
80                 if full>=charge_deadband:
81                     choices=np.r_[choices,current+ETA*charge_deadband]
82                     choices=choices[(choices-current<1e-8)|(choices-current>=ETA*charge_deadband-1e-8)]
83             costs=np.interp(choices,grid,values[t+1])+wear*(choices-current)/ETA
84             following=choices[np.argmin(costs)]
85         else:
86             full=min(-b,LIMIT,max(0,(current-LOW)*ETA))
87             endpoint=current-full/ETA
88             choices=np.r_[current,grid[(grid>=endpoint)&(grid<=current)],endpoint]
89             release=(current-choices)*ETA
90             costs=np.interp(choices,grid,values[t+1])+5*price[t]*(-b-release)+wear*release
91             following=choices[np.argmin(costs)]
92         c[t]=max(0,(following-current)/ETA)
93         d[t]=max(0,(current-following)*ETA)
94         w[t]=max(0,b-c[t]); e[t]=max(0,-b-d[t])

```

```

95     states[t+1]=following
96     fees=np.stack([purchase*price,np.zeros(n),np.zeros(n),5*e*price],axis=-1)
97     return dict(original=purchase.copy(),final=purchase.copy(),charge=c,discharge=d,
98                 emergency=e,surplus=w,states=states,fees=fees,actual=actual.copy(),price=price.copy
99                 ())

```

experiments/exp008/closed_loop.py

```

1  """Optimize fixed purchases against the same causal inventory execution law.
2
3  Analytic reverse differentiation through clipped storage dynamics. Historical
4  paths are scenarios available at midnight, never the current day's actuals.
5  The objective is nonsmooth and nonconvex; L-BFGS-B is a local numerical method.
6  """
7  from time import perf_counter
8  import numpy as np
9  from scipy.optimize import minimize
10
11  ETA=np.sqrt(.9)
12  LOW,HIGH,LIMIT=1200.,10800.,5000/6
13
14
15  def objective(purchase, net_paths, prices, initial_soc, *, charge_mask=None,
16               throughput=.005, variation=.001, terminal=.45, deadband=0.,
17               emergency_weight=5.):
18      paths=np.asarray(net_paths)
19      k,n=paths.shape
20      soc=np.full(k,initial_soc)
21      sa=np.empty((k,n)); sb=np.empty((k,n))
22      local_s=np.zeros((k,n)); local_b=np.zeros((k,n))
23      action=np.zeros((k,n)); action_s=np.zeros((k,n));action_b=np.zeros((k,n))
24      cost=float(np.dot(purchase,prices))
25      for t in range(n):
26          b=purchase[t]-paths[:,t]
27          positive=b>=0
28          can_c=True if charge_mask is None else bool(charge_mask[t])
29          can_d=True if charge_mask is None else not bool(charge_mask[t])
30          available_c=np.maximum(0,(HIGH-soc)/ETA)
31          available_d=np.maximum(0,(soc-LOW)*ETA)
32          c=np.where(positive&can_c,np.minimum(np.minimum(b,LIMIT),available_c),0.)
33          d=np.where((~positive)&can_d,np.minimum(np.minimum(-b,LIMIT),available_d),0.)
34          c=np.where(c>=deadband,c,0.)
35          cb=(positive&can_c&(b<LIMIT)&(b<available_c)&(c>=deadband)).astype(float)
36          cs=np.where(positive&can_c&(available_c<=b)&(available_c<LIMIT)&(c>=deadband),-1/ETA,0.)
37          db=-((~positive)&can_d&(-b<LIMIT)&(-b<available_d)).astype(float)
38          ds=np.where((~positive)&can_d&(available_d<=-b)&(available_d<LIMIT),ETA,0.)
39          e=np.maximum(0,-b-d)
40          # e == 0 kink uses the no-emergency side; exact crossings remain
41          # nonsmooth and no global convergence certificate is claimed.
42          active=e>1e-9
43          local_b[:,t]=emergency_weight*prices[t]*np.where(active,-1-db,0)+throughput*(cb+db)
44          local_s[:,t]=emergency_weight*prices[t]*np.where(active,-ds,0)+throughput*(cs+ds)
45          sa[:,t]=1+ETA*cs-ds/ETA
46          sb[:,t]=ETA*cb-db/ETA
47          action[:,t]=c-d
48          action_b[:,t]=cb-db
49          action_s[:,t]=cs-ds
50          soc=soc+ETA*c-d/ETA
51          cost+=float(np.mean(emergency_weight*prices[t]*e+throughput*(c+d)))
52      cost=float(terminal*np.mean(soc-LOW))
53      diff=np.diff(action,axis=1)
54      cost+=float(variation*6*np.mean(np.abs(diff).sum(1)))
55      tv_grad=np.zeros_like(action)
56      tv_grad[:,1:]=variation*6*np.sign(diff)
57      tv_grad[:,:-1]-=variation*6*np.sign(diff)
58      local_b+=tv_grad*action_b
59      local_s+=tv_grad*action_s

```

```

60     adj=np.full(k,-terminal)
61     grad=np.asarray(prices).copy()
62     for t in range(n-1,-1,-1):
63         grad[t]+=np.mean(local_b[:,t]+adj*sb[:,t])
64         adj=local_s[:,t]+adj*sa[:,t]
65     return cost,grad
66
67
68 def optimize(initial_purchase, net_paths, prices, initial_soc, *, charge_mask=None,
69             throughput=.005, variation=.001, terminal=.45, deadband=0., maxiter=100,
70             emergency_weight=5., original=None):
71     began=perf_counter()
72     prices=np.asarray(prices)
73     floor=np.zeros(len(prices)) if original is None else np.asarray(original)
74     buy_price=prices if original is None else 1.5*prices
75     # Pricing of emergency and purchased energy can differ under adjustment.
76     # Q2 optimization uses prices for both; adjustment is handled by the joint
77     # planner until its own billing-aware closed-loop derivative is implemented.
78     if original is not None:
79         raise NotImplementedError('Closed-loop adjustment objective is not implemented')
80     def fun(g):
81         return objective(g, net_paths, buy_price, initial_soc, charge_mask=charge_mask,
82                         throughput=throughput, variation=variation, terminal=terminal,
83                         deadband=deadband, emergency_weight=emergency_weight)
84     result=minimize(fun, np.maximum(initial_purchase, floor), jac=True, method='L-BFGS-B',
85                    bounds=list(zip(floor, np.full(len(prices), np.inf))),
86                    options={'maxiter':maxiter, 'maxls':30, 'ftol':1e-9, 'gtol':1e-5})
87     return {'purchase':result.x, 'metadata':{'method':'closed_loop_lbfgsb',
88     'success':bool(result.success), 'message':str(result.message),
89     'iterations':int(result.nit), 'evaluations':int(result.nfev),
90     'objective':float(result.fun), 'planning_seconds':perf_counter()-began,
91     'global_optimality_certificate':False}}

```

experiments/exp008/controller_candidate.py

```

1  """Exploratory causal Q2 controllers; fixed forecasts and same exp006 physics.
2
3  This is development on the reported year, not an untouched validation set.
4  No actual value from the execution day is available to "plan_inventory".
5  """
6  from __future__ import annotations
7
8  import argparse
9  import hashlib
10 import json
11 import time
12 from pathlib import Path
13
14 import numpy as np
15 import pandas as pd
16 from scipy.optimize import linprog
17 from scipy.sparse import lil_matrix, coo_matrix
18 from scipy.stats import rankdata
19
20 from experiments.problem2.exp003.data import Data
21 from experiments.problem2.exp004.predict import ForecastStore
22 from experiments.problem2.tree_planning.model import ETA, MIN_SOC, MAX_SOC, POWER_ENERGY
23 from experiments.problem2.tree_planning.risk import TreeResidualScenarios
24 from experiments.problem2.tree_planning.verify import battery_metrics
25
26 OUT = Path(__file__).resolve().parents[2] / 'data/results/exp008/controller_candidates'
27 INITIAL_SOC = 1421.7991105135516
28 BASELINE = 14066257.477300335
29 CHEAPEST = 13534196.0
30
31
32 def charging_mask(spec, net, price):

```

```

33     t = np.arange(len(price))
34     start, stop = spec.get('solar_start', 54), spec.get('solar_stop', 96)
35     return (t < 34) | ((t >= start) & (t < stop)) | (t >= 131)
36
37
38 def plan_inventory(support, price, soc, spec, final=False, error_paths=None, base_forecast=None):
39     """Continuous inventory LP with separate AC flows and physical efficiency."""
40     if spec.get('planner') == 'joint':
41         from experiments.exp008.planner import plan, Settings
42         keys = Settings.__dataclass_fields__
43         settings = Settings(**{k:v for k,v in spec.items() if k in keys})
44         if spec.get('path_type') == 'rank' and len(error_paths) > 1:
45             ranks = (rankdata(error_paths, axis=0)-.5)/len(error_paths)
46             quantile_levels = (np.arange(9)+.5)/9
47             net = np.median(support,axis=1)
48             demands = np.stack([np.interp(ranks[:,t],quantile_levels,support[t]) for t in range(144)
49 ],axis=1)
49             error_paths = demands-net
50             forecast = {'load_kw':np.maximum(net,0)*6, 'pv_kw':np.maximum(-net,0)*6, 'price':price}
51         else:
52             forecast = {'load_kw':base_forecast[:,0], 'pv_kw':base_forecast[:,1], 'price':price}
53             net = (base_forecast[:,0]-base_forecast[:,1])/6
54             allowed_charge = charging_mask(spec,net,price)
55             if spec.get('controller') == 'adaptive_blocks':
56                 reference = plan_inventory(support,price,soc,{ 'quantile':spec.get('quantile',.8), 'wear':
spec.get('mode_wear',.02)},final)
57                 direction = np.sign(reference['charge']-reference['discharge'])
58                 last = 1
59                 for t in range(len(direction)):
60                     last = direction[t] or last
61                     direction[t] = last
62                 hold = spec.get('mode_min_slots',3)
63                 starts = np.r_[0,np.flatnonzero(np.diff(direction))+1,len(direction)]
64                 for a,b in zip(starts[:-1],starts[1:]):
65                     if b-a < hold and a:
66                         direction[a:b] = direction[a-1]
67                 allowed_charge = direction > 0
68                 mask = allowed_charge if spec.get('controller') in ('fixed_blocks','adaptive_blocks') else
None
69                 result = plan(forecast,soc,error_paths=error_paths,settings=settings,final_day=final,
charge_mask=mask)
70                 result.update(net=net,allowed_charge=allowed_charge)
71                 return result
72     n = len(price)
73     # Variable blocks: q, charge, discharge, spill, next SOC.
74     obj = np.r_[price, np.full(n, spec.get('wear', .002)),
75               np.full(n, spec.get('wear', .002)), np.full(n, 1e-7), np.zeros(n)]
76     obj[-1] = 0 if final else -float(price.min()) / ETA
77     eq = lil_matrix((2*n, 5*n))
78     quantile = spec.get('quantile', .7)
79     net = np.quantile(support, quantile, axis=1)
80     rhs = np.r_[net, np.zeros(n)]
81     rhs[n] = soc
82     bounds = [(0, None)] * n + [(0, POWER_ENERGY)] * (2*n) + [(0, None)] * n + [(MIN_SOC, MAX_SOC)]
* n
83     # Pool forecast uncertainty in stored energy and empty capacity instead of
84     # purchasing a separate safety quantile at every expensive ten-minute slot.
85     reserve=spec.get('soc_reserve',0.)
86     headroom=spec.get('soc_headroom',0.)
87     for t in range(n):
88         bottom=min(MIN_SOC+reserve,soc+ETA*POWER_ENERGY*(t+1))
89         top=max(MAX_SOC-headroom,soc-POWER_ENERGY*(t+1)/ETA)
90         if final:
91             bottom=max(MIN_SOC,bottom-reserve*max(0,(t-120)/23))
92         if bottom>top:
93             raise ValueError('Reserve and headroom leave no feasible inventory interval')
94         bounds[4*n+t]=(bottom,top)
95     allowed_charge = charging_mask(spec, net, price)
96     if spec.get('controller') == 'fixed_blocks':
97         for t in range(n):

```

```

98         bounds[(2 if allowed_charge[t] else 1)*n+t] = (0, 0)
99     for t in range(n):
100         eq[t, t] = 1
101         eq[t, n+t] = -1
102         eq[t, 2*n+t] = 1
103         eq[t, 3*n+t] = -1
104         eq[n+t, 4*n+t] = 1
105         eq[n+t, n+t] = -ETA
106         eq[n+t, 2*n+t] = 1/ETA
107         if t:
108             eq[n+t, 4*n+t-1] = -1
109     result = linprog(obj, A_eq=eq.tocsr(), b_eq=rhs, bounds=bounds, method='highs')
110     if not result.success:
111         raise RuntimeError(result.message)
112     x = result.x
113     return {'purchase': x[:n], 'charge': x[n:2*n], 'discharge': x[2*n:3*n],
114            'states': np.r_[soc, x[4*n:]], 'net': net, 'allowed_charge': allowed_charge}
115
116
117 def execute_inventory(plan, actual, price, soc, mode, spec):
118     n = len(price)
119     q = plan['purchase']
120     c, d, e, w = (np.zeros(n) for _ in range(4))
121     states = np.r_[soc, np.zeros(n)]
122     min_charge = spec.get('charge_deadband', 0.)
123     min_discharge = spec.get('discharge_deadband', 0.)
124     reserve = spec.get('reserve', 0.)
125     for t in range(n):
126         # Current observation only. Whole actual array is merely replay input.
127         balance = q[t] + (actual[t, 1] - actual[t, 0])/6
128         if balance >= 0:
129             request = min(balance, POWER_ENERGY, (MAX_SOC-states[t])/ETA)
130             if spec.get('controller') == 'planned_direction' and plan['charge'][t] < 1e-5:
131                 request = 0
132             if spec.get('controller') == 'macro' and not (t < 42 or 60 <= t < 96 or t >= 138):
133                 request = 0
134             if spec.get('controller') in ('fixed_blocks', 'adaptive_blocks') and not plan['
135 allowed_charge'][t]:
136                 request = 0
137             threshold = min_charge if mode == -1 else spec.get('holding_deadband', 0.)
138             c[t] = request if request >= threshold else 0
139             w[t] = balance-c[t]
140         else:
141             floor = MIN_SOC
142             if reserve and price[t] < np.max(price[t:]):
143                 floor = min(MAX_SOC, MIN_SOC + reserve)
144             if 'state_buffer' in spec:
145                 floor = max(MIN_SOC, plan['states'][t+1]-spec['state_buffer'])
146             request = min(-balance, POWER_ENERGY, max(0, (states[t]-floor)*ETA))
147             if spec.get('controller') in ('fixed_blocks', 'adaptive_blocks') and plan['allowed_charge
148 '][t]:
149                 request = 0
150             d[t] = request if request >= min_discharge else 0
151             e[t] = -balance-d[t]
152             states[t+1] = states[t] + ETA*c[t] - d[t]/ETA
153             if c[t] or d[t]:
154                 mode = 1 if c[t] else -1
155     fees = np.stack([q*price, np.zeros(n), np.zeros(n), 5*e*price], axis=-1)
156     detail = dict(original=q, final=q.copy(), charge=c, discharge=d, emergency=e,
157                  surplus=w, states=states, fees=fees, actual=actual.copy(), price=price.copy(),
158                  intended_charge=plan['charge'], intended_discharge=plan['discharge'],
159                  intended_states=plan['states'])
160     return detail, mode
161
162 def risk_cache(data, store):
163     OUT.mkdir(parents=True, exist_ok=True)
164     path = OUT/'risk_inputs.npz'
165     if path.exists():
166         with np.load(path) as z:

```

```

166         return z['supports'].copy(), json.loads((OUT/'risk_audit.json').read_text())
167 risk = TreeResidualScenarios(store.origins, store.values, data.actual, data.fixed_price)
168 support, audit = [], []
169 for day in range(31, 365):
170     values, info = risk.for_day(day)
171     support.append(values)
172     audit.append(info)
173 np.savez_compressed(path, supports=np.stack(support))
174 (OUT/'risk_audit.json').write_text(json.dumps(audit, ensure_ascii=False, indent=2))
175 return np.stack(support), audit
176
177
178 def perfect_information_bound(data):
179     """Full-horizon perfect-information relaxation; never an eligible controller."""
180     started = time.perf_counter()
181     actual = data.actual[31*144:]
182     net = (actual[:,0]-actual[:,1])/6
183     n = len(net)
184     p = np.tile(data.fixed_price,334)
185     t = np.arange(n)
186     rows = np.r_[t,t,t,t,n+t,n+t,n+t,n+t[1:]]
187     cols = np.r_[t,n+t,2*n+t,3*n+t,4*n+t,n+t,2*n+t,4*n+t[:-1]]
188     vals = np.r_[np.ones(n),-np.ones(n),np.ones(n),-np.ones(n),np.ones(n),
189                 np.full(n,-ETA),np.full(n,1/ETA),-np.ones(n-1)]
190     mat = coo_matrix((vals,(rows,cols)),shape=(2*n,5*n)).tocsr()
191     rhs = np.r_[net,np.zeros(n)]
192     rhs[n] = INITIAL_SOC
193     bounds = [(0,None)]*n+[(0,POWER_ENERGY)]*(2*n)+[(0,None)]*n+[(MIN_SOC,MAX_SOC)]*n
194     result = linprog(np.r_[p,np.zeros(4*n)],A_eq=mat,b_eq=rhs,bounds=bounds,method='highs')
195     if not result.success:
196         raise RuntimeError(result.message)
197     summary = {'role':'ineligible perfect-information lower bound','total_cost':float(result.fun),
198              'days':334,'initial_soc':INITIAL_SOC,'final_soc':float(result.x[-1]),
199              'max_savings_vs_exp006_pct':100*(1-result.fun/BASELINE),
200              'elapsed_seconds':time.perf_counter()-started,'uses_future_actuals':True}
201     OUT.mkdir(parents=True,exist_ok=True)
202     (OUT/'perfect_information_bound.json').write_text(json.dumps(summary,indent=2))
203     print(json.dumps(summary),flush=True)
204     return summary
205
206
207 def evaluate(spec, data=None, supports=None, audits=None):
208     data = Data() if data is None else data
209     if supports is None:
210         supports, audits = risk_cache(data, ForecastStore('no_season', seed=42))
211     run_dir = OUT/spec['id']
212     if (run_dir/'summary.json').exists():
213         cached = json.loads((run_dir/'summary.json').read_text())
214         if cached['spec'] != spec:
215             raise ValueError(f"Refusing to reuse run id with changed settings: {spec['id']}")
216         return cached
217     start = time.perf_counter()
218     source_sha256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
219     soc, mode, details, daily, execution_audit = INITIAL_SOC, 1, [], [], []
220     store = ForecastStore('no_season',seed=42) if spec.get('planner') == 'joint' or spec.get('
221 optimize') else None
222     if spec.get('calibration'):
223         from experiments.exp008.forecast_calibration import CalibratedStore
224         store = CalibratedStore(spec['calibration'])
225     stop_day = 31+spec.get('development_days',334)
226     for i, day in enumerate(range(31,stop_day)):
227         errors, forecast = None, None
228         if store is not None:
229             ids = np.flatnonzero(store.origins+144 <= day*144)[-28:]
230             forecast = store.get(day*144)
231             if len(ids):
232                 observed = data.actual[store.origins[ids,None]+np.arange(144)]
233                 diff = (observed-store.values[ids])/6
234                 errors = diff[:,0]-diff[:,1]
235         else:

```

```

235         net = (forecast[:,0]-forecast[:,1])/6
236         errors = supports[i].T-net[None,:]
237     plan = plan_inventory(supports[i], data.fixed_price, soc, spec, final=day == 364,
238         error_paths=errors,base_forecast=forecast)
239     if spec.get('optimize'):
240         from experiments.exp008.closed_loop import optimize
241         net = (forecast[:,0]-forecast[:,1])/6
242         paths = net[None,:] + errors
243         if spec.get('path_type') == 'rank' and len(errors)>1:
244             ranks = (rankdata(errors,axis=0)-.5)/len(errors)
245             levels = (np.arange(9)+.5)/9
246             paths = np.stack([np.interp(ranks[:,t],levels,supports[i,t]) for t in range(144)],
axis=1)
247         max_paths = spec.get('scenarios',28)
248         if len(paths)>max_paths:
249             paths = paths[np.linspace(0,len(paths)-1,max_paths).astype(int)]
250         mask = plan['allowed_charge'] if spec.get('controller') in ('fixed_blocks',
adaptive_blocks') else None
251         result = optimize(plan['purchase'],paths,data.fixed_price,soc,charge_mask=mask,
252             throughput=spec.get('wear',.005),variation=spec.get('variation_penalty
',.001),
253             terminal=0 if day==364 else data.fixed_price.min()/ETA,
254             deadband=spec.get('charge_deadband',0),maxiter=spec.get('maxiter',100)
,
255             emergency_weight=spec.get('emergency_weight',5))
256         plan['purchase'] = result['purchase']
257         plan['optimizer_metadata'] = result['metadata']
258         old_mode = mode
259         if spec.get('impulse_feedback'):
260             from experiments.exp008.impulse_feedback import value_functions, execute as
impulse_execute
261             grid, values, value_meta = value_functions(plan['purchase'], supports[i], data.
fixed_price,
262                 grid_kwh=spec.get('feedback_grid',100.),wear=spec.get('wear',.002),
263                 switching=spec.get('switching',50.),terminal=0 if day==364 else data.fixed_price.min
()/ETA)
264             detail,mode = impulse_execute(plan['purchase'],data.actual[day*144:(day+1)*144],data.
fixed_price,
265                 soc,grid,values,wear=spec.get('wear',.002),switching=spec.get('switching',50.),
initial_mode=mode)
266             plan['optimizer_metadata'] = value_meta
267             elif spec.get('value_feedback'):
268                 from experiments.exp008.battery_feedback import value_functions, execute as
value_execute
269                 mask = plan['allowed_charge'] if spec.get('controller') in ('fixed_blocks',
adaptive_blocks') else None
270                 grid, values, value_meta = value_functions(plan['purchase'], supports[i], data.
fixed_price,
271                     grid_kwh=spec.get('feedback_grid',100.),wear=spec.get('wear',.002),charge_mask=mask,
272                     terminal=0 if day==364 else data.fixed_price.min()/ETA,
273                     charge_deadband=spec.get('charge_deadband',0.))
274                 detail = value_execute(plan['purchase'],data.actual[day*144:(day+1)*144],data.
fixed_price,
275                     soc,grid,values,wear=spec.get('wear',.002),charge_mask=mask,
276                     charge_deadband=spec.get('charge_deadband',0.))
277                 signs=np.sign(detail['charge']-detail['discharge'])
278                 if np.any(signs):
279                     mode=int(signs[signs!=0][-1])
280                 plan['optimizer_metadata'] = value_meta
281             else:
282                 detail, mode = execute_inventory(plan, data.actual[day*144:(day+1)*144], data.
fixed_price, soc, mode, spec)
283                 details.append(detail)
284                 row = {'day':day, 'date':str((pd.Timestamp('2025-01-01')+pd.Timedelta(days=day)).date()),
285                     'total_cost':float(detail['fees'].sum()), 'planned_cost':float(detail['fees'][:,0].
sum()),
286                     'emergency_cost':float(detail['fees'][:,3].sum()), 'initial_soc':soc,
287                     'final_soc':float(detail['states'][-1])}
288                 daily.append(row)
289                 execution_audit.append(**audits[i], 'forecast_origin':day*144,

```

```

290         'execution_initial_soc':soc,'execution_initial_mode':old_mode,
291         'execution_final_soc':row['final_soc'],'execution_final_mode':mode,
292         'purchase_locked_before_actual_read':True,
293         'optimizer_metadata':plan.get('optimizer_metadata'))
294     soc = row['final_soc']
295     arrays = {k:np.stack([d[k] for d in details]) for k in details[0]}
296     battery = battery_metrics(arrays, 1, 172.759999999999976)
297     summary = {'spec':spec,'days':stop_day-31,'total_cost':float(arrays['fees'].sum()),
298               'planned_cost':float(arrays['fees'][:,0].sum()),
299               'emergency_cost':float(arrays['fees'][:,3].sum()),
300               'emergency_kwh':float(arrays['emergency'].sum()),
301               'surplus_kwh':float(arrays['surplus'].sum()),**battery,
302               'elapsed_seconds':time.perf_counter()-start,'evaluation_role':'2025 development; not
untouched validation'}
303     summary['controller_source_sha256_at_start'] = source_sha256
304     baseline = BASELINE
305     if stop_day != 365:
306         baseline = float(pd.read_csv(OUT.parents[1]/'exp006/primary/daily.csv').iloc[:stop_day-31]['
total_cost'].sum())
307     summary['baseline_cost_for_evaluation_days'] = baseline
308     summary['target_eligible'] = stop_day == 365
309     summary['improvement_vs_exp006_pct'] = 100*(1-summary['total_cost']/baseline)
310     summary['improvement_vs_cheapest_pct'] = 100*(1-summary['total_cost']/CHEAPEST) if stop_day==365
else None
311     run_dir.mkdir(parents=True,exist_ok=True)
312     np.savez_compressed(run_dir/'dispatch_2.npz',**arrays)
313     pd.DataFrame(daily).to_csv(run_dir/'daily.csv',index=False)
314     (run_dir/'planning_audit.json').write_text(json.dumps({'days':execution_audit},ensure_ascii=
False,indent=2))
315     (run_dir/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2))
316     print(json.dumps(summary,ensure_ascii=False),flush=True)
317     return summary
318
319
320 def verify_completed():
321     from experiments.problem2.tree_planning.verify import verify_arrays
322     data = Data()
323     rows = []
324     for path in sorted(OUT.glob('*/summary.json')):
325         summary = json.loads(path.read_text())
326         if summary['days'] != 334:
327             continue
328         with np.load(path.parent/'dispatch_2.npz') as archive:
329             arrays = {k:archive[k].copy() for k in archive.files}
330             if summary['spec'].get('planner') == 'joint':
331                 # Scenario means are surrogate intentions, not a single physical
332                 # action; verify only actual physical dispatch for these runs.
333                 arrays = {k:v for k,v in arrays.items() if not k.startswith('intended_')}
334             daily = pd.read_csv(path.parent/'daily.csv').to_dict('records')
335             audits = json.loads((path.parent/'planning_audit.json').read_text())['days']
336             report = verify_arrays(arrays,daily,audits,source_actual=data.actual[31*144:].reshape
(334,144,2),
337                                   source_price=data.fixed_price)
338             report['planning_intentions_are_scenario_means'] = summary['spec'].get('planner') == 'joint'
339             (path.parent/'verification.json').write_text(json.dumps(report,indent=2))
340             rows.append({'id':summary['spec']['id'],'passed':report['passed'],'errors':report['errors']
})
341         (OUT/'verification_summary.json').write_text(json.dumps(rows,indent=2))
342     print(json.dumps({'runs':len(rows),'all_passed':all(r['passed'] for r in rows),'failed':[r for r
in rows if not r['passed']]}))
343
344
345 def main():
346     parser = argparse.ArgumentParser()
347     parser.add_argument('--batch', default='first')
348     args = parser.parse_args()
349     data = Data()
350     if args.batch == 'bound':
351         perfect_information_bound(data)
352     return

```

```

353     if args.batch == 'verify':
354         verify_completed()
355         return
356     supports, audits = risk_cache(data, ForecastStore('no_season', seed=42))
357     if args.batch == 'first':
358         specs = [{'id': f'lp_q{q:.2f}_greedy', 'quantile': q} for q in [.5, .6, .7, .8]]
359     elif args.batch == 'deadband':
360         specs = [{'id': f'lp_q{q:.2f}_db{db}', 'quantile': q, 'charge_deadband': db}
361                  for q in [.5, .6, .7] for db in [50, 150, 300]]
362     elif args.batch == 'macro':
363         specs = [{'id': f'lp_q{q:.2f}_{control}', 'quantile': q, 'controller': control}
364                  for q in [.5, .6, .7, .8] for control in ['planned_direction', 'macro']]
365     elif args.batch == 'blocks':
366         specs = [{'id': f'blocks_q{q:.2f}_solar{start}', 'quantile': q, 'controller': 'fixed_blocks', '
367                  solar_start': start}
368                  for q in [.6, .7, .8, .85] for start in [48, 54, 60, 64]]
369     elif args.batch == 'joint':
370         specs = [{'id': f'joint_{kind}_e{ew}', 'planner': 'joint', 'path_type': kind, 'emergency_weight'
371                  : ew, 'scenarios': 7}
372                  for kind in ['raw', 'rank'] for ew in [5, 7]]
373     elif args.batch == 'joint_blocks':
374         specs = [{'id': f'joint_{kind}_{control}_db{db}', 'planner': 'joint', 'path_type': kind,
375                  'emergency_weight': 5, 'scenarios': 7, 'controller': control, 'solar_start': 60,
376                  'charge_deadband': db, 'holding_deadband': db}
377                  for kind in ['rank'] for control in ['fixed_blocks', 'adaptive_blocks'] for db in
378                  [0, 100]]
379     elif args.batch == 'reserve':
380         specs = ([{'id': f'lp_q{q:.2f}_reserve{r}', 'quantile': q, 'reserve': r}
381                  for q in [.7, .75, .8, .85] for r in [500, 1000, 2000]] +
382                  [{'id': f'lp_q{q:.2f}_statebuffer{r}', 'quantile': q, 'state_buffer': r}
383                  for q in [.75, .8, .85] for r in [0, 500, 1000, 2000]])
384     elif args.batch == 'closed_pilot':
385         specs = [{'id': f'closed_pilot_{kind}_{control}', 'optimize': True, 'path_type': kind,
386                  'quantile': .8, 'controller': control, 'solar_start': 60, 'development_days': 30,
387                  'maxiter': 100, 'scenarios': 28}
388                  for kind in ['raw', 'rank'] for control in ['greedy', 'fixed_blocks']]
389     else:
390         specs = json.loads(Path(args.batch).read_text())
391     rows = [evaluate(s, data, supports, audits) for s in specs]
392     all_rows = [json.loads(p.read_text()) for p in OUT.glob('*/summary.json')]
393     pd.DataFrame([{'**r', 'spec': json.dumps(r['spec'])} for r in all_rows if r['days']==334]).to_csv(
394         OUT/'candidate_summary.csv', index=False)
395     pd.DataFrame([{'**r', 'spec': json.dumps(r['spec'])} for r in all_rows if r['days']!=334]).to_csv(
396         OUT/'pilot_summary.csv', index=False)
397     print('BEST', min(rows, key=lambda r: r['total_cost'])['spec'], flush=True)
398
399 if __name__ == '__main__':
400     main()

```

experiments/exp008/forecast.py

```

1  """Shared, issue-time forecasts for all exp008 annual scenarios.
2
3  Q2 midnight load/PV are exactly the frozen exp004 no-season CNN output.
4  Q3 augments this common forecast with observed-prefix load correction and
5  the current official PV issue. Q4 prices are forecast from past prices.
6  No decision feature or calibration label reads an actual at/after its issue.
7  """
8  from __future__ import annotations
9
10 import argparse
11 import hashlib
12 import json
13 from functools import lru_cache
14 from pathlib import Path
15

```

```

16 import numpy as np
17 import pandas as pd
18
19 from experiments.common.neural_v2.data import Data, ROOT
20 from experiments.problem2.exp004.predict import ForecastStore
21
22 STEPS = 144
23 SCENARIOS = ("2", "3", "4-2", "4-3")
24 ISSUE_SLOTS = (0, 36, 72, 108)
25 OUT = ROOT / "data/results/exp008"
26
27
28 def _integration_basis():
29     """Linear hourly-knot corrections -> ten-minute interval averages.
30
31     The observed issue-time anchor is fixed, so its correction is zero.
32     Fitting in interval space avoids treating hourly forecasts as averages.
33     """
34     basis = np.zeros((144, 24))
35     for h in range(24):
36         knot = np.zeros(25)
37         knot[h + 1] = 1
38         right = np.interp(np.arange(1, 145), np.arange(0, 145, 6), knot)
39         basis[:, h] = (np.r_[0., right[:-1]] + right) / 2
40     return basis
41
42
43 class Forecasts:
44     """Read-only forecast adapter; arrays returned by "get" are copies.
45
46     "day" is zero based (31 = February 1); "slot" is a ten-minute index.
47     Formal forecasts cover the remaining natural day, never a future day.
48     January is an explicitly labelled causal periodic cold start.
49     """
50
51     def __init__(self, data=None, seed=42):
52         self.data = Data() if data is None else data
53         self.seed = int(seed)
54         self.store = ForecastStore("no_season", seed=self.seed)
55         self._basis = _integration_basis()
56         difference = np.diff(np.eye(24), n=2, axis=0)
57         # Fixed regularization: smooth adjacent hourly bias estimates.
58         self._pv_penalty = 12 * np.einsum("ni,nj->ij", difference, difference,
59                                           optimize=False) + 2 * np.eye(24)
60
61     def _observed(self, start, stop, cutoff):
62         if not 0 <= start <= stop <= cutoff:
63             raise ValueError("actual reader requires [start,stop) strictly before issue")
64         return np.asarray(self.data.actual[start:stop], float)
65
66     @lru_cache(maxsize=366)
67     def _midnight(self, day):
68         origin = day * STEPS
69         if day >= 31:
70             return self.store.get(origin)
71         # Appendix 1 is known before operation and supplies the day-zero seed.
72         target = origin + np.arange(STEPS)
73         value = self.data.reference[:, [1, 2]].copy()
74         for channel, lag in ((0, 7), (1, 1)):
75             ids = target - lag * STEPS
76             if lag == 7 and day < 7:
77                 ids = target - STEPS
78             known = (ids >= 0) & (ids < origin)
79             if known.any():
80                 value[known, channel] = self._observed(int(ids[known][0]),
81                                                         int(ids[known][-1]) + 1, origin)[: , channel]
82         return value
83
84     def _integrate(self, hourly, origin, cutoff):
85         if origin > cutoff:

```

```

86         raise ValueError("unreleased official forecast")
87     anchor = self._observed(origin - 1, origin, cutoff)[0, 1] if origin else 0.
88     right = np.interp(np.arange(1, 145), np.arange(0, 145, 6),
89                       np.r_[anchor, np.maximum(hourly, 0)])
90     return (np.r_[anchor, right[:-1]] + right) / 2
91
92 @lru_cache(maxsize=1500)
93 def _pv(self, origin):
94     # Same release hour and lead horizon; every calibration horizon has
95     # completely arrived, including yesterday's 24-hour horizon.
96     history = np.arange(max(origin % 144, origin - 28 * 144), origin, 144)
97     history = history[history + 144 <= origin]
98     residuals = []
99     for old in history:
100         raw = self._integrate(self.data.forecasts[int(old)], int(old), origin)
101         truth = self._observed(int(old), int(old) + 144, origin)[:1, 1]
102         residuals.append(truth - raw)
103     correction = np.zeros(24)
104     if residuals:
105         weight = 2 * (-np.arange(len(history) - 1, -1, -1) / 14)
106         mean_error = np.average(np.asarray(residuals), axis=0, weights=weight)
107         a = self._basis
108         correction = np.linalg.solve(np.einsum("ni,nj->ij", a, a, optimize=False)
109                                     + self._pv_penalty,
110                                     np.einsum("ni,n->i", a, mean_error, optimize=False))
111         correction *= len(history) / (len(history) + 7)
112     hourly = np.asarray(self.data.forecasts[origin], float)
113     # Preserve official zero-generation knots. No annual capacity estimate.
114     corrected = np.where(hourly > 0, np.maximum(0, hourly + correction), 0)
115     value = self._integrate(corrected, origin, origin)
116     return value, {
117         "pv_method": "current_official_issue_with_historical_lead_knot_bias",
118         "pv_issue_origin": origin,
119         "pv_calibration_origins": history.astype(int).tolist(),
120         "pv_calibration_last_label": int(history[-1] + 143) if len(history) else None,
121         "pv_calibration_days": len(history),
122         "pv_correction_knot_kw": correction.tolist(),
123         "pv_calibration_semantics": "smooth hourly knot bias fitted to interval-integrated
124         historical errors",
125         "pv_zero_knots": "official_zero_knots_preserved",
126     }
127
128 def _price_design(self, indices, cutoff):
129     indices = np.asarray(indices, dtype=int)
130     if np.any(indices - 144 < 0) or np.any(indices - 144 >= cutoff):
131         raise ValueError("price lag is not observed")
132     # All reads are bounded prefix views; advanced indices remain within it.
133     past = self._observed(0, cutoff, cutoff)
134     week = np.where(indices >= 7 * 144, indices - 7 * 144, indices - 144)
135     phase = 2 * np.pi * (indices % 144) / 144
136     return np.column_stack((np.ones(len(indices)),
137                             self.data.fixed_price[indices % 144], past[indices - 144, 2],
138                             past[week, 2], np.sin(phase), np.cos(phase),
139                             np.sin(2 * phase), np.cos(2 * phase)))
140
141 @lru_cache(maxsize=1500)
142 def _price(self, origin, stop):
143     if origin < 144:
144         return self.data.fixed_price[origin:stop].copy(), {
145             "price_method": "appendix1_day_zero_cold_start", "price_last_label": None}
146     target = np.arange(origin, stop)
147     xt = self._price_design(target, origin)
148     train = np.arange(max(144, origin - 28 * 144), origin)
149     # These are forecast-time valid lag features for each training label.
150     x = self._price_design(train, origin)
151     y = self._observed(int(train[0]), origin, origin)[:1, 2]
152     weights = 2 * (- (origin - 1 - train) / (14 * 144))
153     prior = np.array([0., 0., .5, .5, 0., 0., 0., 0.])
154     penalty = np.diag([1e-6, 5., 5., 5., 5., 5., 5., 5.])
155     # Explicit contractions avoid this host's Accelerate matmul FPE flags;

```

```

155     # nonfinite values still fail get() instead of being silently ignored.
156     coefficient = np.linalg.solve(np.einsum("ni,nj,n->ij", x, x, weights,
157         optimize=False) + penalty,
158         np.einsum("ni,n,n->i", x, weights, y, optimize=False)
159         + np.diag(penalty) * prior)
160     prediction = np.maximum(1e-4, np.einsum("ni,i->n", xt, coefficient, optimize=False))
161     return prediction, {
162         "price_method": "past_only_recency_weighted_daily_weekly_harmonic_ridge",
163         "price_training_start": int(train[0]), "price_last_label": origin - 1,
164         "price_training_rows": len(train), "price_coefficients": coefficient.tolist(),
165         "known_future_price": False,
166     }
167
168     def get(self, day, slot=0, scenario="2"):
169         if (not isinstance(day, (int, np.integer)) or not 0 <= day < 365
170             or slot not in ISSUE_SLOTS or scenario not in SCENARIOS):
171             raise ValueError("expected day 0..364, slot 0/36/72/108 and scenario 2/3/4-2/4-3")
172         if scenario in ("2", "4-2") and slot:
173             raise ValueError("Q2 / Q4-2 plans may only be released at midnight")
174         day, slot = int(day), int(slot)
175         origin, stop = day * 144 + slot, (day + 1) * 144
176         midnight = self._midnight(day)
177         load, pv = midnight[slot:, 0].copy(), midnight[slot:, 1].copy()
178         audit = {"day": day, "slot": slot, "scenario": scenario,
179             "origin": origin, "information_cutoff_exclusive": origin,
180             "max_observed_index": origin - 1 if origin else None,
181             "seed": self.seed, "base_forecast": "exp004_no_season" if day >= 31 else "
periodic_cold_start",
182             "load_method": "common_midnight_cnn_remaining_trajectory",
183             "load_correction_kw": 0., "official_pv_allowed": scenario in ("3", "4-3"),
184             "known_future_price": False, "target_start": origin, "target_stop": stop,
185             "units": {"load_kw": "kW", "pv_kw": "kW", "price": "yuan/kWh"}}
186         if scenario in ("3", "4-3"):
187             if slot:
188                 begin = max(0, slot - 18)
189                 truth = self._observed(day * 144 + begin, origin, origin)[: , 0]
190                 error = truth - midnight[begin:slot, 0]
191                 weight = 2 * (-np.arange(len(error) - 1, -1, -1) / 6)
192                 bias = float(np.average(error, weights=weight)) * len(error) / (len(error) + 12)
193                 load = np.maximum(0, load + bias * np.exp(-np.arange(len(load)) / 36))
194                 audit.update(load_method="midnight_cnn_plus_observed_prefix_decaying_bias",
195                     load_correction_kw=bias, load_correction_last_label=origin - 1)
196                 full_pv, pv_info = self._pv(origin)
197                 pv = full_pv[:144 - slot].copy()
198                 audit.update(pv_info)
199             if scenario.startswith("4"):
200                 price, price_info = self._price(origin, stop)
201                 price = price.copy()
202                 audit.update(price_info)
203             else:
204                 price = self.data.fixed_price[slot:].copy()
205                 audit["price_method"] = "known_fixed_attachment1_tariff"
206             if not all(np.isfinite(v).all() and np.min(v) >= 0 for v in (load, pv, price)):
207                 raise ValueError("nonfinite or negative forecast")
208             return {"load_kw": load, "pv_kw": pv, "price": price, "audit": audit}
209
210     def net_error_paths(self, day, scenario="2", limit=28):
211         """Joint historical midnight errors; AC kWh, actual minus forecast.
212
213         Returns complete (history_day,144) paths, preserving time dependence.
214         January fallback paths are identified and never described as CNN errors.
215         Price errors are also supplied for joint Q4 scenario construction.
216         """
217         if not isinstance(limit, int) or limit < 1 or not 1 <= day < 365:
218             raise ValueError("need positive history limit and day 1..364")
219         history = np.arange(max(1, int(day) - limit), int(day))
220         kw, prices = [], []
221         for old in history:
222             prediction = self.get(int(old), scenario=scenario)
223             actual = self._observed(int(old) * 144, (int(old) + 1) * 144, int(day) * 144)

```

```

224         kw.append(actual[:, :2] - np.column_stack((prediction["load_kw"], prediction["pv_kw"])))
225         prices.append(actual[:, 2] - prediction["price"])
226     kw = np.asarray(kw)
227     return {"errors_kwh": (kw[:, :, 0] - kw[:, :, 1]) / 6,
228            "errors_kw": kw, "price_errors": np.asarray(prices), "origins": history * 144,
229            "audit": {"information_cutoff_exclusive": int(day) * 144,
230                    "max_observed_index": int(day) * 144 - 1,
231                    "fallback_days": history[history < 31].tolist(),
232                    "source": "frozen_cnn_or_labelled_january_periodic_then_same_issue_adapter",
233                    "semantics": "joint_completed_daily_residual_paths_not_slot_marginals"}}
234
235
236 def export(seed=42, out=OUT):
237     """Archive all requested releases; scoring reads truth only after release."""
238     import time
239     began = time.perf_counter()
240     out = Path(out)
241     out.mkdir(parents=True, exist_ok=True)
242     forecasts = Forecasts(seed=seed)
243     arrays, audits, rows, monthly_rows, lead_rows = {}, [], [], [], []
244     months = pd.date_range("2025-02-01", "2025-12-31").month.to_numpy()
245     def score(error, scenario, slot, target, unit):
246         return {"scenario": scenario, "issue_hour": slot // 6,
247                "target": target, "n": error.size, "unit": unit,
248                "mae": float(np.abs(error).mean()), "rmse": float(np.sqrt(np.mean(error ** 2))),
249                "bias_predicted_minus_actual": float(error.mean()),
250                "absolute_error_sum": float(np.abs(error).sum()),
251                "squared_error_sum": float(np.square(error).sum())}
252     for scenario in SCENARIOS:
253         for slot in ISSUE_SLOTS if scenario in ("3", "4-3") else (0,):
254             values = []
255             for day in range(31, 365):
256                 result = forecasts.get(day, slot, scenario)
257                 values.append(np.column_stack((result["load_kw"], result["pv_kw"], result["price"])))
258             audits.append(result["audit"])
259             value = np.stack(values)
260             arrays[f"scenario_{scenario}_slot_{slot}"] = value
261             # Post-hoc evaluation is deliberately outside get().
262             indices = np.arange(31, 365)[:, None] * 144 + np.arange(slot, 144)
263             truth = forecasts.data.actual[indices]
264             for channel, target in enumerate(("load", "pv", "price", "net_load")):
265                 if target == "price" and not scenario.startswith("4"):
266                     continue
267                 error = ((value[:, :, 0] - value[:, :, 1]) - (truth[:, :, 0] - truth[:, :, 1])
268                        if target == "net_load" else value[:, :, channel] - truth[:, :, channel])
269                 unit = "yuan/kWh" if channel == 2 else "kW"
270                 rows.append(score(error, scenario, slot, target, unit))
271                 for month in range(2, 13):
272                     monthly_rows.append(**score(error[months == month], scenario, slot, target,
273 unit),
274                                           "month": month))
275                 for lead in range((144 - slot) // 6):
276                     lead_rows.append(**score(error[:, lead * 6:(lead + 1) * 6], scenario, slot,
277 target, unit),
278                                     "lead_hour_end": lead + 1, "target_hour_end": slot // 6 + lead
279 + 1))
280             arrays["days"] = np.arange(31, 365)
281             np.savez_compressed(out / "forecasts.npz", **arrays)
282             pd.DataFrame(rows).to_csv(out / "forecast_metrics.csv", index=False)
283             pd.DataFrame(monthly_rows).to_csv(out / "forecast_monthly.csv", index=False)
284             pd.DataFrame(lead_rows).to_csv(out / "forecast_lead.csv", index=False)
285             (out / "forecast_audit.json").write_text(json.dumps(audits, indent=2, ensure_ascii=False) + "\n"
286 )
287             manifest = {"seed": seed, "period": ["2025-02-01", "2025-12-31"],
288                       "day_indices": "zero_based", "target_dimension": ["load_kw", "pv_kw", "price"],
289                       "archive_shapes": {k: list(v.shape) for k, v in arrays.items()},
290                       "source_sha256": {"experiments/exp008/forecast.py": hashlib.sha256(Path(__file__).read_bytes
291 ()).hexdigest(),
292                                         "data/results/exp004/predictions.npz": hashlib.sha256((ROOT / "data/results/exp004/

```

```

288     predictions.npz").read_bytes()).hexdigest()},
289     "raw_data_sha256": forecasts.data.hashes,
290     "archive_sha256": hashlib.sha256((out / "forecasts.npz").read_bytes()).hexdigest(),
291     "cnn_retrained": False, "midnight_q2_exact_frozen_exp004": True,
292     "calibration_hyperparameters": {"pv_history_days": 28, "pv_half_life_days": 14,
293     "pv_second_difference_penalty": 12, "pv_ridge": 2, "pv_shrink_days": 7,
294     "load_observed_window_slots": 18, "load_decay_slots": 36,
295     "price_history_days": 28, "price_half_life_days": 14, "price_ridge": 5},
296     "price_information": "past_actual_prices_only; future realized price is evaluation-only",
297     "selection": "
298     adapter_hyperparameters_fixed_before_dispatch_iteration_not_selected_from_annual_forecast_scores
299     ",
300     "generation_seconds": time.perf_counter() - began}
301 (out / "forecast_provenance.json").write_text(json.dumps(manifest, indent=2, ensure_ascii=False)
302 + "\n")
303 print(json.dumps({"archive": str(out / "forecasts.npz"), "seconds": manifest["generation_seconds"]
304 ], "metrics": rows}))
305
306 if __name__ == "__main__":
307     parser = argparse.ArgumentParser()
308     parser.add_argument("--seed", type=int, default=42)
309     parser.add_argument("--out", type=Path, default=OUT)
310     args = parser.parse_args()
311     export(args.seed, args.out)

```

experiments/exp008/forecast_absolute_extra_trees.py

```

1  """One fixed ExtraTrees absolute load/PV family with causal monthly fitting.
2
3  Shares all 31 historical/calendar features and train/validation days with the
4  original absolute HGB. This is a different model family, not a tuning-isolation
5  claim. Last-seven-day validation remains held out; it selects no parameter.
6  """
7  from __future__ import annotations
8
9  import copy
10 import gc
11 import json
12 from pathlib import Path
13 import shutil
14 from time import perf_counter
15
16 import joblib
17 import numpy as np
18 import pandas as pd
19 import sklearn
20 from sklearn.ensemble import ExtraTreesRegressor
21 from threadpoolctl import threadpool_limits
22
23 from experiments.exp008.forecast_absolute_hgb import (
24     OUT as BASE, AbsoluteHGBStore, ArrayStore, features_for_day, historical_labels,
25     digest, array_hash, save,
26 )
27 from experiments.exp008.forecast_calibration import CalibratedStore, _calibrate, _daylight, CONFIG
28 from experiments.exp008.load_energy_memory import EnergyMemoryStore, correct_day
29 from experiments.exp008.forecast_shape_diagnostic import summary as error_summary
30 from experiments.problem2.exp003.data import Data, ROOT
31 from experiments.problem2.exp004.data import split_days, age_weights
32
33 OUT = ROOT / 'data/results/exp008/forecast_absolute_extra_trees'
34 COMPOSITE = ROOT / 'data/results/exp008/forecast_channel_composite'
35 RAW = 'absolute_extra_trees_raw'
36 RIDGE = 'absolute_extra_trees_ridge28'
37 PRIMARY = 'absolute_extra_trees_ridge28_memory_half'
38 GATE_KEYS = ('rmse_kw', 'high_price_rmse_kw', 'daily_energy_rmse_kwh', 'cumulative_error_rmse_kwh')
39 NUMERIC_ATOL_KW = 1e-8

```

```

40 MODEL_CONFIG = dict(n_estimators=128, min_samples_leaf=10, max_features=1.,
41                     bootstrap=False, random_state=42, n_jobs=2,
42                     criterion='squared_error')
43
44
45 def prepare_protocol():
46     if OUT.exists():
47         raise FileExistsError('One fixed candidate: do not overwrite or refit existing evidence')
48     baseline_protocol = json.loads((BASE / 'protocol.json').read_text())
49     sources = ['experiments/exp008/forecast_absolute_extra_trees.py',
50               'experiments/exp008/forecast_absolute_hgb.py', 'experiments/exp008/forecast_net_hgb.py',
51               'experiments/exp008/forecast_calibration.py', 'experiments/exp008/load_energy_memory.py',
52               'experiments/exp008/forecast_shape_diagnostic.py', 'experiments/problem2/exp004/data.py',
53               'experiments/problem2/exp003/data.py', 'experiments/exp008/audited_store_bridge.py',
54               'experiments/exp008/risk_window.py']
55     inputs = [BASE / p for p in ('protocol.json', 'provenance.json', 'training_audit.json',
56                                'causality_verification.json', 'direct_hgb_raw.npz', 'direct_hgb_ridge28_memory_half.npz')]
57     inputs += [COMPOSITE / p for p in ('protocol.json', 'summary.json', 'memory.npz')]
58     protocol = {'candidate_count': 1, 'model_id': PRIMARY,
59                'family': 'sklearn ExtraTreesRegressor; independently fitted absolute load and PV',
60                'prior_search': {'pattern': 'ExtraTreesRegressor|extra[_ -]?trees|extratrees',
61                                'command': 'rg -n -i PATTERN experiments reports pyproject.toml',
62                                'performed_before_creating_this_module': True, 'matched_existing_experiments': False},
63                'not_a_single_hyperparameter_or_tuning_isolation_claim': True,
64                'features': baseline_protocol['features'], 'feature_count': 31,
65                'feature_reference': 'exact original absolute HGB features_for_day tensor and feature order'
66
67                ,
68                'target': 'absolute actual load and PV kW, same semantics in January and every formal month'
69                ,
70                'model_configuration': MODEL_CONFIG, 'monthly_models': 22,
71                'training': 'same split_days(month): all complete days from day7 through month_start-8; no
72                current-month fitting',
73                'validation': 'same preceding seven complete days, held out; metrics only, no early stopping
74                , model selection or refit',
75                'sample_weights': 'same age_weights(training_days, train_cutoff, half_life=90), repeated for
76                all144 daily slots',
77                'sample_weight_supported_by_fit': True,
78                'features_do_not_include': ['CNN or periodic forecast reference', 'official PV forecast',
79                'future actual load/PV/price', 'future-year shape', 'holiday region assumptions'],
80                'raw_postprocessing': 'both channels nonnegative; PV masked by same past28day positive-PV
81                union expanded20minutes each side',
82                'calibration': 'same unmodified joint-channel Ridge28 fitted on this raw family, then same
83                nonrecursive .5 underlying-load daily memory',
84                'predictive_gate': {'keys': list(GATE_KEYS), 'primary_comparator': 'original absolute HGB
85                complete pipeline',
86                'secondary_comparator': 'best fixed raw-channel composite complete pipeline',
87                'rule': 'all four complete-pipeline metrics strictly improve on original HGB and all
88                audits pass'},
89                'conditional_bridge': 'only after predictive gate: one full334 tree28/q.8/state_buffer500
90                using own issued residuals, no reused risk cache',
91                'full_physical_planning': 'not authorized by this experiment; positive LP only informs
92                parent',
93                'checks': {'selected_daily_future_mutations': [31,32,59,90,151,243,364],
94                'full_monthly_retraining_future_mutations': [2,9], '
95                reload_all22_models_all_formal_predictions': True,
96                'floating_tolerance_kw': NUMERIC_ATOL_KW,
97                'tolerance_reason': 'n_jobs2 parallel prediction accumulation can change floating
98                summation order; tree structures and parameters must match exactly'},
99                'development_on_examined2025_not_independent_test': True,
100                'model_or_parameters_selected_by_formal_truth': False,
101                'source_sha256': {p: digest(ROOT / p) for p in sources},
102                'input_artifact_sha256': {str(p): digest(p) for p in inputs},
103                'source_data_sha256': Data().hashes}
104     OUT.mkdir(parents=True)
105     save(OUT / 'protocol.json', protocol)
106     for name in sources:
107         target = OUT / 'source_archive' / name
108         target.parent.mkdir(parents=True, exist_ok=True)
109         shutil.copy2(ROOT / name, target)
110     return protocol

```

```

97
98
99 def tree_hashes(model):
100     # Contents, not joblib byte layout or parallel prediction addition order.
101     # Structured dtype padding bytes are not model data and may differ after
102     # pickle reload. Hash each named field, preserving all meaningful nodes.
103     return [{'node_fields': {name: array_hash(tree.tree_.__getstate__()['nodes'][name])
104                             for name in tree.tree_.__getstate__()['nodes'].dtype.names},
105             'values': array_hash(tree.tree_.__getstate__()['values']),
106             'random_state': int(tree.random_state)} for tree in model.estimators_]
107
108
109 def predict_models(models, features):
110     with threadpool_limits(limits=1):
111         return np.column_stack([m.predict(features) for m in models])
112
113
114 def physical_output(data, day, values):
115     values = np.maximum(values, 0.)
116     values[:, 1] *= _daylight(data.actual, day)
117     return values
118
119
120 def predict_day(data, day, models):
121     x, _ = features_for_day(data, day)
122     return physical_output(data, day, predict_models(models, x))
123
124
125 def fit_month(data, month, features):
126     train, validation, formal, cutoff = split_days(month)
127     asof = int(formal[0])
128     assert len(validation) == 7 and train[-1] < validation[0] and validation[-1] < asof
129     xtrain = features[train - 7].reshape(-1, 31)
130     xval = features[validation - 7].reshape(-1, 31)
131     ytrain = historical_labels(data, train, cutoff * 144).reshape(-1, 2)
132     yval = historical_labels(data, validation, asof * 144).reshape(-1, 2)
133     weights = np.repeat(age_weights(train, cutoff, 90), 144)
134     models, records = [], []
135     with threadpool_limits(limits=1):
136         for channel, name in enumerate(('load', 'pv')):
137             model = ExtraTreesRegressor(**MODEL_CONFIG)
138             began = perf_counter()
139             model.fit(xtrain, ytrain[:, channel], sample_weight=weights)
140             predicted = model.predict(xval)
141             records.append({'channel': name, 'training_seconds': perf_counter() - began,
142                           'trees': len(model.estimators_), 'node_count': sum(t.tree_.node_count for t in model
143                               .estimators_),
144                           'validation_rmse_kw_before_physical_postprocessing': float(np.sqrt(np.mean((
145                               predicted - yval[:, channel])**2))),
146                           'feature_importance': model.feature_importances_.tolist(),
147                           'tree_hashes': tree_hashes(model), 'resolved_model_parameters': model.get_params()})
148             models.append(model)
149     audit = {'month': month, 'asof_day': asof, 'training_days': train.tolist(),
150             'validation_days': validation.tolist(), 'formal_days': formal.tolist(),
151             'train_label_stop_exclusive': cutoff * 144, 'validation_label_stop_exclusive': asof * 144,
152             'model_information_cutoff_exclusive': asof * 144,
153             'training_feature_hash': array_hash(xtrain), 'validation_feature_hash': array_hash(xval),
154             'training_label_hash': array_hash(ytrain), 'validation_label_hash': array_hash(yval),
155             'training_weights_hash': array_hash(weights), 'training_rows': len(xtrain),
156             'validation_rows': len(xval), 'validation_labels_used_for_fitting_or_model_selection': False
157         ,
158             'models': records}
159     return models, audit
160
161
162 def postprocess(data, values, origins):
163     raw = ArrayStore(values, origins, RAW)
164     ridge = CalibratedStore('ridge_28', data=data, use_cache=False, _base_store=raw,
165                             directory=OUT / 'unused_calibration_cache')
166     memory = EnergyMemoryStore(data=data, base_store=ridge)

```

```

164 ridge.name, memory.name = RIDGE, PRIMARY
165 for audit in ridge.audit:
166     audit['actual_base_model_id'] = RAW
167 for audit in memory.audit:
168     audit['underlying_forecast'] = 'absolute_load_PV_ExtraTrees_then_same_Ridge28'
169 save(OUT / 'ridge28_audit.json', ridge.audit)
170 save(OUT / 'memory_half_audit.json', memory.audit)
171 return {RAW: raw, RIDGE: ridge, PRIMARY: memory}
172
173
174 class AbsoluteExtraTreesStore(ArrayStore):
175     def __init__(self, stage=PRIMARY):
176         meta = json.loads((OUT / 'provenance.json').read_text())
177         path = OUT / f'{stage}.npz'
178         if not meta['complete'] or digest(path) != meta['archives'][stage]:
179             raise ValueError('ExtraTrees archive is incomplete or changed')
180         with np.load(path) as z:
181             super().__init__(z['values'], z['origins'], stage)
182
183
184 def load_models(month):
185     return [joblib.load(OUT / 'models' / f'month{month:02d}_{channel}_seed42.joblib')
186             for channel in ('load', 'pv')]
187
188
189 def causal_checks(data, raw, processed, features, protocol):
190     daily = []
191     for day in protocol['checks']['selected_daily_future_mutations']:
192         changed = copy.copy(data)
193         changed.actual = data.actual.copy()
194         changed.actual[day * 144:] += np.array([70000., 50000.])
195         x0, _ = features_for_day(data, day)
196         x1, _ = features_for_day(changed, day)
197         np.testing.assert_array_equal(x0, x1)
198         month = int((pd.Timestamp('2025-01-01') + pd.Timedelta(days=day)).month)
199         models = load_models(month)
200         a, b = predict_day(data, day, models), predict_day(changed, day, models)
201         np.testing.assert_allclose(a, b, rtol=0, atol=NUMERIC_ATOL_KW)
202         i = day - 31
203         future_raw = raw.values.copy()
204         future_raw[i + 1:] += 80000.
205         r0, _ = _calibrate(data.actual, raw.origins, raw.values, i, CONFIG['ridge_28'])
206         r1, _ = _calibrate(changed.actual, raw.origins, future_raw, i, CONFIG['ridge_28'])
207         np.testing.assert_array_equal(r0, r1)
208         future_ridge = processed[RIDGE].values.copy()
209         future_ridge[i + 1:] += 90000.
210         m0, _ = correct_day(data.actual, raw.origins, processed[RIDGE].values, i)
211         m1, _ = correct_day(changed.actual, raw.origins, future_ridge, i)
212         np.testing.assert_array_equal(m0, m1)
213         daily.append({'day': day, 'origin': day * 144, 'features_exactly_unchanged': True,
214                     'raw_prediction_max_change_kw': float(np.max(np.abs(a-b))),
215                     'daylight_mask_unchanged': bool(np.array_equal(_daylight(data.actual, day), _daylight(
216                         changed.actual, day))),
217                     'ridge_and_memory_exactly_unchanged': True})
218     del models
219     retraining = []
220     for month in protocol['checks']['full_monthly_retraining_future_mutations']:
221         train, validation, formal, cutoff = split_days(month)
222         issue = int(formal[0]) * 144
223         changed = copy.copy(data)
224         changed.actual = data.actual.copy()
225         changed.actual[issue:] += np.array([90000., 60000.])
226         new_features = np.stack([features_for_day(changed, day)[0] for day in range(7, 365)])
227         for days in (train, validation):
228             np.testing.assert_array_equal(features[days - 7], new_features[days - 7])
229         fresh, audit = fit_month(changed, month, new_features)
230         saved = load_models(month)
231         for a, b in zip(saved, fresh):
232             assert a.get_params() == b.get_params()
233             assert tree_hashes(a) == tree_hashes(b)

```

```

233     a = predict_day(data, int(formal[0]), saved)
234     b = predict_day(changed, int(formal[0]), fresh)
235     np.testing.assert_allclose(a, b, rtol=0, atol=NUMERIC_ATOL_KW)
236     retraining.append({'month': month, 'future_mutation_origin': issue,
237                       'train_and_validation_features_exactly_unchanged': True,
238                       'retrained_model_parameters_and_all_tree_structures_exactly_identical': True,
239                       'first_day_prediction_max_change_kw': float(np.max(np.abs(a-b))),
240                       'training_seconds': sum(m['training_seconds'] for m in audit['models'])})
241     print(json.dumps({'causal_retraining_month': month, 'passed': True}), flush=True)
242     del fresh, saved, new_features
243     gc.collect()
244     labels_past = all(r['history_last_label'] is None or r['history_last_label'] < r['origin']
245                      for r in processed[RIDGE].audit)
246     labels_past &= all(r['prior_label_end_exclusive'] is None or r['prior_label_end_exclusive'] <= r
247                      ['origin']
248                      for r in processed[PRIMARY].audit)
249     assert labels_past
250     result = {'passed': True, 'selected_daily_checks': daily, 'full_retraining_checks': retraining,
251              'all334_postprocessing_labels_past': bool(labels_past), 'floating_tolerance_kw':
252              NUMERIC_ATOL_KW,
253              'reference_feature_or_official_PV_used': False}
254     save(OUT / 'causality_verification.json', result)
255     return result
256
257 def metrics(data, stores, origins):
258     truth = data.actual[origins[:, None] + np.arange(144), :2]
259     dates = pd.date_range('2025-02-01', '2025-12-31')
260     annual, monthly, daily = [], [], []
261     for name, values in stores.items():
262         for channel, error in [('load', values[:, :, 0]-truth[:, :, 0]),
263                               ('pv', values[:, :, 1]-truth[:, :, 1]),
264                               ('net', (values[:, :, 0]-values[:, :, 1])-(truth[:, :, 0]-truth[:, :, 1]))]:
265             annual.append({'name': name, 'channel': channel, **error_summary(error, data.fixed_price)
266                           })
267             for month in range(2, 13):
268                 monthly.append({'name': name, 'channel': channel, 'month': month,
269                               **error_summary(error[dates.month == month], data.fixed_price)})
270             if channel == 'net':
271                 for i, date in enumerate(dates):
272                     daily.append({'name': name, 'date': str(date.date()), 'day': i+31,
273                                   **error_summary(error[i:i+1], data.fixed_price)})
274     for name, rows in [('annual_metrics', annual), ('monthly_metrics', monthly), ('daily_metrics',
275                                         daily)]:
276         pd.DataFrame(rows).to_csv(OUT / f'{name}.csv', index=False)
277     return annual
278
279 def source_checks(protocol):
280     assert all(digest(ROOT / p) == sha for p, sha in protocol['source_sha256'].items())
281     assert all(digest(p) == sha for p, sha in protocol['input_artifact_sha256'].items())
282
283 def run():
284     protocol = prepare_protocol()
285     source_checks(protocol)
286     started = perf_counter()
287     data = Data()
288     features = np.stack([features_for_day(data, day)[0] for day in range(7, 365)])
289     base_meta = json.loads((BASE / 'provenance.json').read_text())
290     assert features.shape == (358, 144, 31) and np.isfinite(features).all()
291     assert array_hash(features) == base_meta['feature_tensor_sha256']
292     old_training = {r['month']: r for r in json.loads((BASE / 'training_audit.json').read_text())}
293     values = np.empty((334, 144, 2))
294     training, predictions = [], []
295     for month in range(2, 13):
296         models, audit = fit_month(data, month, features)
297         for key in ('training_days', 'validation_days', 'formal_days', 'training_feature_hash',
298                   'validation_feature_hash', 'training_label_hash', 'validation_label_hash'):
299             assert audit[key] == old_training[month][key], key

```

```

299     audit['exact_historical_features_labels_and_splits_match_original_HGB'] = True
300     xformal = features[np.asarray(audit['formal_days']) - 7].reshape(-1, 31)
301     raw_prediction = predict_models(models, xformal).reshape(-1, 144, 2)
302     for channel, model in enumerate(models):
303         path = OUT / 'models' / f'month{month:02d}_{("load", "pv")[channel]}_seed42.joblib'
304         path.parent.mkdir(parents=True, exist_ok=True)
305         joblib.dump(model, path, compress=3)
306         reloaded = joblib.load(path)
307         assert model.get_params() == reloaded.get_params()
308         assert tree_hashes(model) == tree_hashes(reloaded)
309         with threadpool_limits(limits=1):
310             restored = reloaded.predict(xformal).reshape(-1, 144)
311             difference = float(np.max(np.abs(restored-raw_prediction[:, :, channel])))
312             np.testing.assert_allclose(restored, raw_prediction[:, :, channel], rtol=0, atol=
NUMERIC_ATOL_KW)
313             audit['models'][channel].update(model_path=str(path), model_sha256=digest(path),
314             reload_tree_parameters_exact=True, reload_all_formal_prediction_max_difference_kw=
difference)
315         del reloaded
316     for i, day in enumerate(audit['formal_days']):
317         values[day-31] = physical_output(data, day, raw_prediction[i])
318         predictions.append({'day': day, 'origin': day*144, 'monthly_model': month,
319         'train_label_stop_exclusive': audit['train_label_stop_exclusive'],
320         'validation_label_stop_exclusive': audit['validation_label_stop_exclusive'],
321         'feature_last_actual_index': day*144-1, 'daylight_last_actual_index': day*144-1,
322         'raw_prediction_sha256': array_hash(values[day-31]),
323         'negative_raw_load_slots': int(np.sum(raw_prediction[i, :, 0] < 0)),
324         'negative_raw_pv_slots': int(np.sum(raw_prediction[i, :, 1] < 0)),
325         'positive_raw_pv_outside_causal_daylight_mask_slots': int(np.sum(
326         (raw_prediction[i, :, 1] > 0) & ~_daylight(data.actual, day)))))
327     training.append(audit)
328     save(OUT / 'training_audit.json', training)
329     print(json.dumps({'month': month, 'days': len(audit['training_days']),
330     'validation_days': 7, 'training_seconds': sum(m['training_seconds'] for m in audit['
models']),
331     'saved_models': 2*len(training)}), flush=True)
332     del models, model
333     gc.collect()
334     origins = np.arange(31, 365)*144
335     processed = postprocess(data, values, origins)
336     causal = causal_checks(data, processed[RAW], processed, features, protocol)
337     np.testing.assert_array_equal(origins, AbsoluteHGBStore().origins)
338     with np.load(COMPOSITE / 'memory.npz') as z:
339         np.testing.assert_array_equal(origins, z['origins'])
340         composite_values = z['values'].copy()
341     comparison = {'original_HGB_raw': AbsoluteHGBStore(stage='direct_hgb_raw').values,
342     'original_HGB_full': AbsoluteHGBStore().values, 'best_fixed_channel_composite_full':
composite_values,
343     **{name: store.values for name, store in processed.items()}}
344     scores = metrics(data, comparison, origins)
345     candidate, baseline, composite = [next(r for r in scores if r['name'] == name and r['channel']
== 'net')]
346     for name in (PRIMARY, 'original_HGB_full', 'best_fixed_channel_composite_full')]
347     checks = {k: candidate[k] < baseline[k] for k in GATE_KEYS}
348     gate = all(checks.values()) and causal['passed']
349     truth = data.actual[origins[:, None]+np.arange(144), :2]
350     archives = {}
351     for name, store in processed.items():
352         path = OUT / f'{name}.npz'
353         np.savez_compressed(path, values=store.values, origins=origins, errors_kw=truth-store.values)
354         archives[name] = digest(path)
355     save(OUT / 'prediction_audit.json', predictions)
356     source_checks(protocol)
357     save(OUT / 'dispatch_gate.json', {'passed': gate, 'metric_checks_vs_original_HGB': checks,
358     'candidate': candidate, 'original_HGB': baseline, 'best_fixed_channel_composite': composite,
359     'metric_checks_vs_fixed_composite': {k: candidate[k] < composite[k] for k in GATE_KEYS},
360     'formal_scores_are_development_results_not_independent_test': True})
361     provenance = {'complete': True, 'days': 334, 'primary': PRIMARY, 'archives': archives,
362     'source_sha256': digest(__file__), 'protocol_sha256': digest(OUT / 'protocol.json'),
363     'source_data_sha256': data.hashes, 'feature_tensor_sha256': array_hash(features),

```

```

364         'exact_original_HGB_feature_tensor_match': True, 'models': 22,
365         'source_and_input_hashes_unchanged': True, 'causality_verified': causal['passed'],
366         'all_monthly_models_reloaded_all_formal_predictions_verified': True,
367         'sklearn_version': sklearn.__version__, 'numpy_version': np.__version__,
368         'predictive_gate_passed': gate, 'elapsed_seconds_before_bridge': perf_counter()-started}
369     save(OUT / 'provenance.json', provenance)
370     summary = {'complete': True, 'model_id': PRIMARY, 'predictive_gate_passed': gate,
371               'metrics': scores, 'gate_checks': checks, 'bridge_performed': False,
372               'full_physical_performed': False, 'final_model_selected': False,
373               'source_hashes_verified': True, 'audit_passed': True}
374     save(OUT / 'summary.json', summary)
375     print(json.dumps({'gate': gate, 'candidate': candidate, 'original_HGB': baseline,
376                      'fixed_composite': composite}), flush=True)
377     if gate:
378         from experiments.exp008.audited_store_bridge import run as bridge
379         bridge(AbsoluteExtraTreesStore(), OUT / 'lp_bridge',
380               [OUT / 'protocol.json', OUT / 'provenance.json', OUT / f'{PRIMARY}.npz',
381                OUT / 'causality_verification.json', OUT / 'dispatch_gate.json'])
382         summary['bridge_performed'] = True
383         save(OUT / 'summary.json', summary)
384
385
386 if __name__ == '__main__':
387     run()

```

experiments/exp008/forecast_absolute_hgb.py

```

1  """One direct absolute load/PV HGB candidate; protocol saved before fitting.
2
3  Thirty-one historical/calendar inputs are the old residual HGB inputs minus
4  all four forecast-reference columns. No CNN/periodic output enters training
5  or inference. The fixed primary comparison includes identical Ridge28 and
6  nonrecursive gain-.5 load-memory postprocessing for HGB and joint CNN.
7  """
8  from __future__ import annotations
9
10 import copy
11 import hashlib
12 import json
13 from pathlib import Path
14 import shutil
15 from time import perf_counter
16
17 import joblib
18 import numpy as np
19 import pandas as pd
20 import sklearn
21 from sklearn.ensemble import HistGradientBoostingRegressor
22 from threadpoolctl import threadpool_limits
23
24 from experiments.exp008.forecast_net_hgb import features_for_day as residual_features
25 from experiments.exp008.forecast_calibration import CalibratedStore, _calibrate, _daylight, CONFIG
26 from experiments.exp008.forecast_shape_diagnostic import summary as error_summary
27 from experiments.exp008.load_energy_memory import EnergyMemoryStore, correct_day
28 from experiments.exp008.neural_joint_calibration import JointStore
29 from experiments.problem2.exp003.data import Data, ROOT
30 from experiments.problem2.exp004.data import split_days, age_weights
31
32 OUT = ROOT / 'data/results/exp008/forecast_absolute_hgb'
33 PRIMARY = 'direct_hgb_ridge28_memory_half'
34 COMPARATOR = 'joint_ridge28_load_energy_memory_half'
35
36
37 def digest(path):
38     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
39
40

```

```

41 def array_hash(array):
42     return hashlib.sha256(np.ascontiguousarray(array).tobytes()).hexdigest()
43
44
45 def save(path, value):
46     Path(path).parent.mkdir(parents=True, exist_ok=True)
47     Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + '\n')
48
49
50 def features_for_day(data, day):
51     # The supplied zeros are immediately removed, along with base_is_cnn.
52     # Reuse the exact remaining historical formulas without modifying old HGB.
53     matrix, names = residual_features(data, day, np.zeros((144, 2)))
54     return matrix[:, 3:-1], names[3:-1]
55
56
57 def historical_labels(data, days, cutoff):
58     days = np.asarray(days, int)
59     if np.any((days + 1) * 144 > cutoff):
60         raise ValueError('Label interval is not complete at its fitting cutoff')
61     return np.stack([data.actual[day * 144:(day + 1) * 144, :2] for day in days])
62
63
64 def fit_month(data, month, features, protocol):
65     train, validation, formal, cutoff = split_days(month)
66     asof = int(formal[0])
67     assert len(validation) == 7 and train[-1] < validation[0] and validation[-1] < asof
68     xtrain = features[train - 7].reshape(-1, len(protocol['features']))
69     xval = features[validation - 7].reshape(-1, len(protocol['features']))
70     ytrain = historical_labels(data, train, cutoff * 144).reshape(-1, 2)
71     yval = historical_labels(data, validation, asof * 144).reshape(-1, 2)
72     weights = np.repeat(age_weights(train, cutoff, 90), 144)
73     models, records = [], []
74     with ThreadPoolLimits(limits=1):
75         for channel, name in enumerate(('load', 'pv')):
76             model = HistGradientBoostingRegressor(**protocol['model_configuration'])
77             began = perf_counter()
78             model.fit(xtrain, ytrain[:, channel], sample_weight=weights,
79                     X_val=xval, y_val=yval[:, channel])
80             models.append(model)
81             predicted = model.predict(xval)
82             records.append({'channel': name, 'iterations': int(model.n_iter_),
83                             'training_seconds': perf_counter() - began,
84                             'training_score_by_iteration': model.train_score_.tolist(),
85                             'validation_score_by_iteration': model.validation_score_.tolist(),
86                             'validation_rmse_kw_before_physical_postprocessing': float(np.sqrt(np.mean((
87             predicted - yval[:, channel])**2)))))
88     return models, {'month': month, 'asof_day': asof, 'training_days': train.tolist(),
89                     'validation_days': validation.tolist(), 'formal_days': formal.tolist(),
90                     'train_label_stop_exclusive': cutoff * 144, 'validation_label_stop_exclusive': asof * 144,
91                     'model_information_cutoff_exclusive': asof * 144,
92                     'feature_rule': 'each training/validation/formal feature uses actuals only before its own
93                     daily issue',
94                     'target_semantics': 'absolute load/PV kW throughout January and all formal months',
95                     'periodic_or_CNN_reference_columns': False, 'validation_rows': len(xval),
96                     'training_rows': len(xtrain), 'training_feature_hash': array_hash(xtrain),
97                     'validation_feature_hash': array_hash(xval), 'training_label_hash': array_hash(ytrain),
98                     'validation_label_hash': array_hash(yval), 'models': records}
99
100 def predict_day(data, day, models):
101     x, _ = features_for_day(data, day)
102     with ThreadPoolLimits(limits=1):
103         values = np.column_stack([model.predict(x) for model in models])
104     values = np.maximum(values, 0.)
105     values[:, 1] *= _daylight(data.actual, day)
106     return values
107
108 class ArrayStore:

```

```

109     def __init__(self, values, origins, name):
110         self.values, self.origins = np.asarray(values).copy(), np.asarray(origins).copy()
111         self.name = name
112         self.lookup = {int(origin): i for i, origin in enumerate(self.origins)}
113
114     def get(self, origin):
115         return self.values[self.lookup[int(origin)]]
116
117
118 class AbsoluteHGBStore(ArrayStore):
119     def __init__(self, stage=PRIMARY, directory=OUT):
120         directory = Path(directory)
121         meta = json.loads((directory / 'provenance.json').read_text())
122         path = directory / f'{stage}.npz'
123         if not meta['complete'] or digest(path) != meta['archives'][stage]:
124             raise ValueError('Direct HGB forecast archive is incomplete or changed')
125         with np.load(path) as z:
126             super().__init__(z['values'], z['origins'], stage)
127
128
129 def postprocess(data, values, origins):
130     raw = ArrayStore(values, origins, 'direct_hgb_raw')
131     ridge = CalibratedStore('ridge_28', seed=42, data=data, use_cache=False,
132                             directory=OUT / 'calibration_recomputed', _base_store=raw)
133     memory = EnergyMemoryStore(data=data, base_store=ridge)
134     ridge.name, memory.name = 'direct_hgb_ridge28', PRIMARY
135     for audit in memory.audit:
136         audit['underlying_forecast'] = 'direct_absolute_load_pv_HGB_then_same_fixed_Ridge28'
137     save(OUT / 'ridge28_audit.json', ridge.audit)
138     save(OUT / 'memory_half_audit.json', memory.audit)
139     return {raw.name: raw, ridge.name: ridge, memory.name: memory}
140
141
142 def causality_checks(data, models_by_month, raw, processed, features, protocol):
143     future = []
144     for day in (31, 32, 59, 90, 151, 243, 364):
145         changed = copy.copy(data)
146         changed.actual = data.actual.copy()
147         changed.actual[day * 144:] += np.array([50000., 30000.])
148         a, _ = features_for_day(data, day)
149         b, _ = features_for_day(changed, day)
150         np.testing.assert_array_equal(a, b)
151         month = int((pd.Timestamp('2025-01-01') + pd.Timedelta(days=day)).month)
152         before, after = predict_day(data, day, models_by_month[month]), predict_day(changed, day,
153                                         models_by_month[month])
154         np.testing.assert_array_equal(before, after)
155         index = day - 31
156         future_raw = raw.values.copy()
157         future_raw[index + 1:] += 70000.
158         ridge0, _ = _calibrate(data.actual, raw.origins, raw.values, index, CONFIG['ridge_28'])
159         ridge1, _ = _calibrate(changed.actual, raw.origins, future_raw, index, CONFIG['ridge_28'])
160         np.testing.assert_array_equal(ridge0, ridge1)
161         future_ridge = processed['direct_hgb_ridge28'].values.copy()
162         future_ridge[index + 1:] += 80000.
163         memory0, _ = correct_day(data.actual, raw.origins, processed['direct_hgb_ridge28'].values,
164                                 index)
165         memory1, _ = correct_day(changed.actual, raw.origins, future_ridge, index)
166         np.testing.assert_array_equal(memory0, memory1)
167         future.append({'day': day, 'origin': day * 144, 'features_identical': True,
168                       'raw_model_output_with_daylight_mask_identical': True, 'ridge28_identical': True,
169                       'fixed_memory_identical': True, 'changed_current_and_future_actual_and_future_forecasts':
170                           True})
171     retraining = []
172     for month in (2, 9):
173         train, validation, formal, cutoff = split_days(month)
174         issue = int(formal[0]) * 144
175         changed = copy.copy(data)
176         changed.actual = data.actual.copy()
177         changed.actual[issue:] += np.array([90000., 50000.])
178         x = np.stack([features_for_day(changed, day)[0] for day in range(7, 365)])

```

```

176     for selected in (train, validation):
177         np.testing.assert_array_equal(features[selected - 7], x[selected - 7])
178     fresh, audit = fit_month(changed, month, x, protocol)
179     a = predict_day(data, int(formal[0]), models_by_month[month])
180     b = predict_day(changed, int(formal[0]), fresh)
181     np.testing.assert_array_equal(a, b)
182     retraining.append({'month': month, 'future_actual_mutation_from': issue,
183                       'full_train_and_validation_features_unchanged': True,
184                       'retrained_model_first_day_predictions_identical': True,
185                       'iterations': [r['iterations'] for r in audit['models']]})
186     result = {'passed': True, 'future_actual_and_future_prediction_checks': future,
187              'monthly_full_retraining_checks': retraining,
188              'official_PV_not_loaded_by_model': True, 'CNN_or_periodic_feature_not_used': True,
189              'all_postprocessing_labels_past': all(r['history_last_label'] is None or
190            r['history_last_label'] < r['origin'] for r in processed['direct_hgb_ridge28'].audit)
191            and all(r['prior_label_end_exclusive'] is None or r['prior_label_end_exclusive'] <= r['
192            origin']
193                  for r in processed[PRIMARY].audit)}
194     assert result['all_postprocessing_labels_past']
195     save(OUT / 'causality_verification.json', result)
196     return result
197
198 def score_all(data, values, origins):
199     truth = data.actual[origins[:, None] + np.arange(144), :2]
200     dates = pd.date_range('2025-02-01', '2025-12-31')
201     annual, monthly, daily = [], [], []
202     for name, value in values.items():
203         for channel, error in [('load', value[:, :, 0] - truth[:, :, 0]),
204                               ('pv', value[:, :, 1] - truth[:, :, 1]),
205                               ('net', (value[:, :, 0] - value[:, :, 1]) - (truth[:, :, 0] - truth
206                               [[:, :, 1]]))]:
207             annual.append({'name': name, 'channel': channel, **error_summary(error, data.fixed_price
208             )})
209             for month in range(2, 13):
210                 ids = dates.month == month
211                 monthly.append({'name': name, 'channel': channel, 'month': month,
212                               **error_summary(error[ids], data.fixed_price)})
213             if channel == 'net':
214                 for i, date in enumerate(dates):
215                     daily.append({'name': name, 'date': str(date.date()), 'day': i + 31,
216                                   **error_summary(error[i:i + 1], data.fixed_price)})
217     for name, rows in [('annual_metrics', annual), ('monthly_metrics', monthly), ('daily_metrics',
218     daily)]:
219         pd.DataFrame(rows).to_csv(OUT / f'{name}.csv', index=False)
220     return annual
221
222 def bridge():
223     from experiments.exp008.risk_window import replay, SPEC
224     from experiments.exp008.verify import verify_npz
225     store = AbsoluteHGBStore()
226     data = Data()
227     directory = OUT / 'lp_bridge'
228     case = 'direct_hgb_ridge28_memory_tree28_q08_buffer500'
229     result = replay(case, 28, 'tree', 334, data, store, directory)
230     case_dir = directory / f'{case}_334days'
231     audit = json.loads((case_dir / 'audit.json').read_text())
232     for row in audit:
233         row['forecast_calibration'] = store.name
234         row['residual_source'] = 'periodic_baseline' if row['fallback'] else f'{store.name}
235         _prequential_forecast'
236     save(case_dir / 'audit.json', audit)
237     check = verify_npz(case_dir / 'dispatch_2.npz', audit_path=case_dir / 'audit.json')
238     assert check['passed'], check['errors']
239     result.update(fixed_spec={**SPEC, 'calibration': store.name,
240     evaluation_role='fixed forecast-only LP bridge; not final action-constrained
241     policy',
242     forecast_values_sha256=array_hash(store.values))
243     save(case_dir / 'summary.json', result)

```

```

240     save(case_dir / 'independent_verification.json', check)
241     return result
242
243
244 def run():
245     protocol = json.loads((OUT / 'protocol.json').read_text())
246     if (OUT / 'provenance.json').exists():
247         raise FileExistsError('This fixed candidate is already complete; no refit or parameter
            search')
248     for name, expected in protocol['source_before_training'].items():
249         if digest(ROOT / name) != expected:
250             raise ValueError(f'Protocol source changed before training: {name}')
251     for name in [__file__, *[str(ROOT / name) for name in protocol['source_before_training']]]:
252         target = OUT / 'source_archive' / Path(name).name
253         target.parent.mkdir(parents=True, exist_ok=True)
254         shutil.copy2(name, target)
255     started = perf_counter()
256     data = Data()
257     features = np.stack([features_for_day(data, day)[0] for day in range(7, 365)])
258     assert features.shape == (358, 144, 31) and np.isfinite(features).all()
259     values = np.empty((334, 144, 2))
260     models_by_month, training, predictions = {}, [], []
261     for month in range(2, 13):
262         models, audit = fit_month(data, month, features, protocol)
263         models_by_month[month] = models
264         for channel, model in enumerate(models):
265             path = OUT / 'models' / f'month{month:02d}_{("load", "pv")[channel]}_seed42.joblib'
266             path.parent.mkdir(parents=True, exist_ok=True)
267             joblib.dump(model, path)
268             reloaded = joblib.load(path)
269             x = features[audit['formal_days']][0] - 7]
270             with threadpool_limits(limits=1):
271                 np.testing.assert_array_equal(model.predict(x), reloaded.predict(x))
272                 audit['models'][channel].update(model_path=str(path), model_sha256=digest(path),
            reload_identical=True)
273             training.append(audit)
274             for day in audit['formal_days']:
275                 values[day - 31] = predict_day(data, day, models)
276                 predictions.append({'day': day, 'origin': day * 144, 'month_model': month,
            'training_label_stop': audit['train_label_stop_exclusive'],
277                 'validation_label_stop': audit['validation_label_stop_exclusive'],
278                 'feature_last_actual_index': day * 144 - 1, 'daylight_last_actual_index': day * 144
            - 1,
279                 'raw_prediction_sha256': array_hash(values[day - 31]),
280                 'no_CNN_periodic_official_PV_future_price_input': True})
281             print(json.dumps({'month': month, 'training_days': len(audit['training_days']),
            'validation_days': len(audit['validation_days']), 'iterations': [r['iterations'] for r
            in audit['models']]}, flush=True)
282         origins = np.arange(31, 365) * 144
283         processed = postprocess(data, values, origins)
284         raw = processed['direct_hgb_raw']
285         verified = causality_checks(data, models_by_month, raw, processed, features, protocol)
286         comparison = {'joint_raw': JointStore().values, 'joint_ridge28': JointStore(calibrated=True).
            values,
287                 COMPARATOR: EnergyMemoryStore(data=data).values}
288         comparison.update({name: store.values for name, store in processed.items()})
289         metrics = score_all(data, comparison, origins)
290         archives = {}
291         truth = data.actual[origins[:, None] + np.arange(144), :2]
292         for name, store in processed.items():
293             path = OUT / f'{name}.npz'
294             np.savez_compressed(path, values=store.values, origins=origins, errors_kw=truth - store.
            values)
295             archives[name] = digest(path)
296         save(OUT / 'training_audit.json', training)
297         save(OUT / 'prediction_audit.json', predictions)
298         prior = next(r for r in metrics if r['name'] == COMPARATOR and r['channel'] == 'net')
299         candidate = next(r for r in metrics if r['name'] == PRIMARY and r['channel'] == 'net')
300         gate_metrics = [key for group in ('net', 'high_price', 'daily_accumulation') for key in protocol
            ['dispatch_gate'][group]]

```

```

303 checks = {key: candidate[key] < prior[key] for key in gate_metrics}
304 gate = all(checks.values()) and verified['passed']
305 save(OUT / 'dispatch_gate.json', {'passed': gate, 'primary_candidate': PRIMARY,
306     'primary_comparator': COMPARATOR, 'metric_checks': checks, 'candidate_metrics': candidate,
307     'comparator_metrics': prior, 'model_or_postprocessing_selected_by_formal_scores': False})
308 provenance = {'complete': True, 'seed': 42, 'days': 334, 'fitted_monthly_estimators': 22,
309     'source_sha256': digest(__file__), 'protocol_sha256': digest(OUT / 'protocol.json'),
310     'source_data_sha256': data.hashes, 'feature_names': protocol['features'],
311     'feature_tensor_sha256': array_hash(features), 'archives': archives,
312     'primary': PRIMARY, 'comparison': COMPARATOR, 'dispatch_gate_passed': gate,
313     'sklearn_version': sklearn.__version__, 'numpy_version': np.__version__,
314     'total_seconds_before_bridge': perf_counter() - started,
315     'training_and_predictions_causally_verified': verified['passed'],
316     'comparison_inputs': {str(RESULT): digest(RESULT) for RESULT in
317         [ROOT / 'data/results/exp008/neural_joint/predictions.npz',
318          ROOT / 'data/results/exp008/neural_joint_calibration/joint_ridge28.npz',
319          ROOT / 'data/results/exp008/load_energy_memory/predictions.npz']}},
320     'development_not_independent_test': True}
321 save(OUT / 'provenance.json', provenance)
322 linked = bridge() if gate else None
323 save(OUT / 'summary.json', {'predictive_gate_passed': gate, 'bridge_performed': linked is not
324     None,
325     'primary_metrics': candidate, 'baseline_metrics': prior, 'metric_checks': checks,
326     'bridge_total_cost': linked['total_cost'] if linked else None,
327     'final_model_selected': False, 'final_report_generated': False})
328 print(json.dumps({'gate': gate, 'candidate': candidate, 'baseline': prior, 'bridge_performed':
329     linked is not None}), flush=True)
330
331 if __name__ == '__main__':
332     run()

```

experiments/exp008/forecast_calibration.py

```

1  """Causal output calibration of the unchanged exp004 midnight CNN.
2
3  These candidates are development experiments on the already examined 2025
4  year, not an independent test. Each day's adjustment is fitted exclusively
5  to completed earlier days and never selected using the current day's truth.
6  """
7  from __future__ import annotations
8
9  import argparse
10 import hashlib
11 import json
12 import time
13 from pathlib import Path
14
15 import numpy as np
16 import pandas as pd
17
18 from experiments.problem2.exp003.data import Data, ROOT
19 from experiments.problem2.exp004.predict import ForecastStore
20
21 OUT = ROOT / "data/results/exp008/forecast_calibration"
22 CANDIDATES = (
23     {"name": "base", "method": "none", "window": 0},
24     *({"name": f"global_bias_{w}", "method": "global_bias", "window": w} for w in (7, 14, 28, 56)),
25     *({"name": f"hour_bias_{w}", "method": "hour_bias", "window": w} for w in (14, 28)),
26     *({"name": f"ridge_{w}", "method": "ridge", "window": w} for w in (28, 56)),
27     {"name": "online_select_28", "method": "online_select", "window": 28},
28 )
29 CONFIG = {c["name"]: dict(c) for c in CANDIDATES}
30
31
32 def _history(actual, start, stop, cutoff):
33     if not 0 <= start <= stop <= cutoff:

```

```

34         raise ValueError("calibration cannot access an unobserved actual")
35     return np.asarray(actual[start:stop, :2], dtype=float)
36
37
38 def _daylight(actual, day):
39     origin = day * 144
40     past = _history(actual, max(0, origin - 28 * 144), origin, origin)
41     daylight = (past[:, 1].reshape(-1, 144) > 0).any(axis=0)
42     return np.convolve(daylight.astype(int), np.ones(5), mode="same") > 0
43
44
45 def _features(actual, day, base, channel):
46     """Features for a historical issue use only information at that issue."""
47     origin = day * 144
48     yesterday = _history(actual, origin - 144, origin, origin)[:, channel]
49     weekly = _history(actual, origin - 7 * 144, origin - 6 * 144, origin)[:, channel]
50     last3 = _history(actual, origin - 3 * 144, origin, origin).reshape(3, 144, 2)[:, :, channel].
51         mean(0)
52     phase = 2 * np.pi * np.arange(144) / 144
53     predicted = base[:, channel]
54     return np.column_stack((np.ones(144), predicted / 1000,
55         (yesterday - predicted) / 1000, (weekly - predicted) / 1000,
56         (last3 - predicted) / 1000, base[:, 1 - channel] / 1000,
57         np.sin(phase), np.cos(phase), np.sin(2 * phase), np.cos(2 * phase)))
58
59 def _calibrate(actual, origins, base, index, config):
60     day = int(origins[index] // 144)
61     cutoff = day * 144
62     method, window = config["method"], config["window"]
63     history = np.arange(max(0, index - window), index, dtype=int)
64     info = {"day": day, "origin": cutoff, "method": method,
65         "window_days": window, "history_count": len(history),
66         "history_origins": origins[history].astype(int).tolist(),
67         "max_actual_index": cutoff - 1,
68         "history_last_label": int(origins[history[-1]] + 143) if len(history) else None,
69         "current_truth_used": False}
70     if method == "none" or not len(history):
71         info["cold_start"] = not len(history)
72     return base[index].copy(), info
73     observed = np.stack([_history(actual, int(origins[h]), int(origins[h]) + 144, cutoff)
74         for h in history])
75     residual = observed - base[history]
76     # Half-life is fixed at half of the chosen history window.
77     weights = 2 ** (-np.arange(len(history) - 1, -1, -1) / max(1, window / 2))
78     delta = np.zeros((144, 2))
79     if method in ("global_bias", "hour_bias"):
80         for channel in range(2):
81             valid = np.ones_like(residual[:, :, channel], dtype=bool)
82             if channel == 1:
83                 # Nighttime zeros must not dilute a daylight-only PV correction.
84                 valid = (base[history, :, 1] > 1) | (observed[:, :, 1] > 1)
85             numerator = np.sum(weights[:, None] * residual[:, :, channel] * valid)
86             denominator = np.sum(weights[:, None] * valid)
87             global_bias = float(numerator / max(denominator, 1))
88             if method == "global_bias":
89                 delta[:, channel] = global_bias
90             else:
91                 for hour in range(24):
92                     sl = slice(hour * 6, (hour + 1) * 6)
93                     w = weights[:, None] * valid[:, sl]
94                     # Equivalent to three days of shrinkage toward global bias.
95                     prior_weight = 3 * 6 * weights.mean()
96                     estimate = (np.sum(w * residual[:, sl, channel]) + prior_weight * global_bias)
97                     delta[sl, channel] = estimate / (w.sum() + prior_weight)
98     delta *= len(history) / (len(history) + 3)
99     elif method == "ridge":
100         coefficients = []
101         for channel in range(2):
102             x = np.concatenate([_features(actual, int(origins[h] // 144), base[h], channel)

```

```

103         for h in history])
104     target = residual[:, :, channel].ravel() / 1000
105     w = np.repeat(weights, 144)
106     if channel == 1:
107         active = ((base[history, :, 1] > 1) | (observed[:, :, 1] > 1)).ravel()
108         w *= np.where(active, 1., .1)
109     penalty = np.full(x.shape[1], 20.)
110     penalty[0] = 2.
111     gram = np.einsum("ni,nj,n->ij", x, x, w, optimize=False)
112     rhs = np.einsum("ni,n,n->i", x, w, target, optimize=False)
113     coefficient = np.linalg.solve(gram + np.diag(penalty), rhs)
114     xt = _features(actual, day, base[index], channel)
115     correction = 1000 * np.einsum("ni,i->n", xt, coefficient, optimize=False)
116     bound = max(100., 3 * float(np.sqrt(np.mean(residual[:, :, channel] ** 2))))
117     delta[:, channel] = np.clip(correction, -bound, bound)
118     coefficients.append(coefficient.tolist())
119     info["coefficients"] = coefficients
120 else:
121     raise ValueError(f"unknown calibration method: {method}")
122 output = np.maximum(0., base[index] + delta)
123 output[:, 1] *= _daylight(actual, day)
124 info.update(cold_start=False, mean_output_delta_kw=(output - base[index]).mean(0).tolist())
125 return output, info
126
127
128 class CalibratedStore:
129     """Compatible with exp004 ForecastStore: "get(origin)", values, origins.
130
131     Passing a data object always recomputes rather than reading saved output,
132     permitting independent future-perturbation checks. Saved caches are signed.
133     """
134     def __init__(self, name="hour_bias_28", seed=42, data=None, directory=OUT,
135                  use_cache=True, _base_store=None, _candidate_values=None):
136         if name not in CONFIG:
137             raise ValueError(f"unknown candidate {name}; choose {tuple(CONFIG)}")
138         self.name, self.seed = name, int(seed)
139         self.base_store = _base_store or ForecastStore("no_season", seed=self.seed)
140         self.origins = self.base_store.origins.copy()
141         self.base_values = self.base_store.values.copy()
142         self.lookup = {int(origin): i for i, origin in enumerate(self.origins)}
143         self.config = CONFIG[name]
144         cache = Path(directory) / f"{name}_seed_{self.seed}.npz"
145         meta_path = cache.with_suffix(".json")
146         if data is None and use_cache and cache.exists() and meta_path.exists():
147             meta = json.loads(meta_path.read_text())
148             if meta["source_sha256"] == source_hash() and meta["base_sha256"] == base_hash(self.
149 base_values):
150                 if meta["archive_sha256"] != hashlib.sha256(cache.read_bytes()).hexdigest():
151                     raise RuntimeError("calibration archive hash mismatch")
152                 with np.load(cache) as pack:
153                     self.values = pack["values"].copy()
154                     self.delta = pack["delta"].copy()
155                     self.errors_kw = pack["errors_kw"].copy()
156                 self.audit = meta["days"]
157                 return
158         actual = (Data() if data is None else data).actual
159         if self.config["method"] == "online_select":
160             candidates = _candidate_values
161             if candidates is None:
162                 candidates = {c["name"]: CalibratedStore(c["name"], self.seed, data,
163                 directory, use_cache, _base_store=self.base_store).values
164                 for c in CANDIDATES if c["method"] != "online_select"}
165             self.values, self.audit = _online_select(actual, self.origins, candidates, self.config)
166         else:
167             outputs = [_calibrate(actual, self.origins, self.base_values, index, self.config)
168             for index in range(len(self.origins))]
169             self.values = np.stack([r[0] for r in outputs])
170             self.audit = [r[1] for r in outputs]
171             self.delta = self.values - self.base_values
172             # These arrays are post-hoc scoring labels, never calibration inputs.

```

```

172     truth = actual[self.origins[:, None] + np.arange(144)][..., :2]
173     self.errors_kw = truth - self.values
174
175     def get(self, origin):
176         if origin not in self.lookup:
177             raise ValueError("requires a formal February-December midnight")
178         return self.values[self.lookup[int(origin)]].copy()
179
180     def completed_error_paths(self, origin, limit=28):
181         self.get(origin)
182         ids = np.flatnonzero(self.origins + 144 <= origin)[-limit:]
183         return {"origins": self.origins[ids].copy(), "errors_kw": self.errors_kw[ids].copy(),
184                 "information_cutoff": int(origin), "variant": self.name,
185                 "exploratory": False, "development_on_evaluation_year": True}
186
187
188     def _online_select(actual, origins, candidates, config):
189         """Select today's calibrator by prior complete-day prequential net MSE."""
190         names = list(candidates)
191         values, audits = [], []
192         for i, origin in enumerate(origins):
193             indices = np.arange(max(0, i - config["window"]), i)
194             scores = {}
195             if len(indices) >= 3:
196                 truth = np.stack([_history(actual, int(origins[h]), int(origins[h]) + 144, int(origin))
197                                   for h in indices])
198                 net_truth = truth[:, :, 0] - truth[:, :, 1]
199                 for name, array in candidates.items():
200                     net = array[indices, :, 0] - array[indices, :, 1]
201                     scores[name] = float(np.mean((net - net_truth) ** 2))
202                 selected = min(names, key=lambda name: scores[name])
203             else:
204                 selected = "base"
205             values.append(candidates[selected][i])
206             audits.append({"day": int(origin // 144), "origin": int(origin),
207                           "selected": selected, "historical_net_mse": scores,
208                           "selection_label_origins": origins[indices].astype(int).tolist(),
209                           "current_truth_used": False,
210                           "history_last_label": int(origins[indices[-1]] + 143) if len(indices) else None})
211         return np.stack(values), audits
212
213
214     def source_hash():
215         return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
216
217
218     def base_hash(values):
219         return hashlib.sha256(np.ascontiguousarray(values).tobytes()).hexdigest()
220
221
222     def metrics(prediction, truth):
223         output = []
224         for target in ("load", "pv", "net_load"):
225             p = prediction[:, :, 0] - prediction[:, :, 1] if target == "net_load" else prediction[:, :,
226             int(target == "pv")]
227             y = truth[:, :, 0] - truth[:, :, 1] if target == "net_load" else truth[:, :, int(target == "
228             pv")]
229             e = p - y
230             output.append({"target": target, "n": e.size, "unit": "kW",
231                           "mae": float(np.abs(e).mean()), "rmse": float(np.sqrt(np.square(e).mean())),
232                           "bias": float(e.mean()), "wapec_pct": float(100 * np.abs(e).sum() / np.abs(y).sum()),
233                           "absolute_error_sum": float(np.abs(e).sum()), "squared_error_sum": float(np.square(e).
234                           sum())})
235         return output
236
237
238     def run(seed=42, out=OUT):
239         out = Path(out)
240         out.mkdir(parents=True, exist_ok=True)
241         began = time.perf_counter()

```

```

239 data, base_store = Data(), ForecastStore("no_season", seed=seed)
240 # This list is written before evaluating any candidate.
241 (out / "candidate_protocol.json").write_text(json.dumps({"seed": seed,
242     "candidates": CANDIDATES, "forecast_structure": "unchanged_frozen_no_season_CNN",
243     "evaluation_role": "2025_development_exploration_not_independent_test",
244     "selection_boundary": "all_fit_or_online_selection_labels_are_prior_complete_days",
245     "source_sha256": source_hash()}, indent=2) + "\n")
246 all_rows, daily_rows, monthly_rows, candidate_values = [], [], [], {}
247 truth = data.actual[base_store.origins[:, None] + np.arange(144)][..., :2]
248 dates = pd.date_range("2025-02-01", "2025-12-31")
249 for config in CANDIDATES:
250     t = time.perf_counter()
251     name = config["name"]
252     store = CalibratedStore(name, seed, data, out, False, base_store, candidate_values)
253     seconds = time.perf_counter() - t
254     candidate_values[name] = store.values
255     path = out / f"{name}_seed_{seed}.npz"
256     np.savez_compressed(path, origins=store.origins, values=store.values,
257         delta=store.delta, base_values=store.base_values, errors_kw=store.errors_kw)
258     path.with_suffix(".json").write_text(json.dumps({"config": config, "seed": seed,
259         "source_sha256": source_hash(), "base_sha256": base_hash(store.base_values),
260         "archive_sha256": hashlib.sha256(path.read_bytes()).hexdigest(),
261         "source_data_sha256": data.hashes, "days": store.audit,
262         "delta_definition": "
263         final_output_minus_frozen_CNN_including_nonnegative_and_daylight_projection",
264         "error_definition": "actual_minus_calibrated_prediction;
265         labels_for_scoring_after_release",
266         "generation_seconds": seconds,
267         "evaluation_role": "development_year_exploration_not_independent_test"}, indent=2) + "\n
268 ")
269     for row in metrics(store.values, truth):
270         all_rows.append({"name": name, "seconds": seconds, **row})
271     for i, date in enumerate(dates):
272         for row in metrics(store.values[i:i+1], truth[i:i+1]):
273             daily_rows.append({"name": name, "date": str(date.date()), **row})
274     for month in range(2, 13):
275         ids = dates.month == month
276         for row in metrics(store.values[ids], truth[ids]):
277             monthly_rows.append({"name": name, "month": month, **row})
278     print(json.dumps({"name": name, "seconds": seconds, "net": all_rows[-1]}, ensure_ascii=False
279 ), flush=True)
280 pd.DataFrame(all_rows).to_csv(out / "annual_metrics.csv", index=False)
281 pd.DataFrame(daily_rows).to_csv(out / "daily_metrics.csv", index=False)
282 pd.DataFrame(monthly_rows).to_csv(out / "monthly_metrics.csv", index=False)
283 summary = {"complete": True, "seconds": time.perf_counter() - began,
284     "candidate_count": len(CANDIDATES), "period": ["2025-02-01", "2025-12-31"],
285     "source_sha256": source_hash(), "role": "development_exploration",
286     "independent_test": False, "cnn_retrained": False}
287 (out / "completion.json").write_text(json.dumps(summary, indent=2) + "\n")
288 return summary
289
290 if __name__ == "__main__":
291     parser = argparse.ArgumentParser()
292     parser.add_argument("--seed", type=int, default=42)
293     parser.add_argument("--out", type=Path, default=OUT)
294     args = parser.parse_args()
295     run(args.seed, args.out)

```

experiments/exp008/forecast_hgb_extra_trees_half.py

```

1 """One fixed 50/50 raw HGB/ExtraTrees blend, then shared causal postprocessing.
2
3 Both raw channels and all dates use the same predeclared weight. This candidate
4 was designed after development results were examined; it is not an untouched
5 test. No model fitting, blend-weight selection, or daily channel selection.
6 """

```

```

7 from __future__ import annotations
8
9 import copy
10 import json
11 from pathlib import Path
12 import shutil
13
14 import numpy as np
15 import pandas as pd
16
17 from experiments.exp008.forecast_absolute_hgb import (
18     OUT as HGB, AbsoluteHGBStore, ArrayStore, array_hash, digest, save,
19 )
20 from experiments.exp008.forecast_absolute_extra_trees import (
21     OUT as EXTRA, AbsoluteExtraTreesStore,
22 )
23 from experiments.exp008.forecast_calibration import CalibratedStore, _calibrate, CONFIG
24 from experiments.exp008.load_energy_memory import EnergyMemoryStore, correct_day
25 from experiments.exp008.forecast_shape_diagnostic import summary as score
26 from experiments.exp008.audited_store_bridge import run as bridge
27 from experiments.problem2.exp003.data import Data, ROOT
28
29 OUT = ROOT / 'data/results/exp008/forecast_hgb_extra_trees_half'
30 COMPOSITE = ROOT / 'data/results/exp008/forecast_channel_composite'
31 RAW = 'hgb_extra_trees_half_raw'
32 RIDGE = 'hgb_extra_trees_half_ridge28'
33 PRIMARY = 'hgb_extra_trees_half_ridge28_memory'
34 WEIGHT = .5
35 GATE_KEYS = ('rmse_kw', 'high_price_rmse_kw', 'daily_energy_rmse_kwh',
36             'cumulative_error_rmse_kwh')
37
38
39 class HGBExtraTreesHalfStore(ArrayStore):
40     def __init__(self, stage=PRIMARY):
41         meta = json.loads((OUT / 'provenance.json').read_text())
42         path = OUT / f'{stage}.npz'
43         if not meta['complete'] or digest(path) != meta['archives'][stage]:
44             raise ValueError('The blend archive is incomplete or changed')
45         with np.load(path) as z:
46             super().__init__(z['values'], z['origins'], stage)
47
48
49 def prepare_protocol():
50     if OUT.exists():
51         raise FileExistsError('Existing evidence is immutable')
52     inputs = [directory / name for directory in (HGB, EXTRA)
53              for name in ('protocol.json', 'provenance.json', 'training_audit.json',
54                          'causality_verification.json', 'independent_verification.json')]
55     inputs += [HGB / 'direct_hgb_raw.npz', HGB / 'direct_hgb_ridge28_memory_half.npz',
56              EXTRA / 'absolute_extra_trees_raw.npz',
57              EXTRA / 'absolute_extra_trees_ridge28_memory_half.npz',
58              EXTRA / 'resume_verification.json', COMPOSITE / 'memory.npz',
59              COMPOSITE / 'summary.json']
60     sources = [Path(__file__).resolve(),
61               ROOT / 'experiments/exp008/forecast_absolute_hgb.py',
62               ROOT / 'experiments/exp008/forecast_absolute_extra_trees.py',
63               ROOT / 'experiments/exp008/forecast_calibration.py',
64               ROOT / 'experiments/exp008/load_energy_memory.py',
65               ROOT / 'experiments/exp008/forecast_shape_diagnostic.py',
66               ROOT / 'experiments/exp008/audited_store_bridge.py']
67     protocol = {
68         'candidate_count': 1, 'model_id': PRIMARY, 'raw_weight_HGB': WEIGHT,
69         'raw_weight_ExtraTrees': 1-WEIGHT, 'both_channels_same_weight_all334days': True,
70         'raw_blend': '.5 * signed original absolute HGB raw + .5 * signed ExtraTrees raw',
71         'postprocessing': 'same joint-channel Ridge28 fitted on this blended raw archive, then same
72         nonrecursive .5 load memory',
73         'no_postprocessed_output_blending': True,
74         'no_weight_tuning_or_model_refitting_or_daywise_selection': True,
75         'origin_range': '2025-02-01 through 2025-12-31, 334 complete days',
76         'design_context': 'fixed blend proposed after observing development component metrics',

```

```

76     'development_not_independent_test': True,
77     'raw_model_causality': 'references signed independent audits for 22 original HGB and 22
ExtraTrees monthly models; no retraining here',
78     'postprocessing_mutation_days': [31,32,59,90,151,243,364],
79     'continuation_gate': {'keys': GATE_KEYS, 'comparator': 'original absolute HGB full pipeline',
80                           'rule': 'all four metrics strictly improve and verification passes'},
81     'conditional_bridge': 'one full334 unchanged tree28/q.8/state_buffer500 using own issued
errors',
82     'full_physical_planning': False, 'final_model_selection': False,
83     'input_sha256': {str(p): digest(p) for p in inputs},
84     'source_sha256': {str(p): digest(p) for p in sources},
85     'source_data_sha256': Data().hashes,
86 }
87 OUT.mkdir(parents=True)
88 save(OUT / 'protocol.json', protocol)
89 for p in sources:
90     target = OUT / 'source_archive' / p.relative_to(ROOT)
91     target.parent.mkdir(parents=True, exist_ok=True)
92     shutil.copy2(p, target)
93 return protocol
94
95
96 def source_checks(protocol):
97     for key in ('source_sha256', 'input_sha256'):
98         assert all(digest(path) == sha for path, sha in protocol[key].items())
99     for directory in (HGB, EXTRA):
100         assert json.loads((directory / 'independent_verification.json').read_text())['passed']
101         assert json.loads((directory / 'causality_verification.json').read_text())['passed']
102
103
104 def main():
105     protocol = prepare_protocol()
106     source_checks(protocol)
107     hgb = AbsoluteHGBStore('direct_hgb_raw')
108     extra = AbsoluteExtraTreesStore('absolute_extra_trees_raw')
109     np.testing.assert_array_equal(hgb.origins, extra.origins)
110     np.testing.assert_array_equal(hgb.origins, np.arange(31,365)*144)
111     values = WEIGHT*hgb.values+(1-WEIGHT)*extra.values
112     raw = ArrayStore(values, hgb.origins.copy(), RAW)
113     data = Data()
114     ridge = CalibratedStore('ridge_28', data=data, use_cache=False,
115                             _base_store=raw, directory=OUT/'unused_cache')
116     memory = EnergyMemoryStore(data=data, base_store=ridge)
117     ridge.name, memory.name = RIDGE, PRIMARY
118     for row in ridge.audit:
119         row['actual_base_model_id'] = RAW
120     for row in memory.audit:
121         row['underlying_forecast'] = 'fixed_raw_HGB_ExtraTrees_half_then_Ridge28'
122     truth = data.actual[raw.origins[:,None]+np.arange(144), :2]
123     archives = {}
124     for store in (raw, ridge, memory):
125         path = OUT / f'{store.name}.npz'
126         np.savez_compressed(path, values=store.values, origins=store.origins,
127                             errors_kw=truth-store.values)
128         archives[store.name] = digest(path)
129     save(OUT/'ridge_audit.json', ridge.audit)
130     save(OUT/'memory_audit.json', memory.audit)
131     checks = []
132     for day in protocol['postprocessing_mutation_days']:
133         i = day-31
134         changed = copy.copy(data)
135         changed.actual = data.actual.copy()
136         changed.actual[day*144:] += [70000.,30000.]
137         future_hgb, future_extra = hgb.values.copy(), extra.values.copy()
138         future_hgb[i+1:] += [80000.,20000.]
139         future_extra[i+1:] += [30000.,90000.]
140         future_raw = WEIGHT*future_hgb+(1-WEIGHT)*future_extra
141         np.testing.assert_array_equal(future_raw[:i+1], raw.values[:i+1])
142         a,_ = _calibrate(data.actual, raw.origins, raw.values, i, CONFIG['ridge_28'])
143         b,_ = _calibrate(changed.actual, raw.origins, future_raw, i, CONFIG['ridge_28'])

```

```

144     np.testing.assert_array_equal(a, b)
145     future_ridge = ridge.values.copy()
146     future_ridge[i+1:] += [40000., 90000.]
147     a,_ = correct_day(data.actual, raw.origins, ridge.values, i)
148     b,_ = correct_day(changed.actual, raw.origins, future_ridge, i)
149     np.testing.assert_array_equal(a, b)
150     checks.append({'day':day, 'asymmetric_future_actual_and_both_source_forecasts':True,
151                   'raw_prefix_ridge_and_memory_exactly_unchanged':True})
152     assert all(row['history_last_label'] is None or row['history_last_label'] < row['origin']
153               for row in ridge.audit)
154     assert all(row['prior_label_end_exclusive'] is None or row['prior_label_end_exclusive'] <= row['
155               origin']
156               for row in memory.audit)
157     save(OUT/'causality_verification.json', {'passed':True, 'checks':checks,
158       'all334_postprocessing_labels_before_issue':True,
159       'raw_training_causality_inherited_from_signed_model_audits':True})
160     with np.load(COMPOSITE/'memory.npz') as z:
161         np.testing.assert_array_equal(z['origins'], raw.origins)
162         composite = z['values'].copy()
163     metrics = []
164     for name,v in [('original_HGB_full', AbsoluteHGBStore().values),
165                   ('ExtraTrees_full', AbsoluteExtraTreesStore().values),
166                   ('best_fixed_channel_composite_full', composite),
167                   (RAW, raw.values), (RIDGE, ridge.values), (PRIMARY, memory.values)]:
168         for channel,e in [('load',v[:, :,0]-truth[:, :,0]), ('pv',v[:, :,1]-truth[:, :,1]),
169                           ('net', (v[:, :,0]-v[:, :,1])-(truth[:, :,0]-truth[:, :,1]))]:
170             metrics.append({'name':name, 'channel':channel, **score(e,data.fixed_price)})
171     pd.DataFrame(metrics).to_csv(OUT/'metrics.csv', index=False)
172     net = {r['name']:r for r in metrics if r['channel']=='net'}
173     gate_checks = {k:net[PRIMARY][k] < net['original_HGB_full'][k] for k in GATE_KEYS}
174     gate = all(gate_checks.values())
175     source_checks(protocol)
176     save(OUT/'provenance.json', {'complete':True, 'model_id':PRIMARY, 'archives':archives,
177       'raw_HGB_values_sha256':array_hash(hgb.values),
178       'raw_ExtraTrees_values_sha256':array_hash(extra.values),
179       'raw_exact_half_blend':True, 'protocol_sha256':digest(OUT/'protocol.json'),
180       'source_and_input_hashes_unchanged':True})
181     save(OUT/'summary.json', {'complete':True, 'metrics':metrics,
182       'predictive_gate_passed':gate, 'gate_checks':gate_checks,
183       'causality_verified':True, 'source_hashes_verified':True,
184       'development_not_independent_test':True, 'final_model_selection':False})
185     print(json.dumps({'net_metrics':net, 'gate':gate}, indent=2), flush=True)
186     if gate:
187         bridge(memory, OUT/'lp_bridge', [OUT/'summary.json', OUT/'protocol.json',
188           OUT/'f'{PRIMARY}.npz', OUT/'causality_verification.json',
189           HGB/'provenance.json', EXTRA/'provenance.json'])
190     source_checks(protocol)
191
192 if __name__ == '__main__':
193     main()

```

experiments/exp008/forecast_net_hgb.py

```

1  """One fixed monthly HGB net-demand residual model, with issue-time features.
2
3  The existing CNN stays a baseline and input. This is a bounded development
4  experiment, not an independent test or a replacement of historical archives.
5  The returned two-column values are a NET DEMAND ADAPTER, not separate improved
6  load/PV point forecasts: the learned net correction is assigned to load.
7  """
8  from __future__ import annotations
9
10 import argparse
11 import hashlib
12 import json
13 import time

```

```

14 from pathlib import Path
15
16 import joblib
17 import numpy as np
18 import pandas as pd
19 from sklearn.ensemble import HistGradientBoostingRegressor
20 from threadpoolctl import threadpool_limits
21
22 from experiments.problem2.exp003.data import Data, ROOT, EPOCH
23 from experiments.problem2.exp004.predict import ForecastStore
24
25 OUT = ROOT / "data/results/exp008/forecast_net_hgb"
26 MODEL_CONFIG = dict(loss="squared_error", max_leaf_nodes=15, max_iter=100,
27                     min_samples_leaf=50, l2_regularization=2., learning_rate=.08,
28                     early_stopping=False, random_state=42)
29 HISTORY_HALF_LIFE_DAYS = 90
30
31
32 def _before(data, first, last, issue):
33     if not 0 <= first <= last <= issue:
34         raise ValueError("feature actuals must be strictly before their historical issue")
35     return np.asarray(data.actual[first:last, :2], dtype=float)
36
37
38 def _triple(power):
39     return np.column_stack((power, power[:, 0] - power[:, 1]))
40
41
42 def features_for_day(data, day, cnn_or_periodic):
43     """All historical feature windows are anchored at this day's own issue."""
44     if not 7 <= day < 365:
45         raise ValueError("seven complete days required")
46     issue = int(day) * 144
47     history = _before(data, issue - 7 * 144, issue, issue).reshape(7, 144, 2)
48     blocks = [_triple(cnn_or_periodic), _triple(history[-1]), _triple(history[0]),
49             _triple(history[-3:].mean(0)), _triple(history.mean(0)),
50             _triple(history.std(0))]
51     names = [f"{group}_{channel}" for group in
52             ("base", "yesterday", "previous_week", "mean3", "mean7", "std7")
53             for channel in ("load", "pv", "net")]
54     # std7_net above is a difference of component standard deviations, rather
55     # than std(net); replace it with the actual historical net standard deviation.
56     blocks[-1][:, 2] = (history[:, :, 0] - history[:, :, 1]).std(0)
57     phase = 2 * np.pi * np.arange(144) / 144
58     weekday = (day + 2) % 7 # 2025-01-01 was Wednesday, Monday=0.
59     calendar = np.column_stack((np.sin(phase), np.cos(phase), np.sin(2 * phase),
60     np.cos(2 * phase), np.full(144, np.sin(2 * np.pi * weekday / 7)),
61     np.full(144, np.cos(2 * np.pi * weekday / 7))))
62     names += ["slot_sin", "slot_cos", "slot_sin2", "slot_cos2", "weekday_sin", "weekday_cos"]
63     yesterday = history[-1]
64     statistics = np.r_[yesterday.mean(0), yesterday.std(0), yesterday.min(0), yesterday.max(0),
65     (yesterday[:, 0] - yesterday[:, 1]).mean(),
66     (yesterday[:, 0] - yesterday[:, 1]).std()]
67     names += [f"yesterday_{stat}_{channel}" for stat in ("mean", "std", "min", "max")
68             for channel in ("load", "pv")]
69     names += ["yesterday_mean_net", "yesterday_std_net", "base_is_cnn"]
70     features = np.column_stack((*blocks, calendar,
71     np.broadcast_to(statistics, (144, len(statistics))), np.full(144, day >= 31)))
72     return features, names
73
74
75 def bases_and_features(data, store):
76     days = np.arange(7, 365)
77     bases, arrays, feature_names = [], [], None
78     for day in days:
79         base = store.get(int(day) * 144) if day >= 31 else data.baseline(int(day) * 144)
80         features, feature_names = features_for_day(data, int(day), base)
81         bases.append(base)
82         arrays.append(features)
83     return days, np.stack(bases), np.stack(arrays), feature_names

```

```

84
85
86 def fit_month(data, month, days, bases, features, seed=42):
87     asof = int((pd.Timestamp(2025, month, 1) - EPOCH).days)
88     ids = np.flatnonzero(days < asof)
89     train_days = days[ids]
90     truth = np.stack([_before(data, int(day) * 144, (int(day) + 1) * 144, asof * 144)
91                       for day in train_days])
92     target = ((truth[:, :, 0] - truth[:, :, 1]) - (bases[ids, :, 0] - bases[ids, :, 1])).ravel()
93     weights = np.repeat(2 * (- (asof - 1 - train_days) / HISTORY_HALF_LIFE_DAYS), 144)
94     weights /= weights.mean()
95     config = {**MODEL_CONFIG, "random_state": int(seed)}
96     model = HistGradientBoostingRegressor(**config)
97     started = time.perf_counter()
98     with threadpool_limits(limits=1):
99         model.fit(features[ids].reshape(-1, features.shape[-1]), target, sample_weight=weights)
100     audit = {"month": month, "asof_day": asof, "information_cutoff_exclusive": asof * 144,
101             "training_days": train_days.astype(int).tolist(), "training_rows": len(target),
102             "training_last_label": asof * 144 - 1,
103             "january_periodic_rows": int(np.sum(train_days < 31) * 144),
104             "historical_frozen_cnn_rows": int(np.sum(train_days >= 31) * 144),
105             "iterations": int(model.n_iter_), "training_seconds": time.perf_counter() - started,
106             "config": config, "early_stopping": False}
107     return model, audit
108
109
110 def score(net, truth):
111     e = net - truth
112     return {"target": "net_load", "unit": "kW", "n": e.size,
113            "mae": float(np.abs(e).mean()), "rmse": float(np.sqrt(np.square(e).mean())),
114            "bias_predicted_minus_actual": float(e.mean()),
115            "absolute_error_sum": float(np.abs(e).sum()), "squared_error_sum": float(np.square(e).sum())
116            }
117
118 def run(seed=42, out=OUT):
119     out = Path(out)
120     out.mkdir(parents=True, exist_ok=True)
121     models_dir = out / "models"
122     models_dir.mkdir(exist_ok=True)
123     began = time.perf_counter()
124     data, cnn = Data(), ForecastStore("no_season", seed=seed)
125     protocol = {"model": {**MODEL_CONFIG, "random_state": seed}, "cnn_retrained": False,
126               "one_fixed_candidate": True, "no_annual_actual_feature": True,
127               "training": "monthly_refit_all_prior_completed_days; historical features obey historical
128               issue",
129               "history_weight_half_life_days": HISTORY_HALF_LIFE_DAYS,
130               "january": "causal_weekly_load_yesterday_PV_cold_start_days7_to30",
131               "official_pv_allowed": False, "future_realized_price_allowed": False,
132               "development_year": "2025_already_examined_not_independent_test",
133               "continuation_gate": "only continue mechanism if net RMSE meaningfully improves on ridge
134               about388kW"}
135     (out / "protocol.json").write_text(json.dumps(protocol, indent=2) + "\n")
136     days, bases, features, names = bases_and_features(data, cnn)
137     net_delta = np.empty((334, 144))
138     training, audits = [], []
139     for month in range(2, 13):
140         model, audit = fit_month(data, month, days, bases, features, seed)
141         model_path = models_dir / f"m{month:02d}_seed{seed}.joblib"
142         joblib.dump(model, model_path)
143         audit["model_sha256"] = hashlib.sha256(model_path.read_bytes()).hexdigest()
144         training.append(audit)
145         begin = audit["asof_day"]
146         stop = int((pd.Timestamp(2025, month, 1) + pd.offsets.MonthBegin(1) - EPOCH).days)
147         test_days = np.arange(begin, stop)
148         with threadpool_limits(limits=1):
149             prediction = model.predict(features[test_days - 7].reshape(-1, len(names))).reshape(-1,
150             144)
151         net_delta[test_days - 31] = prediction
152         for day in test_days:

```

```

150         audits.append({"day": int(day), "origin": int(day * 144), "month_model": month,
151                        "model_information_cutoff_exclusive": begin * 144,
152                        "model_training_last_label": begin * 144 - 1,
153                        "feature_last_actual_index": int(day * 144 - 1),
154                        "base_source": "frozen_exp004_no_season", "current_truth_used": False})
155     print(json.dumps({"month": month, "training_seconds": audit["training_seconds"],
156                      "training_rows": audit["training_rows"]}), flush=True)
157     base = cnn.values.copy()
158     net = base[:, :, 0] - base[:, :, 1] + net_delta
159     # Preserve the predicted NET exactly while keeping adapter channels nonnegative.
160     adapter = base.copy()
161     adapter[:, :, 0] += net_delta
162     below_zero = np.minimum(adapter[:, :, 0], 0)
163     adapter[:, :, 0] -= below_zero
164     adapter[:, :, 1] -= below_zero
165     truth_power = data.actual[cnn.origins[:, None] + np.arange(144)]
166     truth_net = truth_power[:, :, 0] - truth_power[:, :, 1]
167     errors = truth_power - adapter
168     path = out / "predictions.npz"
169     np.savez_compressed(path, origins=cnn.origins, values=adapter, base_values=base,
170                        net_kw=net, net_delta_kw=net_delta, delta=adapter - base, errors_kw=errors,
171                        net_errors_kw=truth_net - net)
172     dates = pd.date_range("2025-02-01", "2025-12-31")
173     annual = [{"name": "cnn_base", **score(base[:, :, 0] - base[:, :, 1], truth_net)},
174              {"name": "cnn_net_hgb", **score(net, truth_net)}]
175     pd.DataFrame(annual).to_csv(out / "annual_metrics.csv", index=False)
176     pd.DataFrame({"date": str(date.date()), **score(net[i:i+1], truth_net[i:i+1])}
177                  for i, date in enumerate(dates)).to_csv(out / "daily_metrics.csv", index=False)
178     pd.DataFrame({"month": month, **score(net[dates.month == month], truth_net[dates.month == month
179 ])})
180     for month in range(2, 13)).to_csv(out / "monthly_metrics.csv", index=False)
181     (out / "training_audit.json").write_text(json.dumps(training, indent=2) + "\n")
182     (out / "prediction_audit.json").write_text(json.dumps(audits, indent=2) + "\n")
183     metadata = {"complete": True, "period": ["2025-02-01", "2025-12-31"], "seed": seed,
184                "source_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
185                "base_sha256": hashlib.sha256(cnn.values.tobytes()).hexdigest(),
186                "archive_sha256": hashlib.sha256(path.read_bytes()).hexdigest(), "source_data_sha256": data.
187                hashes,
188                "features": names, "feature_count": len(names), "training_models": 11,
189                "seconds": time.perf_counter() - began, "metrics": annual,
190                "semantics": "values are nonnegative net-demand adapter channels; do not score load/PV
191                separately as improved component forecasts",
192                "net_rmse_below_ridge388": annual[-1]["rmse"] < 388,
193                "development_exploration_not_independent_test": True}
194     (out / "provenance.json").write_text(json.dumps(metadata, indent=2) + "\n")
195     print(json.dumps(metadata, ensure_ascii=False), flush=True)
196     return metadata
197
198 class NetHGBStore:
199     """Signed archive adapter compatible with ForecastStore.get(origin)."""
200     def __init__(self, seed=42, directory=OUT):
201         directory = Path(directory)
202         meta = json.loads((directory / "provenance.json").read_text())
203         path = directory / "predictions.npz"
204         if (not meta["complete"] or meta["seed"] != seed
205             or meta["archive_sha256"] != hashlib.sha256(path.read_bytes()).hexdigest()):
206             raise RuntimeError("HGB forecast archive is incomplete, changed or a different seed")
207         with np.load(path) as z:
208             for name in ("values", "origins", "base_values", "net_kw", "net_delta_kw", "errors_kw"):
209                 setattr(self, name, z[name].copy())
210             self.lookup = {int(origin): i for i, origin in enumerate(self.origins)}
211             self.seed = seed
212
213     def get(self, origin):
214         if origin not in self.lookup:
215             raise ValueError("formal midnight origin required")
216         return self.values[self.lookup[int(origin)]].copy()
217
218     def completed_error_paths(self, origin, limit=28):

```

```

217         self.get(origin)
218         ids = np.flatnonzero(self.origins + 144 <= origin)[-limit:]
219         return {"origins": self.origins[ids].copy(), "errors_kw": self.errors_kw[ids].copy(),
220                "information_cutoff": int(origin), "variant": "cnn_net_hgb",
221                "net_adapter_not_component_forecasts": True, "development_on_evaluation_year": True}
222
223
224 if __name__ == "__main__":
225     parser = argparse.ArgumentParser()
226     parser.add_argument("--seed", type=int, default=42)
227     parser.add_argument("--out", type=Path, default=OUT)
228     args = parser.parse_args()
229     run(args.seed, args.out)

```

experiments/exp008/forecast_override_adapter.py

```

1  """Identity wrapper for store-backed forecast overrides; no model retraining.
2
3  The same wrapped get method feeds current issues and issued_error_paths.
4  January remains its explicitly labelled periodic cold start. The wrapper
5  does not load an old residual cache or change any prediction numerically.
6  """
7  import hashlib
8  import inspect
9  from pathlib import Path
10
11 import numpy as np
12
13
14 def array_sha256(values):
15     return hashlib.sha256(np.ascontiguousarray(values).tobytes()).hexdigest()
16
17
18 class AuditedForecastOverride:
19     def __init__(self, forecasts, model_id, source_paths=()):
20         if not isinstance(model_id, str) or not model_id.strip():
21             raise ValueError('An explicit model_id is required')
22         self.forecasts = forecasts
23         self.model_id = model_id
24         self._values_hash = array_sha256(forecasts.store.values)
25         self._origins_hash = array_sha256(forecasts.store.origins)
26         self.source_paths = tuple(Path(p).resolve() for p in source_paths)
27         self._source_hashes = {str(p): hashlib.sha256(p.read_bytes()).hexdigest()
28                                for p in self.source_paths}
29
30     def __getattr__(self, name):
31         return getattr(self.forecasts, name)
32
33     def forecast_identity(self):
34         # Fingerprint the loaded prediction values, not only the .npz name.
35         # Source artifacts must also retain the version loaded by this object.
36         for path in self.source_paths:
37             if hashlib.sha256(path.read_bytes()).hexdigest() != self._source_hashes[str(path)]:
38                 raise ValueError(f'Forecast provenance artifact changed: {path}')
39         method_path = Path(inspect.getfile(type(self.forecasts))).resolve()
40         return {'model_id': self.model_id,
41                'prediction_values_sha256': self._values_hash,
42                'prediction_origins_sha256': self._origins_hash,
43                'prediction_values_shape': list(self.store.values.shape),
44                'prediction_values_dtype': str(self.store.values.dtype),
45                'prediction_origins_dtype': str(self.store.origins.dtype),
46                'implementation_class': f'{type(self.forecasts).__module__}.{type(self.forecasts).__name__}',
47                'implementation_source': str(method_path),
48                'implementation_source_sha256': hashlib.sha256(method_path.read_bytes()).hexdigest(),
49                'wrapper_source_sha256': hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
50                'source_artifact_sha256': self._source_hashes.copy(),

```

```

51         'seed': int(self.seed), 'calibration': getattr(self.forecasts, 'calibration', None),
52         'history_source': 'same wrapped get at each prior completed target day and same legal
issue',
53         'January_history': 'unchanged explicitly labelled periodic cold start, not joint CNN
output'}
54
55     def get(self, day, slot=0, scenario='2'):
56         result = self.forecasts.get(day, slot=slot, scenario=scenario)
57         result['audit'] = {**result['audit'], 'selected_model_id': self.model_id,
58                             'selected_prediction_values_sha256': self._values_hash}
59         return result
60
61     def net_error_paths(self, day, scenario='2', limit=28):
62         result = self.forecasts.net_error_paths(day, scenario=scenario, limit=limit)
63         result['audit'] = {**result['audit'], 'source': '
same_selected_override_adapter_with_labelled_January_periodic_cold_start',
64                             'model_id': self.model_id, 'prediction_values_sha256': self._values_hash,
65                             'historical_predictions_use_same_override': True}
66         return result

```

experiments/exp008/forecast_shape_diagnostic.py

```

1  """Bounded retrospective forecast diagnostics; no network or dispatch training.
2
3  Seven fixed historical summaries inspect local mean, weekday and PV shape
4  information. Every summary uses completed days before its own midnight.
5  Scores use the examined 2025 development period, not an independent test.
6  """
7
8  from __future__ import annotations
9
10 import hashlib
11 import json
12 from pathlib import Path
13
14 import numpy as np
15 import pandas as pd
16
17 from experiments.exp008.neural_joint_calibration import JointStore
18 from experiments.exp008.forecast_calibration import CalibratedStore
19 from experiments.exp008.load_energy_memory import EnergyMemoryStore
20 from experiments.problem2.exp003.data import Data, ROOT
21 from experiments.problem2.exp004.predict import ForecastStore
22
23 OUT = ROOT / 'data/results/exp008/forecast_shape_diagnostic'
24 WINDOWS = (7, 14, 28)
25
26
27 def robust_mean(values, axis=0):
28     """Fixed 20% winsorized mean, not a score-selected quantile."""
29     low, high = np.quantile(values, [.2, .8], axis=axis, keepdims=True)
30     return np.clip(values, low, high).mean(axis=axis)
31
32
33 def historical_forecast(actual, day, window, weekday_load=False, normalized_pv=False):
34     if day < window or window not in WINDOWS:
35         raise ValueError('requires a complete fixed historical window')
36     stop = day * 144
37     history = np.asarray(actual[stop-window*144:stop, :2]).reshape(window, 24, 6, 2).mean(2)
38     value = robust_mean(history)
39     if weekday_load:
40         same = (np.arange(day-window, day)-day) % 7 == 0
41         value[:, 0] = robust_mean(history[same, :, 0])
42     if normalized_pv:
43         if window != 28:
44             raise ValueError('one predeclared shape window: 28 days')
45         pv = history[:, :, 1]

```

```

46     energy = pv.sum(axis=1) # kWh: one-hour interval mean kW.
47     valid = energy > 1e-9
48     shape = robust_mean(pv[valid] / energy[valid, None]) if valid.any() else np.zeros(24)
49     shape /= max(shape.sum(), 1e-9)
50     value[:, 1] = shape * robust_mean(energy[-7:])
51     return np.repeat(np.maximum(value, 0), 6, axis=0)
52
53
54 def summary(error, price):
55     high = price >= np.quantile(price, .75)
56     energy = error.sum(1) / 6
57     cumulative = np.cumsum(error, axis=1) / 6
58     day_level = error.mean(1)
59     return {'rmse_kw': float(np.sqrt(np.mean(error**2))),
60            'mae_kw': float(np.abs(error).mean()), 'bias_kw': float(error.mean()),
61            'price_weighted_rmse_kw': float(np.sqrt(np.mean(error**2*price)/price.mean())),
62            'high_price_rmse_kw': float(np.sqrt(np.mean(error[:, high]**2))),
63            'high_price_bias_kw': float(error[:, high].mean()),
64            'daily_energy_rmse_kwh': float(np.sqrt(np.mean(energy**2))),
65            'daily_energy_mae_kwh': float(np.abs(energy).mean()),
66            'cumulative_error_rmse_kwh': float(np.sqrt(np.mean(cumulative**2))),
67            'daily_level_mse_kw2': float(np.mean(day_level**2)),
68            'within_day_mse_kw2': float(np.mean((error-day_level[:, None])**2)),
69            'high_price_underprediction_kwh': float(np.maximum(-error[:, high], 0).sum()/6)}
70
71
72 def run():
73     OUT.mkdir(exist_ok=True, parents=True)
74     data = Data()
75     original = ForecastStore('no_season', seed=42)
76     days = original.origins // 144
77     dates = pd.date_range('2025-02-01', '2025-12-31')
78     values = {'original_raw': original.values, 'joint_raw': JointStore().values,
79             'original_ridge28': CalibratedStore('ridge_28').values,
80             'joint_ridge28': JointStore(calibrated=True).values}
81     values['joint_ridge28_load_energy_memory_half'] = EnergyMemoryStore(data=data).values
82     for window in WINDOWS:
83         for weekday in (False, True):
84             name = f'{"weekday" if weekday else "recent"}_hourly_winsor{window}'
85             values[name] = np.stack([historical_forecast(data.actual, int(day), window, weekday)
86                                     for day in days])
87     values['weekday28_pvshape28_energy7'] = np.stack([
88         historical_forecast(data.actual, int(day), 28, True, True) for day in days])
89     with np.load(ROOT/'data/results/exp008/forecast_net_hgb/predictions.npz') as hgb:
90         hgb_net = hgb['net_kw'].copy()
91         assert np.array_equal(hgb['origins'], original.origins)
92     mutation = []
93     for day in (31, 62, 243, 364):
94         changed = data.actual.copy()
95         changed[day*144:] += 50000
96         for window in WINDOWS:
97             for weekday in (False, True):
98                 a = historical_forecast(data.actual, day, window, weekday)
99                 b = historical_forecast(changed, day, window, weekday)
100                 error = float(np.max(np.abs(a-b)))
101                 assert error == 0
102                 mutation.append({'day': day, 'window': window, 'weekday': weekday, 'max_error':
103                                error})
103         a = historical_forecast(data.actual, day, 28, True, True)
104         b = historical_forecast(changed, day, 28, True, True)
105         assert np.array_equal(a, b)
106     # Labels below are exclusively retrospective scoring inputs.
107     truth = data.actual[original.origins[:, None]+np.arange(144), :2]
108     net_truth = truth[:, :, 0]-truth[:, :, 1]
109     annual, monthly, hourly, daily, decomposition = [], [], [], [], []
110     errors = {}
111     for name, prediction in values.items():
112         for j, channel in enumerate(('load', 'pv', 'net')):
113             error = (prediction[:, :, j]-truth[:, :, j] if j < 2 else
114                     prediction[:, :, 0]-prediction[:, :, 1]-net_truth)

```

```

115         errors[name, channel] = error
116     errors['net_hgb', 'net'] = hgb_net-net_truth
117     for (name, channel), error in errors.items():
118         annual.append({'name': name, 'channel': channel, **summary(error, data.fixed_price)})
119         for month in range(2, 13):
120             ids = dates.month == month
121             monthly.append({'name': name, 'channel': channel, 'month': month,
122                             **summary(error[ids], data.fixed_price)})
123         for hour in range(24):
124             sl = slice(hour*6, (hour+1)*6)
125             e = error[:, sl]
126             hourly.append({'name': name, 'channel': channel, 'hour_start': hour,
127                             'mean_tariff': float(data.fixed_price[sl].mean()),
128                             'rmse_kw': float(np.sqrt(np.mean(e**2))), 'mae_kw': float(np.abs(e).mean
129
130             ()),
131                             'bias_kw': float(e.mean()), 'squared_error_sum_kw2': float(np.sum(e**2))
132         })
133     if channel == 'net':
134         for i, date in enumerate(dates):
135             daily.append({'name': name, 'day': int(days[i]), 'date': str(date.date()),
136                             'month': date.month, 'day_of_month': date.day,
137                             **summary(error[i:i+1], data.fixed_price)})
138         for group, ids in [('month_first7', dates.day<=7), ('month_rest', dates.day>7)]:
139             decomposition.append({'name': name, 'group': group, 'days': int(ids.sum()),
140                                     **summary(error[ids], data.fixed_price)})
141     for filename, rows in [('annual', annual), ('monthly', monthly), ('hourly', hourly),
142                             ('daily', daily), ('month_age', decomposition)]:
143         pd.DataFrame(rows).to_csv(OUT/f'{filename}.csv', index=False)
144     np.savez_compressed(OUT/'historical_predictions.npz', origins=original.origins,
145                         **{k: v for k, v in values.items() if 'winsor' in k or 'pvshape' in k})
146     protocol = {'historical_baselines': 7, 'windows': WINDOWS,
147                 'single_fixed_followup_candidate': 'joint_ridge28_load_energy_memory_half',
148                 'robust_mean': '20% winsorized mean of completed daily hourly means',
149                 'weekday': 'load uses strictly same weekday in last window; PV uses all days',
150                 'normalized_pv': '28-day robust mean of unit-energy hourly PV shapes times 7-day
151                 robust energy mean',
152                 'hour_interpretation': 'six ten-minute interval means collapsed to hourly mean, then
153                 repeated',
154                 'forecast_inputs': 'strictly prior complete actual load/PV and known weekday only',
155                 'known_fixed_tariff': 'evaluation metrics only; threshold upper quartile of
156                 attachment1 tariff',
157                 'hgb_semantics': 'score net only; adapter channels are not component forecasts',
158                 'model_fitting_performed': False, 'dispatch_performed': False,
159                 'development_year_not_independent_validation': True,
160                 'source_sha256': hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
161                 'mutation_checks': mutation, 'normalized_shape_mutation_passed': True}
162     (OUT/'protocol.json').write_text(json.dumps(protocol, indent=2))
163     print(pd.DataFrame(annual).query('channel == "net"').to_string(index=False), flush=True)
164
165 if __name__ == '__main__':
166     run()

```

experiments/exp008/hgb_linked_price_forecast.py

```

1  """Single predeclared Q4-2 price-model diagnostic using own issued HGB inputs."""
2  from __future__ import annotations
3
4  import copy
5  from functools import lru_cache
6  import hashlib
7  import json
8  from pathlib import Path
9
10 import numpy as np
11 import pandas as pd
12

```

```

13 from experiments.exp008.absolute_hgb_forecast_adapter import AbsoluteHGBForecasts
14 from experiments.exp008.forecast_absolute_hgb import OUT as HGB_OUT, PRIMARY
15
16 ROOT = Path(__file__).resolve().parents[2]
17 OUT = ROOT / 'data/results/exp008/hgb_linked_price_forecast'
18 FIXED_Q = ROOT / 'data/results/exp008/q4_absolute_hgb_physical/full334/dispatch_4-2.npz'
19
20
21 def digest(path):
22     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
23
24
25 def write(path, value):
26     Path(path).write_text(json.dumps(value,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
27
28
29 class LinkedPriceForecasts(AbsoluteHGBForecasts):
30     """Only price regression changes; common forecast adapter stays untouched."""
31
32     def design(self, indices, cutoff):
33         indices = np.asarray(indices, dtype=int)
34         original = self._price_design(indices, cutoff)
35         daysids, slots = indices//144, indices%144
36         # Each historical row receives its own midnight-issued load/PV,
37         # not the current model retroactively applied to previous days.
38         issued = np.empty((len(indices),2))
39         for day in np.unique(daysids):
40             assert int(day)*144 <= cutoff
41             selected = daysids == day
42             issued[selected] = self._midnight(int(day))[slots[selected]] / 1000.
43         assert np.all(indices-144 < daysids*144)
44         return np.column_stack((original,issued))
45
46 @lru_cache(maxsize=1500)
47 def _price(self, origin, stop):
48     if origin < 144:
49         return super()._price(origin,stop)
50     target = np.arange(origin,stop)
51     train = np.arange(max(144,origin-28*144),origin)
52     x, xt = self.design(train,origin), self.design(target,origin)
53     y = self._observed(int(train[0]),origin,origin)[: ,2]
54     weights = 2*(-(origin-1-train)/(14*144))
55     prior = np.array([0.,0.,.5,.5,0.,0.,0.,0.,0.,0.])
56     penalty = np.array([1e-6,5.,5.,5.,5.,5.,5.,5.,5.,5.])
57     coefficient = np.linalg.solve(
58         np.einsum('ni,nj,n->ij',x,x,weights,optimize=False)+np.diag(penalty),
59         np.einsum('ni,n,n->i',x,weights,y,optimize=False)+penalty*prior)
60     prediction = np.maximum(1e-4,np.einsum('ni,i->n',xt,coefficient,optimize=False))
61     return prediction, {'price_method':
62         'Ridge28_daily_weekly_harmonics_plus_own_issued_HGB_load_PV',
63         'price_training_start':int(train[0]), 'price_last_label':origin-1,
64         'price_training_rows':len(train), 'price_coefficients':coefficient.tolist(),
65         'price_feature_columns':10, 'price_new_feature_scale_kw':1000,
66         'price_historical_feature_origins':(np.unique(train//144)*144).tolist(),
67         'price_historical_feature_model':'own-midnight HGB issued pipeline; January labelled
68         periodic cold start',
69         'known_future_price':False}
70
71 def scores(prediction, actual, q):
72     err = prediction-actual
73     high = actual >= 1.
74     return {
75         'rmse_yuan_per_kwh':float(np.sqrt(np.mean(err**2))),
76         'mae_yuan_per_kwh':float(np.mean(np.abs(err))),
77         'bias_yuan_per_kwh':float(np.mean(err)),
78         'high_realized_price_ge1_rmse':float(np.sqrt(np.mean(err[high]**2))),
79         'high_realized_price_ge1_mae':float(np.mean(np.abs(err[high]))),
80         'high_realized_price_slots':int(high.sum()),
81         'realized_price_weighted_mae':float(np.sum(actual*np.abs(err))/np.sum(actual)),
82         'fixed_purchase_absolute_price_exposure_proxy_yuan':float(np.sum(q*np.abs(err))),

```

```

81         'fixed_purchase_signed_price_exposure_proxy_yuan':float(np.sum(q*err)))
82
83
84 def main():
85     if OUT.exists():
86         raise FileExistsError('No candidate overwrite; version any amended diagnostic')
87     OUT.mkdir(parents=True)
88     protected = [Path(__file__),ROOT/'experiments/exp008/forecast.py',
89                 ROOT/'experiments/exp008/absolute_hgb_forecast_adapter.py',
90                 HGB_OUT/f'{PRIMARY}.npz',HGB_OUT/'protocol.json',HGB_OUT/'provenance.json',FIXED_Q]
91     hashes = {str(p.relative_to(ROOT)):digest(p) for p in protected}
92     # This file is saved before forecasting/scoring or reading realized labels.
93     write(OUT/'protocol.json',{ 'candidate_count':1,'diagnostic_only':True,
94                                'forecast_target':'Q4-2 midnight 144-slot price, full334 comparable days',
95                                'change':'append own-issued HGB load and PV /1000 to the original eight regressors',
96                                'historical_feature_policy':'own historical midnight issuance; January periodic cold start;
97                                no retroactive current-model features',
98                                'lookback_days':28,'label_weight_half_life_days':14,
99                                'original_penalty_preserved':True,'new_feature_penalties':[5.,5.],
100                               'new_feature_prior_coefficients':[0.,0.],
101                               'candidate_chosen_after_prior_development_not_untouched_2025_test':True,
102                               'high_price_definition':'realized price >= 1 yuan/kWh, evaluation only',
103                               'proxy_definition':'sum frozen Q4-2 original-purchase kWh times abs(predicted-realized price
104                               ); no action reoptimization or actual savings claim',
105                               'additional_price_weighted_metric':'sum realized_price * abs(price_error) / sum
106                               realized_price',
107                               'no_dispatch_or_Q2_goal_test':True,'sources_sha256':hashes,
108                               'future_mutation_days':[31,32,60,151,243,364]})
109     (OUT/'source_snapshot.py').write_bytes(Path(__file__).read_bytes())
110     baseline, candidate = AbsoluteHGBForecasts(), LinkedPriceForecasts()
111     pred_old, pred_new, audit = [],[],[]
112     for day in range(31,365):
113         old = baseline.get(day,scenario='4-2')
114         new = candidate.get(day,scenario='4-2')
115         for key in ('load_kw','pv_kw'):
116             np.testing.assert_array_equal(old[key],new[key])
117         origin = day*144
118         train = np.arange(max(144,origin-28*144),origin)
119         x = candidate.design(train,origin)
120         np.testing.assert_array_equal(x[:,8],baseline._price_design(train,origin))
121         assert new['audit']['price_last_label'] < origin
122         assert max(new['audit']['price_historical_feature_origins']) < origin
123         pred_old.append(old['price']); pred_new.append(new['price']); audit.append(new['audit'])
124     pred_old,pred_new = np.asarray(pred_old),np.asarray(pred_new)
125     checks = []
126     for day in (31,32,60,151,243,364):
127         origin = day*144
128         for kind in ('price_only','load_PV_only','future_store_only','combined'):
129             data = copy.copy(candidate.data); data.actual=candidate.data.actual.copy()
130             if kind in ('price_only','combined'):
131                 data.actual[origin:,2] += 10000.
132             if kind in ('load_PV_only','combined'):
133                 data.actual[origin:,2] += [70000.,30000.]
134             changed = LinkedPriceForecasts(data=data)
135             future = changed.store.origins > origin
136             if kind in ('future_store_only','combined'):
137                 changed.store.values[future] += [60000.,20000.]
138             before = candidate.get(day,scenario='4-2')
139             after = changed.get(day,scenario='4-2')
140             for key in ('load_kw','pv_kw','price'):
141                 np.testing.assert_array_equal(before[key],after[key])
142                 np.testing.assert_array_equal(before['audit']['price_coefficients'],after['audit']['
143                 price_coefficients'])
144             checks.append({'day':day,'kind':kind,'passed':True,
145                           'future_store_rows':int(future.sum()) if kind in ('future_store_only','combined')
146                           else 0,
147                           'store_test_nonvacuous':bool(future.any()) if kind in ('future_store_only','combined')
148                           else None})
149     with np.load(FIXED_Q,allow_pickle=False) as pack:
150         q=pack['original'].copy()

```

```

145 actual = baseline.data.actual[31*144:,2].reshape(334,144)
146 previous,updated=scores(pred_old,actual,q),scores(pred_new,actual,q)
147 daily=[]
148 for i,day in enumerate(range(31,365)):
149     for name, pred in (('baseline',pred_old),('linked_hgb',pred_new)):
150         daily.append({'day':day,'month':(pd.Timestamp('2025-01-01')+pd.Timedelta(days=day)).
151             month,
152                 'method':name,**scores(pred[i],actual[i],q[i])})
153 pd.DataFrame(daily).to_csv(OUT/'daily_scores.csv',index=False)
154 np.savez_compressed(OUT/'price_predictions.npz',baseline=pred_old,linked_hgb=pred_new,
155     actual=actual,days=np.arange(31,365),origins=np.arange(31,365)*144)
156 write(OUT/'prediction_audit.json',audit)
157 write(OUT/'causality_audit.json',{'passed':True,'checks':checks,
158     'all334_original_eight_design_columns_unchanged':True,
159     'all334_load_PV_forecasts_unchanged':True,
160     'all334_training_labels_and_feature_issue_origins_strictly_before_target_origin':True,
161     'future_store_day364_has_no_later_rows':True})
162 assert all(digest(ROOT/p)==h for p,h in hashes.items())
163 result={'days':334,'baseline':previous,'linked_hgb':updated,
164     'delta_linked_minus_baseline':{k:updated[k]-previous[k] for k in previous},
165     'all_sources_unchanged':True,'causality_checks_passed':True,
166     'planning_and_billing_not_run':True,'this_is_not_a_Q2_goal_test':True,
167     'all_four_forecast_gates_improved':all(updated[k]<previous[k] for k in (
168         'rmse_yuan_per_kwh','mae_yuan_per_kwh','high_realized_price_ge1_rmse',
169         'fixed_purchase_absolute_price_exposure_proxy_yuan'))}
170 write(OUT/'summary.json',result)
171 print(json.dumps(result,indent=2))
172
173 if __name__ == '__main__':
174     main()

```

experiments/exp008/impulse_feedback.py

```

1 """Inventory value control with an explicit last-direction switching charge.
2
3 Adapted idea: compare an action's complete continuation value with waiting.
4 The auxiliary switching charge is never included in the competition bill.
5 """
6 from time import perf_counter
7 import numpy as np
8
9 ETA=np.sqrt(.9)
10 LOW,HIGH,LIMIT=1200.,10800.,5000/6
11 MODES=(-1,0,1)
12
13
14 def value_functions(purchase,support,price,*,grid_kwh=100.,wear=.002,
15     switching=50.,terminal=.45):
16     began=perf_counter()
17     grid=np.r_[np.arange(LOW,HIGH,grid_kwh),HIGH]
18     n=len(price)
19     values=np.zeros((n+1,3,len(grid)))
20     values[-1]=--terminal*(grid-LOW)[None,:]
21     delta=grid[None,:]-grid[:,None]
22     for t in range(n-1,-1,-1):
23         expected=np.zeros((3,len(grid)))
24         future=values[t+1]
25         for demand in support[t]:
26             b=purchase[t]-demand
27             positive=b>=0
28             direction=1 if positive else -1
29             nextv=future[direction+1]
30             if positive:
31                 available=np.minimum(np.minimum(b,LIMIT),(HIGH-grid)/ETA)
32                 endpoint=grid+ETA*available
33                 eligible=(delta>1e-8)&(delta<=ETA*available[:,None]+1e-8)

```

```

34         costs=nextv[None,:]+wear*delta/ETA
35         end_value=np.interp(endpoint,grid,nextv)+wear*available
36     else:
37         available=np.minimum(np.minimum(-b,LIMIT),(grid-LOW)*ETA)
38         endpoint=grid-available/ETA
39         eligible=(delta<-1e-8)&(delta>=-available[:,None]/ETA-1e-8)
40         costs=nextv[None,:]+5*price[t]*(-b+ETA*delta)-wear*ETA*delta
41         end_value=np.interp(endpoint,grid,nextv)+5*price[t]*(-b-available)+wear*available
42         active=np.minimum(np.min(np.where(eligible, costs, np.inf), axis=1),
43                             np.where(available>1e-8, end_value, np.inf))
44         for mi, mode in enumerate(MODES):
45             idle=future[mi]+5*price[t]*max(-b,0)
46             changed=active+(switching if mode*direction==-1 else 0)
47             expected[mi]=np.minimum(idle, changed)
48         values[t]=expected/support.shape[1]
49     return grid, values, {'method': 'inventory_value_with_last_direction',
50                          'seconds': perf_counter()-began, 'switching_yuan_auxiliary': switching,
51                          'grid_kwh': grid_kwh, 'terminal': terminal,
52                          'disturbance_assumption': 'independent_conditional_marginals'}
53
54
55 def execute(purchase, actual, price, initial_soc, grid, values, *, wear=.002, switching=50., initial_mode=1):
56     n=len(purchase)
57     c,d,e,w=(np.zeros(n) for _ in range(4))
58     states=np.empty(n+1); states[0]=initial_soc
59     mode=initial_mode
60     for t in range(n):
61         b=purchase[t]+(actual[t,1]-actual[t,0])/6
62         current=states[t]
63         positive=b>=0
64         direction=1 if positive else -1
65         future=values[t+1]
66         idle_value=np.interp(current, grid, future[mode+1])+5*price[t]*max(-b,0)
67         if positive:
68             available=min(b, LIMIT, max(0, (HIGH-current)/ETA))
69             endpoint=current+ETA*available
70             choices=np.r_[grid[(grid>current)&(grid<=endpoint)], endpoint]
71             costs=np.interp(choices, grid, future[2])+wear*(choices-current)/ETA
72         else:
73             available=min(-b, LIMIT, max(0, (current-LOW)*ETA))
74             endpoint=current-available/ETA
75             choices=np.r_[grid[(grid>=endpoint)&(grid<current)], endpoint]
76             release=(current-choices)*ETA
77             costs=np.interp(choices, grid, future[0])+5*price[t]*(-b-release)+wear*release
78         if mode*direction==-1:
79             costs+=switching
80             best=int(np.argmin(costs))
81             following=float(choices[best]) if available>1e-8 and costs[best]<idle_value-1e-8 else
82             current
83             c[t]=max(0, (following-current)/ETA); d[t]=max(0, (current-following)*ETA)
84             if c[t]+d[t]>1e-8:
85                 mode=direction
86                 e[t]=max(0, -b-d[t]); w[t]=max(0, b-c[t]); states[t+1]=following
87             fees=np.stack([purchase*price, np.zeros(n), np.zeros(n), 5*e*price], axis=-1)
88     return dict(original=purchase.copy(), final=purchase.copy(), charge=c, discharge=d,
89                 emergency=e, surplus=w, states=states, fees=fees, actual=actual.copy(), price=price.copy
90                 ()), mode

```

experiments/exp008/load_energy_memory.py

```

1 """One fixed causal load-energy memory on joint CNN plus unchanged Ridge28.
2
3 The gain is 0.5, chosen before this candidate's score. Yesterday's residual
4 is measured against yesterday's *underlying joint/Ridge28* issued forecast,
5 not against this memory layer recursively. PV is unchanged. No fit or search.
6 """
7

```

```

8 from __future__ import annotations
9
10 import hashlib
11 import json
12 from pathlib import Path
13
14 import numpy as np
15
16 from experiments.exp008.forecast import Forecasts
17 from experiments.exp008.neural_joint_calibration import JointStore
18 from experiments.problem2.exp003.data import Data, ROOT
19
20 OUT = ROOT / 'data/results/exp008/load_energy_memory'
21 GAIN = .5
22
23
24 def correct_day(actual, origins, base_values, index):
25     origin = int(origins[index])
26     output = base_values[index].copy()
27     prior = np.flatnonzero(origins+144 == origin)
28     audit = {'origin': origin, 'day': origin//144, 'gain': GAIN,
29             'information_cutoff_exclusive': origin,
30             'underlying_forecast': 'joint_CNN_then_fixed_Ridge28',
31             'residual_reference': 'previous_day_underlying_forecast_before_memory_layer',
32             'recursive_corrected_output_residual': False,
33             'prior_forecast_origin': None, 'prior_label_end_exclusive': None,
34             'load_correction_kw': 0., 'pv_unchanged': True,
35             'current_truth_used': False}
36     if len(prior):
37         previous = int(prior[-1])
38         old_origin = int(origins[previous])
39         if previous >= index or old_origin+144 > origin:
40             raise ValueError('memory labels must be completely observed at issue')
41         yesterday = np.asarray(actual[old_origin:old_origin+144, 0], float)
42         residual_kw = float(np.mean(yesterday-base_values[previous, :, 0]))
43         output[:, 0] = np.maximum(0., output[:, 0]+GAIN*residual_kw)
44         audit.update(prior_forecast_origin=old_origin,
45                     prior_label_end_exclusive=old_origin+144,
46                     prior_max_observed_index=old_origin+143,
47                     underlying_prior_daily_energy_residual_kwh=24*residual_kw,
48                     load_correction_kw=GAIN*residual_kw)
49     else:
50         audit['cold_start'] = 'unchanged_underlying_output_no_previous_issued_forecast'
51     return output, audit
52
53
54 class EnergyMemoryStore:
55     """Read-only construction from the signed joint/Ridge28 base archive."""
56
57     def __init__(self, data=None, base_store=None):
58         self.base_store = JointStore(calibrated=True) if base_store is None else base_store
59         self.origins = self.base_store.origins.copy()
60         self.base_values = self.base_store.values.copy()
61         self.lookup = {int(origin): i for i, origin in enumerate(self.origins)}
62         self.name = 'joint_ridge28_load_energy_memory_half'
63         actual = (Data() if data is None else data).actual
64         generated = [correct_day(actual, self.origins, self.base_values, i)
65                     for i in range(len(self.origins))]
66         self.values = np.stack([row[0] for row in generated])
67         self.audit = [row[1] for row in generated]
68         self.delta = self.values-self.base_values
69
70     def get(self, origin):
71         return self.values[self.lookup[int(origin)]] copy()
72
73
74 class EnergyMemoryForecasts(Forecasts):
75     """Planning adapter; inherited historical risk calls these same outputs."""
76
77     def __init__(self, data=None):

```

```

78     super().__init__(data=data)
79     self.store = EnergyMemoryStore(data=self.data)
80     self.calibration = self.store.name
81
82     def get(self, day, slot=0, scenario='2'):
83         result = super().get(day, slot, scenario)
84         result['audit'].update(
85             base_forecast='exp008_joint_shared_cnn' if day >= 31 else 'periodic_cold_start',
86             output_calibration='same_fixed_Ridge28_then_load_energy_memory_half' if day >= 31 else
None,
87             base_architecture_unchanged=day < 31)
88         if day >= 31:
89             result['audit']['load_energy_memory'] = self.store.audit[self.store.lookup[day*144]]
90         return result
91
92
93     def save():
94         OUT.mkdir(parents=True, exist_ok=True)
95         data = Data()
96         store = EnergyMemoryStore(data=data)
97         path = OUT/'predictions.npz'
98         np.savez_compressed(path, origins=store.origins, values=store.values,
99                             base_values=store.base_values, delta=store.delta)
100         protocol = {'candidate_count': 1, 'gain': GAIN,
101                     'base': 'signed_joint_CNN_then_unchanged_Ridge28',
102                     'current_and_historical_pipeline': 'apply_same_memory_to_each_historical_issue',
103                     'model_fitted': False, 'hyperparameter_search': False,
104                     'inputs': 'previous_completed_actual_load_and_underlying_prior_issued_load_forecast'
105                     ,
106                     'pv_unchanged': True, 'source_sha256': hashlib.sha256(Path(__file__).read_bytes()).
hexdigest(),
107                     'base_values_sha256': hashlib.sha256(store.base_values.tobytes()).hexdigest(),
108                     'source_data_sha256': data.hashes,
109                     'archive_sha256': hashlib.sha256(path.read_bytes()).hexdigest(),
110                     'development_on_examined_2025_not_independent_test': True}
111         (OUT/'protocol.json').write_text(json.dumps(protocol, indent=2))
112         (OUT/'prediction_audit.json').write_text(json.dumps(store.audit, indent=2))
113         (OUT/'source_snapshot.py').write_bytes(Path(__file__).read_bytes())
114         return store
115
116 if __name__ == '__main__':
117     save()

```

experiments/exp008/mode_budget_hold1_physical.py

```

1  """Ten-minute modes with a hard daily eight-switch budget and no switch penalty.
2
3  The common purchase and scenario storage/complementarity constraints match
4  the existing physical planner. Scenario recourse remains an optimistic
5  surrogate; only actual greedy execution is prefix-causal.
6  """
7
8  from __future__ import annotations
9
10 import json
11
12 import numpy as np
13
14 from experiments.common.neural_v2.physics import LinearModel
15 from experiments.exp008.planner import ETA, HIGH, LIMIT, LOW
16
17
18 def mode_state(mask, previous_mode, previous_run_slots, minimum=1):
19     mask = np.asarray(mask, bool)
20     previous = previous_mode > 0
21     flips = np.r_[mask[0] != previous, mask[1:] != mask[:-1]]

```

```

22 history = np.r_[np.repeat(previous, min(previous_run_slots, minimum)), mask]
23 changes = np.flatnonzero(history[1:] != history[:-1])+1
24 lengths = np.diff(np.r_[0, changes, len(history)])
25 if len(lengths) > 1 and np.any(lengths[:-1] < minimum):
26     raise AssertionError('A completed planned mode run violates minimum hold')
27 own_changes = np.flatnonzero(mask[1:] != mask[:-1])+1
28 tail = len(mask)-int(own_changes[-1]) if len(own_changes) else len(mask)
29 if not len(own_changes) and mask[0] == previous:
30     tail += previous_run_slots
31 return {'planned_changes_including_boundary': int(np.sum(flips)),
32         'final_planned_mode': 1 if mask[-1] else -1,
33         'final_planned_run_slots': tail,
34         'next_day_required_hold_slots': max(0, minimum-tail),
35         'completed_run_minimum_passed': True}, flips
36
37
38 def plan(net_paths, price, soc, previous_mode=1, previous_run_slots=1,
39         *, hold_slots=1, switching=0., wear=.002, seconds=5., gap=.002, final=False):
40     paths, price = np.asarray(net_paths, float), np.asarray(price, float)
41     k, n = paths.shape
42     if price.shape != (n,) or not np.all(price > 0) or not np.isfinite(paths).all():
43         raise ValueError('finite scenario net kWh and matching positive price required')
44     if not LOW <= soc <= HIGH or previous_mode not in (-1, 1) or previous_run_slots < 1 or
45         hold_slots != 1:
46         raise ValueError('valid SOC, explicit previous mode/run and fixed hold1 required')
47     if switching != 0.:
48         raise ValueError('This candidate fixes the soft switching penalty to zero')
49     m = LinearModel()
50     q_upper = np.maximum(paths.max(0), 0.)+LIMIT
51     q = m.variables((n,), upper=q_upper, cost=price)
52     mode = m.variables((n,), upper=1., integer=True)
53     flip = m.variables((n,), upper=1., cost=switching, integer=True)
54     m.constraint([(index, 1.) for index in flip], upper=8.)
55     c = m.variables((k, n), upper=LIMIT, cost=wear/k)
56     d = m.variables((k, n), upper=LIMIT, cost=wear/k)
57     emergency_upper = np.maximum(paths, 0.)
58     e = m.variables((k, n), upper=emergency_upper, cost=5*price[None, :]/k)
59     w = m.variables((k, n))
60     s = m.variables((k, n), lower=LOW, upper=HIGH)
61     may_charge = m.variables((k, n), upper=1., integer=True)
62     for idx in s[:, -1]:
63         m.objective[int(idx)] = 0. if final else -float(price.min())/ETA/k
64     old = float(previous_mode > 0)
65     for t in range(n):
66         difference = [(mode[t], 1.)]
67         if t:
68             difference.append((mode[t-1], -1.))
69         rhs = old if t == 0 else 0.
70         m.constraint(difference+[(flip[t], -1.)], upper=rhs)
71         m.constraint([(i, -v) for i, v in difference]+[(flip[t], -1.)], upper=-rhs)
72         # The upper inequalities make flip the exact XOR, independently of
73         # objective tolerance and its participation in minimum-hold windows.
74         first = [(flip[t], 1.), (mode[t], -1.)]
75         second = [(flip[t], 1.), (mode[t], 1.)]
76         if t:
77             first.append((mode[t-1], -1.))
78             second.append((mode[t-1], 1.))
79         m.constraint(first, upper=old if t == 0 else 0.)
80         m.constraint(second, upper=2.-old if t == 0 else 2.)
81     for t in range(n):
82         m.constraint([(flip[j], 1.) for j in range(t, min(n, t+hold_slots))], upper=1.)
83     pending = min(n, max(0, hold_slots-previous_run_slots))
84     for t in range(pending):
85         m.constraint([(mode[t], 1.)], old, old)
86     for j in range(k):
87         for t in range(n):
88             m.constraint([(q[t], 1), (c[j, t], -1), (d[j, t], 1),
89                           (e[j, t], 1), (w[j, t], -1)], paths[j, t], paths[j, t])
90             terms = [(s[j, t], 1), (c[j, t], -ETA), (d[j, t], 1/ETA)]
91             if t:

```

```

91         terms.append((s[j, t-1], -1.))
92     rhs = soc if t == 0 else 0.
93     m.constraint(terms, rhs, rhs)
94     m.constraint([(c[j, t], 1), (mode[t], -LIMIT)], upper=0.)
95     m.constraint([(d[j, t], 1), (mode[t], LIMIT)], upper=LIMIT)
96     m.constraint([(c[j, t], 1), (may_charge[j, t], -LIMIT)], upper=0.)
97     m.constraint([(e[j, t], 1), (may_charge[j, t], emergency_upper[j, t])],
98                 upper=emergency_upper[j, t])
99 x, metadata = m.solve(seconds=seconds, gap=gap)
100 if x is None:
101     raise RuntimeError(f'No feasible variable-mode hold1 incumbent: {json.dumps(metadata)}')
102 mask = x[mode] > .5
103 boundary, exact_flips = mode_state(mask, previous_mode, previous_run_slots, hold_slots)
104 if boundary['planned_changes_including_boundary'] > 8:
105     raise AssertionError('Daily planned switch budget exceeded')
106 states = np.column_stack((np.full(k, soc), x[s]))
107 balance = x[q][None, :]+x[d]+x[e]-x[c]-x[w]-paths
108 state_error = np.diff(states, axis=1)-ETA*x[c]+x[d]/ETA
109 check = {'maximum_balance_error_kwh': float(np.abs(balance).max()),
110         'maximum_soc_equation_error_kwh': float(np.abs(state_error).max()),
111         'emergency_charging_slots': int(np.sum((x[c] > 1e-6) & (x[e] > 1e-6))),
112         'simultaneous_charge_discharge_slots': int(np.sum((x[c] > 1e-6) & (x[d] > 1e-6))),
113         'minimum_soc_kwh': float(states.min()), 'maximum_soc_kwh': float(states.max()),
114         'maximum_charge_or_discharge_kwh': float(max(x[c].max(), x[d].max())),
115         'minimum_variable': float(x.min()),
116         'mode_flip_XOR_error': float(np.max(np.abs(x[flip]-exact_flips)))}
117 check['passed'] = (max(check['maximum_balance_error_kwh'], check['maximum_soc_equation_error_kwh'],
118                        check['mode_flip_XOR_error']) < 1e-6 and check['emergency_charging_slots'] == 0 and
119                    check['simultaneous_charge_discharge_slots'] == 0 and states.min() >= LOW-1e-6 and
120                    states.max() <= HIGH+1e-6 and check['maximum_charge_or_discharge_kwh'] <= LIMIT+1e-6 and
121                    check['minimum_variable'] >= -1e-6)
122 if not check['passed']:
123     raise AssertionError(check)
124 metadata.update(method='variable_hold1_daily_cap8_kappa0_physical_scenario_MIP',
125                daily_planned_switch_cap=8, switching_cost=0., exact_binary_XOR=True,
126                scenario_count=k, hold_slots=hold_slots, previous_planned_mode=previous_mode,
127                previous_planned_run_slots=previous_run_slots, inherited_locked_prefix_slots=pending,
128                nonanticipative_recourse_certificate=False, scenario_physical_checks=check,
129                planned_mode_changes=boundary['planned_changes_including_boundary'], **boundary)
130 return {'purchase': np.maximum(x[q], 0.), 'allowed_charge': mask,
131        'planned_flips': exact_flips, 'solved_flip_values': x[flip], 'scenario_charge': x[c], '
132        scenario_discharge': x[d],
133        'scenario_emergency': x[e], 'scenario_surplus': x[w], 'scenario_states': states,
134        'metadata': metadata}

```

experiments/exp008/mode_planning_physical.py

```

1  """Shared hourly modes with scenario emergency charging explicitly forbidden.
2
3  This repairs a physical feasible-set error in the previous scenario surrogate.
4  Scenario recourse still knows its own future path and remains an optimistic
5  approximation; the physical repair does not certify nonanticipativity.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import ast
12 import hashlib
13 import json
14 import shutil
15 from pathlib import Path
16 from time import perf_counter
17
18 import numpy as np
19 import pandas as pd

```

```

20
21 from experiments.common.neural_v2.physics import LinearModel
22 from experiments.exp008.closed_loop import optimize
23 from experiments.exp008.controller_candidate import INITIAL_SOC, Data
24 from experiments.exp008.planner import ETA, HIGH, LIMIT, LOW, execute
25 from experiments.exp008.unified_forecast import UnifiedForecasts
26 from experiments.exp008.verify import battery_metrics, verify_npz
27
28 OUT = Path('data/results/exp008/mode_planning_physical')
29
30
31 def plan(net_paths, price, soc, previous_mode=1, *, block_slots=6,
32         switching=50., wear=.002, seconds=5., gap=.002, final=False):
33     paths=np.asarray(net_paths,float)
34     price=np.asarray(price,float)
35     k,n=paths.shape
36     if price.shape!=(n,) or not np.all(price>0) or not np.isfinite(paths).all():
37         raise ValueError('finite scenario net kwh and matching positive price required')
38     if not LOW<=soc<=HIGH or block_slots<1:
39         raise ValueError('invalid initial SOC or block length')
40     nb=(n+block_slots-1)//block_slots
41     m=LinearModel()
42     # Larger common purchases are entirely wasted in every scenario and are
43     # dominated under strictly positive fixed-plan purchase prices.
44     q_upper=np.maximum(paths.max(axis=0),0.)+LIMIT
45     q=m.variables((n,),upper=q_upper,cost=price)
46     mode=m.variables((nb,),upper=1.,integer=True)
47     flip=m.variables((nb,),cost=switching)
48     c=m.variables((k,n),upper=LIMIT,cost=wear/k)
49     d=m.variables((k,n),upper=LIMIT,cost=wear/k)
50     # With emergency energy unable to charge, q>=0 and discharge>=0,
51     # physically useful emergency energy cannot exceed positive net demand.
52     emergency_upper=np.maximum(paths,0.)
53     e=m.variables((k,n),upper=emergency_upper,cost=5*price[None,:]/k)
54     w=m.variables((k,n))
55     s=m.variables((k,n),lower=LOW,upper=HIGH)
56     may_charge=m.variables((k,n),upper=1.,integer=True)
57     for idx in s[:,-1]:
58         m.objective[int(idx)]=0. if final else -float(price.min())/ETA/k
59     for b in range(nb):
60         terms=[(mode[b],1.)]
61         rhs=float(previous_mode>0) if b==0 else 0.
62         if b:
63             terms.append((mode[b-1],-1.))
64         m.constraint(terms+[(flip[b],-1.)],upper=rhs)
65         m.constraint([(i,-v) for i,v in terms]+[(flip[b],-1.)],upper=-rhs)
66     for j in range(k):
67         for t in range(n):
68             m.constraint([(q[t],1),(c[j,t],-1),(d[j,t],1),(e[j,t],1),(w[j,t],-1)],
69                           paths[j,t],paths[j,t])
70             terms=[(s[j,t],1),(c[j,t],-ETA),(d[j,t],1/ETA)]
71             if t:
72                 terms.append((s[j,t-1],-1.))
73             rhs=soc if t==0 else 0.
74             m.constraint(terms,rhs,rhs)
75             block=t//block_slots
76             m.constraint([(c[j,t],1),(mode[block],-LIMIT)],upper=0.)
77             m.constraint([(d[j,t],1),(mode[block],LIMIT)],upper=LIMIT)
78             m.constraint([(c[j,t],1),(may_charge[j,t],-LIMIT)],upper=0.)
79             m.constraint([(e[j,t],1),(may_charge[j,t],emergency_upper[j,t])],
80                           upper=emergency_upper[j,t])
81     x,metadata=m.solve(seconds=seconds,gap=gap)
82     if x is None:
83         raise RuntimeError(f'No feasible physical shared-mode incumbent: {metadata}')
84     states=np.column_stack((np.full(k,soc),x[s]))
85     balance=x[q][None,:]+x[d]+x[e]-x[c]-x[w]-paths
86     state_error=np.diff(states,axis=1)-ETA*x[c]+x[d]/ETA
87     check={'maximum_balance_error_kwh':float(np.abs(balance).max()),
88           'maximum_soc_equation_error_kwh':float(np.abs(state_error).max()),
89           'emergency_charging_slots':int(np.sum((x[c]>1e-6)&(x[e]>1e-6))),

```

```

90         'simultaneous_charge_discharge_slots':int(np.sum((x[c]>1e-6)&(x[d]>1e-6))),
91         'minimum_soc_kwh':float(states.min()), 'maximum_soc_kwh':float(states.max()),
92         'maximum_charge_or_discharge_kwh':float(max(x[c].max(),x[d].max())),
93         'minimum_variable':float(x.min())
94     check['passed']=(max(check['maximum_balance_error_kwh'],check['maximum_soc_equation_error_kwh'])
95 <1e-6
96
97         and check['emergency_charging_slots']==0
98         and check['simultaneous_charge_discharge_slots']==0
99         and states.min()>=LOW-1e-6 and states.max()<=HIGH+1e-6
100         and check['maximum_charge_or_discharge_kwh']<=LIMIT+1e-6
101         and check['minimum_variable']>=-1e-6)
102 if not check['passed']:
103     raise AssertionError(check)
104 metadata.update(method='shared_mode_scenario_milp_without_emergency_charging',
105                 scenario_count=k,nonanticipative_recourse_certificate=False,
106                 planned_mode_changes=int(np.round(x[flip]).sum()),
107                 scenario_physical_checks=check,
108                 emergency_charge_complementarity='scenario-slot binary',
109                 common_purchase_bound='max(max_scenario_net,0)+AC_charge_limit',
110                 emergency_bound='max(scenario_net,0)')
111 return {'purchase':np.maximum(x[q],0.), 'allowed_charge':(x[mode]>.5)[np.arange(n)//block_slots],
112        'scenario_charge':x[c], 'scenario_discharge':x[d], 'scenario_emergency':x[e],
113        'scenario_surplus':x[w], 'scenario_states':states, 'metadata':metadata}
114
115 def counterexample():
116     paths=np.array([[ -1000.,0.]]*6+[[0.,1000.]])
117     result=plan(paths,np.array([.4,1.]),LOW,block_slots=1,switching=0.,final=True)
118     return {'purchase':result['purchase'].tolist(),
119           'allowed_charge':result['allowed_charge'].tolist(),
120           'scenario_charge':result['scenario_charge'].tolist(),
121           'scenario_emergency':result['scenario_emergency'].tolist(),
122           'metadata':result['metadata']}
123
124 def run(days=3,scenarios=7,checkpoint_days=None,forecast_override=None,case_name=None,
125        prefix_directory=None):
126     directory=OUT/(case_name or (f'refined_{days}days' if scenarios==7 else f'refined_s{scenarios}_{
127 days}days'))
128     directory.mkdir(parents=True,exist_ok=True)
129     if (directory/'summary.json').exists():
130         raise FileExistsError('Completed experiments are immutable')
131     cfg={'days':days, 'calibration':getattr(forecast_override, 'calibration', 'ridge_28'),
132         'scenarios':scenarios, 'block_slots':6,
133         'switching':50., 'wear':.002, 'seconds':5., 'gap':.002,
134         'refinement':True, 'maxiter':120, 'deadband':0.,
135         'repair': 'hard_no_scenario_emergency_charging',
136         'scenario_selection': 'deterministic_equally_spaced_complete_historical_origins',
137         'initialization_scenario_count_also_changed':scenarios!=7,
138         'refinement_uses_all_28_historical_paths':True,
139         'checkpoint_days':checkpoint_days, 'checkpoint_maximum_gap_threshold':.05,
140         'nonanticipative_recourse_certificate':False,
141         'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
142     (directory/'config.json').write_text(json.dumps(cfg,indent=2))
143     (directory/'source_snapshot.py').write_bytes(Path(__file__).read_bytes())
144     data=Data()
145     forecast=forecast_override if forecast_override is not None else UnifiedForecasts(calibration='
146 ridge_28')
147     dependencies=('closed_loop', 'planner', 'unified_forecast', 'forecast', 'forecast_calibration')
148     provenance={'source_data_hashes':data.hashes,
149               'calibrated_forecast_values_sha256':hashlib.sha256(forecast.store.values.tobytes()).
150               hexdigest(),
151               'dependency_source_sha256':{}}
152     for dependency in dependencies:
153         source=Path(f'experiments/exp008/{dependency}.py')
154         provenance['dependency_source_sha256'][str(source)]=hashlib.sha256(source.read_bytes()).
155         hexdigest()
156         (directory/f'{dependency}_snapshot.py').write_bytes(source.read_bytes())
157     (directory/'provenance.json').write_text(json.dumps(provenance,indent=2))
158     soc,mode=INITIAL_SOC,1

```

```

155 rows, details, audits=[],[],[]
156 prefix_count=0
157 if prefix_directory is not None:
158     prefix=Path(prefix_directory)
159     old_cfg=json.loads((prefix/'config.json').read_text())
160     excluded={'days','checkpoint_days','source_sha256'}
161     for key,value in cfg.items():
162         if key not in excluded and value!=old_cfg.get(key):
163             raise AssertionError(f'Prefix configuration mismatch: {key}')
164     old_provenance=json.loads((prefix/'provenance.json').read_text())
165     if old_provenance!=provenance:
166         raise AssertionError('Prefix data, forecasts or dependency sources changed')
167     def plan_ast(source):
168         return ast.dump(next(node for node in ast.parse(source).body
169                             if isinstance(node,ast.FunctionDef) and node.name=='plan'))
170     if plan_ast((prefix/'source_snapshot.py').read_text())!=plan_ast(Path(__file__).read_text())
171 :
172     raise AssertionError('Prefix physical planning function changed')
173 with np.load(prefix/'dispatch.npz',allow_pickle=False) as archive:
174     prefix_count=len(archive['days'])
175     if prefix_count>=days or not np.array_equal(archive['days'],np.arange(31,31+prefix_count
176 ))):
177         raise ValueError('Prefix must be shorter, contiguous and start at day 31')
178     details=[{key:archive[key][i].copy() for key in archive.files if key!='days'}
179              for i in range(prefix_count)]
180     prefix_check=verify_npz(prefix/'dispatch.npz',expected_days=prefix_count,
181                             audit_path=prefix/'planning_audit.json')
182     if not prefix_check['passed']:
183         raise AssertionError(prefix_check['errors'])
184     audits=json.loads((prefix/'planning_audit.json').read_text())
185     rows=[{'day':a['day'],'total_cost':float(d['fees'].sum()),
186           'planned_cost':float(d['fees'][:,0].sum()),
187           'emergency_cost':float(d['fees'][:,3].sum()),
188           'mip':a['mip'],'refinement':a['greedy_refinement']}
189           for a,d in zip(audits,details,strict=True)]
190     for day in range(31,31+prefix_count):
191         shutil.copy2(prefix/f'planning_day{day}.npz',directory/f'planning_day{day}.npz')
192     soc=float(details[-1]['states'][-1])
193     modes=np.sign(np.concatenate([d['charge']-d['discharge'] for d in details]))
194     nonzero=modes[modes!=0]
195     mode=int(nonzero[-1]) if len(nonzero) else 1
196     (directory/'prefix_manifest.json').write_text(json.dumps({
197         'directory':str(prefix),'days':prefix_count,'exact_arrays_reused':True,
198         'continuation_initial_soc':soc,'continuation_initial_mode':mode,
199         'dispatch_sha256':hashlib.sha256((prefix/'dispatch.npz').read_bytes()).hexdigest(),
200         'physical_plan_ast_unchanged':True,'configuration_unchanged':True,
201         'data_forecasts_dependencies_unchanged':True,'independent_verification':prefix_check},
202         indent=2))
203     began=perf_counter()
204     for day in range(31+prefix_count,31+days):
205         issue=forecast.get(day,scenario='2')
206         errors=forecast.net_error_paths(day,'2',limit=28)
207         paths=(issue['load_kw']-issue['pv_kw'])[None,:]/6+errors['errors_kwh']
208         selected=np.linspace(0,len(paths)-1,min(scenarios,len(paths))).astype(int)
209         try:
210             result=plan(paths[selected],data.fixed_price,soc,mode,final=day==364)
211         except (RuntimeError,AssertionError) as error:
212             (directory/'failure.json').write_text(json.dumps({'day':day,'error':str(error),
213                 'completed_days':len(details),'stopped_before_remaining_days':True},indent=2))
214             raise
215     np.savez_compressed(directory/f'planning_day{day}.npz',
216         **{key:value for key,value in result.items() if key!='metadata'},
217         all_net_paths=paths,selected_scenario_indices=selected,initial_soc=np.array(soc),
218         initial_mode=np.array(mode),price=data.fixed_price)
219     refined=optimize(result['purchase'],paths,data.fixed_price,soc,
220         charge_mask=result['allowed_charge'],throughput=.002,variation=0.,
221         terminal=0. if day==364 else .45,maxiter=120,deadband=0.)
222     q=refined['purchase']
223     actual=data.actual[day*144:(day+1)*144]
224     detail=execute(q,actual,data.fixed_price,soc,charge_deadband=0.,charge_mask=result['

```

```

222     allowed_charge'])
223     detail.update(original=q, final=q.copy(), actual=actual.copy(), price=data.fixed_price.copy(),
224                   allowed_charge=result['allowed_charge'].copy())
225     detail['fees']=np.stack((q*data.fixed_price,np.zeros(144),np.zeros(144),
226                             5*detail['emergency']*data.fixed_price),axis=-1)
227     details.append(detail)
228     audits.append({'day':day, 'forecast':issue['audit'], 'history':errors['audit'],
229                   'mip':result['metadata'], 'greedy_refinement':refined['metadata']})
230     rows.append({'day':day, 'total_cost':float(detail['fees'].sum()),
231                 'planned_cost':float(detail['fees'][:,0].sum()),
232                 'emergency_cost':float(detail['fees'][:,3].sum()),
233                 'mip':result['metadata'], 'refinement':refined['metadata']})
234     soc=float(detail['states'][-1])
235     signs=np.sign(detail['charge']-detail['discharge']);nonzero=signs[signs!=0]
236     if len(nonzero):
237         mode=int(nonzero[-1])
238     print('physical day', day, 'cost', rows[-1]['total_cost'], 'seconds', perf_counter()-began, flush=
239     True)
240     if days>30 and len(details)%10==0:
241         progress={key:np.stack([d[key] for d in details]) for key in details[0]}
242         progress['days']=np.arange(31,day+1)
243         np.savez_compressed(directory/'progress_dispatch.npz',**progress)
244         (directory/'progress_audit.json').write_text(json.dumps(audits,indent=2))
245         pd.DataFrame(rows).to_csv(directory/'progress_daily.csv',index=False)
246         if checkpoint_days is not None and len(details)==checkpoint_days:
247             gaps=[audit['mip']['mip_gap'] for audit in audits]
248             checkpoint={'completed_days':len(details),
249                       'feasible_days':sum(audit['mip']['feasible'] for audit in audits),
250                       'mip_gaps':gaps, 'mip_seconds':[audit['mip']['seconds'] for audit in audits],
251                       'maximum_gap_threshold':.05,
252                       'continue':all(gap is not None and gap<=.05 for gap in gaps),
253                       'cost_so_far':sum(row['total_cost'] for row in rows)}
254             (directory/'checkpoint.json').write_text(json.dumps(checkpoint,indent=2))
255             print('CHECKPOINT', json.dumps(checkpoint), flush=True)
256             if not checkpoint['continue']:
257                 return checkpoint
258     arrays={key:np.stack([detail[key] for detail in details]) for key in details[0]}
259     arrays['days']=np.arange(31,31+days)
260     np.savez_compressed(directory/'dispatch.npz',**arrays)
261     pd.DataFrame(rows).to_csv(directory/'daily.csv',index=False)
262     (directory/'planning_audit.json').write_text(json.dumps(audits,indent=2))
263     check=verify_npz(directory/'dispatch.npz',expected_days=days,audit_path=directory/'
264     planning_audit.json')
265     if not check['passed']:
266         raise AssertionError(check['errors'])
267     if prefix_count:
268         with np.load(Path(prefix_directory)/'dispatch.npz',allow_pickle=False) as old:
269             prefix_equal={key:bool(np.array_equal(old[key],arrays[key][:prefix_count])) for key in
270             old.files}
271         if not all(prefix_equal.values()):
272             raise AssertionError(prefix_equal)
273         (directory/'prefix_array_equality.json').write_text(json.dumps(prefix_equal,indent=2))
274     summary={'config':cfg, 'total_cost':float(arrays['fees'].sum()),
275             'battery':battery_metrics(arrays), 'verification':check,
276             'seconds':perf_counter()-began,
277             'scenario_physical_checks':[a['mip']['scenario_physical_checks'] for a in audits],
278             'mip_solver_metadata':[a['mip'] for a in audits]}
279     (directory/'summary.json').write_text(json.dumps(summary,indent=2))
280     print(json.dumps(summary,indent=2), flush=True)
281     return summary
282
283 if __name__=='__main__':
284     parser=argparse.ArgumentParser();parser.add_argument('--check',action='store_true')
285     parser.add_argument('--days',type=int,default=3)
286     parser.add_argument('--scenarios',type=int,default=7)
287     parser.add_argument('--checkpoint-days',type=int)
288     args=parser.parse_args()
289     OUT.mkdir(parents=True,exist_ok=True)
290     if args.check:

```

```

288     report=counterexample()
289     (OUT/'counterexample_fixed.json').write_text(json.dumps(report,indent=2))
290     print(json.dumps(report,indent=2))
291 else:
292     run(args.days,args.scenarios,args.checkpoint_days)

```

experiments/exp008/neural_joint_calibration.py

```

1  """Fair fixed Ridge28 postprocessing for the original and joint CNN archives."""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  from pathlib import Path
8
9  import numpy as np
10 import pandas as pd
11
12 from experiments.exp008.forecast_calibration import CONFIG, CalibratedStore, _calibrate
13 from experiments.problem2.exp003.data import ROOT, Data
14 from experiments.problem2.exp004.predict import ForecastStore
15
16 OUT=ROOT/'data/results/exp008/neural_joint_calibration'
17 JOINT=ROOT/'data/results/exp008/neural_joint'
18
19
20 class JointStore:
21     def __init__(self,calibrated=False):
22         directory=OUT if calibrated else JOINT
23         filename='joint_ridge28.npz' if calibrated else 'predictions.npz'
24         meta=json.loads((directory/'manifest.json').read_text())
25         if not meta['complete']:
26             raise ValueError('All eleven monthly joint outputs must be complete')
27         path=directory/filename
28         expected=meta['joint_calibrated_archive_sha256'] if calibrated else meta['archive_sha256']
29         if hashlib.sha256(path.read_bytes()).hexdigest()!=expected:
30             raise ValueError('Joint forecast archive hash mismatch')
31         with np.load(path) as archive:
32             self.origins=archive['origins'].copy();self.values=archive['values'].copy()
33             self.name='joint_ridge28' if calibrated else 'joint_raw'
34             self.lookup={int(origin):i for i,origin in enumerate(self.origins)}
35
36     def get(self,origin):
37         return self.values[self.lookup[int(origin)]]
38
39
40 def score(values,truth,price):
41     error=values-truth;net=error[:,0]-error[:,1]
42     high=price>=np.quantile(price,.75)
43     energy=net.sum(axis=1)/6;prefix=np.cumsum(net/6,axis=1)
44     return {'net_rmse_kw':float(np.sqrt(np.mean(net**2))),
45             'price_weighted_net_rmse_kw':float(np.sqrt(np.mean(price*net**2)/price.mean())),
46             'load_rmse_kw':float(np.sqrt(np.mean(error[:,0]**2))),
47             'pv_rmse_kw':float(np.sqrt(np.mean(error[:,1]**2))),
48             'net_bias_kw':float(net.mean()),'high_price_threshold':float(np.quantile(price,.75)),
49             'high_price_net_rmse_kw':float(np.sqrt(np.mean(net[:,high]**2))),
50             'high_price_net_mae_kw':float(np.abs(net[:,high]).mean()),
51             'high_price_net_bias_kw':float(net[:,high].mean()),
52             'daily_net_energy_rmse_kwh':float(np.sqrt(np.mean(energy**2))),
53             'daily_net_energy_mae_kwh':float(np.abs(energy).mean()),
54             'intraday_cumulative_error_rmse_kwh':float(np.sqrt(np.mean(prefix**2))),
55             'cumulative_net_energy_error_kwh':float(energy.sum())}
56
57
58 def run():
59     OUT.mkdir(parents=True,exist_ok=True)

```

```

60 cfg={'network_candidate_count':1,'calibrator':'unmodified_exp008_Ridge28',
61      'calibrator_config':CONFIG['ridge_28'],'data_explicit':True,'use_cache':False,
62      'base_store_injection':True,'no_existing_forecast_or_calibration_cache_overwritten':True,
63      'same_postprocessing_for_both_architectures':True,
64      'hyperparameters_changed':False,'development_on_2025_not_independent_validation':True,
65      'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
66      'calibrator_source_sha256':hashlib.sha256((ROOT/'experiments/exp008/forecast_calibration.py
').read_bytes()).hexdigest())
67 (OUT/'protocol.json').write_text(json.dumps(cfg,indent=2))
68 data=Data();original=ForecastStore('no_season',seed=42);joint=JointStore()
69 np.testing.assert_array_equal(original.origins,joint.origins)
70 calibrated={}
71 for name,store in [('original',original),('joint',joint)]:
72     result=CalibratedStore('ridge_28',seed=42,data=data,directory=OUT/name,use_cache=False,
73     _base_store=store)
74     calibrated[name]=result
75     path=OUT/f'{name}_ridge28.npz'
76     np.savez_compressed(path,origins=result.origins,values=result.values,delta=result.delta)
77     (OUT/f'{name}_ridge28_audit.json').write_text(json.dumps({
78         'config':CONFIG['ridge_28'],'days':result.audit,
79         'base_forecast_sha256':hashlib.sha256(store.values.tobytes()).hexdigest(),
80         'archive_sha256':hashlib.sha256(path.read_bytes()).hexdigest()},indent=2))
81 cached_original=CalibratedStore('ridge_28').values
82 original_parity=float(np.max(np.abs(cached_original-calibrated['original'].values)))
83 assert original_parity==0.
84 pollution=[]
85 for day in (31,62,243,364):
86     index=joint.lookup[day*144]
87     values,_=calibrate(data.actual,joint.origins,joint.values,index,CONFIG['ridge_28'])
88     changed=data.actual.copy();changed[day*144:]+=70000.
89     mutated,_=calibrate(changed,joint.origins,joint.values,index,CONFIG['ridge_28'])
90     error=float(np.max(np.abs(values-mutated)))
91     assert error==0.
92     pollution.append({'day':day,'current_and_future_actual_mutation_output_error_kw':error})
93 truth=data.actual[joint.origins[:],None]+np.arange(144)
94 dates=pd.date_range('2025-02-01','2025-12-31')
95 annual=[];monthly=[];daily=[]
96 for name,values in [('original_raw',original.values),('joint_raw',joint.values),
97                     ('original_ridge28',calibrated['original'].values),('joint_ridge28',
98                     calibrated['joint'].values)]:
99     annual.append({'name':name,**score(values,truth,data.fixed_price)})
100     for month in range(2,13):
101         ids=dates.month==month
102         monthly.append({'name':name,'month':month,**score(values[ids],truth[ids],data.
103         fixed_price)})
104     for i,date in enumerate(dates):
105         daily.append({'name':name,'date':str(date.date()),**score(values[i:i+1],truth[i:i+1],
106         data.fixed_price)})
107 pd.DataFrame(annual).to_csv(OUT/'annual_metrics.csv',index=False)
108 pd.DataFrame(monthly).to_csv(OUT/'monthly_metrics.csv',index=False)
109 pd.DataFrame(daily).to_csv(OUT/'daily_metrics.csv',index=False)
110 reference,candidate=annual[2],annual[3]
111 manifest={'complete':True,'days':334,'metrics':annual,
112          'original_recomputed_vs_cached_max_error_kw':original_parity,
113          'future_actual_pollution_checks':pollution,
114          'all_calibration_labels_are_prior_complete_days':all(
115              a['history_last_label'] is None or a['history_last_label']<a['origin']
116              for store in calibrated.values() for a in store.audit),
117          'joint_calibrated_archive_sha256':hashlib.sha256((OUT/'joint_ridge28.npz').read_bytes()).
118          hexdigest(),
119          'joint_calibrated_net_rmse_below_original_calibrated':candidate['net_rmse_kw']<reference['
120          net_rmse_kw'],
121          'joint_calibrated_weighted_rmse_below_original_calibrated':candidate['
122          price_weighted_net_rmse_kw']<reference['price_weighted_net_rmse_kw'],
123          'joint_calibrated_high_price_rmse_below_original_calibrated':candidate['
124          high_price_net_rmse_kw']<reference['high_price_net_rmse_kw']}
125 (OUT/'manifest.json').write_text(json.dumps(manifest,indent=2))
126 print(json.dumps(manifest,indent=2),flush=True)
127 return manifest

```

```

121
122 if __name__ == '__main__':
123     run()

```

experiments/exp008/planner.py

```

1  """Joint historical-path purchase planning; realized dispatch is verified separately.
2
3  The scenario recourse LP is an optimistic planning approximation: its battery
4  states are scenario dependent, not a certified nonanticipative scenario tree.
5  Only purchases are shared. No future realized observations enter this module.
6  """
7  from dataclasses import dataclass
8  from time import perf_counter
9
10 import numpy as np
11
12 from experiments.common.neural_v2.physics import LinearModel
13
14 ETA = float(np.sqrt(.9))
15 LOW, HIGH, LIMIT = 1200., 10800., 5000 / 6
16
17
18 @dataclass(frozen=True)
19 class Settings:
20     scenarios: int = 7
21     residual_scale: float = 1.
22     throughput_penalty: float = .005
23     variation_penalty: float = .001
24     terminal_value: float = 1.
25     emergency_weight: float = 5.
26     future_shortfall_weight: float | None = None
27
28
29 def plan(forecast, initial_soc, error_paths=None, original=None, settings=Settings(),
30         final_day=False, charge_mask=None, next_update_slot=None,
31         future_shortfall_weight=None):
32     """Return shared purchases plus representative inventory/mode intentions.
33
34     next_update_slot is relative to this remaining horizon: 36 means its first
35     36 ten-minute intervals execute before the next legal information release.
36     The optional future weight applies from that index onward, only when both
37     a boundary and a weight are present. It is an opportunity-value proxy, not
38     an alternative billing rule or a nonanticipative recourse certificate.
39     """
40     began = perf_counter()
41     net = (np.asarray(forecast['load_kw']) - forecast['pv_kw']) / 6
42     price = np.asarray(forecast['price'])
43     n = len(net)
44     shortfall_weights = np.broadcast_to(np.asarray(settings.emergency_weight, float), (n,)).copy()
45     future_weight = (getattr(settings, 'future_shortfall_weight', None)
46                     if future_shortfall_weight is None else future_shortfall_weight)
47     if future_weight is not None and (not np.isfinite(future_weight) or future_weight <= 0):
48         raise ValueError('future_shortfall_weight must be finite and positive')
49     if next_update_slot is not None:
50         if (not isinstance(next_update_slot, (int, np.integer))
51             or not 1 <= next_update_slot <= n):
52             raise ValueError('next_update_slot must be a relative horizon index in 1..n')
53         if future_weight is not None:
54             shortfall_weights[next_update_slot:] = future_weight
55     paths = np.asarray(error_paths) if error_paths is not None else np.zeros((1, n))
56     if not len(paths):
57         paths = np.zeros((1, n))
58     if paths.shape[1] == 144:
59         paths = paths[:, -n:]
60     if len(paths) > settings.scenarios:
61         # Deterministic equally spaced complete historical days, retaining

```

```

62     # within-day correlations. No sorting by future or realized cost.
63     paths = paths[np.linspace(0, len(paths)-1, settings.scenarios).astype(int)]
64     demands = net[None, :] + settings.residual_scale * paths
65     k = len(demands)
66     assert demands.shape == (k, n) and price.shape == (n,)
67     assert np.isfinite(demands).all() and np.all(price > 0)
68     assert LOW - 1e-6 <= initial_soc <= HIGH + 1e-6
69     m = LinearModel()
70     g = m.variables((n,), cost=price if original is None else 1.5*price,
71                     lower=0 if original is None else np.asarray(original))
72     # Under the inherited billing rule the original plan is paid in full,
73     # unused energy may be discarded, and reducing it adds a penalty. Thus
74     # a downward revision is dominated by retaining the original purchase.
75     charge_limit = LIMIT if charge_mask is None else LIMIT*np.asarray(charge_mask)[None, :]
76     discharge_limit = LIMIT if charge_mask is None else LIMIT*(1-np.asarray(charge_mask))[None, :]
77     c = m.variables((k, n), upper=charge_limit, cost=settings.throughput_penalty/k)
78     d = m.variables((k, n), upper=discharge_limit, cost=settings.throughput_penalty/k)
79     e = m.variables((k, n), cost=shortfall_weights*price[None, :]/k)
80     w = m.variables((k, n))
81     s = m.variables((k, n), lower=LOW, upper=HIGH)
82     tv = m.variables((k, n), cost=settings.variation_penalty*6/k)
83     terminal = 0 if final_day else settings.terminal_value*price.min()/ETA
84     for index in s[:, -1]:
85         m.objective[int(index)] = -terminal/k
86     for j in range(k):
87         for t in range(n):
88             m.constraint([(g[t], 1), (d[j,t], 1), (e[j,t], 1),
89                           (c[j,t], -1), (w[j,t], -1)], demands[j,t], demands[j,t])
90             transition = [(s[j,t], 1), (c[j,t], -ETA), (d[j,t], 1/ETA)]
91             rhs = initial_soc if t == 0 else 0
92             if t:
93                 transition.append((s[j,t-1], -1))
94             m.constraint(transition, rhs, rhs)
95             if t:
96                 diff = [(c[j,t],1),(d[j,t],-1),(c[j,t-1],-1),(d[j,t-1],1)]
97                 m.constraint(diff + [(tv[j,t],-1)], upper=0)
98                 m.constraint([(ids,-v) for ids,v in diff] + [(tv[j,t],-1)], upper=0)
99     x, meta = m.solve(seconds=20, gap=1e-7)
100     if x is None:
101         raise RuntimeError(f'Joint-path LP failed: {meta}')
102     return {'purchase': np.maximum(x[g], 0), 'charge': np.maximum(x[c].mean(0), 0),
103           'discharge': np.maximum(x[d].mean(0), 0),
104           'states': np.r_[initial_soc, x[s].mean(0)],
105           'metadata': {'*meta', 'planning_seconds': perf_counter()-began,
106                       'method': 'joint_path_lp_surrogate', 'scenarios': k,
107                       'nonanticipative_recourse_certificate': False,
108                       'next_update_slot': None if next_update_slot is None else int(
109 next_update_slot),
110                       'future_shortfall_weight': future_weight,
111                       'planning_emergency_weights': shortfall_weights.tolist(),
112                       'opportunity_proxy_active': bool(future_weight is not None
113 and next_update_slot is not None and next_update_slot < n),
114                       'downward_revision_dominated': original is not None}}
115
116 def execute(purchase, actual_kw, price, initial_soc, *, charge_deadband=20.,
117            discharge_deadband=0., ramp_kw=None, previous_power=0., previous_mode=0,
118            mode_hold_slots=0, planned_mode=None, charge_mask=None):
119     """Causal physical feedback. Emergencies supply unmet load, never charge.
120
121     Ramp clipping is optional: it is softened toward zero if maintaining an
122     earlier direction would require intentional emergency charging/discharging
123     into surplus. Realized ramp violations are retained as diagnostics.
124     """
125     n = len(purchase)
126     charge, discharge, emergency, surplus = (np.zeros(n) for _ in range(4))
127     states = np.empty(n+1)
128     states[0] = initial_soc
129     mode, age, pprev = previous_mode, mode_hold_slots, previous_power
130     for t in range(n):

```

```

131     balance = purchase[t] + (actual_kw[t,1] - actual_kw[t,0])/6
132     if balance >= 0:
133         action = min(balance, LIMIT, max(0, (HIGH-states[t])/ETA))
134         if action < charge_deadband or (mode < 0 and age < mode_hold_slots):
135             action = 0.
136     else:
137         action = -min(-balance, LIMIT, max(0, (states[t]-LOW)*ETA))
138         if -action < discharge_deadband:
139             action = 0.
140     if charge_mask is not None:
141         if (balance >= 0 and not charge_mask[t]) or (balance < 0 and charge_mask[t]):
142             action = 0.
143     if planned_mode is not None and balance >= 0 and planned_mode[t] < 0:
144         action = 0.
145     if ramp_kw is not None:
146         action = np.clip(action, (pprev-ramp_kw)/6, (pprev+ramp_kw)/6)
147         action = min(max(0,action), balance, LIMIT, max(0,(HIGH-states[t])/ETA)) if balance >= 0
148     else max(min(0,action),balance,-LIMIT,-max(0,(states[t]-LOW)*ETA))
149         if charge_mask is not None and ((balance >= 0 and not charge_mask[t]) or
150             (balance < 0 and charge_mask[t])):
151             action = 0.
152     charge[t], discharge[t] = max(action,0), max(-action,0)
153     surplus[t] = max(0,balance-charge[t])
154     emergency[t] = max(0,-balance-discharge[t])
155     states[t+1] = states[t]+ETA*charge[t]-discharge[t]/ETA
156     sign = int(np.sign(action))
157     if sign and sign != mode:
158         mode, age = sign, 1
159     else:
160         age += 1
161     pprev = 6*action
162     return dict(charge=charge, discharge=discharge, emergency=emergency,
163                 surplus=surplus, states=states)

```

experiments/exp008/risk_window.py

```

1  """Bounded causal residual-window ablation; forecast, LP and execution fixed.
2
3  The only new settings are tree7, tree14 and per_slot14. The tree28 control
4  is replayed because the existing ridge28 q=.8 archive has no state buffer.
5  This is development on 2025, not an independent validation sample.
6  """
7  from __future__ import annotations
8
9  import argparse
10 import hashlib
11 import json
12 from pathlib import Path
13 from time import perf_counter
14
15 import numpy as np
16 import pandas as pd
17 from sklearn.tree import DecisionTreeRegressor
18
19 from experiments.exp008.controller_candidate import (
20     INITIAL_SOC, execute_inventory, plan_inventory,
21 )
22 from experiments.exp008.forecast_calibration import CalibratedStore
23 from experiments.exp008.verify import verify_arrays
24 from experiments.problem2.exp003.data import Data
25 from experiments.problem2.tree_planning import risk as original_risk
26 from experiments.problem2.tree_planning.risk import (
27     DT_HOURS, FEATURE_NAMES, QUANTILE_LEVELS, STEPS, TreeResidualScenarios,
28     periodic_baseline, tree_features,
29 )
30
31 ROOT = Path(__file__).resolve().parents[2]

```

```

32 OUT = ROOT / "data/results/exp008/risk_window"
33 CONFIGURATIONS = (("tree7", 7, "tree"), ("tree14", 14, "tree"),
34                  ("per_slot14", 14, "per_slot"))
35 SPEC = {"calibration": "ridge_28", "quantile": .8, "controller": "greedy",
36        "state_buffer": 500.}
37
38
39 def write_json(path, value):
40     Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2) + "\n")
41
42
43 class WindowResidualScenarios(TreeResidualScenarios):
44     """Original tree algorithm with a constructor-level history-day limit.
45
46     "for_day" copies the original implementation rather than modifying a
47     shared module global. Tree hyperparameters, quantile interpolation and
48     periodic fallback are identical. Every actual read uses inherited
49     "_completed_day" and its exclusive issue cutoff.
50     """
51
52     def __init__(self, *args, history_days=28, **kwargs):
53         super().__init__(*args, **kwargs)
54         if history_days not in (7, 14, 28):
55             raise ValueError("This ablation only permits 7, 14 or control 28 days")
56         self.history_days = int(history_days)
57
58     def for_day(self, day):
59         if isinstance(day, (bool, np.bool_)) or not isinstance(day, (int, np.integer)):
60             raise TypeError("day must be a zero-based integer day index")
61         day = int(day)
62         cutoff = day * STEPS
63         if cutoff not in self._lookup:
64             raise ValueError("requested day has no frozen forecast")
65         historical_ids = np.flatnonzero(self.origins + STEPS <= cutoff)[-self.history_days:]
66         fallback = len(historical_ids) < 2
67         features, residuals, training_days = [], [], []
68         if fallback:
69             for historical_day in range(max(1, day-self.history_days), day):
70                 issue_cutoff = historical_day * STEPS
71                 previous = self._completed_day(historical_day-1, issue_cutoff)
72                 weekly = (self._completed_day(historical_day-7, issue_cutoff)
73                          if historical_day >= 7 else None)
74                 predicted = periodic_baseline(previous, weekly)
75                 observed = self._completed_day(historical_day, cutoff)
76                 features.append(tree_features(predicted, self.price144))
77                 residuals.append(((observed[:, 0]-observed[:, 1])
78                                - (predicted[:, 0]-predicted[:, 1])) * DT_HOURS)
79                 training_days.append(historical_day)
80         else:
81             for index in historical_ids:
82                 historical_day = int(self.origins[index] // STEPS)
83                 predicted = self.values[index]
84                 observed = self._completed_day(historical_day, cutoff)
85                 features.append(tree_features(predicted, self.price144))
86                 residuals.append(((observed[:, 0]-observed[:, 1])
87                                - (predicted[:, 0]-predicted[:, 1])) * DT_HOURS)
88                 training_days.append(historical_day)
89         if not training_days:
90             raise ValueError("no completed calibration day exists")
91         x_train, y_train = np.concatenate(features), np.concatenate(residuals)
92         current_forecast = self.values[self._lookup[cutoff]]
93         x_current = tree_features(current_forecast, self.price144)
94         tree, fit_seconds = None, 0.
95         if self.conditioning == "tree":
96             tree = DecisionTreeRegressor(max_depth=5, min_samples_leaf=48, random_state=42)
97             began = perf_counter()
98             tree.fit(x_train, y_train)
99             fit_seconds = perf_counter()-began
100             train_leaves = tree.apply(x_train)
101             leaf_supports = {

```

```

102         int(leaf): np.quantile(y_train[train_leaves == leaf], QUANTILE_LEVELS,
103                                method="linear") for leaf in np.unique(train_leaves)
104     }
105     errors = np.vstack([leaf_supports[int(leaf)] for leaf in tree.apply(x_current)])
106 else:
107     errors = np.quantile(np.stack(residuals), QUANTILE_LEVELS,
108                           axis=0, method="linear").T
109 support = x_current[:, 2, None] + errors
110 info = {
111     "day": day, "cutoff": cutoff, "information_cutoff": cutoff,
112     "cutoff_is_exclusive": True, "training_start_day": min(training_days),
113     "training_end_day": max(training_days), "training_days": len(training_days),
114     "training_origins": [int(value*STEPS) for value in training_days],
115     "sample_count": len(y_train), "issued_history_days": len(historical_ids),
116     "max_observed_index": int((max(training_days)+1)*STEPS-1),
117     "fit_seconds": float(fit_seconds), "tree_nodes": int(tree.tree_.node_count) if tree else
118     0,
119     "tree_leaves": int(tree.tree_.n_leaves) if tree else 0,
120     "conditioning": self.conditioning, "history_days_limit": self.history_days,
121     "fallback": bool(fallback),
122     "residual_source": "periodic_baseline" if fallback else "ridge28_prequential_forecast",
123     "forecast_calibration": "ridge_28", "base_risk_cache_used": False,
124     "feature_names": list(FEATURE_NAMES), "residual_units": "kWh_per_10min",
125     "scenario_units": "kWh_per_10min",
126     "scenario_semantics": "conditional_slot_marginals_not_joint_daily_paths",
127     "quantile_levels": QUANTILE_LEVELS.tolist(), "weights": self.weights.tolist(),
128     "max_depth": 5, "min_samples_leaf": 48, "random_state": 42,
129 }
130 return support, info
131
132 def implementation_check(data, store):
133     common = (store.origins, store.values, data.actual, data.fixed_price)
134     control = WindowResidualScenarios(*common)
135     reference = TreeResidualScenarios(*common)
136     days = (31, 32, 33, 45, 90, 180, 364)
137     for day in days:
138         np.testing.assert_array_equal(control.for_day(day)[0], reference.for_day(day)[0])
139     for name, window, conditioning in CONFIGURATIONS:
140         risk = WindowResidualScenarios(*common, history_days=window, conditioning=conditioning)
141         for day in (31, 45, 180):
142             support, audit = risk.for_day(day)
143             changed_actual = data.actual.copy()
144             changed_actual[day*144:] = 1e7
145             changed_forecast = store.values.copy()
146             changed_forecast[store.origins > day*144] = 2e7
147             changed = WindowResidualScenarios(store.origins, changed_forecast, changed_actual,
148                                                data.fixed_price, history_days=window, conditioning=conditioning)
149             np.testing.assert_array_equal(support, changed.for_day(day)[0])
150             assert audit["training_days"] <= window
151             assert audit["max_observed_index"] < day*144
152             assert all(origin+144 <= day*144 for origin in audit["training_origins"])
153     return {"passed": True, "tree28_exact_original_days": list(days),
154            "future_actual_and_future_forecast_mutation_days": [31, 45, 180],
155            "mutated_configurations": [item[0] for item in CONFIGURATIONS]}
156
157 def distribution_metrics(supports, truth, price):
158     """Post-release diagnostics only; never used to form a day's support."""
159     levels = QUANTILE_LEVELS
160     errors = truth[..., None]-supports
161     pinball = np.maximum(levels*errors, (levels-1)*errors)
162     threshold = np.quantile(supports, .8, axis=2)
163     peak = np.broadcast_to(price >= np.quantile(price, .75), truth.shape)
164     coverage = (truth[..., None] <= supports).mean(axis=(0, 1))
165     return {
166         "support_quantile_levels": levels.tolist(), "empirical_coverage": coverage.tolist(),
167         "mean_absolute_coverage_error": float(np.abs(coverage-levels).mean()),
168         "mean_pinball_kwh": float(pinball.mean()),
169         "price_weighted_pinball_yuan": float((pinball*price[None, :, None]).mean()),
170     }

```

```

171         "central_support_range_coverage": float(((truth >= supports[... , 0])
172                                                    & (truth <= supports[... , -1])).mean()),
173         "central_support_range_nominal": float(levels[-1]-levels[0]),
174         "central_support_range_mean_width_kwh": float((supports[... , -1]-supports[... , 0]).mean()),
175         "lp_q080_coverage": float((truth <= threshold).mean()),
176         "lp_q080_high_tariff_coverage": float((truth <= threshold)[peak].mean()),
177         "lp_q080_mean_predicted_minus_actual_kwh": float((threshold-truth).mean()),
178         "lp_q080_interpolated_underlying_level": float(np.quantile(levels, .8)),
179         "lp_q080_note": "LP takes numpy quantile .8 over nine equally weighted support atoms;
preserved exactly",
180     }
181
182
183 def replay(name, window, conditioning, days, data, store, out):
184     directory = out / f"{name}_{days}days"
185     completed = directory / "summary.json"
186     if completed.exists():
187         return json.loads(completed.read_text())
188     directory.mkdir(parents=True, exist_ok=True)
189     risk = WindowResidualScenarios(store.origins, store.values, data.actual, data.fixed_price,
190                                    history_days=window, conditioning=conditioning)
191     soc, mode = INITIAL_SOC, 1
192     parts, rows, audits, all_supports = [], [], [], []
193     began = perf_counter()
194     for day in range(31, 31+days):
195         support, audit = risk.for_day(day)
196         plan = plan_inventory(support, data.fixed_price, soc, SPEC, final=day == 364)
197         observed = data.actual[day*144:(day+1)*144]
198         detail, mode = execute_inventory(plan, observed, data.fixed_price, soc, mode, SPEC)
199         audit.update(forecast_origin=day*144, purchase_locked_before_actual_read=True)
200         parts.append(detail)
201         all_supports.append(support)
202         audits.append(audit)
203         net = (observed[:, 0]-observed[:, 1])/6
204         coverage = (net <= np.quantile(support, .8, axis=1)).mean()
205         rows.append({"day": day, "date": str((pd.Timestamp("2025-01-01")+pd.Timedelta(days=day)).
date()),
206                    "total_cost": float(detail["fees"].sum()), "planned_cost": float(detail["fees"][:, 0].
sum()),
207                    "emergency_cost": float(detail["fees"][:, 3].sum()),
208                    "emergency_kwh": float(detail["emergency"].sum()), "lp_q080_coverage": float(coverage),
209                    "initial_soc": soc, "final_soc": float(detail["states"][-1])})
210         soc = rows[-1]["final_soc"]
211     arrays = {key: np.stack([part[key] for part in parts]) for key in parts[0]}
212     arrays["days"] = np.arange(31, 31+days)
213     supports = np.stack(all_supports)
214     truth = (arrays["actual"][:, :, 0]-arrays["actual"][:, :, 1])/6
215     verification = verify_arrays(arrays, expected_days=days,
216                                  source_actual=data.actual[31*144:(31+days)*144].reshape(days, 144, 2),
217                                  source_price=data.fixed_price, audit_records=audits)
218     if not verification["passed"]:
219         raise AssertionError(verification["errors"])
220     summary = {"name": name, "history_days": window, "conditioning": conditioning,
221               "fixed_spec": SPEC, "days": days, "start_day": 31,
222               **verification["billing"], "battery": verification["battery_metrics"],
223               "distribution": distribution_metrics(supports, truth, data.fixed_price),
224               "verified": True, "elapsed_seconds": perf_counter()-began,
225               "evaluation_role": "2025 development; fixed first 30 days gate before any annual extension"}
226     np.savez_compressed(directory/"dispatch_2.npz", **arrays)
227     np.savez_compressed(directory/"supports.npz", supports=supports, days=arrays["days"])
228     pd.DataFrame(rows).to_csv(directory/"daily.csv", index=False)
229     write_json(directory/"audit.json", audits)
230     write_json(directory/"verification.json", verification)
231     write_json(completed, summary)
232     print(json.dumps({"name": name, "days": days, "cost": summary["total_cost"],
233                      "emergency_cost": summary["emergency_cost"],
234                      "lp_q080_coverage": summary["distribution"]["lp_q080_coverage"]}), flush=True)
235     return summary
236
237

```

```

238 def run(out=OUT, extend=False):
239     out = Path(out)
240     out.mkdir(parents=True, exist_ok=True)
241     data, store = Data(), CalibratedStore("ridge_28")
242     protocol = {"fixed_spec": SPEC, "control": ["tree28", 28, "tree"],
243               "new_configurations": [list(item) for item in CONFIGURATIONS],
244               "pilot": {"first_day": "2025-02-01", "last_day": "2025-03-02", "days": 30},
245               "annual_gate": "at least 0.5% lower real pilot bill after adding an upper-bound cost for any
246               lower final SOC; physics must pass",
247               "final_soc_charge": "max(0, control_final_soc - candidate_final_soc) * sqrt(.9) * 5 * max(
248               fixed_price)",
249               "no_current_day_truth_window_selection": True,
250               "source_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
251               "original_risk_sha256": hashlib.sha256(Path(original_risk.__file__).read_bytes()).hexdigest
252               ()),
253               "controller_sha256": hashlib.sha256((ROOT/"experiments/exp008/controller_candidate.py").
254               read_bytes()).hexdigest(),
255               "forecast_values_sha256": hashlib.sha256(store.values.tobytes()).hexdigest(),
256               "fixed_controls_note": "ridge28 q=.8 buffer500 control replayed; existing ridge28 q=.8
257               annual archive has no state buffer"}
258     if (out/"protocol.json").exists():
259         if json.loads((out/"protocol.json").read_text()) != protocol:
260             raise RuntimeError("Refusing to overwrite changed risk-window protocol")
261     else:
262         write_json(out/"protocol.json", protocol)
263     write_json(out/"implementation_verification.json", implementation_check(data, store))
264     baseline = replay("tree28", 28, "tree", 30, data, store, out)
265     pilots, eligible = [], []
266     for name, window, conditioning in CONFIGURATIONS:
267         result = replay(name, window, conditioning, 30, data, store, out)
268         soc_cost = max(0, baseline["battery"]["final_soc"]-result["battery"]["final_soc"])*np.sqrt
269         (.9)*5*data.fixed_price.max()
270         adjusted_saving = baseline["total_cost"]-result["total_cost"]-soc_cost
271         gate = bool(result["verified"] and adjusted_saving >= .005*baseline["total_cost"])
272         pilots.append({"name": name, "total_cost": result["total_cost"],
273                       "delta_cost_vs_control": result["total_cost"]-baseline["total_cost"],
274                       "emergency_cost": result["emergency_cost"], "final_soc_depletion_upper_bound_cost":
275                       float(soc_cost),
276                       "conservatively_adjusted_saving": float(adjusted_saving), "annual_gate": gate,
277                       "distribution": result["distribution"], "battery": result["battery"]})
278         if gate:
279             eligible.append((name, window, conditioning))
280     annual = []
281     if extend and eligible:
282         annual.append(replay("tree28", 28, "tree", 334, data, store, out))
283         for args in eligible:
284             annual.append(replay(*args, 334, data, store, out))
285     summary = {"control": baseline, "pilot_candidates": pilots,
286               "annual_eligible": [item[0] for item in eligible],
287               "annual_completed": [row["name"] for row in annual]}
288     write_json(out/"summary.json", summary)
289     print(json.dumps({"annual_eligible": summary["annual_eligible"],
290                       "annual_completed": summary["annual_completed"]}), flush=True)
291     return summary
292
293 if __name__ == "__main__":
294     parser = argparse.ArgumentParser(description=__doc__)
295     parser.add_argument("--out", type=Path, default=OUT)
296     parser.add_argument("--extend-if-improved", action="store_true")
297     arguments = parser.parse_args()
298     run(arguments.out, arguments.extend_if_improved)

```

experiments/exp008/unified_forecast.py

```

1 """Optional causal output calibration shared by all four annual scenarios."""
2 from .forecast import Forecasts

```

```

3 from .forecast_calibration import CalibratedStore
4
5
6 class UnifiedForecasts(Forecasts):
7     def __init__(self, data=None, seed=42, calibration=None):
8         super().__init__(data=data, seed=seed)
9         self.calibration=calibration
10        if calibration:
11            self.store=CalibratedStore(calibration, seed=seed)
12
13    def get(self, day, slot=0, scenario='2'):
14        result=super().get(day, slot, scenario)
15        result['audit']['output_calibration']=self.calibration
16        result['audit']['base_architecture_unchanged']=True
17        if self.calibration:
18            result['audit']['calibration_labels_end_exclusive']=day*144
19        return result

```

experiments/exp008/verify.py

```

1 """Independent physical, billing, chronology and goal checks for exp008.
2
3 No optimizer, forecast or controller modules are imported. Archive checks
4 cannot prove forecast causality; optional issue audit records are checked
5 separately and their exact scope is reported.
6 """
7
8 import argparse
9 import hashlib
10 import json
11 from pathlib import Path
12
13 import numpy as np
14 import pandas as pd
15
16 ROOT = Path(__file__).resolve().parents[2]
17 ETA = float(np.sqrt(0.9))
18 MIN_SOC, MAX_SOC, CAPACITY = 1200.0, 10800.0, 12000.0
19 POWER_ENERGY, TOL = 5000 / 6, 1e-6
20 INITIAL_SOC = 1421.7991105135516
21 INITIAL_MODE, INITIAL_POWER = 1, 172.759999999999976
22 BASELINE_COST = 14066257.477256786
23 BASELINE_REVERSALS, BASELINE_ACTIVE = 2729, 23028
24 BASELINE_THROUGHPUT = 11480847.039823763
25 GOAL_COST = 12940956.8791
26
27
28 def battery_metrics(detail, initial_mode=INITIAL_MODE, initial_power_kw=INITIAL_POWER):
29     """Count actions on the chronological 10-minute trace, never block sums."""
30     c, d = (np.asarray(detail[key], float).ravel() for key in ("charge", "discharge"))
31     states = np.asarray(detail["states"], float)
32     power = 6 * (c - d)
33     modes = np.where(np.abs(c - d) > TOL, np.sign(c - d), 0).astype(int)
34     nonidle = modes[modes != 0]
35     sequence = np.r_[initial_mode, nonidle]
36     ramps = np.abs(np.diff(power))
37     return {
38         "charge_kwh": float(c.sum()), "discharge_kwh": float(d.sum()),
39         "throughput_kwh": float((c + d).sum()),
40         "equivalent_full_cycles": float((ETA * c.sum() + d.sum()) / ETA) / (2 * CAPACITY),
41         "direction_reversals": int(np.sum(nonidle[1:] * nonidle[:-1] == -1)),
42         "direction_reversals_including_warmup_boundary": int(np.sum(sequence[1:] * sequence[:-1] ==
43         -1)),
44         "charging_slots": int(np.sum(c > TOL)), "discharging_slots": int(np.sum(d > TOL)),
45         "active_slots": int(np.sum((c > TOL) | (d > TOL))),
46         "simultaneous_slots": int(np.sum((c > TOL) & (d > TOL))),
47         "max_overlap_kwh": float(np.minimum(c, d).max(initial=0)),

```

```

47     "charge_starts": int(np.sum((modes == 1) & (np.r_[0, modes[:-1]] != 1))),
48     "discharge_starts": int(np.sum((modes == -1) & (np.r_[0, modes[:-1]] != -1))),
49     "power_changes_count": int(np.sum(ramps > TOL)),
50     "power_ramp_total_kw": float(ramps.sum()),
51     "power_ramp_total_kw_including_warmup_boundary": float(ramps.sum() + abs(power[0] -
initial_power_kw)),
52     "power_ramp_max_kw_per_10min": float(ramps.max(initial=0)),
53     "power_ramp_p95_kw_per_10min": float(np.quantile(ramps, .95)) if ramps.size else 0.0,
54     "mean_absolute_power_kw": float(np.abs(power).mean()),
55     "max_charge_kw": float(6 * c.max(initial=0)), "max_discharge_kw": float(6 * d.max(initial=0)
),
56     "initial_soc": float(states[0, 0]), "final_soc": float(states[-1, -1]),
57     "minimum_soc": float(states.min()), "maximum_soc": float(states.max()),
58     "initial_mode": int(initial_mode), "final_mode": int(nonidle[-1]) if nonidle.size else int(
initial_mode),
59     "initial_power_kw": float(initial_power_kw), "final_power_kw": float(power[-1]),
60     "degradation_interpretation": "operating intensity proxies; not calibrated battery ageing or
lifetime",
61 }
62
63
64 def goal_check(total_cost, battery, physics_passed=True, full_evaluation=True):
65     """The fixed exp006 Q2 comparator is never reselected after iteration."""
66     checks = {
67         "cost_reduced_at_least_8pct": bool(full_evaluation and total_cost <= GOAL_COST + TOL),
68         "direction_reversals_reduced": bool(full_evaluation and battery["direction_reversals"] <
BASELINE_REVERSALS),
69         "no_simultaneous_charge_discharge": bool(battery["simultaneous_slots"] == 0),
70         "physical_checks_passed": bool(physics_passed),
71         "full_334_day_evaluation": bool(full_evaluation),
72     }
73     return {
74         "passed": all(checks.values()), "checks": checks,
75         "count_definition": "exp006 non-idle direction reversals; active ten-minute slots are a
duration diagnostic",
76         "stricter_all_operation_metrics_passed": bool(all(checks.values()
and battery["active_slots"] < BASELINE_ACTIVE
and battery["throughput_kwh"] < BASELINE_THROUGHPUT),
77         "active_slots_also_reduced": bool(full_evaluation and battery["active_slots"] <
BASELINE_ACTIVE),
78         "baseline": {"run_id": "exp006/primary", "total_cost": BASELINE_COST,
79                     "direction_reversals": BASELINE_REVERSALS, "active_slots": BASELINE_ACTIVE,
80                     "throughput_kwh": BASELINE_THROUGHPUT},
81         "cost_threshold_yuan": GOAL_COST,
82         "cost_reduction_pct": float(100 * (1 - total_cost / BASELINE_COST)) if full_evaluation else
None,
83         "direction_reversal_reduction_pct": float(100 * (1 - battery["direction_reversals"] /
BASELINE_REVERSALS)) if full_evaluation else None,
84         "active_slot_reduction_pct": float(100 * (1 - battery["active_slots"] / BASELINE_ACTIVE)) if
full_evaluation else None,
85         "throughput_reduction_pct": float(100 * (1 - battery["throughput_kwh"] / BASELINE_THROUGHPUT
)) if full_evaluation else None,
86         "throughput_also_reduced": bool(full_evaluation and battery["throughput_kwh"] <
BASELINE_THROUGHPUT),
87         "cost_reduced_at_least_10pct": bool(full_evaluation and total_cost <= .9 * BASELINE_COST +
TOL),
88         "comparison_scope": "full comparable 334 days" if full_evaluation else "partial run: no
comparison to full-year totals",
89     }
90
91
92
93
94 def verify_arrays(detail, scenario="2", *, expected_days=334, start_day=31,
95                 initial_soc=INITIAL_SOC, initial_mode=INITIAL_MODE,
96                 initial_power_kw=INITIAL_POWER, source_actual=None, source_price=None,
97                 audit_records=None, ramp_limit_kw=None):
98     """Verify arrays with shape (days,144), SOC (days,145), fees (... ,4).
99
100     Q2/4-2 lock purchases at midnight. Q3/4-3 use the frozen original-plan
101     settlement: p*g0 + 1.5*p*(g-g0)+ + .5*p*(g0-g)+ + 5*p*emergency.
102     Actual may retain additional channels; first two are load/PV in kW.

```

```

103 """
104 scenario = str(scenario)
105 if scenario not in ("1", "2", "3", "4-2", "4-3"):
106     raise ValueError("scenario must be 1, 2, 3, 4-2 or 4-3")
107 errors = []
108
109 def check(condition, message):
110     if not bool(condition):
111         errors.append(message)
112
113 required = ("original", "final", "charge", "discharge", "emergency", "surplus", "price", "states",
114            "fees", "actual")
115 for key in required:
116     check(key in detail, f"missing:{key}")
117 if errors:
118     return {"passed": False, "errors": errors, "scenario": scenario}
119 a = {key: np.asarray(detail[key], dtype=float) for key in required}
120 for key in required:
121     expected = ((expected_days, 145) if key == "states" else
122                (expected_days, 144, 4) if key == "fees" else (expected_days, 144))
123     if key == "actual":
124         check(a[key].ndim == 3 and a[key].shape[:2] == (expected_days, 144)
125              and a[key].shape[-1] >= 2, "shape:actual")
126     else:
127         check(a[key].shape == expected, f"shape:{key}")
128         check(np.isfinite(a[key]).all(), f"nonfinite:{key}")
129         check(np.min(a[key], initial=0) >= -TOL, f"negative:{key}")
130 if errors:
131     return {"passed": False, "errors": errors, "scenario": scenario}
132 original, q, c, d, e, w, s, p = (a[key] for key in
133                                ("original", "final", "charge", "discharge", "emergency", "surplus", "states", "price"))
134 check(np.all(p > 0), "nonpositive_tariff")
135 if scenario in ("1", "2", "4-2"):
136     check(np.max(np.abs(original - q)) <= TOL, "midnight_purchase_changed")
137 balance = q + (a["actual"][..., 1] - a["actual"][..., 0]) / 6 + d + e - c - w
138 state_error = np.diff(s, axis=1) - ETA * c + d / ETA
139 continuity = float(np.max(np.abs(s[1:, 0] - s[:-1, -1]), initial=0))
140 max_balance, max_state = float(np.abs(balance).max()), float(np.abs(state_error).max())
141 check(max_balance <= TOL, "supply_demand_balance")
142 check(max_state <= TOL, "battery_state_equation")
143 check(continuity <= TOL, "cross_day_soc_discontinuity")
144 check(abs(float(s[0, 0]) - initial_soc) <= TOL, "initial_soc")
145 check(s.min() >= MIN_SOC - TOL and s.max() <= MAX_SOC + TOL, "soc_bounds")
146 check(max(c.max(), d.max()) <= POWER_ENERGY + TOL, "battery_power_limit")
147 check(not np.any((c > TOL) & (d > TOL)), "simultaneous_charge_discharge")
148 check(not np.any((c > TOL) & (e > TOL)), "emergency_charging")
149 if scenario == "1":
150     check(np.max(e) <= TOL, "q1_emergency_purchase")
151     check(np.max(np.abs(s[:, -1] - s[:, 0])) <= TOL, "q1_daily_soc_equality")
152 expected_fees = np.stack((original * p, 1.5 * p * np.maximum(q - original, 0),
153                          .5 * p * np.maximum(original - q, 0), 5 * p * e), axis=-1)
154 fee_error = float(np.max(np.abs(expected_fees - a["fees"])))
155 check(fee_error <= TOL, "slot_settlement")
156 if source_actual is not None:
157     source = np.asarray(source_actual, float)
158     check(source.shape[:2] == a["actual"].shape[:2] and source.shape[-1] >= 2
159          and np.array_equal(a["actual"][..., :2], source[..., :2]), "source_actual_mismatch")
160 if source_price is not None:
161     check(np.array_equal(p, np.broadcast_to(source_price, p.shape)), "source_tariff_mismatch")
162 # Explicit identifiers, when available, are compared rather than inferred.
163 if "days" in detail:
164     check(np.array_equal(detail["days"], np.arange(start_day, start_day + expected_days)), "day_identifiers")
165 if "origins" in detail:
166     check(np.array_equal(detail["origins"], np.arange(start_day, start_day + expected_days) *
167              144), "midnight_origins")
168 audit_checks = 0
169 if audit_records is not None:
170     expanded = []

```

```

169     outer_origins = []
170     for row in audit_records:
171         outer = row.get("information_cutoff", row.get("origin"))
172         if outer is not None:
173             outer_origins.append(int(outer))
174         expanded.append(row)
175         if isinstance(row.get("forecast"), dict):
176             forecast_audit = row["forecast"]
177             expanded.append(forecast_audit)
178             if outer is not None and forecast_audit.get("origin") is not None:
179                 check(int(outer) == int(forecast_audit["origin"]), "
forecast_and_execution_issue_mismatch")
180                 audit_checks += 1
181         if outer_origins:
182             check(outer_origins == sorted(outer_origins), "audit_chronological_order")
183             check({value // 144 for value in outer_origins}
184                   == set(range(start_day, start_day + expected_days)), "audit_day_coverage")
185     for row in expanded:
186         origin = row.get("information_cutoff", row.get("information_cutoff_exclusive",
187                                                         row.get("origin", row.get("forecast_origin"))))
188         if origin is None:
189             continue
190         origin = int(origin)
191         check(origin % 144 in ((0, 36, 72, 108) if scenario in ("3", "4-3") else (0,)), "
invalid_issue_time")
192         if row.get("max_observed_index") is not None:
193             check(int(row["max_observed_index"]) < origin, "future_actual_used")
194             audit_checks += 1
195         if "forecast_origin" in row:
196             check(int(row["forecast_origin"]) <= origin, "future_forecast_issue_used")
197             audit_checks += 1
198         if "training_origins" in row:
199             check(all(int(value) + 144 <= origin for value in row["training_origins"]), "
incomplete_training_day")
200             audit_checks += 1
201         if "pv_calibration_origins" in row:
202             check(all(int(value) + 144 <= origin for value in row["pv_calibration_origins"]), "
incomplete_pv_calibration_horizon")
203             audit_checks += 1
204         for key in ("pv_calibration_last_label", "load_correction_last_label", "price_last_label
"):
205             if row.get(key) is not None:
206                 check(int(row[key]) < origin, f"future_label:{key}")
207                 audit_checks += 1
208             if row.get("pv_issue_origin") is not None:
209                 check(int(row["pv_issue_origin"]) <= origin, "unreleased_official_pv_forecast")
210                 audit_checks += 1
211             if "known_future_price" in row:
212                 check(not bool(row["known_future_price"]), "future_actual_price_used")
213                 audit_checks += 1
214     battery = battery_metrics(a, initial_mode, initial_power_kw)
215     if ramp_limit_kw is not None:
216         power = 6 * (c - d).ravel()
217         check(np.abs(np.diff(np.r_[initial_power_kw, power])).max(initial=0) <= ramp_limit_kw + TOL,
218               "power_ramp_limit")
219     billing = dict(zip(("planned_cost", "up_cost", "down_cost", "emergency_cost"),
220                       map(float, expected_fees.sum(axis=(0, 1)))))
221     billing["total_cost"] = float(expected_fees.sum())
222     result = {
223         "passed": not errors, "errors": errors, "scenario": scenario,
224         "days": expected_days, "intervals": expected_days * 144, "start_day": start_day,
225         "max_balance_error_kwh": max_balance, "max_soc_error_kwh": max_state,
226         "max_cross_day_soc_error_kwh": continuity, "max_slot_fee_error_yuan": fee_error,
227         "source_actual_and_tariff_checked": source_actual is not None and source_price is not None,
228         "billing": billing, "recomputed_total_cost": billing["total_cost"],
229         "reported_cost_excludes_penalties_and_terminal_value": True,
230         "battery_metrics": battery,
231         "causality_audit": {"provided": audit_records is not None, "checked_fields": audit_checks,
                             "scope": "provided issue/cutoff consistency only; forecast causality
needs independent future-mutation tests"},

```

```

232 }
233 if scenario == "2":
234     result["goal"] = goal_check(billing["total_cost"], battery, not errors,
235                               expected_days == 334 and start_day == 31)
236 return result
237
238
239 def source_arrays(scenario, start_day=31, days=334, root=ROOT):
240     raw = Path(root) / "data/raw"
241     if str(scenario) == "1":
242         reference = pd.read_csv(raw / "附件1.csv").iloc[:, 1:].to_numpy(float)
243         return np.broadcast_to(reference[:, 1:3], (days, 144, 2)), reference[:, 0]
244     arrays = [pd.read_csv(raw / name).iloc[:, 1:].to_numpy(float)
245               for name in ("附件2_小区负载.csv", "附件2_光伏发电实际功率.csv")]
246     actual = np.stack(arrays, axis=-1)[start_day:start_day + days]
247     if str(scenario).startswith("4"):
248         price = pd.read_csv(raw / "附件4.csv").iloc[:, 1:].to_numpy(float)[start_day:start_day + days]
249     else:
250         price = pd.read_csv(raw / "附件1.csv").iloc[:, 1].to_numpy(float)
251     return actual, price
252
253
254 def verify_npz(path, scenario="2", *, expected_days=334, start_day=31,
255               check_sources=True, audit_path=None, **kwargs):
256     path = Path(path)
257     with np.load(path, allow_pickle=False) as archive:
258         detail = {key: archive[key].copy() for key in archive.files}
259     if audit_path is not None:
260         audit = json.loads(Path(audit_path).read_text())
261         kwargs["audit_records"] = audit.get("days", audit.get("records", [])) if isinstance(audit,
262         dict) else audit
263     if check_sources:
264         actual, price = source_arrays(scenario, start_day, expected_days)
265         kwargs.update(source_actual=actual, source_price=price)
266     result = verify_arrays(detail, scenario, expected_days=expected_days, start_day=start_day, **
267     kwargs)
268     result.update(source=str(path), source_sha256=hashlib.sha256(path.read_bytes()).hexdigest())
269     return result
270
271
272 def main():
273     parser = argparse.ArgumentParser(description=__doc__)
274     parser.add_argument("archive", type=Path)
275     parser.add_argument("--scenario", default="2")
276     parser.add_argument("--days", type=int, default=334)
277     parser.add_argument("--start-day", type=int, default=31)
278     parser.add_argument("--initial-soc", type=float, default=INITIAL_SOC)
279     parser.add_argument("--audit", type=Path)
280     parser.add_argument("--output", type=Path)
281     args = parser.parse_args()
282     result = verify_npz(args.archive, args.scenario, expected_days=args.days, start_day=args.
283     start_day,
284                       initial_soc=args.initial_soc, audit_path=args.audit)
285     encoded = json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False) + "\n"
286     if args.output:
287         args.output.parent.mkdir(parents=True, exist_ok=True)
288         args.output.write_text(encoded)
289     print(encoded)
290     raise SystemExit(0 if result["passed"] else 1)
291
292
293 if __name__ == "__main__":
294     main()

```

experiments/problem2/exp003/data.py

```

1 """Question 2 data and causal features; no training-window policy is imposed.
2
3 Index "o" denotes an issue at the start of ten-minute interval "o".
4 Observed values have indices strictly below "o"; targets are "o:o+144".
5 CSV times denote interval ends, including "0:00+1" for 24:00.
6 """
7
8 import hashlib
9 import re
10 from io import BytesIO
11 from pathlib import Path
12
13 import numpy as np
14 import pandas as pd
15
16 ROOT = Path(__file__).resolve().parents[3]
17 HERE = Path(__file__).resolve().parent
18 EPOCH = pd.Timestamp("2025-01-01")
19 STEPS = 144
20 DAYS = 365
21 TARGETS = ("load", "pv")
22 ALLOWED_INPUTS = ("附件1.csv", "附件2_小区负载.csv", "附件2_光伏发电实际功率.csv")
23
24
25 def _integer(value, name, minimum, maximum):
26     if isinstance(value, (bool, np.bool_)) or not isinstance(value, (int, np.integer)):
27         raise ValueError(f"{name} must be an integer")
28     if not minimum <= value <= maximum:
29         raise ValueError(f"{name} must be between {minimum} and {maximum}")
30     return int(value)
31
32
33 def _origins(values, minimum=0):
34     origins = np.asarray(values)
35     if origins.ndim != 1 or (origins.size and origins.dtype.kind not in "iu"):
36         raise ValueError("origins must be a one-dimensional sequence of integer interval indices")
37     if origins.size and (origins.min() < minimum or origins.max() >= DAYS * STEPS):
38         raise ValueError(f"origins must be between {minimum} and {DAYS * STEPS - 1}")
39     return origins.astype(np.int64)
40
41
42 def _end_minutes(value):
43     match = re.fullmatch(r"(\d{1,2}):(\d{2})(?::(\d{2}))?(\+1)?", str(value).strip())
44     if not match:
45         raise ValueError(f"invalid interval-end time: {value!r}")
46     hour, minute, second = (int(part or 0) for part in match.groups()[1:4])
47     if second != 0 or minute >= 60 or hour >= 24:
48         raise ValueError(f"invalid interval-end time: {value!r}")
49     if match.group(4):
50         if hour or minute:
51             raise ValueError("the next-day endpoint must be 0:00+1")
52         return 1440
53     return 60 * hour + minute
54
55
56 def _validate_times(values, name):
57     actual = np.asarray([_end_minutes(value) for value in values])
58     if not np.array_equal(actual, np.arange(10, 1441, 10)):
59         raise ValueError(f"{name}: expected ordered ten-minute interval ends from 00:10 to 0:00+1")
60
61
62 def _numeric(frame, name):
63     try:
64         values = frame.to_numpy(dtype=np.float64)
65     except (TypeError, ValueError) as error:
66         raise ValueError(f"{name}: power and price cells must be numeric") from error
67     if not np.isfinite(values).all() or (values < 0).any():
68         raise ValueError(f"{name}: power and price values must be finite and nonnegative")
69     return values
70

```

```

71
72 class Data:
73     """Read and hash exactly the three files permitted for Question 2.
74
75     "actual" contains load/PV in kW, "reference" retains attachment 1's
76     numeric price/load/PV columns, and "fixed_price" is in yuan/kWh.
77     """
78
79     def __init__(self, root=ROOT):
80         self.root = Path(root)
81         raw = self.root / "data/raw"
82         self.hashes = {}
83         frames = []
84         for name in ALLOWED_INPUTS:
85             content = (raw / name).read_bytes()
86             self.hashes[name] = hashlib.sha256(content).hexdigest()
87             frames.append(pd.read_csv(BytesIO(content)))
88
89         reference, *histories = frames
90         if reference.shape != (STEPS, 4):
91             raise ValueError("附件1.csv: expected shape (144, 4)")
92         if list(reference.columns) != ["时间", "电价", "小区负载", "光伏发电预测功率"]:
93             raise ValueError("附件1.csv: unexpected time/price/load/PV column names or order")
94         _validate_times(reference.iloc[:, 0], ALLOWED_INPUTS[0])
95         self.reference = _numeric(reference.iloc[:, 1:], ALLOWED_INPUTS[0])
96         self.fixed_price = self.reference[:, 0].copy()
97
98         expected_dates = pd.date_range(EPOCH, periods=DAYS)
99         arrays = []
100         for name, frame in zip(ALLOWED_INPUTS[1:], histories):
101             if frame.shape != (DAYS, STEPS + 1):
102                 raise ValueError(f"{name}: expected shape (365, 145)")
103             _validate_times(frame.columns[1:], name)
104             try:
105                 dates = pd.DatetimeIndex(pd.to_datetime(frame.iloc[:, 0], format="%Y-%m-%d", errors=
106 "raise"))
107                 except (TypeError, ValueError) as error:
108                     raise ValueError(f"{name}: invalid dates") from error
109                 if not dates.equals(expected_dates):
110                     raise ValueError(f"{name}: expected every 2025 date exactly once in chronological
111 order")
112                 arrays.append(_numeric(frame.iloc[:, 1:], name).ravel())
113                 self.actual = np.stack(arrays, axis=-1)
114
115     def lag(self, origin, days):
116         origin = _integer(origin, "origin", 0, DAYS * STEPS - 1)
117         days = _integer(days, "days", 1, DAYS)
118         targets = origin + np.arange(STEPS)
119         output = self.reference[targets % STEPS][:, 1:3].copy()
120         ids = targets - days * STEPS
121         known = (ids >= 0) & (ids < origin)
122         output[known] = self.actual[ids[known]]
123         if days > 1:
124             recent = targets - STEPS
125             fallback = ~known & (recent >= 0) & (recent < origin)
126             output[fallback] = self.actual[recent[fallback]]
127         return output
128
129     def baseline(self, origin, kind="periodic"):
130         """Causal yesterday, weekly, or weekly-load/yesterday-PV predictions."""
131         if kind not in ("periodic", "yesterday", "weekly"):
132             raise ValueError(f"unknown baseline kind: {kind!r}")
133         yesterday = self.lag(origin, 1)
134         if kind == "yesterday":
135             return yesterday
136         weekly = self.lag(origin, 7)
137         if kind == "periodic":
138             weekly[:, 1] = yesterday[:, 1]
139         return weekly

```

```

139 def features(self, origins, cutoff):
140     """Return features, bases, training means and scales for any issue slots.
141
142     Shapes are "(n,144,2,16)", "(n,144,2)", "(2,)", "(2,)".
143     "cutoff" is the caller's exclusive scaler-fit boundary. Historical
144     training issues may precede it; callers must ensure it is no later
145     than their model's deployment time. Features require at least one
146     observed interval; issue zero is handled by the reference baseline.
147     """
148     origins = _origins(origins, minimum=1)
149     if not origins.size:
150         raise ValueError("features require at least one origin")
151     cutoff = _integer(cutoff, "cutoff", 1, int(origins.max()))
152     mean = self.actual[:cutoff].mean(axis=0)
153     scale = np.maximum(self.actual[:cutoff].std(axis=0), 1e-6)
154     features, bases = [], []
155     for origin in origins:
156         base = self.baseline(origin)
157         yesterday, weekly = self.lag(origin, 1), self.lag(origin, 7)
158         end = origin + np.arange(1, STEPS + 1)
159         dayphase = end % STEPS / STEPS * 2 * np.pi
160         weekphase = ((end // STEPS + 2) % 7) / 7 * 2 * np.pi
161         issuephase = origin % STEPS / STEPS * 2 * np.pi
162         calendar = np.column_stack((
163             np.sin(dayphase), np.cos(dayphase), np.sin(weekphase), np.cos(weekphase),
164             np.full(STEPS, np.sin(issuephase)), np.full(STEPS, np.cos(issuephase)),
165             np.arange(1, STEPS + 1) / STEPS,
166         ))
167         branches = []
168         for channel in range(2):
169             history = self.actual[max(0, origin - 7 * STEPS):origin, channel]
170             values = np.asarray((self.actual[origin - 1, channel], history[-36:].mean(),
171                                 history[-STEPS:].mean(), history.mean()))
172             stats = np.tile(np.r_[
173                 (values - mean[channel]) / scale[channel],
174                 history[-STEPS:].std() / scale[channel], history.std() / scale[channel],
175             ], (STEPS, 1))
176             branches.append(np.column_stack((
177                 calendar, (yesterday[:, channel] - mean[channel]) / scale[channel],
178                 (weekly[:, channel] - mean[channel]) / scale[channel], stats,
179                 (base[:, channel] - mean[channel]) / scale[channel],
180             )))
181         features.append(np.stack(branches, axis=1))
182         bases.append(base)
183     return np.asarray(features, dtype="float32"), np.asarray(bases, dtype="float32"), mean,
184     scale
185
186 def labels(self, origins, cutoff=None):
187     """Read complete target windows ending at or before exclusive "cutoff"."""
188     origins = _origins(origins)
189     cutoff = DAYS * STEPS if cutoff is None else _integer(cutoff, "cutoff", 0, DAYS * STEPS)
190     if origins.size and np.any(origins + STEPS > cutoff):
191         raise ValueError("a complete 24-hour target window is unavailable at the label cutoff")
192     ids = origins[:, None] + np.arange(STEPS)
193     return self.actual[ids].astype("float32")
194
195 def midnight_origins(month=None, *, start=None, end=None):
196     """Return daily 00:00 issue indices for a month or inclusive date range.
197
198     With no arguments, return the formal evaluation period, February through
199     December 2025. This helper does not prescribe training issue frequency.
200     """
201     if month is not None:
202         month = _integer(month, "month", 1, 12)
203         if start is not None or end is not None:
204             raise ValueError("choose either month or start/end dates")
205         first = pd.Timestamp(2025, month, 1)
206         last = first + pd.offsets.MonthEnd(0)
207     else:

```

```

208     first = pd.Timestamp("2025-02-01" if start is None else start)
209     last = pd.Timestamp("2025-12-31" if end is None else end)
210     if any(value.tzinfo is not None or pd.isna(value) or value != value.normalize())
211         for value in (first, last)):
212         raise ValueError("start/end must be timezone-naive dates at midnight")
213     if not EPOCH <= first <= last < EPOCH + pd.Timedelta(days=DAYS):
214         raise ValueError("start/end must be an ordered inclusive range within 2025")
215     return np.asarray((pd.date_range(first, last) - EPOCH).days, dtype=np.int64) * STEPS

```

experiments/problem2/exp004/data.py

```

1  """Causal midnight arrays, seven-day sequences and fitted seasonal curves.
2
3  No future realized value enters features except for the explicitly marked
4  oracle_season mode. All forecast arrays have shape (144, 2) in kw.
5  """
6
7  import hashlib
8  import json
9  from functools import lru_cache
10 from pathlib import Path
11
12 import numpy as np
13
14 from experiments.problem2.exp003.data import DAYS, EPOCH, ROOT, STEPS, Data
15
16 HERE = Path(__file__).resolve().parent
17 OUT = ROOT / "data/results/exp004"
18 VARIANTS = ("no_season", "causal_season", "oracle_season")
19
20
21 def protocol():
22     return json.loads((HERE / "protocol.json").read_text())
23
24
25 def sha256(path):
26     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
27
28
29 def write_json(path, obj):
30     path = Path(path)
31     path.parent.mkdir(parents=True, exist_ok=True)
32     temporary = path.with_suffix(path.suffix + ".tmp")
33     temporary.write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False) + "\n")
34     temporary.replace(path)
35
36
37 def source_signature(data):
38     sources = {str(p.relative_to(ROOT)): sha256(p) for p in
39               [HERE / name for name in ("data.py", "train.py", "protocol.json")]}
40     inherited = ROOT / "experiments/problem2/exp003/data.py"
41     sources[str(inherited.relative_to(ROOT))] = sha256(inherited)
42     payload = {"code": sources, "data": data.hashes}
43     signature = hashlib.sha256(json.dumps(payload, sort_keys=True).encode()).hexdigest()
44     return signature, payload
45
46
47 def design(days):
48     days = np.asarray(days, dtype=float).ravel()
49     phase = 2 * np.pi * days / 365.25
50     weekday = (days.astype(int) + 2) % 7
51     return np.column_stack([np.ones(len(days)), np.sin(phase), np.cos(phase),
52                             np.sin(2 * phase), np.cos(2 * phase),
53                             *[(weekday == k).astype(float) for k in range(1, 7)]])
54
55
56 class Features:

```

```

57 def __init__(self, data=None):
58     self.data = Data() if data is None else data
59     self.cfg = protocol()
60
61 @lru_cache(maxsize=366)
62 def seasonal_fit(self, cutoff_day):
63     """Fit complete prefix; solve all 144 load/PV slot curves jointly."""
64     if not 1 <= cutoff_day <= DAYS:
65         raise ValueError("seasonal cutoff must be a complete observed day count")
66     x = design(np.arange(cutoff_day))
67     y = self.data.actual[:cutoff_day * STEPS].reshape(cutoff_day, -1)
68     cfg = self.cfg["seasonality"]
69     regularizer = np.diag([1e-8] + [cfg["harmonic_ridge"]] * 4
70                               + [cfg["weekday_ridge"]] * 6)
71     # Explicit contractions avoid spurious Accelerate/NumPy matmul FPE flags
72     # on this host; nonfinite values remain a hard error rather than masked.
73     gram = np.einsum("ni,nj->ij", x, x, optimize=False)
74     rhs = np.einsum("ni,nj->ij", x, y, optimize=False)
75     coef = np.linalg.solve(gram + regularizer, rhs)
76     if not np.isfinite(coef).all():
77         raise RuntimeError("Seasonal fit produced nonfinite coefficients")
78     return coef
79
80 def seasonal_shift(self, day, variant):
81     if variant not in VARIANTS or not 7 <= day < DAYS:
82         raise ValueError("invalid forecast variant or midnight day")
83     cfg = self.cfg["seasonality"]
84     if variant == "no_season" or day < cfg["minimum_days"]:
85         return np.zeros((STEPS, 2), dtype=float)
86     cutoff = DAYS if variant == "oracle_season" else day
87     coef = self.seasonal_fit(cutoff)
88     # Weekday effects are deliberately excluded from the seasonal shift.
89     x = design([day, day - 7, day - 1])
90     x[:, 5:] = 0
91     curves = np.einsum("ni,ij->nj", x, coef, optimize=False).reshape(3, STEPS, 2)
92     shift = np.column_stack((curves[0, :, 0] - curves[1, :, 0],
93                             curves[0, :, 1] - curves[2, :, 1]))
94     scale = np.maximum(self.data.actual[:day * STEPS].std(axis=0), 1)
95     bound = cfg["maximum_shift_scale"] * scale
96     shrink = min(1, cutoff / cfg["shrink_days"])
97     return np.clip(shift * shrink, -bound, bound)
98
99 def base(self, day, variant):
100     periodic = self.data.baseline(day * STEPS)
101     return np.maximum(0, periodic + self.seasonal_shift(day, variant))
102
103 def arrays(self, days, cutoff_day, variant):
104     """Scaler cutoff is the training prefix, never the formal target day."""
105     days = np.asarray(days, dtype=int)
106     if (len(days) == 0 or days.min() < 7 or days.max() >= DAYS
107         or not 7 <= cutoff_day <= days.max()):
108         raise ValueError("invalid feature days/scaler cutoff")
109     observed = self.data.actual[:cutoff_day * STEPS]
110     mean = observed.mean(axis=0)
111     scale = np.maximum(observed.std(axis=0), 1)
112     sequences, contexts, bases, shifts, masks = [], [], [], [], []
113     slotphase = 2 * np.pi * (np.arange(STEPS) + .5) / STEPS
114     for day in days:
115         origin = int(day) * STEPS
116         history = self.data.actual[origin - 7 * STEPS:origin]
117         # Hourly means retain chronological ordering of all seven days.
118         sequences.append((history.reshape(168, 6, 2).mean(axis=1) - mean) / scale)
119         base = self.base(int(day), variant)
120         yesterday = self.data.baseline(origin, "yesterday")
121         weekly = self.data.baseline(origin, "weekly")
122         weekphase = 2 * np.pi * ((day + 2) % 7) / 7
123         calendar = np.column_stack((np.sin(slotphase), np.cos(slotphase),
124                                     np.full(STEPS, np.sin(weekphase)),
125                                     np.full(STEPS, np.cos(weekphase))))
126         contexts.append(np.stack([np.column_stack((

```

```

127         (base[:, k] - mean[k]) / scale[k],
128         (yesterday[:, k] - mean[k]) / scale[k],
129         (weekly[:, k] - mean[k]) / scale[k],
130         np.full(STEPS, (history[:, k].mean() - mean[k]) / scale[k]), calendar,
131     )) for k in range(2)], axis=1))
132     bases.append(base)
133     shifts.append(self.seasonal_shift(int(day), variant))
134     daylight = (self.data.actual[max(0, origin - 28 * STEPS):origin, 1]
135                 .reshape(-1, STEPS) > 0).any(axis=0)
136     # A fixed 20-minute margin permits upcoming sunrise/sunset shifts.
137     mask = np.convolve(daylight.astype(int), np.ones(5), mode="same") > 0
138     masks.append(mask)
139     return {"sequence": np.asarray(sequences, dtype="float32"),
140            "context": np.asarray(contexts, dtype="float32"),
141            "base": np.asarray(bases), "seasonal_shift": np.asarray(shifts),
142            "mask": np.asarray(masks), "mean": mean, "scale": scale}
143
144
145 def split_days(month):
146     import pandas as pd
147
148     if not 2 <= month <= 12:
149         raise ValueError("formal months are February to December")
150     asof = (pd.Timestamp(2025, month, 1) - EPOCH).days
151     end = (pd.Timestamp(2025, month, 1) + pd.offsets.MonthEnd(0) - EPOCH).days + 1
152     cutoff = asof - protocol()["training"]["validation_days"]
153     return np.arange(7, cutoff), np.arange(cutoff, asof), np.arange(asof, end), cutoff
154
155
156 def age_weights(days, cutoff_day, half_life=90):
157     days = np.asarray(days)
158     if half_life <= 0 or np.any(days >= cutoff_day):
159         raise ValueError("sample ages require strictly historical days and positive half-life")
160     weights = 2.0 ** (-(cutoff_day - 1 - days) / half_life)
161     return (weights / weights.mean()).astype("float32")

```

experiments/problem2/exp004/predict.py

```

1  """Read-only forecast interface for the planning team. No solver is called."""
2
3  import json
4
5  import numpy as np
6
7  from experiments.problem2.exp003.data import STEPS, Data
8
9  from .data import OUT, VARIANTS, sha256
10
11
12 class ForecastStore:
13     def __init__(self, variant="causal_season", seed=42, allow_oracle=False, directory=OUT):
14         if variant not in VARIANTS:
15             raise ValueError("Unknown variant")
16         if variant == "oracle_season" and not allow_oracle:
17             raise ValueError("oracle_season uses future actuals; explicitly opt in for exploration only")
18         manifest = json.loads((directory / "prediction_manifest.json").read_text())
19         if not manifest["complete"] or sha256(directory / "predictions.npz") != manifest["archive_sha256"]:
20             raise RuntimeError("Prediction archive is incomplete or changed")
21         with np.load(directory / "predictions.npz") as z:
22             self.origins = z["origins"].copy()
23             self.values = z[f"{variant}_seed_{int(seed)}"].copy()
24         self.variant = variant
25         self.seed = int(seed)
26         self.lookup = {int(o): i for i, o in enumerate(self.origins)}
27

```

```

28 def get(self, origin):
29     """Return an independent float64 (144,2) load/PV array, in kW."""
30     if not isinstance(origin, (int, np.integer)) or isinstance(origin, bool):
31         raise TypeError("origin is an integer index of a 10-minute interval")
32     if origin not in self.lookup:
33         raise ValueError("origin must be a formal 2025 February–December midnight")
34     return self.values[self.lookup[origin]].copy()
35
36 def completed_error_paths(self, origin, limit=28):
37     """Return complete issued forecast errors (actual minus forecast, kW).
38
39     Preserve cross-slot and load/PV dependence. Only paths whose last
40     label is already observed are exposed. Empty history stays empty;
41     this method chooses neither scenarios nor a purchasing policy.
42     """
43     self.get(origin)
44     if not isinstance(limit, int) or limit < 1:
45         raise ValueError("limit must be a positive integer")
46     ids = np.flatnonzero(self.origins + STEPS <= origin)[-limit:]
47     origins = self.origins[ids]
48     actual = Data().actual[origins[:, None] + np.arange(STEPS)]
49     return {"origins": origins.copy(), "errors_kw": actual - self.values[ids],
50           "information_cutoff": int(origin), "variant": self.variant,
51           "exploratory": self.variant == "oracle_season"}

```

experiments/problem2/tree_planning/model.py

```

1  """Discrete SOC planning surrogate and causal, physically projected execution.
2
3  The tree supplies marginal net-demand supports, not joint scenarios. The DP
4  exactly optimizes a discrete open-loop surrogate; the physical controller can
5  curtail actions, so this is not an exact stochastic recourse optimization.
6  Units: AC-side kWh, battery-state kWh, yuan/kWh, and yuan.
7  """
8
9  import time
10 from dataclasses import dataclass
11
12 import numpy as np
13
14 ETA = float(np.sqrt(0.9))
15 MIN_SOC, MAX_SOC, POWER_ENERGY = 1200.0, 10800.0, 5000 / 6
16 MODES = np.array([-1, 0, 1])
17
18 @dataclass(frozen=True)
19 class Config:
20     grid_kwh: float = 100.0
21     throughput_yuan_per_kwh: float = 0.002
22     reversal_yuan: float = 0.05
23     deadband_kwh: float = 2.0
24     terminal_value: bool = True
25
26
27
28 DEFAULT_CONFIG = Config()
29
30
31 def stage_cost(net_support, shifts, price):
32     """Exact newsvendor minimizer for equally weighted, discrete supports.
33
34     Conditional on a fixed battery action,  $q = \max(0, \text{empirical } Q_{0.8})$ .
35     This statement is about the open-loop surrogate, not realized recourse.
36     """
37     support = np.asarray(net_support, dtype=float)
38     shifts = np.asarray(shifts, dtype=float)
39     quantile = np.quantile(support, 0.8, axis=-1, method="inverted_cdf")
40     purchase = np.maximum(0, quantile[:, None] + shifts[None, :])

```

```

41 shortage = np.maximum(
42     support[:, None, :] + shifts[None, :, None] - purchase[:, :, None], 0
43 ).mean(axis=-1)
44 return purchase, np.asarray(price)[:, None] * (purchase + 5 * shortage)
45
46
47 def plan_day(net_support, price, initial_soc, config=DEFAULT_CONFIG, initial_mode=0,
48             final_evaluation_day=False):
49     began = time.perf_counter()
50     support, price = np.asarray(net_support, float), np.asarray(price, float)
51     if (support.ndim != 2 or support.shape[0] != len(price)
52         or support.shape[1] < 1 or not np.isfinite(support).all()
53         or not np.isfinite(price).all() or np.any(price <= 0)):
54         raise ValueError("Expected finite net supports and positive matching prices")
55     if not MIN_SOC <= initial_soc <= MAX_SOC or initial_mode not in MODES:
56         raise ValueError("Invalid initial battery state or direction")
57     if (config.grid_kwh <= 0 or config.grid_kwh > 600
58         or config.throughput_yuan_per_kwh < 0 or config.reversal_yuan < 0
59         or config.deadband_kwh < 0):
60         raise ValueError("Invalid grid or secondary penalty")
61     n_steps = len(price)
62     grid = np.unique(np.r_[np.arange(MIN_SOC, MAX_SOC, config.grid_kwh),
63                           MAX_SOC, float(initial_soc)])
64     # Keep the exact initial SOC instead of rounding or resetting stored energy.
65     difference = grid[None, :] - grid[:, None]
66     charge = np.maximum(difference, 0) / ETA
67     discharge = np.maximum(-difference, 0) * ETA
68     feasible = np.maximum(charge, discharge) <= POWER_ENERGY + 1e-8
69     counts = feasible.sum(axis=1)
70     width = int(counts.max())
71     next_index = np.zeros((len(grid), width), dtype=int)
72     valid = np.arange(width)[None, :] < counts[:, None]
73     for state in range(len(grid)):
74         eligible = np.flatnonzero(feasible[state])
75         next_index[state, :len(eligible)] = eligible
76     delta = grid[next_index] - grid[:, None]
77     c, d = np.maximum(delta, 0) / ETA, np.maximum(-delta, 0) * ETA
78     action_mode = np.sign(delta).astype(int)
79     # Uniform-grid differences repeat. Evaluate each AC energy shift once.
80     shifts, inverse = np.unique(np.round(c - d, 9), return_inverse=True)
81     inverse = inverse.reshape(delta.shape)
82     purchases, costs = stage_cost(support, shifts, price)
83     throughput = config.throughput_yuan_per_kwh * (c + d)
84     terminal_lambda = (float(price.min()) / ETA
85                       if config.terminal_value and not final_evaluation_day else 0.0)
86     value = np.tile(-terminal_lambda * (grid - MIN_SOC), (3, 1))
87     choices = np.empty((n_steps, 3, len(grid)), dtype=np.int32)
88     for slot in range(n_steps - 1, -1, -1):
89         updated = np.empty_like(value)
90         stage = costs[slot, inverse] + throughput
91         for mode_index, previous_mode in enumerate(MODES):
92             following_mode = np.where(action_mode == 0, previous_mode, action_mode) + 1
93             future = value[following_mode, next_index]
94             switching = config.reversal_yuan * (action_mode * previous_mode == -1)
95             candidates = np.where(valid, stage + future + switching, np.inf)
96             selected = candidates.argmin(axis=1)
97             updated[mode_index] = candidates[np.arange(len(grid)), selected]
98             choices[slot, mode_index] = selected
99         value = updated
100     state = int(np.flatnonzero(grid == initial_soc)[0])
101     mode = int(initial_mode)
102     objective = float(value[mode + 1, state])
103     plan, intended_charge, intended_discharge = (np.zeros(n_steps) for _ in range(3))
104     states = np.empty(n_steps + 1)
105     states[0] = initial_soc
106     for slot in range(n_steps):
107         column = choices[slot, mode + 1, state]
108         plan[slot] = purchases[slot, inverse[state, column]]
109         intended_charge[slot], intended_discharge[slot] = c[state, column], d[state, column]
110         action = action_mode[state, column]

```

```

111         if action:
112             mode = int(action)
113             state = int(next_index[state, column])
114             states[slot + 1] = grid[state]
115         return {
116             "purchase": plan, "charge": intended_charge, "discharge": intended_discharge,
117             "states": states,
118             "metadata": {"planning_seconds": time.perf_counter() - began,
119                         "grid_states": len(grid), "max_actions_per_state": width,
120                         "objective_surrogate_yuan": objective,
121                         "terminal_value_yuan_per_soc_kwh": terminal_lambda,
122                         "exact_for_discrete_surrogate_only": True},
123         }
124
125
126 def execute_plan(plan, actual_kw, price, initial_soc, config=DEFAULT_CONFIG, initial_mode=0,
127                 greedy=False):
128     """Fixed purchases; observe current slot only; never emergency-charge.
129
130     Positive actions are clipped by available excess electricity. Negative
131     actions are clipped by the load deficit. Actual SOC and last non-idle
132     direction carry forward, including across days when called in sequence.
133     """
134     purchase = np.asarray(plan["purchase"], dtype=float)
135     actual, price = np.asarray(actual_kw, dtype=float), np.asarray(price, dtype=float)
136     if (actual.shape != (len(purchase), 2) or not np.isfinite(actual).all()
137         or np.any(actual < 0) or not MIN_SOC <= initial_soc <= MAX_SOC):
137         raise ValueError("Invalid physical observations or starting SOC")
138     n_steps = len(purchase)
139     c, d, emergency, surplus = (np.zeros(n_steps) for _ in range(4))
140     states = np.empty(n_steps + 1)
141     states[0] = initial_soc
142     previous_mode, reversals = initial_mode, 0
143     for slot in range(n_steps):
144         balance = purchase[slot] + (actual[slot, 1] - actual[slot, 0]) / 6
145         if balance >= 0:
146             request = POWER_ENERGY if greedy else plan["charge"][slot]
147             c[slot] = min(request, balance, POWER_ENERGY,
148                          max(0, (MAX_SOC - states[slot]) / ETA))
149             if not greedy and c[slot] < config.deadband_kwh:
150                 c[slot] = 0
151             surplus[slot] = balance - c[slot]
152         else:
153             request = POWER_ENERGY if greedy else plan["discharge"][slot]
154             d[slot] = min(request, -balance, POWER_ENERGY,
155                          max(0, (states[slot] - MIN_SOC) * ETA))
156             if not greedy and d[slot] < config.deadband_kwh:
157                 d[slot] = 0
158             emergency[slot] = -balance - d[slot]
159             states[slot + 1] = states[slot] + ETA * c[slot] - d[slot] / ETA
160             mode = int(np.sign(c[slot] - d[slot]))
161             if mode:
162                 reversals += int(previous_mode * mode == -1)
163                 previous_mode = mode
164     fees = np.column_stack((purchase * price, np.zeros((n_steps, 2)),
165                          5 * emergency * price))
166     detail = {"original": purchase.copy(), "final": purchase.copy(), "charge": c,
167              "discharge": d, "emergency": emergency, "surplus": surplus,
168              "states": states, "fees": fees, "actual": actual.copy(), "price": price.copy()}
169     balance_error = purchase + (actual[:, 1] - actual[:, 0]) / 6 + d + emergency - c - surplus
170     state_error = np.diff(states) - ETA * c + d / ETA
171     simultaneous = int(((c > 1e-6) & (d > 1e-6)).sum())
172     assert max(np.abs(balance_error).max(), np.abs(state_error).max()) < 1e-6
173     assert MIN_SOC - 1e-6 <= states.min() <= states.max() <= MAX_SOC + 1e-6
174     assert max(c.max(), d.max()) <= POWER_ENERGY + 1e-6
175     assert simultaneous == 0 and not ((c > 1e-6) & (emergency > 1e-6)).any()
176     metrics = {"total_cost": float(fees.sum()), "planned_cost": float(fees[:, 0].sum()),
177              "emergency_cost": float(fees[:, 3].sum()), "simultaneous_slots": simultaneous,
178              "throughput_kwh": float((c + d).sum()), "direction_reversals": reversals,
179              "equivalent_full_cycles": float((ETA * c.sum() + d.sum() / ETA) / 24000),
180

```

```

181         "initial_soc": float(states[0]), "final_soc": float(states[-1]),
182         "final_mode": previous_mode,
183         "max_balance_error": float(np.abs(balance_error).max()),
184         "max_state_error": float(np.abs(state_error).max())
185     return detail, metrics

```

experiments/problem2/tree_planning/risk.py

```

1  """Causal, lightweight tree-conditioned net-load residual distributions.
2
3  The frozen exp004 load/PV forecasts are inputs, in kW; this module does not
4  change or regenerate them. "for_day" returns per-slot net-load supports in
5  kWh for ten-minute intervals. The nine columns are conditional marginal
6  quantiles with equal weights, not nine temporally correlated daily paths.
7
8  Historical forecast labels are read only after their complete day is known.
9  When fewer than two issued forecast days are available, the explicitly
10 identified periodic-baseline fallback uses only previously completed days.
11 That fallback is not represented as an exp004 forecast-error distribution.
12 """
13
14 from time import perf_counter
15
16 import numpy as np
17 from sklearn.tree import DecisionTreeRegressor
18
19 STEPS = 144
20 DT_HOURS = 1.0 / 6.0
21 HISTORY_DAYS = 28
22 SUPPORT_SIZE = 9
23 QUANTILE_LEVELS = (np.arange(SUPPORT_SIZE, dtype=float) + 0.5) / SUPPORT_SIZE
24 FEATURE_NAMES = ("slot_sin", "slot_cos", "forecast_net_kwh", "forecast_pv_kwh", "price")
25
26
27 def tree_features(forecast_kw, price144):
28     """Pure (144, 5) features; accepts forecasts/prices only, never actuals.
29
30     Column order: slot sine/cosine, predicted load minus PV in kWh, predicted
31     PV in kWh, and the fixed tariff in yuan/kWh. Slot zero starts at midnight.
32     """
33     forecast = np.asarray(forecast_kw, dtype=float)
34     price = np.asarray(price144, dtype=float)
35     if forecast.shape != (STEPS, 2) or price.shape != (STEPS,):
36         raise ValueError("forecast_kw must be (144,2), price144 must be (144,)")
37     if not np.isfinite(forecast).all() or not np.isfinite(price).all():
38         raise ValueError("forecasts and prices must be finite")
39     phase = np.arange(STEPS, dtype=float) * (2.0 * np.pi / STEPS)
40     return np.column_stack((
41         np.sin(phase), np.cos(phase),
42         (forecast[:, 0] - forecast[:, 1]) * DT_HOURS,
43         forecast[:, 1] * DT_HOURS, price,
44     ))
45
46
47 def periodic_baseline(previous_day_kw, previous_week_kw=None):
48     """Pure causal fallback: mean yesterday/week load; yesterday PV.
49
50     The caller must supply only completed days preceding the issue date.
51     Before one full week exists, the load prediction is yesterday's load.
52     """
53     yesterday = np.asarray(previous_day_kw, dtype=float)
54     weekly = yesterday if previous_week_kw is None else np.asarray(previous_week_kw, dtype=float)
55     if yesterday.shape != (STEPS, 2) or weekly.shape != (STEPS, 2):
56         raise ValueError("baseline inputs must each contain one complete (144,2) day")
57     if not np.isfinite(yesterday).all() or not np.isfinite(weekly).all():
58         raise ValueError("baseline history must be finite")
59     return np.column_stack((0.5 * (yesterday[:, 0] + weekly[:, 0]), yesterday[:, 1]))

```

```

60
61
62 class TreeResidualScenarios:
63     """Fit a small residual tree independently at each requested midnight.
64
65     Parameters are frozen forecast "origins" (interval indices), forecast
66     "values" shaped (number of origins, 144, 2), "actual" shaped
67     (number of complete days * 144, 2), and fixed daily "price144". The
68     application supplies the 365-day actual archive; shorter archives also
69     support small synthetic checks. Days are zero-based (February 1 is 31).
70
71     Actuals are retained as an array reference without scanning or copying
72     their contents during construction. Only selected completed-day slices
73     are read later. Treat all constructor inputs as immutable thereafter.
74     """
75
76     def __init__(self, origins, values, actual, price144, conditioning="tree"):
77         if conditioning not in ("tree", "per_slot"):
78             raise ValueError("conditioning must be tree or per_slot")
79         self.conditioning = conditioning
80         origins = np.asarray(origins)
81         if origins.ndim != 1 or origins.dtype.kind not in "iu":
82             raise ValueError("origins must be a one-dimensional integer array")
83         if np.any(origins < 0) or np.any(origins % STEPS):
84             raise ValueError("origins must be nonnegative midnight interval indices")
85         if len(np.unique(origins)) != len(origins):
86             raise ValueError("forecast origins must be unique")
87         forecast = np.asarray(values)
88         if forecast.shape != (len(origins), STEPS, 2):
89             raise ValueError("values must have shape (len(origins),144,2)")
90         if not np.isfinite(forecast).all():
91             raise ValueError("frozen forecasts must be finite")
92         observed = np.asarray(actual)
93         if observed.ndim != 2 or observed.shape[1] != 2 or observed.shape[0] % STEPS:
94             raise ValueError("actual must contain complete days, shaped (days*144,2)")
95         if np.any(origins + STEPS > observed.shape[0]):
96             raise ValueError("forecast origins must fit the actual archive's calendar")
97         price = np.asarray(price144, dtype=float)
98         if price.shape != (STEPS,) or not np.isfinite(price).all():
99             raise ValueError("price144 must contain 144 finite tariff values")
100         order = np.argsort(origins)
101         self.origins = origins[order].astype(np.int64, copy=True)
102         self.values = np.asarray(forecast[order], dtype=float)
103         self.price144 = price.copy()
104         self._actual = observed
105         self._lookup = {int(origin): index for index, origin in enumerate(self.origins)}
106         self.weights = np.full(SUPPORT_SIZE, 1.0 / SUPPORT_SIZE)
107
108     def _completed_day(self, day, cutoff):
109         """The sole actual-value reader; enforce an exclusive information cut."""
110         start, stop = int(day) * STEPS, (int(day) + 1) * STEPS
111         if start < 0 or stop > cutoff:
112             raise ValueError("attempt to read a day not complete at the information cutoff")
113         observed = np.asarray(self._actual[start:stop], dtype=float)
114         if observed.shape != (STEPS, 2) or not np.isfinite(observed).all():
115             raise ValueError("selected completed actual day must be finite and complete")
116         return observed
117
118     def for_day(self, day):
119         """Return "(net_load_kwh[144,9], audit_info)" before "day" starts.
120
121         At most the 28 most recent complete issued forecast days are used.
122         If there are fewer than two, use up to 28 known periodic-baseline
123         error days instead. A day-zero request without any calibration data
124         raises instead of inventing a residual distribution.
125         """
126         if isinstance(day, (bool, np.bool_)) or not isinstance(day, (int, np.integer)):
127             raise TypeError("day must be a zero-based integer day index")
128         day = int(day)
129         cutoff = day * STEPS

```

```

130     if cutoff not in self._lookup:
131         raise ValueError("requested day has no frozen forecast")
132     historical_ids = np.flatnonzero(self.origins + STEPS <= cutoff)[-HISTORY_DAYS:]
133     fallback = len(historical_ids) < 2
134     features, residuals, training_days = [], [], []
135
136     if fallback:
137         # The target day d is already known at today's cutoff; the
138         # baseline issued on d still sees only indices below d*144.
139         for historical_day in range(max(1, day - HISTORY_DAYS), day):
140             issue_cutoff = historical_day * STEPS
141             previous = self._completed_day(historical_day - 1, issue_cutoff)
142             weekly = (self._completed_day(historical_day - 7, issue_cutoff)
143                      if historical_day >= 7 else None)
144             predicted = periodic_baseline(previous, weekly)
145             observed = self._completed_day(historical_day, cutoff)
146             features.append(tree_features(predicted, self.price144))
147             residuals.append(((observed[:, 0] - observed[:, 1])
148                               - (predicted[:, 0] - predicted[:, 1])) * DT_HOURS)
149             training_days.append(historical_day)
150         else:
151             for index in historical_ids:
152                 historical_day = int(self.origins[index] // STEPS)
153                 predicted = self.values[index]
154                 observed = self._completed_day(historical_day, cutoff)
155                 features.append(tree_features(predicted, self.price144))
156                 residuals.append(((observed[:, 0] - observed[:, 1])
157                                   - (predicted[:, 0] - predicted[:, 1])) * DT_HOURS)
158                 training_days.append(historical_day)
159
160     if not training_days:
161         raise ValueError("no completed day exists to calibrate even the periodic fallback")
162     x_train, y_train = np.concatenate(features), np.concatenate(residuals)
163     current_forecast = self.values[self._lookup[cutoff]]
164     x_current = tree_features(current_forecast, self.price144)
165     tree = None
166     fit_seconds = 0.0
167     if self.conditioning == "tree":
168         tree = DecisionTreeRegressor(max_depth=5, min_samples_leaf=48, random_state=42)
169         began = perf_counter()
170         tree.fit(x_train, y_train)
171         fit_seconds = perf_counter() - began
172         train_leaves = tree.apply(x_train)
173         leaf_supports = {
174             int(leaf): np.quantile(y_train[train_leaves == leaf], QUANTILE_LEVELS, method="
175 linear")
176         for leaf in np.unique(train_leaves)
177         }
178         current_leaves = tree.apply(x_current)
179         errors = np.vstack([leaf_supports[int(leaf)] for leaf in current_leaves])
180     else:
181         # Identical completed history and fallback; condition only on slot.
182         errors = np.quantile(np.stack(residuals), QUANTILE_LEVELS,
183                               axis=0, method="linear").T
184     scenarios = x_current[:, 2, None] + errors
185     info = {
186         "day": day,
187         "cutoff": cutoff,
188         "information_cutoff": cutoff,
189         "cutoff_is_exclusive": True,
190         "training_start_day": min(training_days),
191         "training_end_day": max(training_days),
192         "training_days": len(training_days),
193         "training_origins": [int(value * STEPS) for value in training_days],
194         "sample_count": len(y_train),
195         "issued_history_days": len(historical_ids),
196         "max_observed_index": int((max(training_days) + 1) * STEPS - 1),
197         "fit_seconds": float(fit_seconds),
198         "tree_nodes": int(tree.tree_.node_count) if tree is not None else 0,
199         "tree_leaves": int(tree.tree_.n_leaves) if tree is not None else 0,

```

```

199         "conditioning": self.conditioning,
200         "fallback": bool(fallback),
201         "residual_source": "periodic_baseline" if fallback else "frozen_exp004_forecast",
202         "fallback_reason": "fewer_than_two_completed_issued_forecast_days" if fallback else None
203     },
204     "fallback_formula": "load=(yesterday+previous_week)/2; pv=yesterday; early_week_load=
yesterday" if fallback else None,
205     "fallback_same_forecast_source": False if fallback else None,
206     "feature_names": list(FEATURE_NAMES),
207     "residual_units": "kWh_per_10min",
208     "scenario_units": "kWh_per_10min",
209     "scenario_semantics": "conditional_slot_marginals_not_joint_daily_paths",
210     "quantile_levels": QUANTILE_LEVELS.tolist(),
211     "weights": self.weights.tolist(),
212     "max_depth": 5,
213     "min_samples_leaf": 48,
214     "random_state": 42,
215 }
return scenarios, info

```

experiments/problem2/tree_planning/verify.py

```

1  """Independent archive checks: no planner, executor, or optimization imports."""
2
3  import argparse
4  import json
5  from pathlib import Path
6
7  import numpy as np
8  import pandas as pd
9
10 ETA = float(np.sqrt(0.9))
11 TOL = 1e-6
12
13
14 def battery_metrics(detail, initial_mode=1, initial_power_kw=0.0):
15     charge = np.asarray(detail["charge"]).ravel()
16     discharge = np.asarray(detail["discharge"]).ravel()
17     power = 6 * (charge - discharge)
18     modes = np.sign(charge - discharge).astype(int)
19     nonidle = modes[np.abs(charge - discharge) > TOL]
20     direction_sequence = np.r_[initial_mode, nonidle]
21     reversals_with_warmup = int(np.sum(direction_sequence[1:] * direction_sequence[:-1] == -1))
22     reversals = int(np.sum(nonidle[1:] * nonidle[:-1] == -1))
23     ramps = np.abs(np.diff(power))
24     boundary_ramp = float(abs(power[0] - initial_power_kw))
25     return {
26         "charge_kwh": float(charge.sum()), "discharge_kwh": float(discharge.sum()),
27         "throughput_kwh": float((charge + discharge).sum()),
28         "equivalent_full_cycles": float((ETA * charge.sum() + discharge.sum() / ETA) / 24000),
29         "direction_reversals": reversals,
30         "direction_reversals_including_warmup_boundary": reversals_with_warmup,
31         "simultaneous_slots": int(np.sum((charge > TOL) & (discharge > TOL))),
32         "charging_slots": int(np.sum(charge > TOL)),
33         "discharging_slots": int(np.sum(discharge > TOL)),
34         "active_slots": int(np.sum((charge > TOL) | (discharge > TOL))),
35         "power_changes_count": int(np.sum(ramps > TOL)),
36         "power_ramp_total_kw": float(ramps.sum()),
37         "power_ramp_total_kw_including_warmup_boundary": float(ramps.sum() + boundary_ramp),
38         "power_ramp_max_kw_per_10min": float(ramps.max(initial=0)),
39         "power_ramp_p95_kw_per_10min": float(np.quantile(ramps, .95)),
40         "mean_absolute_power_kw": float(np.abs(power).mean()),
41         "max_charge_kw": float(6 * charge.max(initial=0)),
42         "max_discharge_kw": float(6 * discharge.max(initial=0)),
43         "initial_soc": float(detail["states"][0, 0]),
44         "final_soc": float(detail["states"][-1, -1]),
45         "initial_mode": int(initial_mode),

```

```

46     "final_mode": int(nonidle[-1]) if len(nonidle) else int(initial_mode),
47     "initial_power_kw": float(initial_power_kw), "final_power_kw": float(power[-1]),
48 }
49
50
51 def verify_arrays(detail, daily, audit, expected_days=334, start_day=31,
52                 initial_soc=1421.7991105135516, initial_mode=1,
53                 initial_power_kw=172.75999999999976, source_actual=None, source_price=None):
54     """Recompute physical, settlement, chronology and mode checks from arrays."""
55     errors = []
56
57     def check(condition, message):
58         if not bool(condition):
59             errors.append(message)
60
61     shapes = {key: (expected_days, 144) for key in
62               ("original", "final", "charge", "discharge", "emergency", "surplus", "price")}
63     shapes.update(states=(expected_days, 145), fees=(expected_days, 144, 4),
64                  actual=(expected_days, 144, 2))
65     for key, shape in shapes.items():
66         check(key in detail and detail[key].shape == shape, f"shape:{key}")
67     if errors:
68         return {"passed": False, "errors": errors}
69     for key, values in detail.items():
70         check(np.isfinite(values).all(), f"nonfinite:{key}")
71         check(np.min(values) >= -TOL, f"negative:{key}")
72     q, c, d, e, w, s, p = (detail[key] for key in
73                            ("final", "charge", "discharge", "emergency", "surplus", "states", "
74                             price"))
75     check(np.array_equal(detail["original"], q), "midnight_purchase_changed")
76     check(np.all(p > 0), "nonpositive_tariff")
77     if source_actual is not None:
78         check(np.array_equal(detail["actual"], source_actual), "source_actual_mismatch")
79     if source_price is not None:
80         check(np.array_equal(p, np.broadcast_to(source_price, p.shape)), "source_tariff_mismatch")
81     if all(key in detail for key in ("intended_charge", "intended_discharge", "intended_states")):
82         ic, ide, ist = (detail[key] for key in
83                        ("intended_charge", "intended_discharge", "intended_states"))
84         check(ic.shape == c.shape and ide.shape == d.shape and ist.shape == s.shape,
85              "intended_shapes")
86         check(np.max(np.abs(np.diff(ist, axis=1) - ETA * ic + ide / ETA)) <= TOL,
87              "intended_soc_equation")
88         check(np.min(ist) >= 1200 - TOL and np.max(ist) <= 10800 + TOL, "intended_soc_bounds")
89         check(max(np.max(ic), np.max(ide)) <= 5000 / 6 + TOL, "intended_power_limit")
90         check(not np.any((ic > TOL) & (ide > TOL)), "intended_simultaneous_actions")
91     balance = q + (detail["actual"][:, :, 1] - detail["actual"][:, :, 0]) / 6 + d + e - c - w
92     soc_error = np.diff(s, axis=1) - ETA * c + d / ETA
93     continuity = float(np.max(np.abs(s[1:, 0] - s[:-1, -1]), initial=0))
94     max_balance = float(np.max(np.abs(balance)))
95     max_state = float(np.max(np.abs(soc_error)))
96     check(max_balance <= TOL, "supply_demand_balance")
97     check(max_state <= TOL, "battery_state_equation")
98     check(continuity <= TOL, "cross_day_soc_discontinuity")
99     check(abs(float(s[0, 0]) - initial_soc) <= TOL, "initial_soc")
100    check(np.min(s) >= 1200 - TOL and np.max(s) <= 10800 + TOL, "soc_bounds")
101    check(max(np.max(c), np.max(d)) <= 5000 / 6 + TOL, "battery_power_limit")
102    check(not np.any((c > TOL) & (d > TOL)), "simultaneous_charge_discharge")
103    check(not np.any((c > TOL) & (e > TOL)), "emergency_charging")
104    expected_fees = np.stack((q * p, np.zeros_like(q), np.zeros_like(q), 5 * e * p), axis=-1)
105    fee_error = float(np.max(np.abs(expected_fees - detail["fees"])))
106    check(fee_error <= TOL, "slot_settlement")
107    expected_day_ids = list(range(start_day, start_day + expected_days))
108    check(len(daily) == expected_days, "daily_row_count")
109    check(len(audit) == expected_days, "audit_row_count")
110    if len(daily) == len(audit) == expected_days:
111        check([int(row["day"]) for row in daily] == expected_day_ids, "daily_order")
112        check([int(row["day"]) for row in audit] == expected_day_ids, "audit_order")
113        mode = initial_mode
114        daily_cost_error = 0.0
115        for index, (row, info) in enumerate(zip(daily, audit)):

```

```

115     day = start_day + index
116     check(int(info["information_cutoff"]) == day * 144, f"midnight_cutoff:{day}")
117     check(int(info["max_observed_index"]) < day * 144, f"future_actual_used:{day}")
118     check(all(int(o) + 144 <= day * 144 for o in info["training_origins"]),
119           f"incomplete_training_day:{day}")
120     check(int(info["forecast_origin"]) == day * 144, f"forecast_origin:{day}")
121     check(abs(float(info["execution_initial_soc"]) - s[index, 0]) <= TOL,
122           f"execution_initial_soc:{day}")
123     check(int(info["execution_initial_mode"]) == mode, f"cross_day_initial_mode:{day}")
124     if not info.get("controlled_replay_of_primary", False) and "planning_initial_soc" in
info:
125         check(abs(float(info["planning_initial_soc"]) - s[index, 0]) <= TOL,
126               f"planning_initial_soc:{day}")
127         check(int(info["planning_initial_mode"]) == mode, f"planning_initial_mode:{day}")
128         if day == 364 and "terminal_value_yuan_per_soc_kwh" in info:
129             check(info["terminal_value_yuan_per_soc_kwh"] == 0, "last_day_terminal_value")
130             observed_modes = np.sign(c[index] - d[index]).astype(int)
131             nonidle = observed_modes[np.abs(c[index] - d[index]) > TOL]
132             mode = int(nonidle[-1]) if len(nonidle) else mode
133             check(int(info["execution_final_mode"]) == mode, f"cross_day_final_mode:{day}")
134             daily_cost_error = max(daily_cost_error,
135                                   abs(float(row["total_cost"]) - float(expected_fees[index].sum()))
)
136             check(abs(float(row["initial_soc"]) - s[index, 0]) <= TOL, f"daily_initial_soc:{day}")
137             check(abs(float(row["final_soc"]) - s[index, -1]) <= TOL, f"daily_final_soc:{day}")
138             check(daily_cost_error <= TOL, "daily_settlement")
139         else:
140             daily_cost_error = None
141         return {
142             "passed": not errors, "errors": errors, "days": expected_days,
143             "intervals": expected_days * 144, "max_balance_error_kwh": max_balance,
144             "max_soc_error_kwh": max_state, "max_cross_day_soc_error_kwh": continuity,
145             "max_slot_fee_error_yuan": fee_error, "max_daily_cost_error_yuan": daily_cost_error,
146             "recomputed_total_cost": float(expected_fees.sum()),
147             "reported_cost_excludes_penalties_and_terminal_value": True,
148             "source_actual_and_tariff_checked": source_actual is not None and source_price is not None,
149             "battery_metrics": battery_metrics(detail, initial_mode, initial_power_kw),
150             "causality_scope": "archive cutoff audit plus separately tested future-mutation invariance",
151         }
152
153
154 def verify_directory(path):
155     from experiments.problem2.exp003.data import Data
156
157     path = Path(path)
158     with np.load(path / "dispatch_2.npz") as z:
159         detail = {key: z[key] for key in z.files}
160     daily = pd.read_csv(path / "daily.csv").to_dict("records")
161     audit = json.loads((path / "planning_audit.json").read_text())["days"]
162     data = Data()
163     return verify_arrays(detail, daily, audit, source_actual=data.actual[31 * 144:].reshape(334,
164     144, 2),
165                        source_price=data.fixed_price)
166
167 if __name__ == "__main__":
168     parser = argparse.ArgumentParser(description=__doc__)
169     parser.add_argument("directory", type=Path)
170     result = verify_directory(parser.parse_args().directory)
171     print(json.dumps(result, ensure_ascii=False, indent=2))
172     raise SystemExit(0 if result["passed"] else 1)

```